DEVOPS ENGINEERING MASTERY

PRODUCTION LINUX SERVER ENGINEERING

Linux Server Administration, Deployment & Production Operations

VOLUME 04 • FIRST EDITION

```
nahid@prod-server:~$ sudo systemctl status nginx
● nginx.service – active (running)
nahid@prod-server:~$ sudo ufw status
Status: active  •  22/tcp (OpenSSH)  •  80,443/tcp
nahid@prod-server:~$ pm2 status && sudo apt update && sudo apt upgrade -y
```

Ubuntu Server 24.04 LTS (Noble Numbat) — Primary hands-on environment

First Edition • Published 8 November 2026 • Dhaka, Bangladesh

24 Chapters • 24 Guided Labs • 3 Major Projects • 120+ MCQs

Production Linux Server Engineering

DEVOPS ENGINEERING MASTERY — Volume 04
Linux Server Administration, Deployment & Production Operations

First Edition — Published 8 November 2026.

Hands-on environment: Ubuntu Server 24.04 LTS (Noble Numbat, released April 2024, supported until April 2029). All commands in this book were written for Ubuntu 24.04 LTS unless a section explicitly says otherwise. Version-specific instructions are labelled as such; general Linux concepts are distinguished from Ubuntu-specific implementation. Cloud VM comparisons (AWS EC2, Microsoft Azure Virtual Machines, Google Compute Engine, IBM Cloud Virtual Servers) are high-level and operational; provider details vary by region and plan, and official provider documentation is the authority — no pricing or service limits are stated in this book.

How to use this book. Read the parts in order — each part builds on the previous one. Keep an Ubuntu Server virtual machine beside you and do every hands-on exercise and every lab in the companion Lab Workbook before moving on. Do not copy commands blindly; read the explanation under each command first. Destructive commands are always flagged with a WARNING box and a safe alternative. Sample terminal outputs are illustrative — your output will differ in dates, usernames, PIDs and sizes, and that is expected. Never practise on a production server: use a disposable VM or an isolated lab environment, and take a backup or snapshot before any risky operation.

A note on honesty. Reading a book alone does not make anyone an expert. This book teaches the foundations and the practical judgment to progress toward professional engineering — but the judgment itself comes from doing the labs, breaking a test VM, fixing it, and repeating. There is no shortcut for that practice.

Conventions. Bangla is the explanatory language; technical terms, command names, APIs and configuration keys stay in English. `monospace` marks commands, file paths and code. **CONCEPT** explains an idea, **REAL WORLD** shows an industry scenario, **WARNING** marks risk, **PRO TIP** gives professional advice, and **LAB** marks hands-on work. All domains, IP addresses and credentials in examples are fictional (technova.example, 203.0.113.x, CHANGE-ME-...).

© 2026 DevOps Engineering Mastery series. All rights reserved. This book is for personal learning. Commands that change a system should first be practised in a virtual machine or a disposable test server, never directly on a production machine. The author and publisher accept no liability for data loss caused by running commands without a backup.

Who this book is for

A learner who knows JavaScript, React, Next.js, Node.js, Express and MongoDB, has basic computer and programming knowledge, and wants to become capable of managing real production infrastructure — starting from foundational Linux administration and reaching deployment, security, backup and incident response.

Learning outcomes

By the end of this book you will be able to: install and configure Ubuntu Server; manage SSH, users, groups, sudo and access; administer filesystems, partitions, mounts and LVM; manage processes, signals and resources; maintain packages and updates; write and manage systemd services; configure Nginx, virtual hosts, reverse proxies and TLS; deploy Node.js applications with PM2 and systemd and explain when to choose each; operate PostgreSQL and MongoDB servers; diagnose disk, CPU, memory and network problems; configure UFW and nftables firewalls; harden a Linux server and SSH; design backups and disaster recovery; automate maintenance with cron; manage logs with journald and logrotate; deploy a MERN application to production; manage environment variables and secrets safely; choose deployment strategies and roll back safely; and respond to production incidents with runbooks.

সূচিপত্র — Contents

PART I — Linux Server Administration

Chapter 01 — Understanding Production Servers and Server Architecture 7

Chapter 02 — Ubuntu Server Installation and Initial Configuration 14

Chapter 03 — SSH, Users, Groups, Sudo and Access Management 21

Chapter 04 — Linux Filesystems, Partitions, Mounts and LVM 28

Chapter 05 — Processes, Signals, Jobs and Resource Management 35

Chapter 06 — Package Management, Updates and System Maintenance 42

PART II — Server Services and Application Infrastructure

Chapter 07 — systemd Architecture and Service Management 50

Chapter 08 — Nginx Architecture and Web Server Configuration 57

Chapter 09 — Virtual Hosts, Reverse Proxy and TLS Configuration 63

Chapter 10 — Deploying Node.js Applications on Linux 70

Chapter 11 — Process Management with PM2 and systemd 76

Chapter 12 — Database Server Operations: PostgreSQL and MongoDB Fundamentals 84

PART III — Production Operations and Security

Chapter 13 — Disk Management, Storage Health and Inode Troubleshooting 92

Chapter 14 — CPU, Memory and Process Performance Analysis 98

Chapter 15 — Linux Server Networking and Network Troubleshooting 104

Chapter 16 — Firewalls: UFW, nftables and Network Access Control 110

Chapter 17 — Linux Security Hardening and SSH Protection 116

Chapter 18 — Backup, Restore and Disaster Recovery Fundamentals 122

Chapter 19 — Cron, Scheduled Tasks and Maintenance Automation 127

Chapter 20 — Server Logs, Journald, Logrotate and Health Checks 132

PART IV — Deployment, Reliability and Operations

Chapter 21 — Production Deployment of a MERN Application 139

Chapter 22 — Environment Variables, Secrets and Configuration Management 148

Chapter 23 — Deployment Strategies, Rollback and Downtime Reduction 156

Chapter 24 — Production Incident Response, Runbooks and Server Architecture 163

Appendices

Appendix A — Glossary (150+ Terms) 171

Appendix B — References 178

Appendix C — Next Steps 179

Guided labs (24), major projects (3) and the capstone are in the separate *Linux Server Lab Workbook*. MCQs, assignments, examinations and the answer key are in the separate *Assessment and Answer Guide*. Commands, checklists and decision trees are in the separate *Quick Reference Handbook*.

PART I • DEVOPS ENGINEERING MASTERY

Linux Server Administration

প্রোডাকশন সার্ভার, Ubuntu ইনস্টলেশন, SSH ও অ্যাক্সেস, ফাইলসিস্টেম, প্রসেস ও প্যাকেজ ব্যবস্থাপনা।

অধ্যায় ১

প্রোডাকশন সার্ভার এবং সার্ভার আর্কিটেকচার বোঝা

১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) প্রোডাকশন সার্ভার (Production Server) — অর্থাৎ আসল ব্যবহারকারীদের সেবা দেওয়া কম্পিউটার — এবং ডেভেলপমেন্ট মেশিনের পার্থক্য ব্যাখ্যা করতে; (২) সার্ভারের হার্ডওয়্যার ও সফটওয়্যার স্তরগুলো চিনতে; (৩) CPU, RAM, Disk, Network কীভাবে একসঙ্গে কাজ করে তা বোঝাতে; (৪) একটি Node.js অ্যাপের অনুরোধ (request) সার্ভারের ভেতর দিয়ে কী পথে যায় তা আঁকতে; (৫) On-Premise, VPS, ক্লাউড ও Bare Metal-এর মধ্যে সিদ্ধান্ত নিতে।

১.২ প্রোডাকশনে কেন এটি গুরুত্বপূর্ণ

আপনার ল্যাপটপে Next.js অ্যাপ চললেই তা প্রোডাকশন-প্রস্তুত নয়। ল্যাপটপ বন্ধ হলে, Wi-Fi গেলে বা একসঙ্গে হাজার ব্যবহারকারী এলে অ্যাপ থেমে যায়। প্রোডাকশন সার্ভার ২৪ ঘণ্টা চালু থাকে, একসঙ্গে অনেক অনুরোধ সামলায় এবং ব্যর্থ হলেও দ্রুত ফিরে আসে। সার্ভার আর্কিটেকচার না বুঝলে আপনি সমস্যার মূল কারণ খুঁজে পাবেন না — অ্যাপ ধীর হলে বুঝবেন না সেটা CPU-র দোষ, নাকি RAM শেষ, নাকি ডিস্ক ধীর, নাকি নেটওয়ার্ক। এই বইয়ের বাকি সব অধ্যায় এই ভিত্তির ওপর দাঁড়িয়ে আছে।

১.৩ পূর্বশর্ত জ্ঞান

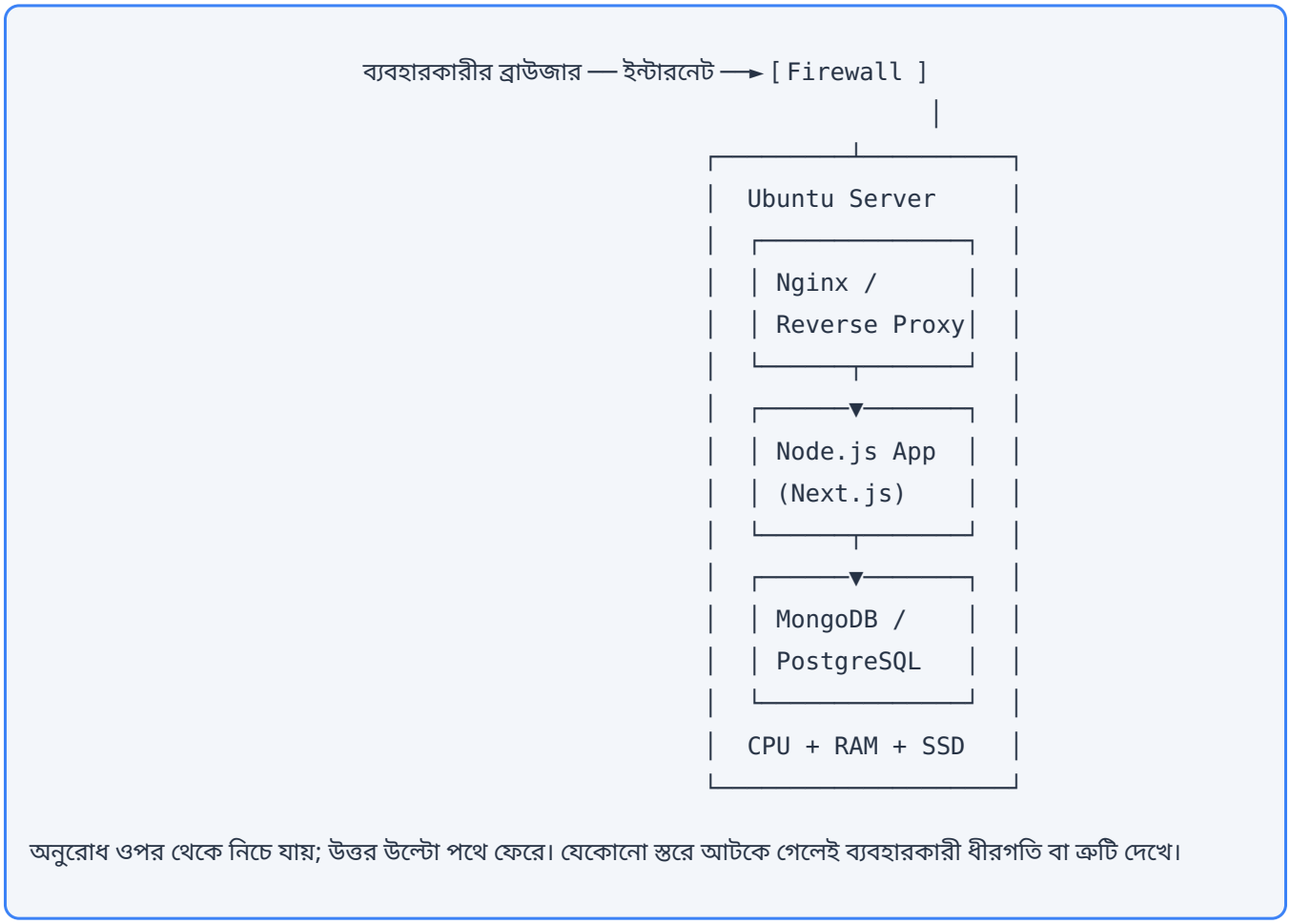
JavaScript, React/Next.js, Node.js/Express এবং MongoDB-র প্রাথমিক ধারণা থাকলেই চলবে। Linux-এর পূর্ব অভিজ্ঞতা প্রয়োজন নেই। টার্মিনাল (Terminal) — কালো পর্দায় লেখা দিয়ে কম্পিউটারকে নির্দেশ দেওয়ার উপায় — এই অধ্যায়ে পরিচয় করিয়ে দেওয়া হবে, বিস্তারিত ব্যবহার পরের অধ্যায়গুলোতে আসবে।

১.৪ প্রথম নীতি থেকে ব্যাখ্যা

সার্ভার আসলে একটি বিশেষ কম্পিউটার ছাড়া আর কিছুই নয়। যেকোনো কম্পিউটারের চারটি মৌলিক কাজ: **গণনা করা** (CPU), **সাময়িকভাবে মনে রাখা** (RAM / Memory), **স্থায়ীভাবে সংরক্ষণ করা** (Disk / Storage) এবং **অন্যের সঙ্গে যোগাযোগ করা** (Network)। আপনার ফোনেও এই চারটিই আছে। পার্থক্য শুধু উদ্দেশ্যে: ল্যাপটপ একজন ব্যবহারকারীর জন্য, সার্ভার হাজারো ব্যবহারকারীর জন্য একটানা সেবা দেয়। তাই সার্ভারে থাকে শক্তিশালী CPU, বেশি RAM, দ্রুত SSD ডিস্ক, নির্ভরযোগ্য নেটওয়ার্ক এবং ব্যর্থতা সামলানোর ব্যবস্থা (Redundancy)।

> **CONCEPT:** ক্লায়েন্ট-সার্ভার মডেল মনে রাখুন: ব্রাউজার (Client) অনুরোধ পাঠায়, সার্ভার উত্তর দেয়। ইন্টারনেট মূলত এই সরল আদান-প্রদানেরই বিশাল বিস্তার।

১.৫ আর্কিটেকচার / ধারণাগত চিত্র



অনুরোধ ওপর থেকে নিচে যায়; উত্তর উল্টো পথে ফেরে। যেকোনো স্তরে আটকে গেলেই ব্যবহারকারী ধীরগতি বা ত্রুটি দেখে।

১.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

হার্ডওয়্যার স্তর

CPU (Central Processing Unit) — কম্পিউটারের মস্তিষ্ক; হিসাব করে। বেশি Core মানে একসঙ্গে বেশি কাজ। Node.js একক থ্রেডে চলে, তাই CPU-বহুল কাজে Worker বা একাধিক প্রসেস লাগে। **RAM** — চলমান প্রোগ্রাম ও ডেটা রাখে; বিদ্যুৎ গেলে মুছে যায়। RAM শেষ হলে সিস্টেম ধীর হয়ে যায় বা প্রসেস মারা যায়। **Disk** — স্থায়ী সংরক্ষণ। SSD (Solid State Drive) HDD-র চেয়ে অনেক দ্রুত; ডেটাবেসের জন্য SSD প্রায় বাধ্যতামূলক। **Network** — Bandwidth (কত ডেটা যায়) ও Latency (কত দেরিতে পৌঁছায়) দুটোই গুরুত্বপূর্ণ।

সফটওয়্যার স্তর

সবচেয়ে নিচে **Kernel** — Linux-এর হৃদয়, যা হার্ডওয়্যারের সঙ্গে কথা বলে। তার ওপরে **Operating System** (Ubuntu Server), তার ওপরে **সার্ভিস** — যেমন systemd দিয়ে চালানো Node.js অ্যাপ, Nginx, ডেটাবেস। সবচেয়ে ওপরে আপনার অ্যাপ্লিকেশন কোড। এই স্তরবিন্যাসের নাম **Stack**। কোনো স্তর ভেঙে গেলে ওপরের স্তরও ভাঙে — তাই সমস্যা নির্ণয়ে নিচ থেকে ওপরে পরীক্ষা করুন।

হোস্টিং ধরন

ধরন	অর্থ	কখন ভালো

Bare Metal	পুরো আসল মেশিন আপনার	সর্বোচ্চ নিয়ন্ত্রণ ও কর্মক্ষমতা দরকার হলে
VPS	এক মেশিন ভাগ করে ভার্চুয়াল সার্ভার	শেখা ও ছোট প্রোডাকশনের জন্য সেরা শুরু
Cloud VM	ক্লাউডে চাহিদামতো ভাড়া করা সার্ভার	স্কেল করা ও স্বয়ংক্রিয়তা দরকার হলে
On-Premise	নিজের অফিস/ডেটা সেন্টারে মেশিন	কঠোর ডেটা নিয়ন্ত্রণ দরকার হলে

Uptime মানে সার্ভার কতক্ষণ নিরবচ্ছিন্ন চালু ছিল। ৯৯.৯% আপটাইম মানে বছরে প্রায় ৯ ঘণ্টা বন্ধ থাকা মেনে নেওয়া হয়। প্রোডাকশনে লক্ষ্য: যত কম Downtime (বন্ধ সময়) তত ভালো।

কাজের উদাহরণ: ধীর সাইটের বোতল-গলা খোঁজা

ধরুন app.technova.example-এর চেকআউট পাতা হঠাৎ ধীর হয়ে গেল। অভিজ্ঞ প্রকৌশলী আন্দাজ করেন না — তিনি স্তর ধরে পরীক্ষা করেন: প্রথমে `uptime` দিয়ে load average, তারপর `free -h` দিয়ে RAM, তারপর `df -h` দিয়ে ডিস্ক, আর শেষে ব্রাউজার থেকে নেটওয়ার্ক। চারটির লক্ষণ আলাদা, তাই ভুল স্তরে ব্যবস্থা নিলে সমস্যা থেকেই যায় — RAM শেষ হলে CPU বাড়িয়ে লাভ হয় না।

সন্দেহভাজন স্তর	লক্ষণ	প্রথমে কী দেখবেন	সম্ভাব্য ব্যবস্থা
CPU	সব পাতা ধীর, load average core সংখ্যার চেয়ে অনেক বেশি	`uptime`, `nproc`	ভারী কাজ আলাদা প্রসেসে ভাগ করা; কোডের অসীম লুপ খোঁজা
RAM	মাঝে মাঝে প্রসেস মারা যায়, সাইট কখনো খোলে কখনো নয়	`free -h`-এর available কলাম	মেমরি-খেকো প্রসেস চিহ্নিত করা; প্রয়োজনে RAM বাড়ানো
Disk	লগইন বা আপলোডে দেরি, ডেটাবেস লেখায় আটকে যায়	`df -h`, ডিস্ক ব্যবহারের শতাংশ	পুরোনো লগ ও অস্থায়ী ফাইল পরিষ্কার করা; আলাদা পার্টিশন ভাবা
Network	লেখা দ্রুত আসে, ছবি ও ফাইল ধীরে নামে	অন্য নেটওয়ার্ক থেকে পাতা খুলে তুলনা করা	বড় ফাইল ক্যাশ করা; নেটওয়ার্ক পথ পরীক্ষা করা

সময়ের ধরনটিও একটি সূত্র — দুপুরে ধীর হলে হয়তো তখন একটি নির্ধারিত ব্যাকআপ চলছে। "কখন থেকে এমন হচ্ছে" প্রশ্নটি করুন এবং Baseline নোটের সঙ্গে মিলিয়ে দেখুন।

১.৭ গুরুত্বপূর্ণ কমান্ড ও তার বিশ্লেষণ

এই অধ্যায়ে শুধু পরিচয় — বিস্তারিত ব্যবহার পরের অধ্যায়ে। তবু তিনটি কমান্ড এখনই চিনে রাখুন:

```
uname -a        # কার্নেল ও সিস্টেমের পরিচয় দেখায়
lscpu           # CPU-র তথ্য: কয়টি core, কী মডেল
free -h         # RAM কত আছে, কত ব্যবহৃত হচ্ছে (-h মানে মানুষের পড়ার মতো একক)
```

নমুনা আউটপুট — চিত্রণমূলক:

```
নমুনা আউটপুট — চিত্রণমূলক
$ free -h
```

```
               total        used        free
Mem:           3.8Gi       1.2Gi       2.6Gi
```

এখানে মোট RAM ৩.৮ GiB, ব্যবহৃত ১.২ GiB — অর্থাৎ এখনো যথেষ্ট জায়গা আছে।

```
nproc
# CPU core সংখ্যা দেখায়; load average-এর সঙ্গে তুলনা করতে কাজে লাগে
# যেমন ২ core-এ load average ৪.০ মানে CPU-র জন্য অপেক্ষা জমে আছে

lsblk
# ডিস্ক, পার্টিশন ও মাউন্ট পয়েন্টের গাছ-চিত্র দেখায়

ip -br addr
# প্রতিটি নেটওয়ার্ক ইন্টারফেসের IP সংক্ষেপে দেখায়
# -br = brief, সংক্ষিপ্ত রূপ; পূর্ণ বিবরণের জন্য ip addr show

uptime
# সার্ভার কতক্ষণ চালু আছে, কতজন ব্যবহারকারী লগইনে আছে
# ও ১, ৫ ও ১৫ মিনিটের load average দেখায়
```

সকালে `nproc` দিয়ে core সংখ্যা জেনে রাখুন, দিনে কয়েকবার `uptime` দিয়ে load average মেলান — দ্বিগুণ হলে চাপে আছে।

১.৮ কনফিগারেশন উদাহরণ

একটি ছোট প্রোডাকশন সার্ভারের যৌক্তিক কনফিগারেশন (ল্যাবের জন্য): ২ vCPU, ৪ GB RAM, ৫০ GB SSD, Ubuntu Server 24.04 LTS। এতে একটি Next.js অ্যাপ, Nginx ও MongoDB একসঙ্গে পরীক্ষামূলকভাবে চালানো যায়। মনে রাখবেন: ল্যাব কনফিগারেশন প্রোডাকশনের অনুমান নয় — আসল ট্রাফিক মেপে তবেই আকার নির্ধারণ করুন।

১.৯ বাস্তব প্রোডাকশন পরিস্থিতি

REAL WORLD: কাল্পনিক কোম্পানি Technova-র ই-কমার্স সাইট app.technova.example — ঈদের অফারে হঠাৎ ট্রাফিক দশ গুণ বেড়ে গেল। সার্ভারের RAM শেষ হয়ে Node.js প্রসেস মারা যেতে লাগল। কারণ ছিল আর্কিটেকচার না বোঝা: একটিমাত্র ছোট VPS-এ অ্যাপ, ডেটাবেস ও ক্যাশ সব একসঙ্গে চলছিল। সমাধান ছিল স্তর আলাদা করা ও মনিটরিং যোগ করা — যা এই বইয়ের পরের খণ্ডগুলোতে আসবে।

আরেকটি দৃশ্য: ডিস্ক ১০০% ভরে গেলে ডেটাবেস লেখা বন্ধ করে দেয়, লগ লেখা যায় না, অ্যাপ ক্র্যাশ করে। তাই ডিস্ক ব্যবহার পর্যবেক্ষণ করা প্রোডাকশনের দৈনন্দিন কাজ।

REAL WORLD: Technova-র সহায়তা দল এক শুক্রবার রাতে অভিযোগ পেল — app.technova.example-এর ছবি ধীরে নামছে, কিন্তু লেখা ঠিকঠাক আসছে। প্রথমে সবাই ভেবেছিল সার্ভার দুর্বল। পরীক্ষায় দেখা গেল CPU ও RAM স্বাভাবিক, কিন্তু ডিস্ক ৯২% ভরা — অস্থায়ী ফোল্ডারে পুরোনো আপলোড জমে গিয়েছিল। দল অস্থায়ী ফাইল পরিষ্কার করে সাইট ফিরিয়ে আনল, আর পরে আলাদা জায়গা ও ৮০%-এ সতর্কবার্তার ব্যবস্থা করল। শিক্ষা: ছবি ধীর, লেখা দ্রুত মানে ডিস্ক বা নেটওয়ার্কের দিকে তাকান, CPU-র দিকে নয়।

১.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: আপনার প্রথম সার্ভারের স্বাস্থ্য দেখা (ল্যাব VM-এ)

1. ল্যাব VM চালু করে লগইন করুন।
2. `uname -a` চালিয়ে Ubuntu সংস্করণ নিশ্চিত করুন।
3. `lscpu` দিয়ে CPU core সংখ্যা লিখে রাখুন।
4. `free -h` ও `df -h` দিয়ে RAM ও ডিস্ক দেখুন।
5. ফলাফল একটি নোটে লিখুন — এটাই আপনার সার্ভারের Baseline (স্বাভাবিক অবস্থার রেকর্ড)। পরে কিছু অস্বাভাবিক মনে হলে এর সঙ্গে তুলনা করবেন।

১.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল ধারণা:** "সার্ভার মানেই বড় কালো বাক্স।" — আসলে যেকোনো কম্পিউটারই সার্ভার হতে পারে; ভূমিকাই পার্থক্য গড়ে।
- **ভুল:** ল্যাপটপের সেটিংস হুবহু প্রোডাকশনে নকল করা। ল্যাপটপ ঘুমায়, সার্ভার ঘুমায় না।
- **ভুল:** RAM বেশি হলেই সব ঠিক — CPU, ডিস্ক I/O বা নেটওয়ার্কও বোতল-গলা (Bottleneck) হতে পারে।
- **ভুল:** "একবার সেটআপ করলেই শেষ।" — সার্ভার জীবন্ত ব্যবস্থা; আপডেট, লগ ও মনিটরিং চলতেই থাকে।

১.১২ নিরাপত্তা বিবেচনা

সার্ভার ইন্টারনেটে উন্মুক্ত মানেই আক্রমণকারীরা খুঁজে পাবে। প্রথম দিন থেকে: অপ্রয়োজনীয় পোর্ট বন্ধ রাখুন, শক্তিশালী পাসওয়ার্ড ব্যবহার করুন, SSH-তে কি-ভিত্তিক লগইন শিখুন (অধ্যায় ৩)। কখনোই পরীক্ষার সার্ভারকে আসল ডেটা দিয়ে ইন্টারনেটে খুলে দেবেন না।

১.১৩ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	প্রথম পরীক্ষা
সাইট খুলছে না	সার্ভার বন্ধ / নেটওয়ার্ক বিচ্ছিন্ন	সার্ভার চালু কি না, ping যায় কি না দেখুন
অ্যাপ খুব ধীর	CPU/RAM/ডিস্ক চাপ	free, df, uptime দিয়ে সম্পদ দেখুন
হঠাৎ ক্র্যাশ	RAM শেষ (OOM)	লগে Out of Memory খুঁজুন
সাইট মাঝে মাঝে খোলে, মাঝে মাঝে নয়	RAM প্রায় শেষ, প্রসেস মাঝেমধ্যে মারা যাচ্ছে	`free -h`-এর available দেখুন; লগে Out of Memory খুঁজুন
ছবি ও ফাইল ধীরে নামে, লেখা দ্রুত আসে	ডিস্ক প্রায় ভরা বা নেটওয়ার্ক পথে দেরি	`df -h` দিয়ে ডিস্কের শতাংশ দেখুন; অন্য নেটওয়ার্ক থেকে তুলনা করুন
সকালে ঠিক, দুপুরে ধীর	দুপুরের নির্ধারিত কাজ (ব্যাকআপ বা রিপোর্ট) সম্পদ খাচ্ছে	কখন ধীর হয় তার সময় লিখুন; ওই সময়ে কী কাজ চলে, মিলিয়ে দেখুন
নতুন ডেপ্লয়ের পর থেকে ধীর	নতুন কোডে ভারী লুপ বা অদক্ষ ডেটাবেস কোয়েরি	আগের সংস্করণের Baseline-এর সঙ্গে তুলনা করুন; পরিবর্তনটি ফিরিয়ে পরীক্ষা করুন

১.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: Baseline রেকর্ড রাখুন; স্তর আলাদা রাখুন; পরিবর্তনের আগে ব্যাকআপ নিন। **অ্যান্টি-প্যাটার্ন:** এক সার্ভারে সবকিছু গাদাগাদি করে চালানো; কোনো মনিটরিং ছাড়া প্রোডাকশনে যাওয়া; "আমার ল্যাপটপে চলেছিল" — এই যুক্তিতে প্রোডাকশন সিদ্ধান্ত নেওয়া।

১.১৫ অধ্যায়ের সারাংশ

সার্ভার একটি বিশেষ উদ্দেশ্যে ব্যবহৃত কম্পিউটার: CPU গণনা করে, RAM সাময়িক মনে রাখে, Disk স্থায়ীভাবে সংরক্ষণ করে, Network যোগাযোগ করে। Kernel হার্ডওয়্যার চালায়, তার ওপরে OS ও অ্যাপ চলে। প্রোডাকশন মানে একটানা, নির্ভরযোগ্য সেবা — আর তার জন্য দরকার আর্কিটেকচার বোঝা, Baseline জানা ও প্রতিটি স্তর আলাদাভাবে পরীক্ষা করার অভ্যাস।

১.১৬ মূল প্রযুক্তিগত পরিভাষা

Production Server: আসল ব্যবহারকারীদের সেবা দেওয়া সার্ভার।

Kernel: অপারেটিং সিস্টেমের মূল অংশ; হার্ডওয়্যারের সঙ্গে সরাসরি কথা বলে।

Uptime / Downtime: চালু থাকার সময় / বন্ধ থাকার সময়।

Bottleneck: ব্যবস্থার সবচেয়ে ধীর অংশ, যা পুরো গতিকে আটকে রাখে।

VPS: Virtual Private Server — ভাগ করা হার্ডওয়্যারে ভার্চুয়াল সার্ভার।

Baseline: স্বাভাবিক অবস্থার রেকর্ড, যার সঙ্গে পরে তুলনা করা হয়।

১.১৭ জ্ঞান যাচাই প্রশ্ন

1. প্রোডাকশন সার্ভার ও ডেভেলপমেন্ট ল্যাপটপের তিনটি মৌলিক পার্থক্য কী?
2. CPU, RAM, Disk ও Network-এর ভূমিকা নিজের ভাষায় ব্যাখ্যা করুন।
3. Kernel কী এবং কেন এটি অপারেটিং সিস্টেমের হৃদয় বলা হয়?
4. Bare Metal, VPS ও Cloud VM-এর মধ্যে কখন কোনটি বেছে নেবেন?
5. ৯৯.৯% আপটাইম মানে বছরে কতক্ষণ Downtime মেনে নেওয়া যায়, এবং কেন ১০০% বাস্তবসম্মত নয়?
6. একটি অনুরোধ ব্রাউজার থেকে ডেটাবেস পর্যন্ত কোন পথে যায়, তা ধাপে ধাপে লিখুন।

- একটি সাইট ধীর — CPU, RAM, ডিস্ক ও নেটওয়ার্কের মধ্যে কোনটি দায়ী, তা ধাপে ধাপে কীভাবে নির্ণয় করবেন? প্রতিটি ধাপে কোন কমান্ড চালাবেন, তা লিখুন।

- আপনার ল্যাব সার্ভারে ২টি CPU core থাকলে load average ৪.০ মানে কী, এবং কেন এটি চাপের লক্ষণ? core সংখ্যা ৪ হলে একই সংখ্যার অর্থ কি বদলে যাবে?

১.১৮ ব্যবহারিক অনুশীলন

> **LAB:** সব অনুশীলন ডিসপোজেবল ল্যাব VM-এ করুন। প্রোডাকশন সার্ভারে নয়।

1. ল্যাব VM-এ `uname -a`, `lscpu`, `free -h`, `df -h` চালিয়ে একটি Baseline নোট তৈরি করুন।
2. একটি চিত্র আঁকুন: আপনার Next.js অ্যাপের অনুরোধ কীভাবে ব্রাউজার থেকে সার্ভার, Nginx, Node.js ও MongoDB পর্যন্ত যায়।
3. তিনটি হোস্টিং ধরনের (VPS, Cloud VM, Bare Metal) সুবিধা-অসুবিধার তুলনামূলক ছক বানান এবং Technova-র মতো ছোট স্টার্টআপের জন্য কোনটি দিয়ে শুরু করবেন, যুক্তিসহ লিখুন।

- একটি কাল্পনিক ঘটনার নির্ণয়-নোট লিখুন: Baseline-এ available ছিল ২.৬ GiB, এখন তা প্রায় শূন্য, আর ডিস্ক ৯৫% ভরা। কোন স্তর আগে পরীক্ষা করবেন, কেন, এবং প্রথম তিনটি ব্যবস্থা কী হবে — লিখুন।

অধ্যায় ২

Ubuntu Server ইনস্টলেশন ও প্রাথমিক কনফিগারেশন

২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Ubuntu Server 24.04 LTS ISO দিয়ে ল্যাব VM ইনস্টল করতে; (২) ইনস্টলারের প্রতিটি ধাপের অর্থ বুঝতে; (৩) প্রথম বুটের পর hostname, timezone, নেটওয়ার্ক ও আপডেট ঠিক করতে; (৪) একটি নিরাপদ প্রাথমিক কনফিগারেশন চেকলিস্ট প্রয়োগ করতে।

২.২ প্রোডাকশনে কেন এটি গুরুত্বপূর্ণ

ভুলভাবে ইনস্টল করা সার্ভার পরে বারবার ভোগায়: ভুল timezone-এ লগের সময় মেলে না, ভুল পার্টিশনে ডিস্ক ভরে যায়, নেটওয়ার্ক ভুল হলে SSH-ই ঢোকা যায় না। প্রোডাকশন সার্ভারের ৮০% ঝামেলা শুরু হয় ইনস্টলের সময়ের ছোট ছোট সিদ্ধান্ত থেকে। LTS (Long Term Support) বেছে নেওয়া মানে ২০২৯ সালের এপ্রিল পর্যন্ত নিরাপত্তা আপডেট পাওয়া — প্রোডাকশনে স্থিতিশীলতাই প্রথম শর্ত।

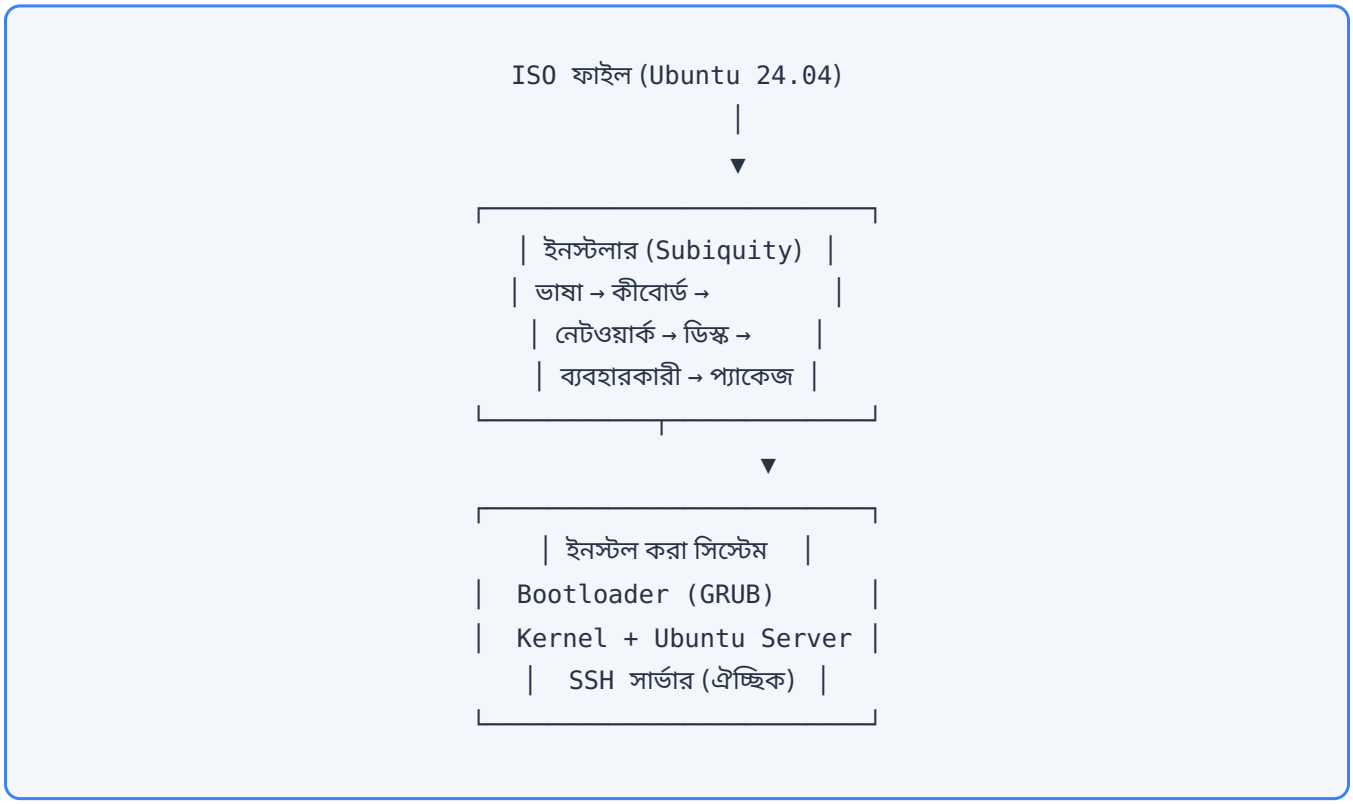
২.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১-এর সার্ভার আর্কিটেকচার ধারণা, ISO ফাইল (অপারেটিং সিস্টেমের ইনস্টলেশন ছবি) ও Virtual Machine (আসল কম্পিউটারের ভেতরে চলা নকল কম্পিউটার) সম্পর্কে সাধারণ ধারণা। VirtualBox বা ক্লাউডে VM বানানোর অভিজ্ঞতা থাকলে ভালো, না থাকলেও ধাপগুলো অনুসরণ করা যাবে।

২.৪ প্রথম নীতি থেকে ব্যাখ্যা

অপারেটিং সিস্টেম ইনস্টল করা মানে খালি ডিস্কে তিনটি জিনিস বসানো: (১) **Bootloader** — কম্পিউটার চালু হলে প্রথমে চলা ছোট প্রোগ্রাম, যা Kernel খুঁজে চালু করে; (২) **Kernel ও সিস্টেম ফাইল** — Linux-এর মূল অংশ; (৩) **কনফিগারেশন** — সার্ভারটির নাম, নেটওয়ার্ক ঠিকানা, ব্যবহারকারী। ইনস্টলার আসলে এই কাজগুলো ধাপে ধাপে করে দেয়। Ubuntu Server-এ গ্রাফিক্যাল ডেস্কটপ থাকে না — শুধু টার্মিনাল — কারণ সার্ভারে ডেস্কটপ সম্পদের অপচয় এবং আক্রমণের বাড়তি পথ।

২.৫ আর্কিটেকচার / ধারণাগত চিত্র



২.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

ইনস্টলারের ধাপসমূহ

ভাষা ও কীবোর্ড: English ও US কীবোর্ড সাধারণ পছন্দ; সার্ভারের লগ ও ত্রুটি-বার্তা ইংরেজিতেই পড়া সহজ। **নেটওয়ার্ক:** DHCP (স্বয়ংক্রিয়ভাবে IP ঠিকানা দেওয়ার ব্যবস্থা) ল্যাবে ঠিক আছে; প্রোডাকশনে সাধারণত Static IP (স্থির ঠিকানা) ব্যবহার হয় যাতে ঠিকানা বদলে না যায়। **Storage:** "Use an entire disk" ল্যাবের জন্য নিরাপদ পছন্দ; LVM (Logical Volume Management) টিক দিয়ে রাখলে পরে ডিস্ক বাড়ানো সহজ হয় — বিস্তারিত অধ্যায় ৪-এ। **Profile:** আপনার নাম, সার্ভারের নাম (hostname) ও ব্যবহারকারীর নাম দিন। **SSH:** "Install OpenSSH server" টিক দিন — দূর থেকে সার্ভার চালানোর জন্য এটি অপরিহার্য।

প্রথম বুটের পর

লগইন করে প্রথমে সিস্টেম আপডেট করুন, hostname ও timezone ঠিক করুন এবং SSH চলছে কি না দেখুন। Ubuntu 24.04-এ নেটওয়ার্ক কনফিগার করে **Netplan** — `/etc/netplan/` ফাইলে YAML আকারে লেখা থাকে। ভুল YAML-এ একটি স্পেসও নেটওয়ার্ক ভেঙে দিতে পারে, তাই পরিবর্তনের আগে ফাইলের কপি রাখুন।

দ্বিতীয় দৃশ্য: ইনস্টল-সময়ের সিদ্ধান্তের দীর্ঘমেয়াদি পরিণতি

দুটি সার্ভারের কথা ভাবুন। প্রথমটিতে ইনস্টলের সময় LVM টিক দেওয়া হয়নি, দ্বিতীয়টিতে দেওয়া হয়েছে। ছয় মাস পর দুটিতেই ডিস্ক প্রায় ভরে গেল। প্রথম সার্ভারে জায়গা বাড়াতে হলে পার্টিশন নিয়ে ঝুঁকিপূর্ণ কাজ করতে হয়, কখনো ডেটা সরাতে হয়; দ্বিতীয়টিতে LVM দিয়ে Logical Volume বাড়ানো যায় — বিস্তারিত অধ্যায় ৪-এ আসবে। একইভাবে, প্রথম সার্ভারে SSH টিক দেওয়া হয়নি বলে পরে কনসোল দিয়ে গিয়ে SSH ইনস্টল করতে হলো, আর দ্বিতীয়টিতে প্রথম দিন থেকেই দূর থেকে কাজ করা গেল। ইনস্টলার শুধু একটি ফরম নয়, ভবিষ্যতের রক্ষণাবেক্ষণের ভিত্তি।

ইনস্টল-সময়ের পছন্দ	সুবিধা	পরে অসুবিধা, যদি ভুল হয়
LVM সহ পুরো ডিস্ক ব্যবহার	পরে ডিস্কের আকার নমনীয়ভাবে বাড়ানো যায়	শুরুতে ধারণা না থাকলে স্তরগুলো বুঝতে সময় লাগে
LVM ছাড়া পুরো ডিস্ক ব্যবহার	কাঠামো সহজ, শুরুতে বোঝা সহজ	পরে জায়গা বাড়ানো কঠিন ও ঝুঁকিপূর্ণ
OpenSSH server সহ ইনস্টল	প্রথম দিন থেকেই দূর থেকে সার্ভার চালানো যায়	ফায়ারওয়াল ও কী-লগইন না শিখে ইন্টারনেটে খুলে দেওয়া বিপজ্জনক
হাতে ভাগ করা পার্টিশন	`/var/log`-এর মতো অংশ আলাদা রাখা যায়	ভুল আকার দিলে একটি পার্টিশন ভরে যায়, অন্যটি ফাঁকা থাকে

ল্যাবে দুটি পথই একবার করে পরীক্ষা করুন: একটি VM LVM সহ, আরেকটি ছাড়া — দুটির `lsblk` আউটপুট পাশাপাশি রেখে পার্থক্যটি নিজের চোখে দেখুন।

২.৭ গুরুত্বপূর্ণ কমান্ড ও তার বিশ্লেষণ

```
sudo apt update && sudo apt upgrade -y
# apt = প্যাকেজ ব্যবস্থাপক; update = প্যাকেজ তালিকা হালনাগাদ করে
# upgrade = ইনস্টল করা প্যাকেজ নতুন সংস্করণে তোলে; sudo = প্রশাসক হিসেবে চালানো

hostnamectl
# সার্ভারের নাম, OS, Kernel সংস্করণ দেখায়

timedatectl
# সময়, timezone ও NTP (ইন্টারনেট সময় মেলানো) অবস্থা দেখায়

sudo timedatectl set-timezone Asia/Dhaka
# বাংলাদেশের সময় অঞ্চল নির্ধারণ করে

ip addr show
# নেটওয়ার্ক ইন্টারফেস ও IP ঠিকানা দেখায়

systemctl status ssh
# SSH সার্ভিস চলছে কি না দেখায়
```

```
cat /etc/os-release
# কোন Ubuntu সংস্করণ ইনস্টল আছে, তা দেখায়
# NAME, VERSION ও সংস্করণ-নম্বরসহ বিস্তারিত তথ্য থাকে

lsb_release -a
# সংস্করণ, কোডনাম (যেমন Noble) ও বিবরণ সংক্ষেপে দেখায়
# -a = সব তথ্য (all)

sudo systemctl is-enabled ssh
# SSH সার্ভিস বুটে স্বয়ংক্রিয়ভাবে চালু হবে কি না, তা দেখায়
# enabled মানে বুটে চালু হবে, disabled মানে হবে না

ip route show
```

```
# ডিফল্ট গেটওয়ে (যে পথে ইন্টারনেটে যায়) দেখায়
# Static IP ঠিক করার পর এই পথটি মিলিয়ে নিন
```

ইনস্টলের পর যাচাই করুন: `cat /etc/os-release` দিয়ে 24.04 LTS নিশ্চিত করুন, আর `systemctl is-enabled ssh` দিয়ে দেখুন রিবুটের পরও SSH চালু হবে কি না।

২.৮ কনফিগারেশন উদাহরণ

hostname বদলানো ও যাচাই:

```
sudo hostnamectl set-hostname technova-web-01
hostnamectl
# নতুন নাম কার্যকর হতে পুনরায় লগইন করুন
```

Static IP-র Netplan উদাহরণ (ল্যাব, TEST-NET IP) — পরিবর্তনের আগে কপি রাখুন:

```
# /etc/netplan/50-cloud-init.yaml (উদাহরণ)
network:
  version: 2
  ethernets:
    eth0:
      addresses: [203.0.113.10/24]
      routes:
        - to: default
          via: 203.0.113.1
      nameservers:
        addresses: [1.1.1.1, 8.8.8.8]
```

প্রয়োগ: `sudo netplan apply` — তবে সতর্ক থাকুন: ভুল কনফিগারেশনে দূরের সার্ভার থেকে সংযোগ বিচ্ছিন্ন হয়ে যেতে পারে।

> **WARNING:** দূরবর্তী (remote) সার্ভারে `netplan apply` বা SSH কনফিগারেশন বদলানোর আগে কনসোল অ্যাক্সেস (ক্লাউড প্যানেল/VNC) হাতের কাছে রাখুন। ভুল হলে SSH বন্ধ হয়ে গেলে ফিরে ঢোকার পথ থাকবে না।

২.৯ বাস্তব প্রোডাকশন পরিস্থিতি

> **REAL WORLD:** Technova-র নতুন সার্ভার technova-web-01-এর timezone ভুল করে UTC রাখা হয়েছিল, আর অ্যাপ Asia/Dhaka ধরে লগ লিখত। ঘটনা তদন্তে দুই ঘণ্টার ব্যবধান দেখে দল ভেবেছিল আক্রমণ রাত ৩টায় হয়েছে, আসলে হয়েছিল রাত ৯টায়। শিক্ষা: প্রথম দিনেই timezone, NTP ও hostname ঠিক করুন — এগুলো "পরে করব" বলার জিনিস নয়।

> **REAL WORLD:** Technova-র স্টেজিং সার্ভার technova-web-02 (203.0.113.11) ইনস্টলের সময় OpenSSH server-এ টিক দিতে ভুলে গিয়েছিল দল। সার্ভারটি ছিল অন্য শহরের একটি ল্যাবে, তাই কেউ দূর থেকে ঢুকতে পারছিল না। শেষ পর্যন্ত একজনকে সশরীরে গিয়ে কনসোলে SSH ইনস্টল করতে হলো, আর ডেপ্লয় পরীক্ষা পিছিয়ে গেল এক দিন। শিক্ষা: চেকলিস্ট লিখে টিক দিয়ে এগোন — SSH, LVM, hostname, timezone — কোনোটিই স্মৃতির ওপর ছাড়বেন না; প্রথম বুটের পরপরই দূর থেকে লগইন করে দেখুন।

২.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: ল্যাব VM ইনস্টল ও প্রথম কনফিগারেশন

1. Ubuntu Server 24.04 LTS ISO ডাউনলোড করে VM বানান (২ vCPU, ৪ GB RAM, ২৫ GB ডিস্ক)।
2. ইনস্টলারে ভাষা English, কীবোর্ড US, Storage-এ "Use an entire disk" + LVM, Profile-এ hostname `technova-web-01` দিন, OpenSSH server টিক দিন।
3. ইনস্টল শেষে রিবুট করে লগইন করুন।
4. `sudo apt update && sudo apt upgrade -y` চালান।
5. `sudo timedatectl set-timezone Asia/Dhaka` ও `timedatectl` দিয়ে যাচাই করুন।
6. `systemctl status ssh` দিয়ে SSH চলছে কি না দেখুন।
7. `ip addr show` দিয়ে IP লিখে রাখুন — এটাই আপনার সার্ভারের ঠিকানা।

২.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল:** Desktop সংস্করণ সার্ভারে ইনস্টল করা — বাড়তি সফটওয়্যার সম্পদ খায় ও নিরাপত্তা ঝুঁকি বাড়ায়।
- **ভুল:** ইনস্টলের পর আপডেট না করা — নতুন সার্ভারেও পুরোনো দুর্বলতা থাকতে পারে।
- **ভুল ধারণা:** "hostname শুধু সাজানোর জিনিস।" — লগ, মনিটরিং ও অটোমেশনে hostname-ই সার্ভারের পরিচয়।
- **ভুল:** root ব্যবহারকারী দিয়ে দৈনন্দিন কাজ করা — পরের অধ্যায়ে এর নিরাপদ বিকল্প আসবে।

২.১২ নিরাপত্তা বিবেচনা

ইনস্টলের সময়ই শক্তিশালী পাসওয়ার্ড দিন; "password123" ধরনের কিছু নয়। SSH ইনস্টল করলেই তা ইন্টারনেটে খুলে দেবেন না — প্রথমে ফায়ারওয়াল ও কি-ভিত্তিক লগইন (অধ্যায় ৩) ঠিক করুন। স্বয়ংক্রিয় নিরাপত্তা আপডেট (unattended-upgrades) চালু রাখা ভালো অভ্যাস — বিস্তারিত অধ্যায় ৬-এ।

২.১৩ সমস্যা সমাধান নির্দেশিকা

সমস্যা	কারণ	সমাধান
ইনস্টলার ডিস্ক দেখছে না	VM-এ ডিস্ক যুক্ত হয়নি	VM সেটিংসে ভার্চুয়াল ডিস্ক যুক্ত করুন
নেটওয়ার্ক নেই / IP পাচ্ছে না	DHCP বন্ধ বা কেবল বিচ্ছিন্ন	VM নেটওয়ার্ক NAT/Bridged পরীক্ষা করুন; `ip addr show` দেখুন
সময় ভুল দেখাচ্ছে	timezone/NTP ঠিক নয়	`timedatectl` ও `set-timezone Asia/Dhaka`

SSH-তে ঢোকা যাচ্ছে না	SSH সার্ভিস বন্ধ বা ফায়ারওয়াল	`systemctl status ssh`; পোর্ট ২২ খোলা কি না দেখুন
ইনস্টলের পর রিবুটে GRUB বা বুট আটকে আছে	Bootloader ঠিকমতো বসেনি বা বুট-ক্রম ভুল	VM-এ ISO সরানো হয়েছে কি না দেখুন; বুট-ক্রমে ডিস্ক আগে আছে কি না যাচাই করুন
`apt update`-এ নেটওয়ার্ক ত্রুটি দেখাচ্ছে	DNS বা গেটওয়ে ঠিক নয়	`ip route show` দিয়ে গেটওয়ে ও Netplan কনফিগ মিলিয়ে দেখুন
কীবোর্ডে কিছু অক্ষর ভুল আসছে	ইনস্টলে ভুল কীবোর্ড লেআউট বেছে নেওয়া হয়েছে	কীবোর্ড সেটিং যাচাই করুন; পাসওয়ার্ড টাইপের আগে লেআউট নিশ্চিত করুন
hostname বদলানোর পরও পুরোনো নাম দেখাচ্ছে	পুরোনো সেশনে বদল কার্যকর হয়নি	পুনরায় লগইন করুন; `hostnamectl` দিয়ে বর্তমান নাম যাচাই করুন

২.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: LTS সংস্করণ ব্যবহার; ইনস্টলের পরপরই আপডেট; hostname/timezone/NTP প্রথম দিনেই ঠিক করা; প্রতিটি পরিবর্তনের আগে কনফিগ ফাইলের ব্যাকআপ। **অ্যান্টি-প্যাটার্ন:** প্রোডাকশনে non-LTS বা পরীক্ষামূলক সংস্করণ; কনফিগারেশন মুখস্থ না লিখে কোথাও নথিভুক্ত না করা; একই সার্ভার বারবার হাতে ইনস্টল করা — পরে অটোমেশন শিখলে এ কাজ স্ক্রিপ্টে করবেন।

২.১৫ অধ্যায়ের সারাংশ

Ubuntu Server 24.04 LTS ইনস্টল করা মানে Bootloader, Kernel ও কনফিগারেশন সাজানো। ইনস্টলারের প্রতিটি ধাপ — ভাষা, নেটওয়ার্ক, ডিস্ক/LVM, ব্যবহারকারী, SSH — ভবিষ্যতের স্থিতিশীলতা নির্ধারণ করে। প্রথম বুটের পর আপডেট, hostname, timezone ও SSH যাচাই করাই প্রাথমিক কনফিগারেশনের মূল কাজ।

২.১৬ মূল প্রযুক্তিগত পরিভাষা

LTS: Long Term Support — দীর্ঘদিন নিরাপত্তা আপডেট পাওয়া সংস্করণ।

ISO: ইনস্টলেশন ডিস্কের ফাইল-ছবি।

Bootloader (GRUB): চালু হওয়ার সময় Kernel লোড করা প্রোগ্রাম।

DHCP / Static IP: স্বয়ংক্রিয়ভাবে IP পাওয়া / হাতে স্থির IP নির্ধারণ করা।

Netplan: Ubuntu-তে নেটওয়ার্ক কনফিগার করার ব্যবস্থা।

NTP: Network Time Protocol — ইন্টারনেট থেকে সঠিক সময় মেলানো।

২.১৭ জ্ঞান যাচাই প্রশ্ন

1. প্রোডাকশনে LTS সংস্করণ কেন বেছে নেওয়া হয়?
2. Bootloader কী করে এবং না থাকলে কী হবে?
3. DHCP ও Static IP-র পার্থক্য কী, প্রোডাকশনে কোনটি কেন?
4. সার্ভারে ডেস্কটপ পরিবেশ না থাকার দুটি যুক্তি দিন।
5. ইনস্টলের পর প্রথম পাঁচটি কাজের তালিকা বানান ও প্রতিটির কারণ লিখুন।
6. Netplan কনফিগারেশন বদলানোর আগে কেন ব্যাকআপ দরকার, একটি বাস্তব ঝুঁকি দিয়ে ব্যাখ্যা করুন।

- LVM টিক দেওয়া ও না দেওয়ার মধ্যে ছয় মাস পরে কী পার্থক্য হয়? ডিস্ক বাড়ানোর দৃশ্য দিয়ে ব্যাখ্যা করুন।
- নতুন সার্ভারে প্রথমে `apt update` না করে সরাসরি প্যাকেজ ইনস্টল করলে কী সমস্যা হতে পারে, এবং কেন update আগে লাগে?

২.১৮ ব্যবহারিক অনুশীলন

LAB: ডিসপোজেবল VM-এ করুন; ইনস্টল শেষে VM-এর স্ন্যাপশট নিয়ে রাখুন যাতে ভুল হলে ফিরে আসা যায়।

1. Ubuntu Server 24.04 LTS দিয়ে একটি ল্যাব VM ইনস্টল করুন (hostname `technova-web-01`, OpenSSH সহ)।
2. প্রথম বুটের পর আপডেট, timezone Asia/Dhaka ও SSH অবস্থা যাচাই করে ফলাফল নোট করুন।
3. `/etc/netplan/` ফাইলের কপি নিয়ে তারপর (ল্যাবে) একটি Static IP কনফিগার করে দেখুন; ভুল হলে কপি দিয়ে ফিরিয়ে আনার (rollback) ধাপ লিখুন।

- একটি ইনস্টল-চেকলিস্ট বানান — ইনস্টলের আগে, সময় ও প্রথম বুটের পরের কাজ তিন ভাগে সাজিয়ে, প্রতিটি ধাপের পাশে যাচাইয়ের কমান্ডসহ। তারপর ল্যাব VM-এর সঙ্গে মিলিয়ে দেখুন কোনো ধাপ বাদ পড়েছিল কি না।

অধ্যায় ৩

SSH, ব্যবহারকারী, গ্রুপ, Sudo ও অ্যাক্সেস ব্যবস্থাপনা

৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) SSH (Secure Shell) দিয়ে নিরাপদে দূরের সার্ভারে লগইন করতে; (২) SSH কী (key) তৈরি করে পাসওয়ার্ড-লগইন বন্ধ করতে; (৩) ব্যবহারকারী (user) ও গ্রুপ (group) বানিয়ে অনুমতি (permission) নিয়ন্ত্রণ করতে; (৪) sudo-র মাধ্যমে নিরাপদে প্রশাসকের কাজ করতে; (৫) SSH কনফিগারেশন শক্ত করে আক্রমণের পথ কমাতে।

৩.২ প্রোডাকশনে কেন এটি গুরুত্বপূর্ণ

প্রোডাকশন সার্ভারের সবচেয়ে বেশি আক্রান্ত দরজাটি হলো SSH। ইন্টারনেটে খোলা সার্ভারে প্রতিদিন হাজার হাজার স্বয়ংক্রিয় লগইন-চেষ্টা আসে — দুর্বল পাসওয়ার্ড থাকলে তা ভেঙে যাওয়া সময়ের ব্যাপার। একইভাবে, সবাইকে root (সর্বময় প্রশাসক) অ্যাক্সেস দিলে একটি ভুল কমান্ডেই পুরো সার্ভার মুছে যেতে পারে। অ্যাক্সেস ব্যবস্থাপনাই তাই প্রোডাকশন নিরাপত্তার প্রথম ও সবচেয়ে গুরুত্বপূর্ণ স্তর।

৩.৩ পূর্বশর্ত জ্ঞান

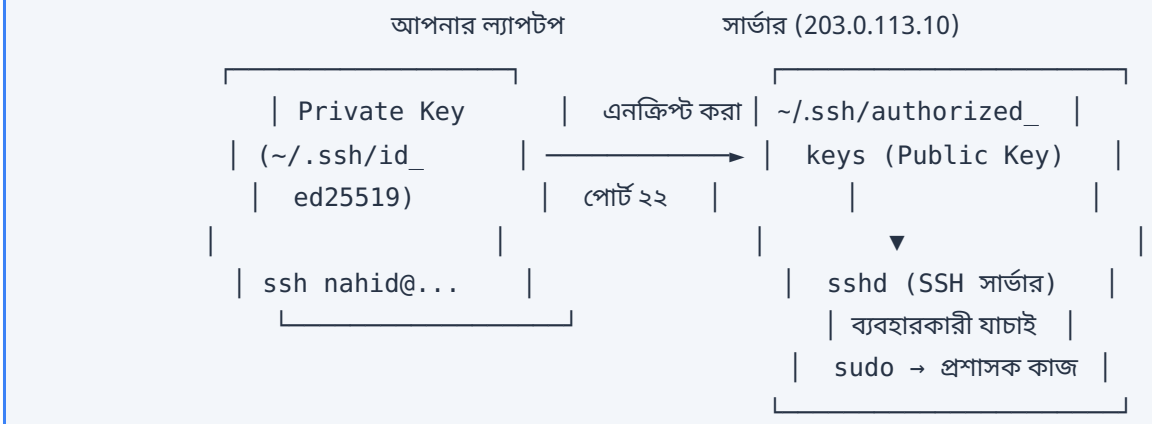
অধ্যায় ২-এর ইনস্টল করা Ubuntu সার্ভার, টার্মিনালে কমান্ড চালানোর প্রাথমিক অভ্যাস এবং ফাইল/ফোল্ডারের সাধারণ ধারণা। পাবলিক-কী ক্রিপ্টোগ্রাফি এখানেই সহজভাবে ব্যাখ্যা করা হবে — আগের জ্ঞান লাগবে না।

৩.৪ প্রথম নীতি থেকে ব্যাখ্যা

দূরের কম্পিউটার চালানো মানে দুটি প্রশ্নের উত্তর: **আপনি কে?** (Authentication — পরিচয় যাচাই) এবং **আপনি কী করতে পারবেন?** (Authorization — অনুমতি)। পাসওয়ার্ড হলো "জানা গোপন কথা" দিয়ে পরিচয় দেওয়া — চুরি করা যায়, আন্দাজ করা যায়। SSH কী হলো "নিজের কাছে রাখা গোপন চাবি" দিয়ে পরিচয় দেওয়া: আপনার কম্পিউটারে থাকে **Private Key** (গোপন চাবি, কাউকে দেবেন না), সার্ভারে থাকে **Public Key** (খোলা চাবি, সবাই দেখতে পারে)। গোপন চাবি ছাড়া খোলা চাবির তালা খোলে না — এটাই পাবলিক-কী পদ্ধতির মূল কথা। আর **sudo** মানে: সাধারণ ব্যবহারকারী হয়েও নির্দিষ্ট প্রশাসনিক কাজ সাময়িকভাবে করা — পুরো root লগইন ছাড়াই।

> **CONCEPT:** Principle of Least Privilege — প্রত্যেককে শুধু ততটুকু ক্ষমতা দিন যতটুকু তার কাজে লাগে, তার বেশি নয়। এটাই অ্যাক্সেস ব্যবস্থাপনার সোনালি নিয়ম।

৩.৫ আর্কিটেকচার / ধারণাগত চিত্র



Private Key কখনো নেটওয়ার্কে যায় না — শুধু প্রমাণ যায় যে আপনার কাছে চাবিটি আছে।

৩.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

SSH কী তৈরি ও ব্যবহার

ed25519 হলো আধুনিক, দ্রুত ও নিরাপদ কী ধরন — Ubuntu 24.04-এ এটাই সুপারিশ করা হয়। কী তৈরি হয় আপনার ল্যাপটপে; তারপর Public Key সার্ভারের `~/.ssh/authorized_keys` ফাইলে বসাতে হয়। `ssh-copy-id` কমান্ড এই কাজটি নিরাপদে করে দেয়।

ব্যবহারকারী, গ্রুপ ও অনুমতি

Linux-এ প্রতিটি ফাইলের মালিক একজন ব্যবহারকারী ও একটি গ্রুপ। অনুমতি তিন ভাগে: **মালিক (owner)**, **গ্রুপ (group)**, **অন্যরা (others)** — প্রত্যেকের জন্য পড়া (r=4), লেখা (w=2), চালানো (x=1)। যেমন `-rw-r--r--` মানে মালিক পড়তে-লিখতে পারে, বাকিরা শুধু পড়তে পারে। `chmod 640 file` মানে মালিক rw, গ্রুপ r, অন্যরা কিছুই না। ডিরেক্টরিতে x মানে "ভেতরে ঢোকা যায়"।

sudo ও sshd কনফিগারেশন

`sudo` ব্যবহারকারীরা থাকে `sudo` গ্রুপে; `/etc/sudoers` ফাইলে তাদের নিয়ম লেখা থাকে — সরাসরি সম্পাদনা না করে `visudo` দিয়ে সম্পাদনা করুন, কারণ visudo ভুল ধরিয়ে দেয়। SSH সার্ভারের কনফিগ `/etc/ssh/sshd_config`-এ থাকে: `PasswordAuthentication no` ও `PermitRootLogin no` করলে পাসওয়ার্ড-আক্রমণ ও root-লগইন বন্ধ হয়। পরিবর্তনের পর `systemctl reload ssh` লাগে — তবে তার আগে একটি চালু SSH সেশন খোলা রাখুন (নিচে দেখুন)।

কাজের উদাহরণ: নতুন ডেভেলপারকে সীমিত অ্যাক্সেস দেওয়া

ধরুন Technova-তে নতুন ডেভেলপার সারা যোগ দিলেন। তাঁকে পুরো sudo দেওয়া ঝুঁকিপূর্ণ, আবার কোনো অ্যাক্সেস না দিলে তিনি অ্যাপ রিস্টার্টও করতে পারবেন না। সঠিক পথ: প্রথমে তাঁর নিজের অ্যাকাউন্ট বানান (`sudo adduser sara`), তারপর ডেভেলপারদের গ্রুপে যোগ করুন (`sudo usermod -aG developers sara`), আর অ্যাপের ফোল্ডার এমনভাবে সাজান যাতে গ্রুপের সদস্যরা পড়তে ও লিখতে পারে, অন্যরা নয়। সবচেয়ে সীমিত অংশটি হলো sudo: পুরো sudo না দিয়ে শুধু একটি কমান্ডের অনুমতি দিন — `visudo`-তে লেখা `sara ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart technova-app` মানে সারা শুধু অ্যাপ রিস্টার্ট করতে পারবেন, আর কিছু নয়।

অ্যাক্সেসের ধরন	কী করতে পারে	কখন দেওয়া হয়
সাধারণ ব্যবহারকারী	নিজের ফাইল ও অনুমোদিত ফোল্ডারে কাজ	প্রত্যেক মানুষের দৈনন্দিন কাজের জন্য
সীমিত sudo (একটি কমান্ড)	শুধু নির্দিষ্ট কাজ, যেমন অ্যাপ রিস্টার্ট	ডেভেলপারকে নির্দিষ্ট দায়িত্ব দিতে
পূর্ণ sudo	প্রশাসকের প্রায় সব কাজ	শুধু বিশ্বস্ত প্রশাসকদের
root সরাসরি লগইন	সবকিছু, কোনো বাধা ছাড়া	প্রোডাকশনে কখনোই নয় — এটি নিষিদ্ধ রাখুন

দ্বিতীয় দৃশ্য: সারা চাকরি ছাড়লেন। অ্যাকাউন্ট মোছার আগে নিশ্চিত করুন তাঁর SSH কী `authorized_keys` থেকে সরানো হয়েছে কি না, এবং তাঁর নামে চলমান কোনো প্রসেস বা নির্ধারিত কাজ আছে কি না।

৩.৭ গুরুত্বপূর্ণ কমান্ড ও তার বিশ্লেষণ

```
ssh-keygen -t ed25519 -C "nahid@technova.example"
# -t ed25519 = কী-র ধরন; -C = মন্তব্য (কোন কী, চেনার জন্য)

ssh-copy-id nahid@203.0.113.10
# Public Key সার্ভারে পৌঁছে দেয়

ssh nahid@203.0.113.10
# SSH দিয়ে লগইন; nahid = ব্যবহারকারী, IP = সার্ভারের ঠিকানা

sudo adduser deploy
# নতুন ব্যবহারকারী বানায় (deploy নামে)

sudo usermod -aG sudo deploy
# deploy-কে sudo গ্রুপে যোগ করে (-aG = আগের গ্রুপ রেখে যোগ করা)

ls -l
# ফাইলের মালিক ও অনুমতি দেখায়

chmod 600 ~/.ssh/id_ed25519
# Private Key শুধু মালিকই পড়তে পারে — এটা বাধ্যতামূলক, নইলে SSH কী মানবে না

sudo visudo
# sudo নিয়ম নিরাপদে সম্পাদনা করে
```

```
id deploy
# ব্যবহারকারীর UID, প্রধান গ্রুপ ও সব গ্রুপ একসঙ্গে দেখায়
# sudo গ্রুপে আছে কি না, এখানেই বোঝা যায়

groups deploy
# ব্যবহারকারী কোন কোন গ্রুপের সদস্য, শুধু সেই তালিকা দেখায়

sudo -l -U deploy
# deploy কী কী sudo কমান্ড চালাতে পারবে, তার তালিকা দেখায়
# -l = তালিকা (list), -U = অন্য ব্যবহারকারীর জন্য

ssh -v nahid@203.0.113.10
# লগইনের বিস্তারিত ধাপ দেখায় (-v = verbose, বিস্তারিত)
# কী পাঠানো হচ্ছে, কোথায় আটকাচ্ছে — সমস্যা নির্ণয়ে অমূল্য
```

```
last -n 10
# শেষ ১০টি লগইনের রেকর্ড দেখায়; অচেনা লগইন চোখে পড়লে তদন্ত করুন
```

সীমিত sudo দেওয়ার পর `sudo -l -U deploy` চালান — এতে প্রমাণ হয় deploy ঠিক ততটুকুই করতে পারবেন যতটুকু দিয়েছেন। `last` নিয়মিত দেখুন।

৩.৮ কনফিগারেশন উদাহরণ

নিরাপদ `/etc/ssh/sshd_config` অংশ (উদাহরণ):

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
AllowUsers nahid deploy
# শুধু এই দুই ব্যবহারকারীই SSH-তে ঢুকতে পারবে
```

পরিবর্তন প্রয়োগের আগে কনফিগ পরীক্ষা করুন: `sudo sshd -t` — ভুল থাকলে জানিয়ে দেবে। তারপর `sudo systemctl reload ssh`।

> **WARNING:** `PasswordAuthentication no` করার আগে অবশ্যই কী-ভিত্তিক লগইন পরীক্ষা করে নিন এবং একটি SSH সেশন খোলা রাখুন। কী কাজ না করলে ও পাসওয়ার্ড বন্ধ হয়ে গেলে আপনি নিজেই ঢুকতে পারবেন না। Rollback: খোলা সেশন থেকে কনফিগ আগের অবস্থায় ফিরিয়ে `systemctl reload ssh` করুন।

৩.৯ বাস্তব প্রোডাকশন পরিস্থিতি

> **REAL WORLD:** Technova-র ডেভেলপার রাকিব চাকরি ছাড়ার পর তার অ্যাকাউন্ট মুছে ফেলা হয়নি। তিন মাস পর তার পুরোনো পাসওয়ার্ড দিয়ে কেউ লগইন-চেষ্টা করে। ভাগ্য ভালো, পাসওয়ার্ড-লগইন বন্ধ ছিল বলে চেষ্টা ব্যর্থ হয়। শিক্ষা: কর্মী বিদায়ের দিনই অ্যাকাউন্ট নিষ্ক্রিয় করুন (`sudo deluser rakib` বা `usermod -L rakib`) এবং তার কী `authorized_keys` থেকে সরান।

> **REAL WORLD:** Technova-তে একসময় সব ডেভেলপার একটি শেয়ার করা অ্যাকাউন্ট — deploy — দিয়ে লগইন করতেন, আর তার Private Key ছিল দলের চ্যাটে। একদিন একটি ভুল কমান্ডে অ্যাপের কনফিগ মুছে গেল, কিন্তু কে চালিয়েছিল, তা বোঝা গেল না — লগে শুধু deploy লেখা ছিল। দল সিদ্ধান্ত নিল: প্রত্যেকের আলাদা অ্যাকাউন্ট ও আলাদা কী হবে, শেয়ার করা কী বাতিল হবে। শিক্ষা: শেয়ার করা পরিচয় মানে দায়িত্ব কারো নয় — আলাদা পরিচয় ছাড়া নিরাপত্তা তদন্ত অসম্ভব।

৩.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: কী-ভিত্তিক লগইন চালু করা (ল্যাবে)

1. ল্যাপটপে: `ssh-keygen -t ed25519 -C "nahid@technova.example"` — পাসফ্রেজ (passphrase) দিন, এটি কী-র বাড়তি তালা।

2. `ssh-copy-id nahid@203.0.113.10` দিয়ে Public Key পাঠান।
3. নতুন টার্মিনালে `ssh nahid@203.0.113.10` — পাসওয়ার্ড ছাড়া (বা শুধু পাসফ্রেজে) ঢুকছে কি না দেখুন।
4. সার্ভারে `sudo adduser deploy` দিয়ে ডেপ্লয় ব্যবহারকারী বানান ও `sudo usermod -aG sudo deploy` দিন।
5. `chmod 600 ~/.ssh/id_ed25519` দিয়ে Private Key-র অনুমতি ঠিক করুন।
6. শুধু এরপর sshd_config শক্ত করার কথা ভাবুন — আগে নয়।

৩.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল:** Private Key ইমেইল/চ্যাটে পাঠানো বা Git-এ কমিট করা — এটা বাড়ির চাবি ফটোকপি করে বিলি করার মতো।
- **ভুল:** `usermod -G sudo deploy` (`-a` ছাড়া) — আগের সব গ্রুপ মুছে যায়।
- **ভুল ধারণা:** "SSH পোর্ট বদলালেই নিরাপদ।" — পোর্ট বদলানো আক্রমণ কমায়, কিন্তু নিরাপত্তা নয়; কী-ভিত্তিক লগইনই আসল সুরক্ষা।
- **ভুল:** `chmod 777` দিয়ে সব সমস্যা মেটানো — এতে সবাই সব করতে পারে; এটি নিরাপত্তা ভেঙে দেওয়ার শর্টকাট।

৩.১২ নিরাপত্তা বিবেচনা

root দিয়ে সরাসরি লগইন নিষিদ্ধ করুন; পাসওয়ার্ড-লগইন বন্ধ করে শুধু কী রাখুন; Private Key-তে পাসফ্রেজ দিন; `AllowUsers` দিয়ে কে ঢুকতে পারবে সীমিত করুন; fail2ban-এর মতো টুল (পরের খণ্ডে বিস্তারিত) বারবার ভুল লগইন-চেষ্টাকারী IP ব্লক করে। কখনোই নিরাপত্তা বন্ধ করে সমস্যা "সমাধান" করবেন না — যেমন SSH খুলে রাখা বা ফায়ারওয়াল বন্ধ করা।

৩.১৩ সমস্যা সমাধান নির্দেশিকা

সমস্যা	কারণ	সমাধান
Permission denied (publickey)	কী সার্ভারে নেই বা অনুমতি ভুল	`~/.ssh` 700 ও `authorized_keys` 600 কি না দেখুন; `ssh -v` দিয়ে বিস্তারিত দেখুন
sudo: user is not in sudoers	ব্যবহারকারী sudo গ্রুপে নেই	অন্য sudo ব্যবহারকারী দিয়ে `usermod -aG sudo নাম` করুন
কী দিয়েও পাসওয়ার্ড চাইছে	sshd_config-এ Pubkey বন্ধ বা ফাইল মালিকানা ভুল	`sshd -t` ও লগ (`journalctl -u ssh`) দেখুন
নিজেই লকআউট	ভুল কনফিগ + পাসওয়ার্ড বন্ধ	ক্লাউড কনসোল/VNC দিয়ে ঢুকে কনফিগ ফিরিয়ে আনুন
নতুন ব্যবহারকারী নিজের home ফোল্ডারে লিখতে পারছে না	ফোল্ডারের মালিকানা বা অনুমতি ভুল	`ls -ld /home/নাম` দিয়ে মালিক ও অনুমতি দেখুন; প্রয়োজনে মালিকানা ঠিক করুন
`AllowUsers` -এ নাম না থাকায় লগইন ব্যর্থ	sshd_config-এ ওই ব্যবহারকারী অনুমোদিত নয়	sshd_config-এ নাম যোগ করে `sshd -t` দিয়ে যাচাই করে reload করুন
sudo পাসওয়ার্ড বারবার চাইছে	প্রতিবার sudo-তে পাসওয়ার্ড লাগে — এটাই স্বাভাবিক আচরণ	এটি ত্রুটি নয়; তবে সীমিত sudo নিয়মে NOPASSWD থাকলে শুধু সেই কমান্ডে পাসওয়ার্ড লাগে না

`ssh -v`-তে কী পাঠানোই হচ্ছে না	ল্যাপটপে কী ফাইলটি ভুল জায়গায় বা ভুল নামে আছে	`ls ~/.ssh/` দিয়ে কী আছে দেখুন; প্রয়োজনে `-i পথ` দিয়ে কী-র পথ নির্দিষ্ট করুন

৩.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: প্রত্যেক মানুষের আলাদা অ্যাকাউন্ট ও আলাদা কী; অ্যাক্সেসের নিয়মিত পর্যালোচনা; পরিবর্তনের আগে কনফিগ ব্যাকআপ; সব প্রশাসনিক কাজ sudo দিয়ে, root লগইন নয়। **অ্যান্টি-প্যাটার্ন:** একটি শেয়ার করা root পাসওয়ার্ড; `chmod 777`; Private Key শেয়ার করা; বিদায়ী কর্মীর অ্যাকাউন্ট রেখে দেওয়া।

৩.১৫ অধ্যায়ের সারাংশ

SSH হলো সার্ভারের নিরাপদ দরজা; কী-ভিত্তিক লগইন পাসওয়ার্ডের চেয়ে অনেক নিরাপদ। ব্যবহারকারী ও গ্রুপ দিয়ে "কে কী করতে পারে" ঠিক হয়, sudo দিয়ে প্রশাসনিক কাজ সাময়িকভাবে করা যায়। নিরাপত্তার মূলমন্ত্র: কম ক্ষমতা, আলাদা পরিচয়, এবং পরিবর্তনের আগে নিজের ঢোকার পথ নিশ্চিত করা।

৩.১৬ মূল প্রযুক্তিগত পরিভাষা

SSH: Secure Shell — এনক্রিপ্ট করা দূরবর্তী লগইন ও কমান্ড চালানোর ব্যবস্থা।

Public / Private Key: খোলা চাবি (সার্ভারে থাকে) / গোপন চাবি (আপনার কাছে থাকে)।

sshd: সার্ভারে চলা SSH সার্ভার প্রোগ্রাম।

sudo: নির্দিষ্ট ব্যবহারকারীকে প্রশাসকের কাজ সাময়িকভাবে করতে দেওয়া।

Permission (rwx): পড়া-লেখা-চালানোর অনুমতি।

Least Privilege: যতটুকু দরকার, শুধু ততটুকু ক্ষমতা দেওয়ার নীতি।

৩.১৭ জ্ঞান যাচাই প্রশ্ন

1. পাসওয়ার্ড-লগইনের চেয়ে কী-ভিত্তিক লগইন কেন নিরাপদ, দুটি কারণ দিন।
2. Private Key ও Public Key কোথায় থাকে এবং কোনটি কখনো শেয়ার করা যায় না?
3. `chmod 640` বলতে কী বোঝায়? সংখ্যাগুলো ব্যাখ্যা করুন।
4. `usermod -aG sudo deploy`-তে `-a` না দিলে কী ক্ষতি হবে?
5. PasswordAuthentication বন্ধ করার আগে কোন দুটি জিনিস নিশ্চিত করতে হবে?
6. একজন কর্মী চাকরি ছাড়লে অ্যাক্সেস বন্ধ করতে কী কী ধাপ নেবেন?

- একজন ডেভেলপারকে শুধু অ্যাপ রিস্টার্ট করার অনুমতি দিতে sudoers-এ কীভাবে সীমিত নিয়ম লিখবেন, এবং কেন তাঁকে পুরো sudo দেওয়া ঝুঁকিপূর্ণ?
- `ssh -v` আউটপুটের কোন অংশ দেখে বুঝবেন সমস্যাটি সার্ভারে পৌঁছানোয়, নাকি কী প্রত্যাখ্যাত হওয়ায়? দুটি ক্ষেত্রের লক্ষণ আলাদা করে লিখুন।

৩.১৮ ব্যবহারিক অনুশীলন

> **LAB:** ল্যাব VM-এ করুন। sshd_config বদলানোর আগে কনসোল অ্যাক্সেস ও খোলা SSH সেশন হাতের কাছে রাখুন।

1. ed25519 কী তৈরি করে `ssh-copy-id` দিয়ে ল্যাব সার্ভারে কী-লগইন চালু করুন।
2. `deploy` নামে ব্যবহারকারী বানিয়ে sudo গ্রুপে যোগ করুন; তারপর `deploy` দিয়ে লগইন করে `sudo whoami` চালিয়ে যাচাই করুন।
3. একটি ফাইলে `chmod 640` ও `chmod 600` প্রয়োগ করে `ls -l`-এ পার্থক্য দেখুন এবং ব্যাখ্যা লিখুন।
4. (ঐচ্ছিক, ল্যাবে) `sshd_config`-এর ব্যাকআপ নিয়ে `PermitRootLogin no` করুন, `sshd -t` দিয়ে যাচাই করে reload করুন; root লগইন ব্যর্থ হচ্ছে কি না পরীক্ষা করুন।

- ল্যাবে `developers` গ্রুপ বানিয়ে দুটি পরীক্ষামূলক ব্যবহারকারীকে তাতে যোগ করুন; একটি ফোল্ডারের গ্রুপ-মালিকানা developers করে `chmod 770` দিন এবং গ্রুপের সদস্য লিখতে পারে কি না, বাইরের কেউ পড়তে পারে কি না — যাচাই করে লিখুন।

অধ্যায় ৪

Linux ফাইলসিস্টেম, পার্টিশন, মাউন্ট ও LVM

৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Linux ফাইলসিস্টেমের গাছ-কাঠামো (tree) ব্যাখ্যা করতে; (২) পার্টিশন (partition), মাউন্ট (mount) ও ফাইলসিস্টেমের পার্থক্য বলতে; (৩) `df`, `du`, `lsblk` দিয়ে ডিস্ক অবস্থা নির্ণয় করতে; (৪) LVM দিয়ে ডিস্কের জায়গা নমনীয়ভাবে বাড়াতে; (৫) ডিস্ক ভরে যাওয়া প্রতিরোধের পরিকল্পনা করতে।

৪.২ প্রোডাকশনে কেন এটি গুরুত্বপূর্ণ

ডিস্ক ভরে গেলে প্রোডাকশন অ্যাপ নিঃশব্দে মারা যায়: ডেটাবেস লিখতে পারে না, লগ লেখা বন্ধ হয়, আপলোড ব্যর্থ হয়। সবচেয়ে খারাপ দিক — ডিস্ক ১০০% ভরা অবস্থায় অনেক সময় SSH-তেও ঢোকা যায় না বা লগইন ব্যর্থ হয়। ফাইলসিস্টেম বোঝা মানে এই বিপদ আগেই দেখা ও আলাদা পার্টিশনে ঝুঁকি ভাগ করে দেওয়া: লগের জন্য আলাদা জায়গা থাকলে লগ ভরে গেলেও অ্যাপের ডিস্ক বাঁচে।

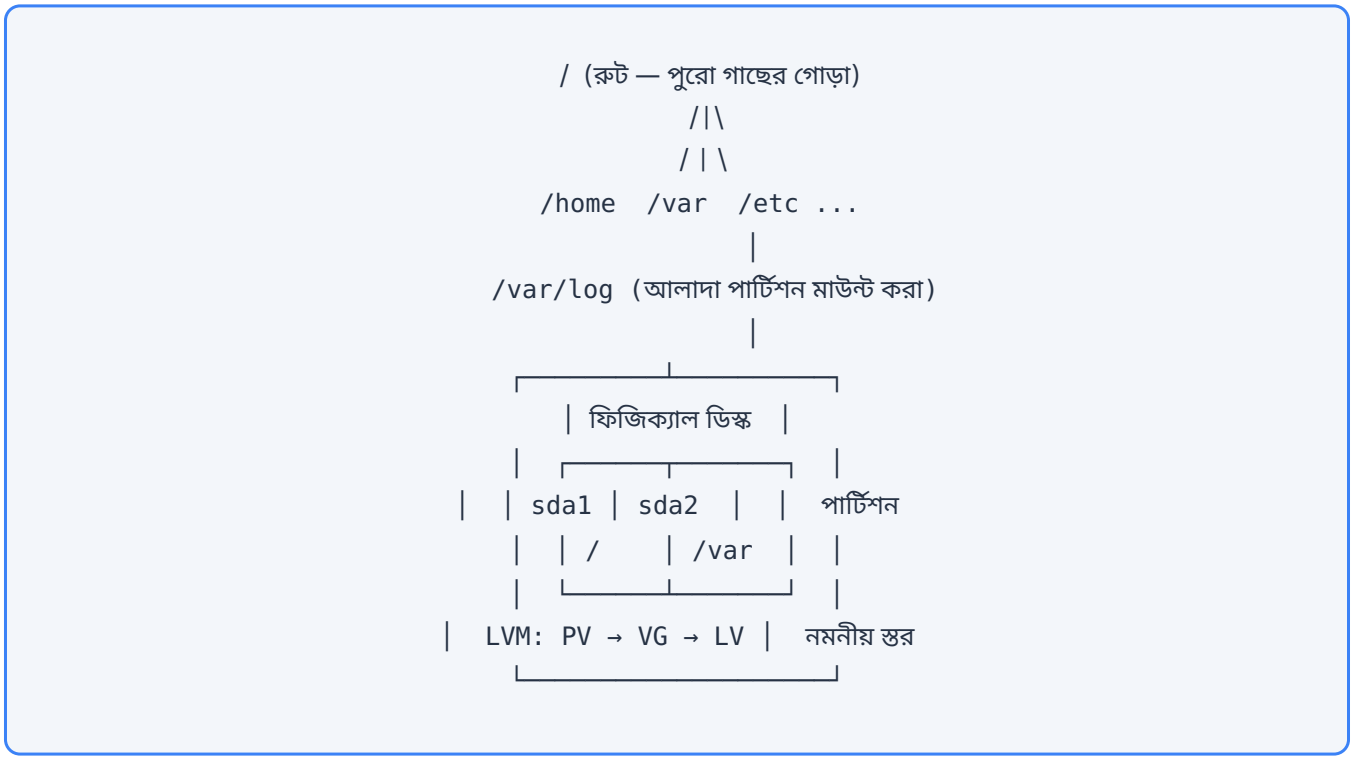
৪.৩ পূর্বশর্ত জ্ঞান

টার্মিনালে ফাইল দেখা (`ls`, `cd`) ও অধ্যায় ২-এর ইনস্টল-সময়ের Storage ধাপের ধারণা। পার্টিশন বা LVM-এর আগের অভিজ্ঞতা লাগবে না।

৪.৪ প্রথম নীতি থেকে ব্যাখ্যা

ডিস্ক একটি বড় খালি গুদাম। **পার্টিশন** মানে গুদামে দেয়াল তুলে আলাদা কক্ষ বানানো — এক কক্ষ ভরে গেলেও অন্য কক্ষ নিরাপদ। **ফাইলসিস্টেম** (যেমন ext4, XFS) মানে প্রতিটি কক্ষে বই সাজানোর নিয়ম — কোথায় কী আছে, তার সূচিপত্র। **মাউন্ট** মানে কক্ষটিকে বাড়ির একটি দরজার সঙ্গে যুক্ত করা: যেমন `/var/log`-এ একটি পার্টিশন মাউন্ট করলে সেই ফোল্ডারে যা লেখা হয়, তা আসলে ওই পার্টিশনে যায়। Linux-এ সবকিছু একটাই গাছ (`/` — রুট) থেকে শুরু হয়; Windows-এর মতো C:, D: ড্রাইভের ধারণা এখানে নেই। **LVM** (Logical Volume Management) মানে দেয়ালগুলোকে নমনীয় করা — পরে কক্ষের আকার বদলানো যায় ডেটা না মুছে।

৪.৫ আর্কিটেকচার / ধারণাগত চিত্র



৪.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

গুরুত্বপূর্ণ ডিরেক্টরি

পথ	কী থাকে
`/etc`	কনফিগারেশন ফাইল (nginx, ssh)
`/var/log`	লগ ফাইল — ডিস্ক ভরার প্রধান কারণ
`/var/lib`	ডেটাবেসের ডেটা
`/home`	ব্যবহারকারীদের ব্যক্তিগত ফাইল
`/tmp`	সাময়িক ফাইল; রিবুটে মুছে যেতে পারে
`/opt`	হাতে ইনস্টল করা সফটওয়্যার

ফাইলসিস্টেম ও মাউন্ট

ext4 হলো Ubuntu-র ডিফল্ট ফাইলসিস্টেম — নির্ভরযোগ্য ও বহুল ব্যবহৃত। নতুন ডিস্ক ব্যবহারের আগে তাতে ফাইলসিস্টেম বানাতে হয় (`mkfs.ext4`) — এটি ডিস্কের সব ডেটা মুছে দেয়। **মাউন্ট** করা হয় `mount` কমান্ডে; স্থায়ীভাবে মাউন্ট করতে `/etc/fstab` ফাইলে লিখতে হয় — ভুল fstab-এ সার্ভার বুটই হতে পারে না, তাই পরিবর্তনের পর `mount -a` দিয়ে পরীক্ষা করুন।

LVM-এর তিন স্তর

PV (Physical Volume) — আসল ডিস্ক/পার্টিশন; **VG** (Volume Group) — একাধিক PV মিলিয়ে বড় পুকুর; **LV** (Logical Volume) — পুকুর থেকে কেটে নেওয়া নমনীয় কক্ষ। LV-র আকার `lvextend` দিয়ে বাড়ানো যায়, তারপর ফাইলসিস্টেম বাড়াতে `resize2fs` লাগে। এটাই প্রোডাকশনে ডিস্ক বাড়ানোর নিরাপদ পথ — ডেটা না মুছে।

কাজের উদাহরণ: আপলোডের জন্য আলাদা জায়গা বানানো

ধরুন Technova-র অ্যাপে ব্যবহারকারীদের ছবি `/opt/technova-app/uploads`-এ জমা হয় — অর্থাৎ মূল `/` পার্টিশনেই। একদিন একটি বড় প্রচারণায় হাজারো ছবি এলো, `/` ভরে গেল, আর ডেটাবেস লেখা বন্ধ করে দিল। সমস্যাটি আপলোডের আকারে নয়, জায়গার ভাগে। সঠিক সমাধান হলো আপলোডের জন্য আলাদা Logical Volume বানিয়ে `/data/uploads`-এ মাউন্ট করা — তাহলে আপলোড বাড়লেও সিস্টেমের ডিস্ক নিরাপদ থাকে।

> **WARNING:** `mkfs.ext4` ধ্বংসাত্মক কমান্ড — এটি ডিস্কের সব ডেটা মুছে দেয়। শুধুমাত্র ডিসপোজেবল ল্যাব VM-এ, স্ন্যাপশট নিয়ে, এবং `lsblk` দিয়ে কোন ডিস্কে চালাচ্ছেন তা তিনবার যাচাই করে চালান। প্রোডাকশন ডিস্কে কখনো পরীক্ষা করবেন না।

ভাগ করার ধরন	সুবিধা	কখন বেছে নেবেন
সবকিছু এক পার্টিশনে	সহজ, শুরুতে ঝামেলা কম	শুধু শেখার ল্যাবে
`/var/log` আলাদা	লগ ভরে গেলেও অ্যাপ ও ডেটাবেস বাঁচে	যেকোনো প্রোডাকশন সার্ভারে
ডেটা (`/var/lib` বা `/data`) আলাদা	ডেটা বাড়লে শুধু ওই অংশ বাড়ানো যায়	ডেটাবেস বা আপলোড থাকলে
ব্যাকআপ আলাদা ডিস্কে	মূল ডিস্ক নষ্ট হলেও ব্যাকআপ থাকে	সবসময় — একই ডিস্কে ব্যাকআপ ব্যাকআপ নয়

দ্বিতীয় দৃশ্য: `df -h` দেখাচ্ছে ৬০% ভরা, অথচ নতুন ফাইল লেখা যাচ্ছে না — কারণ হতে পারে inode শেষ হয়ে যাওয়া। প্রতিটি ফাইলের জন্য একটি inode লাগে, আর লাখো ছোট ফাইল থাকলে inode শেষ হয়ে যায়, জায়গা থাকলেও। তখন `df -i` দিয়ে inode দেখতে হয়।

৪.৭ গুরুত্বপূর্ণ কমান্ড ও তার বিশ্লেষণ

```
df -h
# প্রতিটি মাউন্ট করা ফাইলসিস্টেমে কত জায়গা আছে/ব্যবহৃত হচ্ছে

du -sh /var/log
# কোনো ফোল্ডার কত জায়গা খাচ্ছে (-s = মোট, -h = পড়ার মতো একক)

lsblk
# ডিস্ক, পার্টিশন ও মাউন্ট পয়েন্টের গাছ-চিত্র

sudo lvs; sudo vgs; sudo pvs
# LVM-র Logical Volume, Volume Group, Physical Volume দেখায়
```

```
mount | grep /var
# কী কোথায় মাউন্ট করা আছে দেখায়
```

নমুনা আউটপুট — চিত্রণমূলক:

```
নমুনা আউটপুট — চিত্রণমূলক
$ df -h
Filesystem      Size  Used Avail Use%
/dev/sda2        25G   12G   13G  48%
$ du -sh /var/log
1.2G    /var/log
```

```
sudo blkid
# প্রতিটি ডিস্ক/পার্টিশনের UUID ও ফাইলসিস্টেমের ধরন দেখায়
# fstab-এ UUID লেখার আগে এখান থেকে নকল করুন

findmnt
# কী কোথায় মাউন্ট করা আছে, গাছ-আকারে দেখায়
# mount-এর চেয়ে পড়তে সহজ

df -i
# প্রতিটি ফাইলসিস্টেমে inode কত ব্যবহৃত হচ্ছে, তা দেখায়
# -i = inode; লাখো ছোট ফাইলের সমস্যা এখানে ধরা পড়ে

du -h --max-depth=1 /var | sort -rh | head -5
# /var-এর ভেতরে সবচেয়ে বড় ৫টি ফোল্ডার দেখায়
# --max-depth=1 = এক স্তর গভীরে; sort -rh = বড় থেকে ছোট
```

`findmnt` বলে কোন জায়গা কোথায় মাউন্ট করা, `df -i` বলে ফাইলের সংখ্যার সীমা কতটুকু বাকি। `du`-র এই রূপটি বড় ফোল্ডার খুঁজতে দ্রুততম পথ।

৪.৮ কনফিগারেশন উদাহরণ

`/etc/fstab`-এ একটি ডিস্ক স্থায়ীভাবে মাউন্ট করার উদাহরণ — UUID (ডিস্কের স্থায়ী পরিচয়) ব্যবহার করুন, `/dev/sda2` নয়, কারণ ডিভাইসের নাম বদলে যেতে পারে:

```
# /etc/fstab (উদাহরণ — UUID আপনার ডিস্কেরটি দিয়ে বদলান)
UUID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx  /data  ext4  defaults  0  2
```

UUID জানতে: `sudo blkid`। ফাইল সম্পাদনার আগে কপি রাখুন: `sudo cp /etc/fstab /etc/fstab.bak` এবং পরে `sudo mount -a` দিয়ে পরীক্ষা করুন।

৪.৯ বাস্তব প্রোডাকশন পরিস্থিতি

REAL WORLD: Technova-র সার্ভারে এক রাতে লগ ফাইল ১৮ GB হয়ে গেল — একটি বাগ প্রতি সেকেন্ডে ত্রুটি লিখছিল। `/` পার্টিশন ১০০% ভরে গিয়ে MongoDB বন্ধ হয়ে গেল। সমাধান দুই ধাপে: তাৎক্ষণিকভাবে পুরোনো লগ ঘুরিয়ে (logrotate) জায়গা খালি করা, আর স্থায়ীভাবে `/var/log` আলাদা পার্টিশনে নেওয়া ও লগের আকারে সীমা বসানো। শিক্ষা: ডিস্ক ব্যবহারের অ্যালার্ট ৮০%-এই বসান, ১০০%-এ নয়।

> **REAL WORLD:** Technova-র ব্যাকআপ চলছিল একই ডিস্কে, যেখানে মূল ডেটাও ছিল। একদিন সেই ডিস্কে ত্রুটি দেখা দিল — মূল ডেটা ও ব্যাকআপ দুটোই একই ডিস্কে থাকায় পুনরুদ্ধারের উপায় রইল না। দল সেদিন থেকে ব্যাকআপ আলাদা ডিস্কে (203.0.113.20 ঠিকানার ব্যাকআপ সার্ভারে) রাখতে শুরু করল, আর প্রতি সপ্তাহে একটি ব্যাকআপ ফিরিয়ে আনা যায় কি না, তা পরীক্ষা করতে লাগল। শিক্ষা: ব্যাকআপের সত্যিকারের পরীক্ষা হলো পুনরুদ্ধার; মূল ডিস্কে ব্যাকআপ রাখা মানে কোনো ব্যাকআপ না রাখা।

৪.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: ডিস্ক ভরে যাওয়ার কারণ খোঁজা (ল্যাবে)

1. `df -h` চালিয়ে কোন ফাইলসিস্টেম কত শতাংশ ভরা, দেখুন।
2. সবচেয়ে ভরা ফাইলসিস্টেমে `sudo du -sh /*` চালিয়ে বড় ফোল্ডার খুঁজুন।
3. সন্দেহভাজন ফোল্ডারে আরও গভীরে যান: `sudo du -sh /var/log/* | sort -rh | head -10`।
4. বড় লগ ফাইল পেলে `ls -lh` দিয়ে আকার নিশ্চিত করুন — কিন্তু প্রোডাকশনে লগ মুছে ফেলার আগে কারণ বুঝুন ও ব্যাকআপ ভাবুন।
5. `lsblk` দিয়ে ডিস্ক-পার্টিশন-মাউন্টের মানচিত্রটি নোট করুন।

৪.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল ধারণা:** "ডিস্ক ভরলে শুধু নতুন ফাইল লেখা বন্ধ হয়।" — আসলে চলমান অ্যাপ, ডেটাবেস ও লগইনও ভেঙে যেতে পারে।
- **ভুল:** `/etc/fstab` ভুল লিখে রিবুট করা — সার্ভার বুট আটকে যেতে পারে। সবসময় `mount -a` দিয়ে পরীক্ষা করুন।
- **ভুল:** ডিস্কের নাম (`/dev/sdb`) দিয়ে fstab লেখা — রিবুটে নাম বদলে যেতে পারে; UUID ব্যবহার করুন।
- **ভুল:** `rm -rf` দিয়ে বড় ফোল্ডার মুছে জায়গা খালি করা — মুছে ফেলা ডেটা ফেরে না; আগে নিশ্চিত হোন কী মুছছেন।

৪.১২ নিরাপত্তা বিবেচনা

`/tmp` ও লগে সংবেদনশীল তথ্য (পাসওয়ার্ড, টোকেন) লেখা এড়ান। ব্যাকআপ ডিস্ক আলাদা রাখুন — একই ডিস্কে ব্যাকআপ রাখলে ডিস্ক নষ্ট হলে ব্যাকআপও যায়। মাউন্ট অপশনে `noexec` / `nosuid` (যেমন `/tmp`-এ) আক্রমণকারীর কোড চালানো কঠিন করে — প্রোডাকশনে বিবেচনা করুন।

৪.১৩ সমস্যা সমাধান নির্দেশিকা

সমস্যা	কারণ	সমাধান
df-তে ১০০% ভরা	লগ/ডেটা/অস্থায়ী ফাইল	du দিয়ে বড় ফোল্ডার খুঁজুন; logrotate পরীক্ষা করুন
ফাইল মুছেও জায়গা খালি হচ্ছে না	চলমান প্রসেস ফাইলটি ধরে আছে	প্রসেসটি রিস্টার্ট করলে জায়গা ফেরে; `lsof | grep deleted` দিয়ে দেখুন
বুটে মাউন্ট ব্যর্থ	fstab ভুল	রিকভারি মোডে fstab.bak দিয়ে ফিরিয়ে আনুন
LVM বাড়ানো যাচ্ছে না	VG-তে ফাঁকা জায়গা নেই	আগে PV যোগ করুন বা VG বাড়ান, তারপর LV বাড়ান

`df -h`-তে জায়গা আছে, তবু ফাইল লেখা যাচ্ছে না	inode শেষ হয়ে গেছে (লাখো ছোট ফাইল)	`df -i` দিয়ে inode দেখুন; অপ্রয়োজনীয় ছোট ফাইল (যেমন ক্যাশ) পরিষ্কার করুন
মাউন্টের পর ফোল্ডারটি খালি দেখাচ্ছে	আগের ফাইলগুলো নতুন মাউন্টের নিচে ঢাকা পড়েছে	মাউন্ট খুলে (umount) দেখুন পুরোনো ফাইল আছে কি না; প্রয়োজনে ফাইল সরিয়ে আবার মাউন্ট করুন
fstab-এ UUID ভুলে বুটে emergency mode	ভুল UUID বা ভুল মাউন্ট পয়েন্ট লেখা হয়েছে	রিকভারি মোডে `/etc/fstab.bak` দিয়ে ফিরিয়ে আনুন; পরে `mount -a` দিয়ে পরীক্ষা করুন
LV বাড়ানোর পরও `df -h`-তে আকার বাড়েনি	`lvextend`-এর পর `resize2fs` চালানো হয়নি	ফাইলসিস্টেম বাড়াতে `sudo resize2fs /dev/…/…` চালান (ল্যাবে অনুশীলন করুন)

৪.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: `/var/log` ও ডেটার জন্য আলাদা পার্টিশন/LV; ৮০%-এ অ্যালার্ট; নিয়মিত `du` পর্যালোচনা; LVM দিয়ে ইনস্টল করা যাতে পরে বাড়ানো যায়। **অ্যান্টি-প্যাটার্ন:** সবকিছু এক পার্টিশনে; ডিস্ক ৯৫% ভরা অবস্থায় "পরে দেখব" বলা; fstab-এর ব্যাকআপ ছাড়া সম্পাদনা; ল্যাবে না পরীক্ষা করে প্রোডাকশনে `mkfs` চালানো।

৪.১৫ অধ্যায়ের সারাংশ

Linux-এ সবকিছু `/` থেকে শুরু হওয়া একটিই গাছ। পার্টিশন গুদামে দেয়াল, ফাইলসিস্টেম সাজানোর নিয়ম, মাউন্ট সেই কক্ষকে গাছের সঙ্গে যুক্ত করা, আর LVM দেয়ালকে নমনীয় করে। ডিস্ক পর্যবেক্ষণ (`df`, `du`, `lsblk`) প্রোডাকশনের দৈনন্দিন অভ্যাস — কারণ ডিস্ক ভরা মানেই অ্যাপ বন্ধ।

৪.১৬ মূল প্রযুক্তিগত পরিভাষা

Partition: ডিস্কের আলাদা করা অংশ।

Filesystem (ext4): ডিস্কে ফাইল সাজানোর নিয়ম ও সূচিপত্র।

Mount: ফাইলসিস্টেমকে ডিরেক্টরি গাছের সঙ্গে যুক্ত করা।

fstab: বুটের সময় কী কোথায় মাউন্ট হবে, তার তালিকা।

LVM (PV/VG/LV): ডিস্কের জায়গা নমনীয়ভাবে ভাগ করার ব্যবস্থা।

UUID: ডিস্ক/পার্টিশনের স্থায়ী পরিচয় নম্বর।

৪.১৭ জ্ঞান যাচাই প্রশ্ন

1. পার্টিশন, ফাইলসিস্টেম ও মাউন্টের পার্থক্য নিজের ভাষায় লিখুন।
2. `/var/log` আলাদা পার্টিশনে রাখা কেন নিরাপদ?
3. `df -h` ও `du -sh`-এর কাজের পার্থক্য কী?
4. LVM-র PV, VG, LV — তিন স্তরের কাজ কী কী?
5. fstab-এ ডিভাইসের নামের বদলে UUID কেন ব্যবহার করা হয়?
6. ফাইল মুছে ফেলার পরও ডিস্ক খালি না হওয়ার কারণ কী হতে পারে?

- inode শেষ হলে `df -h`-তে জায়গা থাকলেও ফাইল লেখা যায় না কেন? `df -i`-র ভূমিকাসহ ব্যাখ্যা করুন।
- Logical Volume বাড়ানোর পর `resize2fs` না করলে কী হবে, এবং কেন এই দুটি ধাপই আলাদাভাবে লাগে?

৪.১৮ ব্যবহারিক অনুশীলন

> **WARNING:** `mkfs` ও `rm -rf` ধ্বংসাত্মক কমান্ড — শুধুমাত্র ডিসপোজেবল ল্যাব VM-এ, স্ন্যাপশট নিয়ে, এবং কোন ডিস্কে চালাচ্ছেন তা তিনবার যাচাই করে চালান।

1. ল্যাব VM-এ `df -h`, `lsblk` ও `sudo du -sh /var/*` চালিয়ে ডিস্কের মানচিত্র ও বড় ফোল্ডারগুলোর তালিকা বানান।
2. VM-এ একটি অতিরিক্ত ভার্চুয়াল ডিস্ক যুক্ত করে (ল্যাবে) তাতে ext4 ফাইলসিস্টেম বানিয়ে `/data`-তে মাউন্ট করুন এবং fstab-এ UUID দিয়ে স্থায়ী করুন; রিবুট করে যাচাই করুন।
3. একটি LVM Logical Volume-র আকার বাড়ানোর ধাপগুলো (lvextend + resize2fs) ল্যাবে অনুশীলন করুন অথবা ধাপে ধাপে লিখে প্রতিটি কমান্ডের কাজ ব্যাখ্যা করুন।

- ল্যাব VM-এ `df -i` চালিয়ে inode ব্যবহার লিখুন; তারপর `/tmp`-এর একটি পরীক্ষামূলক ফোল্ডারে অনেকগুলো ছোট খালি ফাইল বানিয়ে inode ব্যবহার কীভাবে বাড়ে, তা দেখুন — শেষে ফোল্ডারটি মুছে ফেলুন। প্রোডাকশনে এই পরীক্ষা করবেন না।

অধ্যায় ৫

প্রসেস, সিগন্যাল, জব ও রিসোর্স ব্যবস্থাপনা

৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) প্রসেস (process) কী এবং কীভাবে তৈরি হয় তা ব্যাখ্যা করতে; (২) `ps`, `top`/`htop`, `systemctl` দিয়ে চলমান প্রসেস দেখতে ও নিয়ন্ত্রণ করতে; (৩) সিগন্যাল (signal) দিয়ে প্রসেস থামানো, পুনরায় চালু করা ও রিলোড করা; (৪) ব্যাকগ্রাউন্ড জব ও systemd সার্ভিসের পার্থক্য বুঝতে; (৫) CPU/RAM চাপ নির্ণয় করে সঠিক ব্যবস্থা নিতে।

৫.২ প্রোডাকশনে কেন এটি গুরুত্বপূর্ণ

প্রোডাকশনে অ্যাপ মানেই চলমান প্রসেস। অ্যাপ "ডাউন" মানে আসলে তার প্রসেস মারা গেছে বা আটকে গেছে। কোন প্রসেস কত CPU/RAM খাচ্ছে, কোনটি আটকে আছে, কোনটি বারবার রিস্টার্ট হচ্ছে — এসব না জানলে আপনি অন্ধভাবে সার্ভার রিবুট করবেন, আর সমস্যা ফিরে আসবে। systemd দিয়ে প্রসেসকে সার্ভিস হিসেবে চালানো মানে: সার্ভার রিবুট হলেও অ্যাপ নিজে চালু হবে, মারা গেলে নিজে ফিরে আসবে।

৫.৩ পূর্বশর্ত জ্ঞান

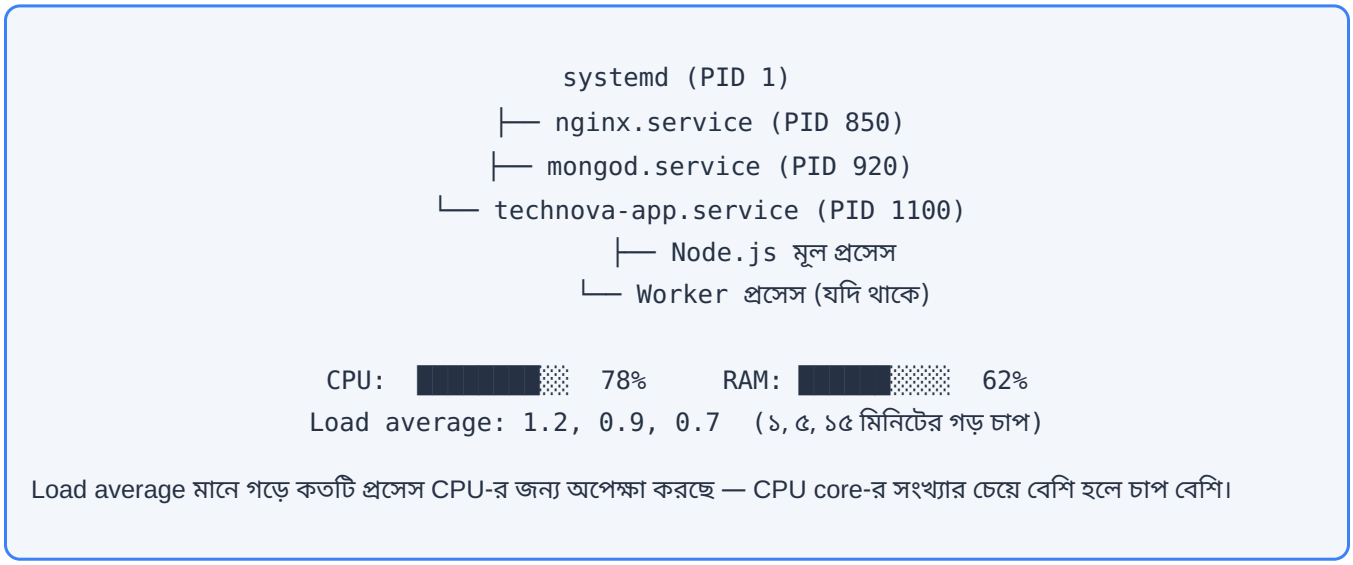
টার্মিনালে কমান্ড চালানো, অধ্যায় ১-এর CPU/RAM ধারণা এবং অধ্যায় ৩-এর sudo ধারণা। Node.js অ্যাপ চালানোর অভিজ্ঞতা থাকলে উদাহরণগুলো আরও জীবন্ত লাগবে।

৫.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রসেস হলো চলমান প্রোগ্রামের একটি জীবন্ত দৃষ্টান্ত — একই Node.js কোড থেকে দুটি প্রসেস চালালে তারা দুটি আলাদা জীবন পায়, আলাদা মেমরি পায়। প্রতিটি প্রসেসের একটি নম্বর থাকে: **PID** (Process ID)। প্রথম প্রসেস (PID 1) হলো **systemd** — সিস্টেমের অভিভাবক, যে সব সার্ভিস চালু করে ও দেখভাল করে। প্রসেসগুলো গাছের মতো জন্ম নেয়: একটি প্রসেস (parent) আরেকটি প্রসেস (child) তৈরি করে। **সিগন্যাল** হলো প্রসেসকে পাঠানো সংক্ষিপ্ত বার্তা: SIGTERM মানে "দয়া করে থামো", SIGKILL মানে "এখনই থামো, কোনো কথা নয়", SIGHUP মানে "কনফিগ আবার পড়ো"। আর **জব** (job) হলো আপনার শেলের (টার্মিনালের) ভেতরে চালানো কাজ — ফোরগ্রাউন্ডে চললে টার্মিনাল আটকে থাকে, ব্যাকগ্রাউন্ডে পাঠালে আপনি অন্য কাজ করতে পারেন।

> **CONCEPT:** SIGTERM বনাম SIGKILL মনে রাখুন: TERM হলো ভদ্র অনুরোধ — প্রসেস ফাইল বন্ধ করে, ডেটা সংরক্ষণ করে থামতে পারে। KILL হলো জোর — প্রসেস কিছুই গুছিয়ে যেতে পারে না, ডেটা নষ্ট হতে পারে। তাই সবসময় আগে TERM, কাজ না হলে পরে KILL।

৫.৫ আর্কিটেকচার / ধারণাগত চিত্র



Load average মানে গড়ে কতটি প্রসেস CPU-র জন্য অপেক্ষা করছে — CPU core-র সংখ্যার চেয়ে বেশি হলে চাপ বেশি।

৫.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

প্রসেস দেখা ও বোঝা

`ps aux` সব প্রসেসের তালিকা দেয়: USER, PID, %CPU, %MEM, COMMAND। `top` বা `htop` (ইনস্টল থাকলে) জীবন্তভাবে দেখায় — কোন প্রসেস সবচেয়ে বেশি সম্পদ খাচ্ছে। `uptime` দিয়ে load average দেখা যায়। মনে রাখবেন: Linux-এ ফাঁকা RAM দেখে ভয় পাবেন না — Linux অব্যবহৃত RAM ক্যাশ হিসেবে ব্যবহার করে; আসল বিপদ হলো Swap ব্যবহার শুরু হওয়া (ডিস্ককে RAM হিসেবে ব্যবহার — খুব ধীর)।

systemd সার্ভিস

systemd-র Unit ফাইল (`/etc/systemd/system/technova-app.service`) লিখে Node.js অ্যাপকে সার্ভিস বানানো যায়: বুটে চালু হবে, মারা গেলে `Restart=always` দিয়ে ফিরে আসবে, লগ যাবে journal-এ (`journalctl -u technova-app`)। `systemctl start/stop/restart/status/enable` দিয়ে সার্ভিস নিয়ন্ত্রণ করা হয় — `enable` মানে বুটে চালু হওয়ার ব্যবস্থা, `start` মানে এখনই চালু করা।

জব নিয়ন্ত্রণ ও অগ্রাধিকার

`command &` দিয়ে ব্যাকগ্রাউন্ডে চালানো যায়; `jobs` দিয়ে তালিকা, `fg` দিয়ে সামনে আনা, `bg` দিয়ে ব্যাকগ্রাউন্ডে পাঠানো যায়। তবে প্রোডাকশনে ব্যাকগ্রাউন্ড জব নয় — systemd সার্ভিস ব্যবহার করুন, কারণ টার্মিনাল বন্ধ হলে জব মারা যায়, সার্ভিস মরে না। `nice` দিয়ে প্রসেসের অগ্রাধিকার কমানো-বাড়ানো যায় — গুরুত্বপূর্ণ প্রসেসকে বেশি CPU দেওয়ার উপায়।

দ্বিতীয় দৃশ্য: বারবার রিস্টার্ট হওয়া সার্ভিসের নির্ণয়

ধরুন `systemctl status technova-app` দেখাচ্ছে সার্ভিসটি বারবার রিস্টার্ট হচ্ছে — চালু হয়, কয়েক সেকেন্ডে মরে, আবার চালু হয়। `Restart=always` থাকায় এই চক্র চলতেই থাকে। নির্ণয়ের ক্রম: প্রথমে `journalctl -u technova-app --since "1 hour ago" -n 100` দিয়ে শেষ এক ঘণ্টার লগ পড়ুন — প্রথম ত্রুটিটিই আসল কারণ। তারপর `systemctl show technova-app --property=ExecStart` দিয়ে দেখুন Unit ফাইলে লেখা পথটি আসলেই আছে কি না।

প্রথম ত্রুটির ধরন	সম্ভাব্য কারণ	পরবর্তী যাচাই

ফাইল বা পথ পাওয়া যাচ্ছে না	`ExecStart` বা `WorkingDirectory`-তে ভুল পথ	`ls -l` দিয়ে পথটি আছে কি না দেখুন
পোর্ট ইতিমধ্যে ব্যবহৃত হচ্ছে	পুরোনো প্রসেস এখনো পোর্ট 3000 ধরে আছে	`ps aux | grep node` দিয়ে পুরোনো প্রসেস খুঁজুন
পরিবেশ-চলক (environment variable) নেই	Unit ফাইলে `Environment` লাইন বাদ পড়েছে	Unit ফাইলটি পড়ে প্রয়োজনীয় চলকগুলো মেলান
অনুমতি নেই (Permission denied)	সার্ভিসের ব্যবহারকারীর ফোল্ডারে লেখার অধিকার নেই	`ls -ld` দিয়ে ফোল্ডারের মালিক ও অনুমতি দেখুন

আরেকটি দৃশ্য: load average বেশি, কিন্তু CPU ব্যবহার কম — প্রসেসগুলো ডিস্কের উত্তরের অপেক্ষায়, একে বলে I/O wait। তখন `top`-এ `wa` মান দেখুন।

৫.৭ গুরুত্বপূর্ণ কমান্ড ও তার বিশ্লেষণ

```
ps aux | grep node
# চলমান Node প্রসেস খুঁজুন; grep = লেখায় খোঁজা

top
# জীবন্ত সম্পদ-তালিকা; q চেপে বের হন

uptime
# কতক্ষণ চালু আছে ও load average দেখায়

kill 1100
# PID 1100-কে SIGTERM পাঠায় (ভদ্রভাবে থামতে বলে)

kill -9 1100
# SIGKILL — শেষ উপায় হিসেবে ব্যবহার করুন

systemctl status technova-app
systemctl restart technova-app
journalctl -u technova-app -n 50
# সার্ভিসের অবস্থা, রিস্টার্ট ও শেষ ৫০ লাইন লগ

free -h
# RAM ও Swap ব্যবহার দেখায়
```

```
ps -o pid,ppid,%cpu,%mem,cmd --sort=-%cpu | head
# সবচেয়ে বেশি CPU খাওয়া প্রসেসগুলো সাজিয়ে দেখায়
# -o = কোন কলাম দেখাবে; --sort=-%cpu = CPU অনুযায়ী বড় থেকে ছোট

ps --forest -o pid,ppid,cmd
# প্রসেসগুলোকে গাছ-আকারে দেখায় — কোনটি কার সন্তান, বোঝা যায়
# --forest = গাছের আকারে

pgrep -a node
# node নামের সব প্রসেসের PID ও কমান্ড দেখায়
# -a = PID-এর সঙ্গে কমান্ডও দেখাবে

systemctl is-active technova-app
```

```
# সার্ভিসটি এখন চালু আছে কি না, এক শব্দে জানায় (active/inactive/failed)

journalctl -u technova-app --since "1 hour ago" -n 100
# শেষ এক ঘণ্টার লগের শেষ ১০০ লাইন দেখায়
# --since = কখন থেকে; ক্র্যাশের সময়-ক্রম সাজাতে অমূল্য
```

`ps --forest` কাজে লাগে যখন মূল প্রসেস মারা গেলেও সন্তান-প্রসেস বেঁচে থাকে।

৫.৮ কনফিগারেশন উদাহরণ

Node.js অ্যাপের systemd Unit ফাইল (উদাহরণ):

```
# /etc/systemd/system/technova-app.service
[Unit]
Description=Technova Next.js App
After=network.target

[Service]
User=deploy
WorkingDirectory=/opt/technova-app
ExecStart=/usr/bin/node /opt/technova-app/server.js
Restart=always
RestartSec=5
Environment=NODE_ENV=production

[Install]
WantedBy=multi-user.target
```

প্রয়োগ: `sudo systemctl daemon-reload && sudo systemctl enable --now technova-app`। `daemon-reload` মানে systemd-কে নতুন Unit ফাইল পড়তে বলা।

৫.৯ বাস্তব প্রোডাকশন পরিস্থিতি

REAL WORLD: app.technova.example হঠাৎ সাড়া দেওয়া বন্ধ করল। `top`-এ দেখা গেল একটি Node প্রসেস CPU ১০০% খাচ্ছে — একটি অসীম লুপ (infinite loop) বাগ। দল সার্ভার রিবুট না করে প্রথমে `journalctl`-এ ত্রুটি দেখল, তারপর `systemctl restart technova-app` দিয়ে সার্ভিস ফিরিয়ে আনল এবং বাগটি ঠিক করল। শিক্ষা: রিস্টার্ট চিকিৎসা নয়, নির্ণয় আগে — লগ দেখুন, কারণ খুঁজুন, তারপর ব্যবস্থা নিন।

REAL WORLD: Technova-তে এক ডেপ্লয়ের পর app.technova.example খুলছিল না, অথচ `systemctl status technova-app` বলছিল সার্ভিস active। পরে `journalctl -u technova-app --since "1 hour ago"` পড়ে দেখা গেল নতুন অ্যাপ পোর্ট 3000-এ নয়, অন্য পোর্টে শুনছে — আর পুরোনো প্রসেসটি তখনো 3000 ধরে বসে আছে। অর্থাৎ দুটি প্রসেস, দুটি পোর্ট, আর Nginx ভুল পোর্টে অনুরোধ পাঠাচ্ছিল। সমাধান ছিল পুরোনো প্রসেসটি `kill` (আগে SIGTERM) দিয়ে থামানো ও ডেপ্লয়-চেকলিস্টে পোর্ট যাচাই যোগ করা। শিক্ষা: active মানেই সুস্থ নয় — সঠিক পোর্টে শুনছে কি না, আলাদাভাবে যাচাই করুন।

৫.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: একটি Node অ্যাপকে systemd সার্ভিস বানানো (ল্যাবে)

1. `/opt/technova-app/server.js`-এ একটি ছোট Express সার্ভার লিখুন (পোর্ট 3000)।
2. ওপরের Unit ফাইলটি `/etc/systemd/system/technova-app.service`-এ বসান।
3. `sudo systemctl daemon-reload` ও `sudo systemctl enable --now technova-app` চালান।
4. `systemctl status technova-app` ও `curl http://localhost:3000` দিয়ে যাচাই করুন।
5. সার্ভার রিবুট করে দেখুন অ্যাপ নিজে চালু হয় কি না — এটাই প্রোডাকশন আচরণ।
6. `journalctl -u technova-app -f` দিয়ে লাইভ লগ দেখুন।

৫.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল:** প্রোডাকশনে `node server.js &` বা `nohup` দিয়ে অ্যাপ চালানো — রিবুটে মরে, ক্র্যাশে ফেরে না, লগ ছড়িয়ে যায়। systemd ব্যবহার করুন।
- **ভুল:** প্রথমেই `kill -9` চালানো — ডেটা নষ্ট হতে পারে; আগে SIGTERM দিন, সময় দিন।
- **ভুল ধারণা:** "RAM ৯০% ভরা মানেই বিপদ।" — ক্যাশসহ দেখতে হয়; `free -h`-এর available কলামটিই আসল চিত্র দেয়।
- **ভুল:** PID না দেখে নামে আন্দাজে kill করা — ভুল প্রসেস মারা যেতে পারে।

৫.১২ নিরাপত্তা বিবেচনা

সার্ভিস `User=deploy` দিয়ে চালান — root দিয়ে অ্যাপ চালালে অ্যাপের কোনো দুর্বলতা দিয়ে আক্রমণকারী পুরো সার্ভার পেয়ে যাবে। অপ্রয়োজনীয় প্রসেস বন্ধ রাখুন; প্রতিটি চলমান প্রসেসই আক্রমণের সম্ভাব্য পথ। `kill` করার আগে PID ও কমান্ড যাচাই করুন।

৫.১৩ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	কারণ	প্রথম পদক্ষেপ
অ্যাপ সাড়া দিচ্ছে না	প্রসেস মারা গেছে বা আটকে আছে	`systemctl status` ও `journalctl -u` দেখুন
CPU ১০০%	অসীম লুপ বা ভারী কাজ	`top`-এ কোন প্রসেস, লগে কী লেখা দেখুন
RAM শেষ / OOM	মেমরি লিক বা অতিরিক্ত লোড	`free -h`, `dmesg | grep -i oom`
সার্ভিস বারবার রিস্টার্ট হচ্ছে	স্টার্টআপেই ক্র্যাশ (ভুল পথ/পরিবেশ)	`journalctl -u`-এ প্রথম ত্রুটিটি পড়ুন
সার্ভিস active, তবু পোর্টে সাড়া নেই	অ্যাপ অন্য পোর্টে শুনছে বা পুরোনো প্রসেস পোর্ট ধরে আছে	`pgrep -a node` দিয়ে কোন প্রসেস কোন পোর্টে, তা যাচাই করুন
`journalctl`-এ সার্ভিসের লগ খুঁজে পাওয়া যাচ্ছে না	Unit ফাইলে লগের গন্তব্য বদলে গেছে বা সার্ভিসটি কখনো চালুই হয়নি	`systemctl status` দিয়ে সার্ভিসের অবস্থা ও `systemctl is-enabled` দেখুন

`systemctl restart`-এর পরও পুরোনো প্রসেস বেঁচে আছে	সন্তান-প্রসেস (child) মূল প্রসেসের সঙ্গে মরেনি	`ps --forest` দিয়ে গাছ দেখে বেঁচে থাকা শাখাটি চিহ্নিত করুন
load average বেশি, কিন্তু CPU ব্যবহার কম	প্রসেসগুলো ডিস্কের উত্তরের অপেক্ষায় (I/O wait)	`top`-এ `wa` মান দেখুন; `df -h` ও ডিস্কের অবস্থা পরীক্ষা করুন

৫.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: সব প্রোডাকশন অ্যাপ systemd সার্ভিস হিসেবে; `Restart=always`; লগ journal-এ; নিয়মিত load ও মেমরি পর্যবেক্ষণ; রিস্টার্টের আগে লগ পড়া। **অ্যান্টি-প্যাটার্ন:** টার্মিনালে `&` দিয়ে অ্যাপ চালানো; kill -9 প্রথম অস্ত্র বানানো; কারণ না খুঁজে বারবার রিবুট করা; root দিয়ে অ্যাপ চালানো।

৫.১৫ অধ্যায়ের সারাংশ

প্রসেস হলো চলমান প্রোগ্রাম, PID তার পরিচয়, systemd (PID 1) তাদের অভিভাবক। SIGTERM ভদ্র অনুরোধ, SIGKILL শেষ উপায়। প্রোডাকশন অ্যাপ systemd সার্ভিস হিসেবে চলে — বুটে চালু হয়, ক্র্যাশে ফিরে আসে, লগ journal-এ রাখে। সম্পদ নির্ণয়ে `top`, `free`, `uptime` ও `journalctl` আপনার প্রধান হাতিয়ার।

৫.১৬ মূল প্রযুক্তিগত পরিভাষা

Process / PID: চলমান প্রোগ্রাম / তার পরিচয় নম্বর।

systemd: PID 1 প্রসেস; সার্ভিস চালু ও দেখভাল করে।

Signal (SIGTERM / SIGKILL): প্রসেসকে পাঠানো নিয়ন্ত্রণ-বার্তা।

Load Average: CPU-র জন্য অপেক্ষমান প্রসেসের গড় সংখ্যা।

OOM (Out of Memory): RAM শেষ হয়ে গেলে Kernel প্রসেস মেরে ফেলে।

Journal: systemd-র লগ ব্যবস্থা (`journalctl` দিয়ে পড়া যায়)।

৫.১৭ জ্ঞান যাচাই প্রশ্ন

1. প্রসেস ও প্রোগ্রামের পার্থক্য কী?
2. PID 1 কেন বিশেষ এবং সেটি কী করে?
3. SIGTERM ও SIGKILL-এর পার্থক্য এবং কখন কোনটি ব্যবহার করবেন?
4. `systemctl enable` ও `systemctl start`-এর পার্থক্য কী?

5. RAM ৯০% দেখালেও সিস্টেম সুস্থ থাকতে পারে কেন?
6. প্রোডাকশনে `node server.js &`-এর বদলে systemd কেন ভালো? তিনটি কারণ দিন।

- সার্ভিস active দেখালেও অ্যাপ সাড়া না দেওয়ার তিনটি সম্ভাব্য কারণ লিখুন, এবং প্রতিটি যাচাইয়ে কোন কমান্ড ব্যবহার করবেন, তা বলুন।
- `journalctl --since "1 hour ago"` ব্যবহার করে একটি ক্র্যাশের সময়-ক্রম কীভাবে সাজাবেন? প্রথম ত্রুটি কেন সবচেয়ে গুরুত্বপূর্ণ, তা ব্যাখ্যা করুন।

৫.১৮ ব্যবহারিক অনুশীলন

> **LAB:** ল্যাব VM-এ করুন। kill অনুশীলন শুধু নিজের পরীক্ষামূলক প্রসেসের ওপর করুন, অন্য সার্ভিসের ওপর নয়।

1. একটি ছোট Node অ্যাপ লিখে systemd সার্ভিস হিসেবে চালান; রিবুট করে স্বয়ংক্রিয় চালু হওয়া যাচাই করুন।
2. `sleep 300 &` দিয়ে একটি ব্যাকগ্রাউন্ড জব বানিয়ে `jobs`, `ps` ও `kill` (SIGTERM) দিয়ে নিয়ন্ত্রণ অনুশীলন করুন।
3. `top` ও `free -h` দিয়ে আপনার ল্যাব সার্ভারের একটি সম্পদ-প্রতিবেদন লিখুন: load average, RAM available ও সবচেয়ে বেশি CPU খাওয়া প্রসেস।

- ল্যাবে Unit ফাইলের `ExecStart`-এর পথ ভুল করে `daemon-reload` ও `restart` করে বারবার রিস্টার্ট হওয়ার আচরণ পর্যবেক্ষণ করুন। `journalctl`-এর প্রথম ত্রুটিটি লিখে রাখুন, তারপর পথটি ঠিক করে সার্ভিস সুস্থ করুন।

অধ্যায় ৬

প্যাকেজ ব্যবস্থাপনা, আপডেট ও সিস্টেম রক্ষণাবেক্ষণ

৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) APT দিয়ে প্যাকেজ ইনস্টল, আপডেট ও সরাতে; (২) `update` বনাম `upgrade`-এর পার্থক্য ব্যাখ্যা করতে; (৩) নিরাপত্তা আপডেট নিরাপদে প্রয়োগ করতে; (৪) অপ্রয়োজনীয় প্যাকেজ ও লগ পরিষ্কার করে ডিস্ক বাঁচাতে; (৫) একটি সাপ্তাহিক রক্ষণাবেক্ষণ রুটিন তৈরি করতে।

৬.২ প্রোডাকশনে কেন এটি গুরুত্বপূর্ণ

প্রোডাকশন সার্ভারের বেশিরভাগ নিরাপত্তা-দুর্বলতা ঠিক হয় আপডেটে — আপডেট না করা সার্ভার জানা দুর্বলতা নিয়ে ইন্টারনেটে দাঁড়িয়ে থাকে। আবার ভুল সময়ে ভুল আপডেট অ্যাপ ভেঙে দিতে পারে। তাই রক্ষণাবেক্ষণ মানে দুই দিকের ভারসাম্য: নিরাপত্তা আপডেট দ্রুত, বড় পরিবর্তন পরীক্ষা করে। যে সার্ভারের যত্ন নেওয়া হয় না, সেটি ধীরে ধীরে ডিস্ক ভরে, লগ জমে ও পুরোনো প্যাকেজে ভরে অস্থির হয়ে ওঠে।

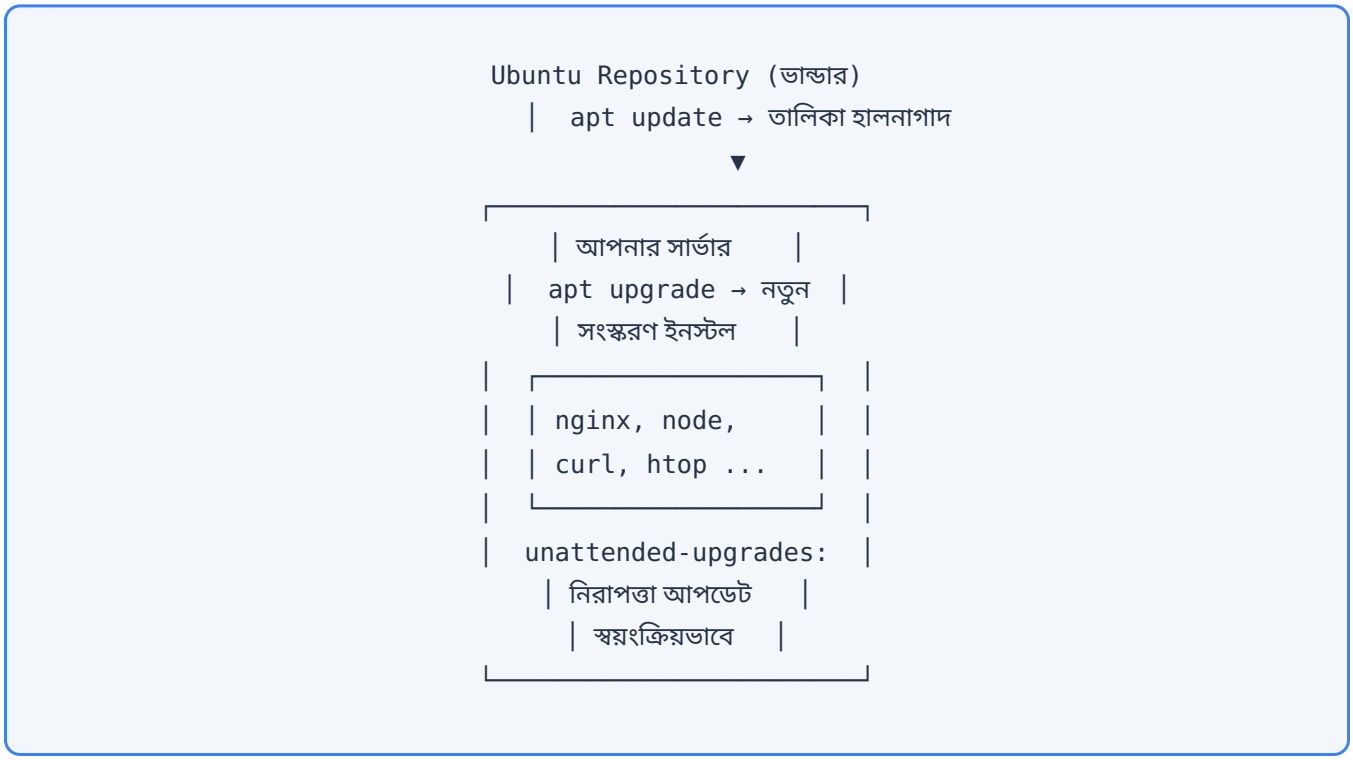
৬.৩ পূর্বশর্ত জ্ঞান

টার্মিনালে sudo দিয়ে কমান্ড চালানো (অধ্যায় ৩), systemd সার্ভিসের ধারণা (অধ্যায় ৫) এবং ডিস্ক পর্যবেক্ষণ (অধ্যায় ৪)। প্যাকেজ ব্যবস্থাপকের আগের অভিজ্ঞতা লাগবে না — npm-এর সঙ্গে তুলনা করে বোঝানো হবে।

৬.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্যাকেজ হলো সফটওয়্যারের গোছানো বাক্স — প্রোগ্রাম, তার কনফিগ ও নির্ভরতা (dependency) একসঙ্গে। npm যেমন JavaScript প্যাকেজ দেয়, **APT** (Advanced Package Tool) তেমনি Ubuntu-র সিস্টেম প্যাকেজ দেয় — Nginx, Node.js, curl সবই প্যাকেজ। প্যাকেজগুলো থাকে **Repository** (ভান্ডার)-এ — Ubuntu-র অফিসিয়াল সার্ভারে। `apt update` মানে ভান্ডারের তালিকা হালনাগাদ করা (কী নতুন এসেছে জানা), `apt upgrade` মানে আসলে নতুন সংস্করণ ইনস্টল করা। দুটি আলাদা কাজ — update ছাড়া upgrade জানেই না কী নতুন আছে। **রক্ষণাবেক্ষণ** মানে নিয়মিত যত্ন: আপডেট, পরিষ্কার, লগ ঘোরানো (logrotate) ও স্বাস্থ্য পরীক্ষা।

৬.৫ আর্কিটেকচার / ধারণাগত চিত্র



৬.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

APT-র মূল কমান্ড

`apt install নাম` ইনস্টল করে, `apt remove নাম` সরায় (কনফিগ রেখে দেয়), `apt purge নাম` কনফিগসহ সরায়। `apt search` দিয়ে খোঁজা যায়, `apt show` দিয়ে বিস্তারিত দেখা যায়। `apt autoremove` অপ্রয়োজনীয় নির্ভরতা সরায়, `apt autoclean` পুরোনো ডাউনলোড করা ফাইল মুছে ডিস্ক বাঁচায়।

আপডেটের ধরন ও ঝুঁকি

নিরাপত্তা আপডেট — দ্রুত প্রয়োগ করুন; এগুলো দুর্বলতা বন্ধ করে। **সাধারণ আপডেট** — পরীক্ষা করে প্রয়োগ করুন। **Kernel আপডেট** — কার্যকর হতে সাধারণত রিবুট লাগে; `reboot required` বার্তা দেখলে রক্ষণাবেক্ষণ-সময়ে (maintenance window) রিবুটের পরিকল্পনা করুন। প্রোডাকশনে কখনোই ব্যস্ত সময়ে `apt upgrade -y` চালিয়ে দেবেন না — আগে ল্যাবে বা স্টেজিংয়ে পরীক্ষা করুন।

স্বয়ংক্রিয় আপডেট ও লগ ঘোরানো

unattended-upgrades চালু থাকলে নিরাপত্তা আপডেট স্বয়ংক্রিয়ভাবে ইনস্টল হয় — ছোট সার্ভারের জন্য ভালো। **logrotate** লগ ফাইল বড় হলে পুরোনো অংশ সংরক্ষণ করে নতুন ফাইল শুরু করে — নইলে লগই ডিস্ক খেয়ে ফেলে (অধ্যায় ৪-এর ঘটনা মনে করুন)। `journalctl --vacuum-size=500M` দিয়ে systemd লগের আকার সীমিত করা যায়।

কাজের উদাহরণ: আপডেটের আগে-পরে যাচাই তালিকা

নিরাপদ আপডেটের রহস্য হলো আপডেটটি নিজে নয় — তার আগে ও পরের যাচাই। Technova-র দল প্রতি সপ্তাহে একই ক্রম মেনে চলে: প্রথমে স্ন্যাপশট বা ব্যাকআপ, তারপর `apt list --upgradable` দিয়ে কী বদলাবে তা পড়া, তারপর

নিরাপত্তা আপডেট প্রয়োগ, আর শেষে সার্ভিস পরীক্ষা। ব্যাকআপ ছাড়া আপডেট মানে ফেরার পথ না রাখা; তালিকা না পড়ে আপডেট মানে কী বদলাচ্ছে না জানা; আপডেটের পর যাচাই না করা মানে ভাঙা অ্যাপ অভিযোগে ধরা পড়া।

পর্যায়	কাজ	যাচাই
আপডেটের আগে	স্ন্যাপশট/ব্যাকআপ নেওয়া	ব্যাকআপটি সত্যিই নেওয়া হয়েছে কি না দেখুন
আপডেটের আগে	`apt list --upgradable` পড়া	কোন প্যাকেজগুলো বদলাবে, তার তালিকা লিখে রাখুন
আপডেটের সময়	নিরাপত্তা আপডেট আগে, বড় পরিবর্তন পরে	ব্যস্ত সময় এড়িয়ে maintenance window বেছে নিন
আপডেটের পরে	সার্ভিস ও সাইট পরীক্ষা	`systemctl status` ও ব্রাউজার থেকে সাইট খুলে দেখুন
আপডেটের পরে	রিবুট প্রয়োজন কি না দেখা	`ls /var/run/reboot-required` — থাকলে পরিকল্পনা করে রিবুট করুন

দ্বিতীয় দৃশ্য: `unattended-upgrades` চালু আছে কি না, তা না জেনে দল ভাবছিল আপডেট হচ্ছে না। `cat /etc/apt/apt.conf.d/20auto-upgrades` পড়লেই বোঝা যায়। শিক্ষা: অনুমান নয়, কনফিগ ফাইল পড়ে যাচাই করুন; বড় সংস্করণ-পরিবর্তন সবসময় হাতে পরীক্ষা করে করুন।

৬.৭ গুরুত্বপূর্ণ কমান্ড ও তার বিশ্লেষণ

```
sudo apt update
# ভান্ডারের তালিকা হালনাগাদ করে (কিছু ইনস্টল করে না)

sudo apt upgrade -y
# ইনস্টল করা প্যাকেজ নতুন সংস্করণে তোলে (-y = নিশ্চিতকরণ ছাড়াই)

sudo apt install nginx htop curl -y
# একসঙ্গে একাধিক প্যাকেজ ইনস্টল

sudo apt autoremove --purge -y
# অপ্রয়োজনীয় প্যাকেজ ও তাদের কনফিগ সরায়

sudo apt autoclean
# পুরোনো ডাউনলোড করা প্যাকেজ ফাইল মুছে

apt list --upgradable
# কোন প্যাকেজগুলোর নতুন সংস্করণ এসেছে, দেখায়

sudo journalctl --vacuum-size=500M
# systemd লগ ৫০০ MB-এ সীমিত করে
```

নমুনা আউটপুট — চিত্রণমূলক:

```
নমুনা আউটপুট — চিত্রণমূলক
$ apt list --upgradable
```

```
nginx/24.04 1.24.0-2ubuntu7.1 amd64 [upgradable from: 1.24.0-2ubuntu7]
# অর্থ: nginx-এর নতুন সংস্করণ পাওয়া যাচ্ছে
```

```
apt-cache policy nginx
# কোন সংস্করণ ইনস্টল আছে, কোন সংস্করণ ভান্ডারে পাওয়া যাচ্ছে — দুটোই দেখায়
# আপডেটের আগে কী বদলাবে, তা বুঝতে প্রথমে এই কমান্ডটি চালান

dpkg -l | grep nginx
# ইনস্টল করা প্যাকেজের তালিকায় nginx আছে কি না, তার সংস্করণসহ দেখায়
# -l = তালিকা (list); grep = লেখায় খোঁজা

ls /var/run/reboot-required
# ফাইলটি থাকলে বুঝবেন Kernel বা গুরুত্বপূর্ণ আপডেটের পর রিবুট লাগবে
# ফাইলটি না থাকলে রিবুটের প্রয়োজন নেই

cat /etc/apt/apt.conf.d/20auto-upgrades
# স্বয়ংক্রিয় আপডেট চালু আছে কি না, তার কনফিগ দেখায়
# unattended-upgrades-এর সেটিং এখানেই লেখা থাকে
```

`apt-cache policy` আপডেটের আগের গুরুত্বপূর্ণ যাচাই: ইনস্টল করা সংস্করণ ও ভান্ডারের সংস্করণ এক কি না, তা দেখা যায়। ভিন্ন হলে তালিকাটি লিখে রাখুন, যাতে কিছু ভাঙলে কোন সংস্করণ থেকে কোথায় গিয়েছিলেন, তা বলতে পারেন।

৬.৮ কনফিগারেশন উদাহরণ

স্বয়ংক্রিয় নিরাপত্তা আপডেট চালু করা:

```
sudo apt install unattended-upgrades -y
sudo dpkg-reconfigure -plow unattended-upgrades
# প্রশ্নে Yes বেছে নিন — নিরাপত্তা আপডেট স্বয়ংক্রিয় হবে
```

logrotate-এর উদাহরণ (অ্যাপের লগের জন্য, `/etc/logrotate.d/technova-app`):

```
/var/log/technova-app/*.log {
    daily
    rotate 14
    compress
    missingok
    notifempty
}
# প্রতিদিন ঘোরাবে, ১৪ দিন রাখবে, পুরোনোগুলো সংকুচিত (compress) করবে
```

৬.৯ বাস্তব প্রোডাকশন পরিস্থিতি

REAL WORLD: Technova-র সার্ভারে ছয় মাস আপডেট হয়নি। একটি জানা Nginx দুর্বলতার খবর আসার পর দল তড়িঘড়ি আপডেট করতে গিয়ে দেখল — একসঙ্গে ২০০+ প্যাকেজের আপডেটে একটি লাইব্রেরি বদলে গিয়ে অ্যাপের একটি নির্ভরতা ভেঙে গেল। শিক্ষা: আপডেট নিয়মিত ছোট ছোট করে করুন, ছয় মাস জমিয়ে নয়। আর বড় আপডেটের আগে স্ন্যাপশট/ব্যাকআপ নিন, যাতে ফিরে আসা যায়।

REAL WORLD: Technova-র দল এক শুক্রবার রাত ১১টায় — ব্যবহারকারী কম থাকার সময়ে — একটি Kernel আপডেট প্রয়োগ করল। আপডেটের পর `ls /var/run/reboot-required` ফাইলটি দেখা গেল, অর্থাৎ রিবুট লাগবে। দল প্রথমে `systemctl status technova-app nginx` দিয়ে সার্ভিস যাচাই করল, তারপর পরিকল্পনা করে রিবুট করল এবং পরে আবার সাইট খুলে দেখল — পুরো কাজ ২০ মিনিটে শেষ হলো। শিক্ষা: maintenance window-এর আসল কাজ যাচাইয়ের ক্রমটি মেনে চলা; রিবুটের বার্তা দেখে ভয় পাবেন না, পরিকল্পনা করে রিবুট করুন।

৬.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: নিরাপদ সাপ্তাহিক রক্ষণাবেক্ষণ (ল্যাবে অনুশীলন)

1. স্ন্যাপশট/ব্যাকআপ নিন (ল্যাব VM-এ স্ন্যাপশট)।
2. `sudo apt update` ও `apt list --upgradable` দিয়ে কী আপডেট আসছে দেখুন।
3. নিরাপত্তা আপডেট আলাদা করে চিনুন; `sudo apt upgrade -y` চালান।
4. `systemctl status technova-app nginx` দিয়ে সার্ভিসগুলো ঠিক আছে কি না দেখুন।
5. `sudo apt autoremove --purge -y && sudo apt autoclean` দিয়ে পরিষ্কার করুন।
6. `df -h` দিয়ে ডিস্ক ও `sudo journalctl --disk-usage` দিয়ে লগের আকার দেখুন।
7. রিবুট প্রয়োজন কি না দেখুন: `ls /var/run/reboot-required` — থাকলে পরিকল্পনা করে রিবুট করুন।

৬.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল ধারণা:** "`apt update` করলেই আপডেট হয়ে যায়।" — না, update শুধু তালিকা হালনাগাদ করে; upgrade-ই আসল ইনস্টল।
- **ভুল:** ব্যস্ত সময়ে বিনা পরীক্ষায় বড় আপডেট চালানো।
- **ভুল:** অজানা PPA/ভান্ডার যোগ করে প্যাকেজ ইনস্টল করা — অফিসিয়াল ভান্ডারই নিরাপদ; তৃতীয় পক্ষের উৎস যাচাই ছাড়া যোগ করবেন না।
- **ভুল:** `apt remove` ও `purge`-এর পার্থক্য না জানা — remove কনফিগ রেখে দেয়, পরে আবার ইনস্টল করলে পুরোনো কনফিগ ফিরে আসে।

৬.১২ নিরাপত্তা বিবেচনা

অফিসিয়াল Ubuntu ভান্ডার ও বিশ্বস্ত উৎস ছাড়া প্যাকেজ ইনস্টল করবেন না; ডাউনলোড করা `.deb` ফাইল যাচাই ছাড়া ইনস্টল করবেন না। নিরাপত্তা আপডেট দ্রুত প্রয়োগ করুন — আক্রমণকারীরা জানা দুর্বলতা খুঁজে বেড়ায়। আপডেটের পর সার্ভিসগুলো চলছে কি না যাচাই করুন। কখনোই নিরাপত্তা আপডেট বন্ধ করে "স্থিতিশীলতা" কিনবেন না — এটি ভুল সুরক্ষা।

৬.১৩ সমস্যা সমাধান নির্দেশিকা

সমস্যা	কারণ	সমাধান
apt lock আটকে আছে	আরেকটি apt প্রসেস চলছে	অপেক্ষা করুন; চলমান প্রসেস শেষ হলে আবার চেষ্টা করুন (lock ফাইল হাতে মুছবেন না)

প্যাকেজ পাওয়া যাচ্ছে না	তালিকা পুরোনো বা নাম ভুল	`apt update` করে `apt search` নাম দিয়ে যাচাই করুন
আপডেটের পর অ্যাপ ভেঙেছে	নির্ভরতা বদলেছে	লগ দেখুন; স্ন্যাপশট/ব্যাকআপ থেকে ফিরে যান (rollback); ল্যাবে পরীক্ষা করে আবার করুন
ডিস্কে জায়গা কমছে	পুরোনো প্যাকেজ/লগ জমেছে	`apt autoclean`, `journalctl --vacuum-size`, logrotate পরীক্ষা করুন
`apt update`-এ ভান্ডার বা GPG ত্রুটি	ভান্ডারের তালিকা বা স্বাক্ষর-কীতে সমস্যা	ত্রুটির বার্তাটি পুরো পড়ুন; অজানা ভান্ডার যোগ করে থাকলে তা যাচাই করুন
আপডেটের মাঝপথে apt আটকে গেছে	আগের আপডেট শেষ হয়নি বা dpkg মাঝপথে থেমেছে	চলমান প্রসেস শেষ হওয়ার অপেক্ষা করুন; প্রয়োজনে `sudo dpkg --configure -a` চালান — lock ফাইল হাতে মুছবেন না
unattended-upgrades চলছে বলে apt lock	স্বয়ংক্রিয় আপডেট এখন চলছে	কিছুক্ষণ অপেক্ষা করুন; `ps aux | grep unattended` দিয়ে চলমান কাজটি দেখুন
`/boot` পার্টিশন ভরে যাচ্ছে	পুরোনো Kernel সংস্করণ জমে আছে	`dpkg -l | grep linux-image` দিয়ে পুরোনো Kernel দেখুন; `apt autoremove --purge` দিয়ে পরিষ্কার করুন

৬.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: সাপ্তাহিক রক্ষণাবেক্ষণ রুটিন; নিরাপত্তা আপডেট স্বয়ংক্রিয়; বড় আপডেটের আগে ব্যাকআপ/স্ন্যাপশট ও স্টেজিং পরীক্ষা; আপডেটের পর সার্ভিস যাচাই; নিয়মিত পরিষ্কার। **অ্যান্টি-প্যাটার্ন:** মাসের পর মাস আপডেট না করা; ব্যস্ত সময়ে আপডেট; ব্যাকআপ ছাড়া আপডেট; অজানা উৎস থেকে প্যাকেজ ইনস্টল।

৬.১৫ অধ্যায়ের সারাংশ

APT হলো Ubuntu-র প্যাকেজ ব্যবস্থাপক: `update` তালিকা হালনাগাদ করে, `upgrade` নতুন সংস্করণ ইনস্টল করে। নিরাপত্তা আপডেট দ্রুত, বড় পরিবর্তন পরীক্ষা করে — এই ভারসাম্যই রক্ষণাবেক্ষণের মূল। নিয়মিত পরিষ্কার, লগ ঘোরানো ও স্বাস্থ্য পরীক্ষা সার্ভারকে স্থিতিশীল রাখে। Part I এখানেই শেষ — এখন আপনি সার্ভার বোঝেন, ইনস্টল করতে পারেন, নিরাপদে ঢুকতে পারেন, ডিস্ক সামলাতে পারেন, প্রসেস নিয়ন্ত্রণ করতে পারেন এবং সিস্টেমের যত্ন নিতে পারেন।

৬.১৬ মূল প্রযুক্তিগত পরিভাষা

Package / APT: গোছানো সফটওয়্যার-বাক্স / Ubuntu-র প্যাকেজ ব্যবস্থাপক।

Repository: প্যাকেজের অফিসিয়াল ভান্ডার।

Dependency: একটি প্যাকেজ চালাতে প্রয়োজনীয় অন্য প্যাকেজ।

unattended-upgrades: নিরাপত্তা আপডেট স্বয়ংক্রিয়ভাবে ইনস্টল করার ব্যবস্থা।

logrotate: লগ ফাইল ঘুরিয়ে পুরোনো লগ সংরক্ষণ/মুছে ফেলার ব্যবস্থা।

Maintenance Window: ব্যবহারকারী কম থাকার সময়, যখন রক্ষণাবেক্ষণ করা নিরাপদ।

৬.১৭ জ্ঞান যাচাই প্রশ্ন

1. `apt update` ও `apt upgrade`-এর পার্থক্য কী?
2. নিরাপত্তা আপডেট দ্রুত করা কেন জরুরি, একটি যুক্তি দিন।
3. `apt remove` ও `apt purge`-এর পার্থক্য কী?
4. Kernel আপডেটের পর কেন রিবুট লাগতে পারে?
5. logrotate না থাকলে কী সমস্যা হবে, অধ্যায় ৪-এর ঘটনার আলোকে ব্যাখ্যা করুন।
6. একটি সাপ্তাহিক রক্ষণাবেক্ষণ চেকলিস্ট বানান — অন্তত পাঁচটি কাজসহ।

- `apt-cache policy nginx` আউটপুট থেকে কীভাবে বুঝবেন কোন সংস্করণ ইনস্টল আছে ও কোনটি ভান্ডারে পাওয়া যাচ্ছে? দুটি সংস্করণ ভিন্ন হলে এর অর্থ কী?
- আপডেটের মাঝে dpkg আটকে গেলে প্রথমে কী করবেন, এবং কেন lock ফাইলটি হাতে মুছে ফেলা বিপজ্জনক?

৬.১৮ ব্যবহারিক অনুশীলন

LAB: ল্যাব VM-এ করুন; আপডেটের আগে স্ন্যাপশট নিন। প্রোডাকশনে ব্যস্ত সময়ে আপডেট করবেন না।

1. ল্যাব VM-এ `apt update`, `apt list --upgradable` ও `apt upgrade` চালিয়ে আপডেটের আগে-পরে সার্ভিসের অবস্থা (`systemctl status`) নোট করুন।
2. `htop` ও `curl` ইনস্টল করে `apt show htop` দিয়ে তার বিস্তারিত পড়ুন; তারপর `apt autoremove` ও `apt autoclean` চালান।
3. unattended-upgrades চালু করুন এবং একটি সাপ্তাহিক রক্ষণাবেক্ষণ চেকলিস্ট (আপডেট, পরিষ্কার, ডিস্ক, লগ, সার্ভিস যাচাই) নিজের ভাষায় লিখুন।

- ল্যাব VM-এ কোনো প্যাকেজ আপডেট না করে শুধু `apt update` ও `apt-cache policy nginx` চালিয়ে একটি আপডেট-পরিকল্পনা লিখুন: কোন প্যাকেজের নতুন সংস্করণ এসেছে, কখন (কোন maintenance window-এ) আপডেট করবেন, আর কিছু ভাঙলে কীভাবে ফিরে যাবেন — লিখুন।

PART II • DEVOPS ENGINEERING MASTERY

Server Services and Application Infrastructure

systemd, Nginx, Virtual Host, Reverse Proxy, TLS, Node.js ডেপ্লয়মেন্ট, PM2 ও ডেটাবেস সার্ভার।

অধ্যায় ৭

systemd Architecture and Service Management

৭.১ শেখার উদ্দেশ্য (Learning Objectives)

এই অধ্যায় শেষে আপনি পারবেন: systemd কী এবং কেন Linux-এর প্রায় সব আধুনিক ডিস্ট্রিবিউশনে এটি PID 1 হিসেবে চলে তা ব্যাখ্যা করতে; unit, service, target, timer, socket — এই ধারণাগুলোর পার্থক্য বলতে; `systemctl` ও `journalctl` দিয়ে সার্ভিস চালু, বন্ধ, পুনরায় চালু, সক্রিয় ও লগ দেখতে; নিজের Node.js অ্যাপের জন্য একটি সঠিক systemd unit ফাইল লিখতে; সার্ভিস ব্যর্থ হলে কেন ব্যর্থ হলো তা নির্ণয় করতে; এবং boot-এর সময় কোন সার্ভিস কখন চালু হবে তা নিয়ন্ত্রণ করতে।

৭.২ প্রোডাকশনে কেন এটি গুরুত্বপূর্ণ

আপনার অ্যাপ যদি সার্ভার রিস্টার্টের পর নিজে থেকে চালু না হয়, অথবা ক্র্যাশ করলে কেউ তাকে আবার না তোলে, তাহলে সেটি প্রোডাকশন অ্যাপ নয় — সেটি একটি পরীক্ষামূলক প্রক্রিয়া। প্রোডাকশনে সার্ভার মাঝরাতে রিবুট হতে পারে, কার্নেল আপডেট হতে পারে, মেমোরি শেষ হয়ে প্রক্রিয়া মারা যেতে পারে। systemd হলো সেই অভিভাবক যে নিশ্চিত করে: নির্ধারিত সার্ভিসগুলো ঠিক সময়ে চালু হবে, ক্র্যাশ করলে আবার চালু হবে, লগ এক জায়গায় জমা হবে এবং কোন সার্ভিস কিসের ওপর নির্ভরশীল তা স্পষ্ট থাকবে। একজন ডেভেলপার হিসেবে আপনি `node server.js` চালিয়ে টার্মিনাল বন্ধ করলেই অ্যাপ মরে যায় — এই অভিজ্ঞতা থেকেই systemd-এর প্রয়োজন বোঝা যায়।

> **REAL WORLD:** একটি ই-কমার্স সাইটে রাত ৩টায় ডেটাবেস সার্ভার রিবুট হয়েছিল। PostgreSQL systemd unit-এর কারণে নিজে থেকে চালু হয়েছিল, কিন্তু অ্যাপ সার্ভিসের unit-এ ডেটাবেসের ওপর নির্ভরতা (`After=postgresql.service`) লেখা ছিল না। ফলে অ্যাপ ডেটাবেসের আগেই চালু হয়ে সংযোগ ব্যর্থ হয়ে থেমে গিয়েছিল — সকাল পর্যন্ত সাইট ডাউন ছিল। নির্ভরতা লেখা থাকলে এই ঘটনা ঘটত না।

৭.৩ পূর্বশর্ত জ্ঞান (Prerequisite Knowledge)

আপনার জানা থাকা দরকার: Linux-এ প্রক্রিয়া (process) কী; টার্মিনালে কমান্ড চালানো; ফাইল অনুমতি (permission) ও `sudo`-এর ধারণা; এবং একটি Node.js অ্যাপ কীভাবে `node` দিয়ে চলে। PID (Process ID) সম্পর্কে ধারণা থাকলে ভালো হয় — প্রতিটি চলমান প্রোগ্রামকে কার্নেল একটি নম্বর দেয়, সেটিই PID। আগের অধ্যায়গুলোতে Linux ফাইল সিস্টেম ও ব্যবহারকারী ব্যবস্থাপনা পড়া থাকলে এই অধ্যায় সহজ লাগবে।

৭.৪ প্রথম নীতি থেকে ব্যাখ্যা (First Principles)

কম্পিউটার চালু হলে কার্নেল (kernel — অপারেটিং সিস্টেমের মূল অংশ) প্রথমে একটি প্রক্রিয়া চালায়, যাকে PID 1 বলা হয়। এই প্রথম প্রক্রিয়াই বাকি সবকিছু চালু করার দায়িত্ব নেয় — নেটওয়ার্ক, ডিস্ক, লগিং, অন্যান্য সার্ভিস। পুরনো দিনে এই কাজ করত SysV init, যা ছিল ধারাবাহিক স্ক্রিপ্টের সমষ্টি: একটা শেষ হলে আরেকটা। systemd সেই ধারণাকে বদলে দিয়েছে — এটি সমান্তরালে (parallel) সার্ভিস চালু করে, নির্ভরতা বোঝে এবং প্রতিটি সার্ভিসকে একটি unit হিসেবে দেখে। Unit হলো systemd-এর মৌলিক একক — একটি সার্ভিস, একটি মাউন্ট পয়েন্ট, একটি টাইমার — সবই unit। Service unit হলো সবচেয়ে পরিচিত: এটি বলে দেয় কোন প্রোগ্রাম চালাতে হবে, কোন ব্যবহারকারী হিসেবে চালাতে হবে, ব্যর্থ হলে কী করতে হবে। অর্থাৎ systemd শুধু "প্রোগ্রাম চালানো" নয়, এটি সিস্টেমের জীবনচক্রের ব্যবস্থাপক।

৭.৫ আর্কিটেকচার / ধারণাগত চিত্র

Boot প্রবাহ (ধারণাগত):

BIOS / UEFI → Bootloader → Linux Kernel → **systemd (PID 1)** → Targets (multi-user.target) → Units সমান্তরালে চালু

systemd ──┬── nginx.service (ওয়েব সার্ভার)

├── postgresql.service (ডেটাবেস)

├── app.service (আপনার Node.js অ্যাপ — After=postgresql.service)

├── journald (লগ সংগ্রহ) └── timers (নির্ধারিত কাজ)

নির্ভরতা: app → ডেটাবেস; nginx → নেটওয়ার্ক। একটি ব্যর্থ হলে systemd জানে কাকে আবার চালু করতে হবে।

৭.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Unit ফাইলের গঠন

একটি service unit ফাইলে তিনটি প্রধান অংশ থাকে। `[Unit]` অংশে বর্ণনা ও নির্ভরতা (Description, After, Requires, Wants) লেখা হয়। `[Service]` অংশে আসল কাজ — কোন কমান্ড চালাতে হবে (ExecStart), কোন ব্যবহারকারী (User), কোন ডিরেক্টরি থেকে (WorkingDirectory), পরিবেশ চলক (Environment), ব্যর্থ হলে পুনরায় চালু করবে কি না (Restart)। `[Install]` অংশে বলা হয় কোন target-এর সঙ্গে এই সার্ভিস যুক্ত হবে (WantedBy=multi-user.target)। `Requires` মানে কঠোর নির্ভরতা — নির্ভর সার্ভিস ব্যর্থ হলে এটিও চালু হবে না; `Wants` মানে নরম নির্ভরতা — চেষ্টা করবে, না পারলেও চালু হবে। `After` শুধু ক্রম নির্ধারণ করে, নির্ভরতা তৈরি করে না — এই পার্থক্যটি অনেকেই গুলিয়ে ফেলেন।

Target কী

Target হলো সার্ভিসের একটি গুচ্ছের নাম — যেমন `multi-user.target` মানে সাধারণ সার্ভার অবস্থা (নেটওয়ার্কসহ, কিন্তু গ্রাফিক্যাল ইন্টারফেস ছাড়া), `graphical.target` মানে ডেস্কটপসহ অবস্থা। সার্ভারে আমরা প্রায় সবসময় multi-user.target ব্যবহার করি।

journald ও লগ

systemd-এর লগ ব্যবস্থার নাম journald। সার্ভিসের stdout/stderr স্বয়ংক্রিয়ভাবে journal-এ জমা হয়। `journalctl -u app.service` দিয়ে শুধু সেই সার্ভিসের লগ দেখা যায়। এটি আলাদা লগ ফাইল খোঁজার ঝামেলা কমায়, তবে ডিস্ক পূর্ণ হয়ে গেলে journal-ও সমস্যা করতে পারে — তাই log rotation ও `SystemMaxUse` সম্পর্কে সচেতন থাকুন।

Restart নীতি

`Restart=always` মানে যেকোনোভাবে থামলে আবার চালু হবে; `on-failure` মানে শুধু অস্বাভাবিকভাবে থামলে; `RestartSec=5` মানে আবার চালুর আগে ৫ সেকেন্ড অপেক্ষা। ক্র্যাশ-লুপ এড়াতে `StartLimitBurst` ও `StartLimitIntervalSec` ব্যবহার করা হয় — বারবার ব্যর্থ হলে systemd আর চেষ্টা থামিয়ে দেয়, যাতে লগে ঝড় না ওঠে।

অন্যান্য unit ধরন ও override (drop-in) ব্যবহার

Service ছাড়াও systemd-এর আরও কয়েকটি unit ধরন প্রোডাকশনে কাজে লাগে। `timer` unit একটি নির্দিষ্ট সময়ে বা নির্দিষ্ট ব্যবধানে একটি service চালায় — cron-এর বিকল্প হিসেবে এটি ভালো, কারণ timer-এর প্রতিটি চালনার লগ journald-এ থাকে এবং ব্যর্থ হলে systemd তা জানায়। `Type=oneshot` হলো এমন সার্ভিসের জন্য যা একবার চলে শেষ হয়ে যায়, যেমন রাতের ব্যাকআপ স্ক্রিপ্ট — এর সঙ্গে `RemainAfterExit=yes` দিলে systemd তাকে সক্রিয় হিসেবে মনে রাখে। Node.js অ্যাপের জন্য `Type=simple`-ই সঠিক, কারণ Node সামনে থেকে চলে।

একটি গুরুত্বপূর্ণ অভ্যাস হলো override ব্যবহার করা। প্যাকেজের সঙ্গে আসা unit ফাইল (যেমন `/lib/systemd/system/nginx.service`) সরাসরি সম্পাদনা করবেন না — প্যাকেজ আপডেট হলে আপনার পরিবর্তন হারিয়ে যাবে। তার বদলে `sudo systemctl edit app` চালান; এটি `/etc/systemd/system/app.service.d/override.conf` নামে একটি ছোট ফাইল তৈরি করে, যেখানে শুধু বদলানো নির্দেশগুলো লেখা থাকে। যেমন Restart-এর ব্যবধান বাড়াতে চাইলে:

```
[Service]
RestartSec=10
MemoryMax=512M
```

এই দুটি লাইনই যথেষ্ট — বাকি সবকিছু মূল unit ফাইল থেকে আসে। পরিবর্তনের পর `sudo systemctl daemon-reload` দিয়ে আবার পড়ান এবং `systemctl cat app` দিয়ে দেখুন মূল ফাইল ও override একসঙ্গে কেমন দেখাচ্ছে।

Unit ধরন	কাজ	উদাহরণ
service	দীর্ঘ সময় চলা প্রক্রিয়া	app.service, nginx.service
timer	নির্ধারিত সময়ে service চালানো	প্রতি রাতে ব্যাকআপ চালানো
socket	অনুরোধ এলে service চালু করা	কম ব্যবহৃত API সার্ভিস
target	সার্ভিসের গুচ্ছের নাম	multi-user.target
path	ফাইল বদলালে service চালানো	কনফিগ বদলালে পুনরায় লোড

৭.৭ গুরুত্বপূর্ণ কমান্ড ও বিশ্লেষণ

```
systemctl status nginx          # সার্ভিসের অবস্থা, সাম্প্রতিক লগ, PID দেখায়
systemctl start app             # এখনই চালু করে (বুটে চালু হবে না)
systemctl enable app            # বুটের সময় চালু হওয়ার ব্যবস্থা করে
systemctl disable app           # বুটে চালু হওয়া বন্ধ করে
systemctl restart app           # বন্ধ করে আবার চালু করে
systemctl reload app            # প্রক্রিয়া না মেরে কনফিগ পুনরায় পড়ায় (যদি সমর্থন করে)
systemctl daemon-reload         # unit ফাইল বদলালে systemd-কে আবার পড়তে বলে — ভুললে পুরনো কনফিগই চলবে
journalctl -u app -f            # লাইভ লগ দেখে (-f মানে follow)
journalctl -u app --since "1 hour ago"   # গত এক ঘণ্টার লগ
systemctl list-units --failed   # কোন সার্ভিসগুলো ব্যর্থ হয়েছে
systemctl cat app               # unit ফাইলের বিষয়বস্তু দেখায়
ps -ef | grep node              # চলমান Node প্রক্রিয়া যাচাই (নমুনা)
```

`start` বনাম `enable`-এর পার্থক্য মনে রাখুন: start শুধু এখনকার জন্য, enable ভবিষ্যৎ বুটের জন্য। দুটোই দরকার হলে `systemctl enable --now app` ব্যবহার করুন।

নমুনা আউটপুট — চিত্রণমূলক (আসল সার্ভারে মান ভিন্ন হবে):

```
● app.service - Technova API Server (Node.js)
   Loaded: loaded (/etc/systemd/system/app.service; enabled)
   Active: active (running) since Sun 2026-11-08 10:00:00 +06
 Main PID: 1234 (node)
```

```
systemctl is-active app         # সার্ভিস এখন চলছে কি না — active / inactive / failed
systemctl is-enabled app        # বুটে চালু হবে কি না — enabled / disabled
systemctl is-failed app         # ব্যর্থ হয়েছে কি না যাচাই
systemd-analyze blame           # বুটে কোন সার্ভিস কত সময় নিয়েছে (ধীর বুটের কারণ খুঁজতে)
systemctl edit app              # override ফাইল খোলে — মূল unit না বদলে সেটিং বদলান
systemctl show app -p Restart -p MemoryMax    # নির্দিষ্ট সেটিংয়ের বর্তমান মান দেখা
```

`is-active` ও `is-enabled` স্ক্রিপ্টে ব্যবহারের জন্য খুব উপযোগী — এরা শুধু একটি শব্দ উত্তর দেয়, তাই ডেপ্লয় স্ক্রিপ্টে শর্ত হিসেবে লেখা যায়।

৭.৮ কনফিগারেশন উদাহরণ

ধরুন আপনার অ্যাপ `/opt/technova/app`-এ আছে, ব্যবহারকারী `technova`, পোর্ট ৩০০০-এ চলে। Unit ফাইল `/etc/systemd/system/app.service`:

```
[Unit]
Description=Technova API Server (Node.js)
After=network.target postgresql.service
Wants=postgresql.service

[Service]
Type=simple
User=technova
Group=technova
WorkingDirectory=/opt/technova/app
ExecStart=/usr/bin/node /opt/technova/app/server.js
Restart=on-failure
RestartSec=5
Environment=NODE_ENV=production
Environment=PORT=3000
# নিরাপত্তা কঠোরকরণ (sandboxing)
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=full

[Install]
WantedBy=multi-user.target
```

এরপর: `sudo systemctl daemon-reload && sudo systemctl enable --now app` এবং `systemctl status app` দিয়ে যাচাই করুন। `Type=simple` মানে ExecStart-এর প্রক্রিয়াটিই মূল প্রক্রিয়া; Node অ্যাপের জন্য এটিই সাধারণত সঠিক।

৭.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১: সার্ভার রিবুটের পর অ্যাপ নিজে চালু হয়নি। কারণ সাধারণত `enable` করা হয়নি, অথবা unit ফাইলে `WantedBy` ভুল। পরিস্থিতি ২: অ্যাপ চালু হয় কিন্তু ৩০ সেকেন্ড পর মরে যায় — সম্ভবত পরিবেশ চলক (যেমন DATABASE_URL) সেট করা হয়নি, অথবা পোর্ট ইতিমধ্যে ব্যবহৃত হচ্ছে। পরিস্থিতি ৩: ডেপ্লয়ের পর নতুন কোড চলছে না — সম্ভবত `daemon-reload` না করে শুধু restart দেওয়া হয়েছে, অথবা পুরনো প্রক্রিয়া আটকে আছে। প্রতিটি ক্ষেত্রে প্রথম কাজ `journalctl -u app -n 100` দিয়ে লগ পড়া, অনুমান নয়।

> **REAL WORLD — TechNova:** TechNova-র API (app.technova.example, 203.0.113.10) একবার রিবুটের পর চালু হয়ে সঙ্গে সঙ্গে মারা যাচ্ছিল, কারণ ডেটাবেস তখনো প্রস্তুত হয়নি এবং অ্যাপ প্রথম সংযোগ ব্যর্থ হলেই বন্ধ হয়ে যাচ্ছিল। দলটি দুটি কাজ করল: unit ফাইলে `After=postgresql.service`-এর পাশাপাশি `Restart=on-failure` ও `RestartSec=10` যোগ করল এবং override ফাইলে সেটি রাখল, যাতে মূল ফাইল অপরিবর্তিত থাকে। এরপর অ্যাপ প্রথম চেষ্টায় ব্যর্থ হলেও দশ সেকেন্ড পর আবার চেষ্টা করে, ততক্ষণে ডেটাবেস প্রস্তুত — সকালের প্রথম ব্যবহারকারীর আগেই সাইট ফিরে আসে। শিক্ষা: নির্ভরতা ক্রম ঠিক করে, restart নীতি সময়ের ব্যবধান ঠিক করে — দুটো একসঙ্গে দরকার।

৭.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

> **LAB:** এই অনুশীলন শুধু disposable VM বা ল্যাবে করুন — প্রোডাকশন সার্ভারে পরীক্ষা করবেন না।

১. একটি সাধারণ অ্যাপ তৈরি করুন: `mkdir -p /tmp/lab-app && echo "console.log('hello'); setInterval(()=>{}, 1000);" > /tmp/lab-app/server.js`। ২. unit ফাইল লিখুন (উপরের উদাহরণ অনুসরণ করে, পথ বদলে)। ৩. `sudo systemctl daemon-reload`। ৪. `sudo systemctl enable --now lab-app`। ৫. `systemctl status lab-app` ও `journalctl -u lab-app -f` দিয়ে যাচাই। ৬. ইচ্ছাকৃতভাবে প্রক্রিয়াটি মেরে দেখুন (`sudo pkill -f lab-app`) — Restart নীতির কারণে এটি ফিরে আসা উচিত। ৭. শেষে `sudo systemctl disable --now lab-app` দিয়ে পরিষ্কার করুন।

৭.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল ১: `systemctl start` করলেই বুটে চালু হবে ভাবা — enable ছাড়া হয় না। ভুল ২: unit ফাইল বদলে `daemon-reload` না করা। ভুল ৩: root ব্যবহারকারী হিসেবে অ্যাপ চালানো — নিরাপত্তা ঝুঁকি; সবসময় আলাদা ব্যবহারকারী ব্যবহার করুন। ভুল ৪: `After`-কে নির্ভরতা ভাবা — শুধু ক্রম নির্ধারণ করে। ভুল ৫: `Restart=always` দিয়ে কনফিগ ভুল ঢেকে রাখা — অ্যাপ বারবার ক্র্যাশ করলে কারণ খুঁজুন, পুনরায় চালুকে সমাধান ভাববেন না। ভুল ৬: লগ ফাইলে `console.log` ভরিয়ে journal পূর্ণ করা।

৭.১২ নিরাপত্তা বিবেচনা

সার্ভিস কখনো root হিসেবে চালাবেন না, যদি না একান্ত প্রয়োজন হয়। `User` / `Group` নির্দিষ্ট করুন, `NoNewPrivileges=true` ও `PrivateTmp=true` ব্যবহার করুন। Unit ফাইলে পাসওয়ার্ড বা গোপন কী (secret) লিখবেন না — পরিবেশ চলকের জন্য `EnvironmentFile=/etc/technova/app.env` ব্যবহার করুন এবং সেই ফাইলের অনুমতি `600` ও মালিকানা সার্ভিস ব্যবহারকারীর রাখুন। `/etc/systemd/system/`-এ লেখার অধিকার শুধু root-এর থাকা উচিত।

৭.১৩ সমস্যা নির্ণয় (Troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
সার্ভিস failed অবস্থায়	ExecStart পথ ভুল / অনুমতি নেই	`journalctl -u app -n 50`; পথ ও অনুমতি যাচাই
বুটে চালু হয় না	enable করা হয়নি	`systemctl is-enabled app`; `enable` করুন
বারবার restart হচ্ছে	অ্যাপ ক্র্যাশ করছে / পোর্ট ব্যবহৃত	লগ পড়ুন; `ss -tlnp` দিয়ে পোর্ট দেখুন
পরিবর্তন কাজ করছে না	daemon-reload হয়নি	`daemon-reload` তারপর restart
লগে কিছু নেই	অ্যাপ stderr-এ লিখছে না	অ্যাপের লগিং যাচাই; `StandardOutput=journal` নিশ্চিত করুন
exit code 203 / EXEC	ExecStart-এর প্রোগ্রাম পাওয়া যায়নি বা চালানো যায়নি	`systemctl cat app` দিয়ে পথ দেখুন; `ls -l /usr/bin/node` দিয়ে ফাইল আছে কি না যাচাই করুন
timer চলছে না	timer enable করা হয়নি, অথবা service-এর নাম ভুল	`systemctl list-timers`; timer ও service — দুটোই enable করুন
সার্ভিস active, কিন্তু পোর্ট শুনছে না	অ্যাপ ভুল পোর্টে বা ভুল ঠিকানায় শুনছে	`ss -tlnp | grep 3000`; অ্যাপের PORT ও listen ঠিকানা যাচাই করুন
journal ডিস্ক পূর্ণ করছে	অতিরিক্ত লগ বা SystemMaxUse সেট নেই	`journalctl --disk-usage`; লগ স্তর কমান ও rotation সেট করুন

৭.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: প্রতিটি সার্ভিসের জন্য আলাদা ব্যবহারকারী; নির্ভরতা স্পষ্ট করে লেখা; `Restart=on-failure` + `RestartSec`; পরিবর্তনের আগে unit ফাইলের ব্যাকআপ (`sudo cp app.service app.service.bak`); `nginx -t`-এর মতো যাচাই ধাপ। অ্যান্টি-প্যাটার্ন: root হিসেবে সব চালানো; `Restart=always` দিয়ে ভাঙা অ্যাপ ঢাকা; unit ফাইলে গোপন তথ্য রাখা; লগ না দেখে বারবার restart দেওয়া।

৭.১৫ অধ্যায় সারাংশ

systemd হলো Linux-এর PID 1 — সিস্টেমের প্রথম ও প্রধান ব্যবস্থাপক। Unit ফাইল দিয়ে আপনি ঘোষণা করেন কী চালাতে হবে, কীভাবে চালাতে হবে, কিসের পরে চালাতে হবে এবং ব্যর্থ হলে কী করতে হবে। `systemctl` দিয়ে সার্ভিস নিয়ন্ত্রণ করা হয়, `journalctl` দিয়ে লগ দেখা হয়। সঠিক নির্ভরতা, সীমিত ব্যবহারকারী ও restart নীতি — এই তিনটি ঠিক থাকলে সার্ভার রিবুট বা ক্র্যাশের পরও আপনার অ্যাপ নিজে থেকে ফিরে আসবে।

৭.১৬ মূল প্রযুক্তিগত পরিভাষা

শব্দ	অর্থ
systemd	Linux-এর init ব্যবস্থা ও সার্ভিস ব্যবস্থাপক, PID 1 হিসেবে চলে।

PID 1	কার্নেল চালু করা প্রথম প্রক্রিয়া; সব প্রক্রিয়ার অভিভাবক।
Unit	systemd-এর ব্যবস্থাপনার একক — service, target, timer ইত্যাদি।
Target	সার্ভিসের গুচ্ছের নাম; যেমন multi-user.target।
journald / journalctl	systemd-এর লগ সংগ্রহ ব্যবস্থা ও তা পড়ার সরঞ্জাম।
daemon-reload	unit ফাইল বদলালে systemd-কে নতুন কনফিগ পড়তে বলা।

৭.১৭ জ্ঞান যাচাই প্রশ্ন

> **CHECK:** নিচের প্রশ্নগুলোর উত্তর না দেখে দেওয়ার চেষ্টা করুন।

1. PID 1 কী এবং systemd কেন PID 1 হিসেবে চলে?
2. `systemctl start` ও `systemctl enable`-এর পার্থক্য কী?
3. `After` ও `Requires`-এর মধ্যে পার্থক্য কী? একটি উদাহরণ দিন।
4. unit ফাইল বদলানোর পর কোন কমান্ডটি চালাতেই হবে এবং কেন?
5. `Restart=on-failure` ও `Restart=always`-এর মধ্যে কোনটি কখন ব্যবহার করবেন?
6. journalctl দিয়ে একটি নির্দিষ্ট সার্ভিসের গত এক ঘণ্টার লগ কীভাবে দেখবেন?
7. systemd timer ও cron-এর মধ্যে পার্থক্য কী? কোন ক্ষেত্রে timer বেছে নেবেন?
8. প্যাকেজের unit ফাইল সরাসরি না বদলে drop-in override ব্যবহার করা কেন ভালো? একটি উদাহরণ দিন।
9. অ্যাপ root হিসেবে চালানো কেন নিরাপত্তা ঝুঁকি?

৭.১৮ ব্যবহারিক অনুশীলন

1. ল্যাব VM-এ একটি সাধারণ Node.js অ্যাপের জন্য systemd unit ফাইল লিখুন, enable করুন, রিবুট দিয়ে যাচাই করুন যে বুটে চালু হয়।
2. ইচ্ছাকৃতভাবে অ্যাপের কোডে ভুল করে ক্র্যাশ করান; journalctl দিয়ে কারণ নির্ণয় করুন এবং StartLimit আচরণ পর্যবেক্ষণ করুন।
3. একটি timer unit তৈরি করুন যা প্রতি ঘণ্টায় একটি স্ক্রিপ্ট চালায় (cron-এর বিকল্প হিসেবে systemd timer বুঝতে)।
4. ল্যাবে একটি timer unit তৈরি করুন যা প্রতিদিন একটি লগ লাইন লেখে এমন oneshot service চালায়; `systemctl list-timers` দিয়ে যাচাই করুন এবং একটি override দিয়ে RestartSec বদলে দেখুন।
5. আপনার unit ফাইলে `ProtectSystem` ও `PrivateTmp` যোগ করে অ্যাপটি এখনো ঠিকমতো চলে কি না যাচাই করুন।

অধ্যায় ৮

Nginx Architecture and Web Server Configuration

৮.১ শেখার উদ্দেশ্য (Learning Objectives)

এই অধ্যায় শেষে আপনি পারবেন: Nginx কী এবং কেন এটি প্রোডাকশনে এত জনপ্রিয় তা ব্যাখ্যা করতে; event-driven আর্কিটেকচার কীভাবে হাজার হাজার একযোগ সংযোগ (concurrent connection) সামলায় তা বুঝতে; একটি static সাইট ও একটি Node.js অ্যাপের সামনে Nginx বসাতে; `nginx.conf` ও server block-এর গঠন পড়তে ও লিখতে; এবং কনফিগ পরীক্ষা করে নিরাপদে পুনরায় লোড করতে।

৮.২ প্রোডাকশনে কেন এটি গুরুত্বপূর্ণ

Node.js অ্যাপ সরাসরি ইন্টারনেটে উন্মুক্ত করা যায়, কিন্তু সেটি ভালো অভ্যাস নয়। Node একা static ফাইল পরিবেশন, TLS সমাপ্তি (TLS termination), হার সীমাবদ্ধতা (rate limiting) ও একাধিক অ্যাপের মধ্যে রাউটিং — সবকিছু দক্ষভাবে করতে পারে না। Nginx হলো সেই দরজার প্রহরী: সে বাইরের সব অনুরোধ গ্রহণ করে, static ফাইল নিজে দিয়ে দেয়, বাকিগুলো Node অ্যাপে পাঠায়, TLS পরিচালনা করে এবং আক্রমণাত্মক ট্র্যাফিক থামায়। প্রায় প্রতিটি প্রোডাকশন Node ডেপ্লয়মেন্টের সামনে Nginx বা তার মতো একটি reverse proxy থাকে।

৮.৩ পূর্বশর্ত জ্ঞান (Prerequisite Knowledge)

HTTP অনুরোধ-উত্তর (request-response), পোর্টের ধারণা (80 = HTTP, 443 = HTTPS), DNS-এর মৌলিক ধারণা এবং অধ্যায় ৭-এর systemd জ্ঞান প্রয়োজন। `curl` দিয়ে পরীক্ষা করার অভ্যাস থাকলে ভালো হয়। Nginx ইনস্টল করা থাকে `sudo apt install nginx` দিয়ে — Ubuntu 24.04-এ এটি systemd সার্ভিস হিসেবে চলে।

৮.৪ প্রথম নীতি থেকে ব্যাখ্যা (First Principles)

একটি ওয়েব সার্ভারের কাজ সহজ: নেটওয়ার্ক থেকে অনুরোধ নেওয়া, সঠিক উত্তর খুঁজে পাঠানো। কঠিন অংশ হলো একসঙ্গে হাজার হাজার অনুরোধ সামলানো। পুরনো Apache-এর প্রক্রিয়া-প্রতি-সংযোগ (process-per-connection) মডেলে প্রতিটি সংযোগের জন্য একটি প্রক্রিয়া বা থ্রেড লাগত — মেমোরি দ্রুত শেষ হতো। Nginx ব্যবহার করে event-driven, non-blocking মডেল: অল্প কয়েকটি worker প্রক্রিয়া একটি event loop-এ হাজার হাজার সংযোগের ঘটনা (event) পর্যবেক্ষণ করে, কেউ ডেটা পাঠালেই শুধু তখন কাজ করে। এটি Node.js-এর event loop ধারণার সঙ্গে মিলে যায় — তাই Node ডেভেলপারদের কাছে Nginx স্বাভাবিক মনে হয়। Master প্রক্রিয়া কনফিগ পড়ে ও worker-দের পরিচালনা করে; worker-রাই আসল অনুরোধ সামলায়। একটি worker মারা গেলেও master আরেকটি চালু করে।

৮.৫ আর্কিটেকচার / ধারণাগত চিত্র

Nginx আর্কিটেকচার:

Client (ব্রাউজার) → পোর্ট 80 / 443 → **Nginx Master Process** → Worker Process × N

Worker ──→ Static ফাইল (/var/www/html) — সরাসরি উত্তর

Worker ──→ Reverse Proxy → Node.js অ্যাপ (127.0.0.1:3000)

Worker ──→ Cache / Rate Limit / TLS সমাপ্তি

একটি worker একসঙ্গে হাজারো সংযোগ ধরে রাখতে পারে, কারণ সে অপেক্ষমাণ সংযোগে বসে থাকে না — event এলেই জেগে ওঠে।

৮.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

কনফিগ গঠন: context ও directive

Nginx কনফিগ গঠিত হয় context (প্রসঙ্গ) ও directive (নির্দেশ) দিয়ে। `events { }` সংযোগ পরিচালনা, `http { }` সব HTTP সেটিং, তার ভেতরে `server { }` একটি ভার্চুয়াল সাইট, তার ভেতরে `location { }` একটি URL পথের নিয়ম। `listen 80;` মানে ৮০ পোর্টে শোনা; `server_name app.technova.example;` মানে এই নামের অনুরোধ এখানে আসবে; `root /var/www/html;` মানে static ফাইল কোথায়।

location মিল (matching)

Nginx location বেছে নেয় নির্দিষ্ট নিয়মে: `= /exact` (নির্ভুল মিল), `^~ /static/` (উপসর্গ মিল, regex এড়িয়ে), সাধারণ উপসর্গ, তারপর regex। ভুল ক্রমে location লিখলে ভুল নিয়ম কার্যকর হতে পারে — তাই static পথের জন্য `^~` ব্যবহার করা নিরাপদ।

Worker ও সংযোগ

`worker_processes auto;` মানে CPU কোর অনুযায়ী worker তৈরি হবে। `worker_connections 1024;` মানে প্রতি worker কত সংযোগ ধরতে পারবে। ছোট সার্ভারে এগুলো না বদলালেও চলে, কিন্তু বড় ট্র্যাফিকে এগুলোই সীমা নির্ধারণ করে।

Static বনাম Dynamic

ছবি, CSS, JavaScript — এগুলো ডিস্ক থেকে সরাসরি দেওয়া Nginx-এর সবচেয়ে দ্রুত কাজ। Dynamic অনুরোধ (API) Nginx নিজে তৈরি করতে পারে না; সেগুলো `proxy_pass` দিয়ে Node অ্যাপে যায় — সেটি অধ্যায় ৯-এর বিষয়। এখানে মূল নীতি: যা Nginx পারে, তা অ্যাপকে দিয়ে করাবেন না।

আরও location মিল ও নির্দেশনা

location বাছাইয়ের ক্রমটি হাতে-কলমে বোঝা জরুরি, কারণ ভুল ক্রমে লেখা নিয়ম ভুল ফাইল পরিবেশন করতে পারে। প্রথমে নির্ভুল মিল (`= /health`) দেখা হয় — এটি শুধু একটি নির্দিষ্ট পথের জন্য, যেমন health check। তারপর উপসর্গ মিল যার আগে `^~` আছে (`^~ /static/`) — এটি পেলে আর regex দেখা হয় না, তাই static পথের জন্য এটি সবচেয়ে

নিরাপদ। তারপর সাধারণ উপসর্গ (`/images/`), সবশেষে regex location (`~ \.php$`) — regex-এর মধ্যে প্রথম মিলটিই জেতে। এই ক্রম মনে রাখলে বোঝা যায় কেন `/static/` নিয়মটি regex-এর আগে লেখা উচিত।

আরও কয়েকটি দরকারি নির্দেশনা জানা উচিত। `gzip on;` দিলে Nginx টেক্সট উত্তর (HTML, CSS, JavaScript) সংকুচিত করে পাঠায়, ফলে পাতা দ্রুত লোড হয় — তবে ছবির মতো আগে থেকে সংকুচিত ফাইলে gzip লাভ নেই। `client_max_body_size 10m;` সীমা নির্ধারণ করে কত বড় ফাইল আপলোড করা যাবে; সীমা না বাড়ালে বড় ছবি আপলোডে `413 Request Entity Too Large` আসে — আপলোড ফর্মের সমস্যা মানেই এই নির্দেশনা দেখুন। `error_page 404 /404.html;` দিয়ে নিজের তৈরি ভুল-পাতা দেখানো যায়, আর `expires` ও `Cache-Control` একসঙ্গে ব্যবহার করলে ব্রাউজার ক্যাশের আচরণ স্পষ্ট থাকে।

location ধরন	মিলের নিয়ম	কখন ব্যবহার করবেন
`= /health`	শুধু নির্ভুল পথ	health check, একক পাতা
`^~ /static/`	উপসর্গ, regex এড়িয়ে যায়	static ফাইল — সবচেয়ে নিরাপদ
`/images/`	সাধারণ উপসর্গ	সাধারণ ডিরেক্টরি নিয়ম
`~ \.(jpg|png)$`	regex মিল	ফাইলের ধরন অনুযায়ী নিয়ম

৮.৭ গুরুত্বপূর্ণ কমান্ড ও বিশ্লেষণ

```
sudo nginx -t                # কনফিগ সিনট্যাক্স পরীক্ষা — reload-এর আগে অবশ্যই
sudo systemctl reload nginx  # প্রক্রিয়া না মেরে নতুন কনফিগ কার্যকর (graceful)
sudo systemctl restart nginx # সম্পূর্ণ পুনরায় চালু (reload ব্যর্থ হলে)
sudo systemctl status nginx  # অবস্থা ও লগ দেখা
sudo nginx -T                # কার্যকর সম্পূর্ণ কনফিগ দেখায় (ডিবাগে অমূল্য)
curl -I http://203.0.113.10/ # হেডারসহ উত্তর যাচাই (নমুনা IP)
tail -f /var/log/nginx/error.log   # ভুল লগ লাইভ দেখা
```

সোনালি নিয়ম: `nginx -t` ছাড়া কখনো reload নয়। `-t` ব্যর্থ হলে পুরনো কনফিগই চলতে থাকে — সাইট বাঁচে।

```
curl -I http://203.0.113.10/static/main.css   # Cache-Control ও Content-Type হেডার দেখা
curl -H "Host: app.technova.example" http://127.0.0.1/health   # Host হেডার দিয়ে virtual host যাচাই
sudo nginx -s reload            # systemctl ছাড়া সরাসরি reload সংকেত (ল্যাবে জানা ভালো)
grep -c " 404 " /var/log/nginx/technova.access.log   # আজ কতগুলো 404 হয়েছে গোনা (নমুনা)
```

`curl -I` শুধু হেডার আনে — ক্যাশ, Content-Type বা রিডাইরেক্ট যাচাইয়ের দ্রুততম উপায়। `-H "Host: ..."` দিয়ে আসল DNS ছাড়াই ল্যাবে virtual host পরীক্ষা করা যায়।

৮.৮ কনফিগারেশন উদাহরণ

```
# /etc/nginx/sites-available/technova
server {
    listen 80;
    server_name app.technova.example;
    root /var/www/technova;
    index index.html;
```

```
    location / {
        try_files $uri $uri/ =404;
    }

    location ^~ /static/ {
        alias /var/www/technova/static/;
        expires 30d;
        add_header Cache-Control "public, max-age=2592000";
    }

    access_log /var/log/nginx/technova.access.log;
    error_log /var/log/nginx/technova.error.log;
}
```

সক্রিয় করতে: `sudo ln -s /etc/nginx/sites-available/technova /etc/nginx/sites-enabled/` তারপর `sudo nginx -t && sudo systemctl reload nginx`। `try_files` মানে: ফাইল থাকলে দাও, না থাকলে 404 — Single Page Application-এর ক্ষেত্রে এটি `try_files $uri /index.html;` হয়।

৮.৯ বাস্তব প্রোডাকশন পরিস্থিতি

একটি React অ্যাপের build ফাইল Nginx দিয়ে পরিবেশন করা হচ্ছে। ডেপ্লয়ের সময় নতুন build এলে পুরনো CSS ব্রাউজার ক্যাশে আটকে থাকে — তাই ফাইলের নামে hash (যেমন `main.a1b2c3.js`) ব্যবহার করা হয়, আর `index.html`-এ ক্যাশ বন্ধ রাখা হয়। আরেকটি পরিস্থিতি: ট্র্যাফিক হঠাৎ বাড়লে Nginx-এর `worker_connections` শেষ হয়ে ৫০২/৫০৩ আসে — তখন worker সংখ্যা ও ফাইল descriptor সীমা (`worker_rlimit_nofile`) বাড়াতে হয়, শুধু সার্ভার বড় করলেই হয় না।

> **REAL WORLD — TechNova:** TechNova-র মার্কেটিং সাইটে (app.technova.example, 203.0.113.10) একবার নতুন ছবি আপলোড করার পরও ব্যবহারকারীরা পুরনো ছবি দেখছিল। কারণ ছিল `location /images/`-এ এক বছরের `expires` দেওয়া, কিন্তু ছবির ফাইলের নাম বদলানো হয়নি — ব্রাউজার পুরনো ফাইলটিই ক্যাশ থেকে দেখাচ্ছিল। সমাধান ছিল দুটি: ছবির নামে hash যোগ করা (যেমন `hero.a1b2c3.jpg`) এবং `index.html`-এ ক্যাশ বন্ধ রাখা। এরপর থেকে ডেপ্লয়ে ফাইলের নাম বদলালেই ব্রাউজার নতুন ফাইল আনে। শিক্ষা: দীর্ঘ ক্যাশ তখনই নিরাপদ, যখন ফাইলের নাম বদলে সংস্করণ চেনা যায় — নাম এক রেখে বিষয়বস্তু বদলালে ক্যাশই শত্রু হয়।

৮.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

১. `sudo apt update && sudo apt install -y nginx`। ২. `curl http://203.0.113.10/` দিয়ে স্বাগত পাতা যাচাই (নমুনা IP — আপনার সার্ভারের IP ব্যবহার করুন)। ৩. নিজের একটি `index.html` তৈরি করে `/var/www/technova/`-এ রাখুন। ৪. উপরের server block লিখে sites-enabled-এ লিংক করুন। ৫. `sudo nginx -t` তারপর reload। ৬. ব্রাউজার বা curl দিয়ে যাচাই করুন। ৭. ইচ্ছাকৃতভাবে কনফিগে ভুল করে `nginx -t` কী বলে দেখুন — তারপর ভুল ঠিক না করে reload দিলে কী হতো ভাবুন।

৮.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল ১: `root` ও `alias` গুলিয়ে ফেলা — `alias`-এ পথ শেষে `/` থাকা জরুরি। ভুল ২: `nginx -t` না করে restart দেওয়া — ভুল কনফিগে সার্ভার বন্ধ হয়ে যায়। ভুল ৩: sites-available-এ লিখে sites-enabled-এ লিংক না করা। ভুল ৪:

ভাবা যে Nginx নিজে Node অ্যাপ চালায় — Nginx শুধু অনুরোধ পাঠায়, অ্যাপ চালায় systemd/PM2। ভুল ৫: সব ফাইলে দীর্ঘ ক্যাশ দেওয়া — `index.html`-এ ক্যাশ দিলে নতুন ডেপ্লয় ব্যবহারকারী পায় না।

৮.১২ নিরাপত্তা বিবেচনা

সার্ভার সংস্করণ লুকান (`server_tokens off;`), অপ্রয়োজনীয় হেডার ফাঁস করবেন না। ডিরেক্টরি তালিকা (autoindex) বন্ধ রাখুন। `/var/www`-এর ফাইল অনুমতি সঠিক রাখুন — Nginx ব্যবহারকারী (www-data) শুধু পড়তে পারলেই যথেষ্ট, লিখতে পারা উচিত নয়। গোপন ফাইল (`.env`, `.git`) যেন ওয়েবে উন্মুক্ত না হয় — location দিয়ে তা আটকান।

```
location ~ /\. { deny all; }   # ডট-ফাইল (.env, .git) উন্মুক্ত হওয়া আটকায়
```

৮.১৩ সমস্যা নির্ণয় (Troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
404 সব পাতায়	root পথ ভুল / ফাইল নেই	`nginx -T` দিয়ে root যাচাই; ফাইল আছে কি না দেখুন
403 Forbidden	অনুমতি নেই / index ফাইল নেই	`ls -l` ও ফাইল মালিকানা যাচাই
502 Bad Gateway	পেছনের অ্যাপ চলছে না	অধ্যায় ৯ দেখুন; অ্যাপের পোর্ট যাচাই করুন
reload-এ ব্যর্থ	সিনট্যাক্স ভুল / পোর্ট ব্যবহৃত	`nginx -t` আউটপুট পড়ুন; `ss -tlnp`
পুরনো ফাইল দেখাচ্ছে	ব্রাউজার / Nginx ক্যাশ	হার্ড রিফ্রেশ; Cache-Control যাচাই
413 Request Entity Too Large	client_max_body_size সীমা ছোট	সীমা বাড়ান; আপলোডের আকার যাচাই করুন
ভুল location-এর নিয়ম কার্যকর হচ্ছে	location ক্রম বা regex অগ্রাধিকার ভুল	`nginx -T` দিয়ে ক্রম পড়ুন; static-এ `^~` ব্যবহার করুন
gzip কাজ করছে না	gzip বন্ধ বা ফাইল আগে থেকে সংকুচিত	gzip on যাচাই; `curl -H "Accept-Encoding: gzip" -I` দিয়ে হেডার দেখুন
নতুন ডেপ্লয়ের পরও পুরনো CSS	ফাইলের নাম একই, ক্যাশ দীর্ঘ	hash-সহ ফাইলের নাম ব্যবহার করুন; index.html-এ ক্যাশ বন্ধ রাখুন

৮.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: প্রতি সাইটে আলাদা server block ফাইল; `nginx -t` তারপর reload; কনফিগ পরিবর্তনের আগে ব্যাকআপ; access ও error লগ আলাদা রাখা; static-এ দীর্ঘ ক্যাশ, HTML-এ ক্যাশ নয়। অ্যান্টি-প্যাটার্ন: সবকিছু `nginx.conf`-এ গাদাগাদি করা; `if` দিয়ে জটিল রাউটিং (Nginx-এ `if` বিপজ্জনক বলে পরিচিত); root হিসেবে ফাইল পরিবেশন করা।

৮.১৫ অধ্যায় সারাংশ

Nginx হলো event-driven, হালকা ও দ্রুত ওয়েব সার্ভার — master/worker মডেলে হাজার হাজার সংযোগ সামলায়। Static ফাইল সে নিজে দেয়, dynamic অনুরোধ পেছনের অ্যাপে পাঠায়। server ও location block দিয়ে আপনি ঘোষণা করেন কোন নামের কোন পথে কী হবে। আর সব পরিবর্তনের আগে `nginx -t` — এই অভ্যাসটি প্রোডাকশন বাঁচায়।

৮.১৬ মূল প্রযুক্তিগত পরিভাষা

শব্দ	অর্থ
Nginx	Event-driven ওয়েব সার্ভার ও reverse proxy।
Worker Process	অনুরোধ সামলানো Nginx প্রক্রিয়া; master তাকে পরিচালনা করে।
Server Block	একটি সাইটের কনফিগ (server { }) — Apache-এর VirtualHost-এর সমতুল্য।
Location Block	একটি URL পথের নিয়ম (location { })।
try_files	ফাইল খোঁজার ক্রম নির্ধারণ করা directive।
Reverse Proxy	ক্লায়েন্টের অনুরোধ পেছনের সার্ভারে পাঠিয়ে উত্তর ফেরত দেওয়া ব্যবস্থা।

৮.১৭ জ্ঞান যাচাই প্রশ্ন

1. Nginx-এর event-driven মডেল process-per-connection মডেলের চেয়ে কেন বেশি সংযোগ সামলাতে পারে?
2. Master ও worker প্রক্রিয়ার ভূমিকা কী কী?
3. `server_name` কী কাজে লাগে?
4. `root` ও `alias`-এর পার্থক্য কী?
5. `nginx -t` ছাড়া reload করা কেন বিপজ্জনক?
6. SPA-এর জন্য `try_files $uri /index.html;` কেন দরকার হয়?
7. `^~ /static/` ও সাধারণ `/static/` location-এর মধ্যে মিলের ক্রমে কী পার্থক্য হয়?
8. বড় ফাইল আপলোডে 413 ভুল এলে কোন নির্দেশনা দেখবেন এবং কেন সীমা না বাড়িয়ে আগে আপলোডের আকার যাচাই করবেন?
9. Static ফাইলে দীর্ঘ ক্যাশ দিলে কী সমস্যা হতে পারে এবং সমাধান কী?

৮.১৮ ব্যবহারিক অনুশীলন

1. ল্যাব VM-এ Nginx ইনস্টল করে একটি static HTML সাইট পরিবেশন করুন; আলাদা server block ফাইল ব্যবহার করুন।
2. `/static/` পথে ৩০ দিনের ক্যাশ ও `index.html`-এ ক্যাশ-বন্ধ হেডার সেট করে curl দিয়ে হেডার যাচাই করুন।
3. ইচ্ছাকৃতভাবে কনফিগ ভুল করে `nginx -t`-এর বার্তা পড়ুন, তারপর ঠিক করুন — reload না করে।
4. ল্যাবে একটি `= /health` নির্ভুল location ও একটি `^~ /static/` location যোগ করুন; curl দিয়ে যাচাই করুন প্রতিটি পথে কোন নিয়ম কার্যকর হচ্ছে এবং gzip হেডার আসছে কি না।
5. `.env` ফাইল ওয়েবে উন্মুক্ত হয় কি না পরীক্ষা করুন এবং location নিয়ম দিয়ে আটকান।

অধ্যায় ৯

Virtual Hosts, Reverse Proxy and TLS Configuration

৯.১ শেখার উদ্দেশ্য (Learning Objectives)

এই অধ্যায় শেষে আপনি পারবেন: এক সার্ভারে একাধিক ডোমেইন (virtual host) চালাতে; Nginx-কে reverse proxy হিসেবে বসিয়ে Node অ্যাপে অনুরোধ পাঠাতে; TLS/HTTPS কী, কেন দরকার এবং কীভাবে কাজ করে তা ব্যাখ্যা করতে; Certbot ও Let's Encrypt দিয়ে বিনামূল্যে সার্টিফিকেট নিয়ে স্বয়ংক্রিয় নবায়নের ব্যবস্থা করতে; এবং HTTP থেকে HTTPS-এ নিরাপদ রিডাইরেক্ট সেট করতে।

৯.২ প্রোডাকশনে কেন এটি গুরুত্বপূর্ণ

একটি সার্ভারে শুধু একটি সাইট থাকে না — `app.technova.example`, `api.technova.example`, হয়তো একটি ডকুমেন্টেশন সাইট। Virtual host ছাড়া প্রতিটির জন্য আলাদা সার্ভার লাগত, যা অপচয়। আর TLS ছাড়া আপনার ব্যবহারকারীর পাসওয়ার্ড, টোকেন — সবকিছু পথে পড়া যায় (plain text-এ যায়)। ব্রাউজার এখন HTTP সাইটকে "Not Secure" দেখায়, আর অনেক API ও ব্রাউজার বৈশিষ্ট্য HTTPS ছাড়া চলেই না। তাই virtual host + reverse proxy + TLS — এই তিনটি একসঙ্গে প্রোডাকশন ডেপ্লয়মেন্টের মেরুদণ্ড।

৯.৩ পূর্বশর্ত জ্ঞান (Prerequisite Knowledge)

অধ্যায় ৮-এর Nginx server block, DNS-এ A রেকর্ডের ধারণা (ডোমেইন নাম → IP), HTTP হেডারের ধারণা এবং পোর্ট ৮০/৪৪৩-এর পার্থক্য জানা প্রয়োজন। একটি কাল্পনিক ডোমেইন দিয়ে অনুশীলন করবেন — আসল ডোমেইন কেনার আগে ল্যাবে `/etc/hosts`-এ নাম ম্যাপ করে পরীক্ষা করা যায়।

৯.৪ প্রথম নীতি থেকে ব্যাখ্যা (First Principles)

Virtual host-এর মূল প্রশ্ন: একটি IP ঠিকানায় অনেক সাইট থাকলে সার্ভার কীভাবে বোঝে কোন সাইট চাওয়া হয়েছে? উত্তর: HTTP অনুরোধের `Host` হেডারে ডোমেইন নাম থাকে। Nginx সেই নাম পড়ে সঠিক server block বেছে নেয় — এটিই name-based virtual hosting। Reverse proxy-এর মূল ধারণা: ক্লায়েন্ট সরাসরি অ্যাপের সঙ্গে কথা বলে না; মাঝখানে Nginx বসে অনুরোধ নেয়, পেছনের অ্যাপ (যেমন 127.0.0.1:3000-এ চলা Node) থেকে উত্তর নিয়ে ক্লায়েন্টকে দেয়। ক্লায়েন্ট জানেও না পেছনে কী আছে। TLS-এর মূল ধারণা: দুই পক্ষের মধ্যে গোপন কথা বলার জন্য প্রথমে একটি নিরাপদ চাবি বিনিময় (handshake) হয়, তারপর সব বার্তা এনক্রিপ্ট করা হয়। সার্টিফিকেট হলো পরিচয়পত্র — একটি বিশ্বস্ত কর্তৃপক্ষ (Certificate Authority, যেমন Let's Encrypt) নিশ্চিত করে যে এই ডোমেইনের মালিক আসলেই সে-ই।

৯.৫ আর্কিটেকচার / ধারণাগত চিত্র

এক সার্ভার, দুই সাইট, একটি TLS স্তর:

Client → HTTPS (443) → Nginx: Host হেডার পড়ে

├─ app.technova.example → proxy_pass → Node অ্যাপ :3000 (127.0.0.1)

└─ api.technova.example → proxy_pass → Node API :4000 (127.0.0.1)

HTTP (80) → 301 Redirect → HTTPS (443) — সবসময় এনক্রিপ্টেড পথে

TLS সমাপ্তি Nginx-এ হয়; Nginx থেকে অ্যাপ পর্যন্ত (localhost) ট্র্যাফিক অভ্যন্তরীণ থাকে।

৯.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Virtual host কীভাবে বাছাই হয়

অনুরোধ এলে Nginx `server_name`-এর সঙ্গে Host হেডার মেলায়। মিল না পেলে `default_server` হিসেবে চিহ্নিত block উত্তর দেয়; কোনোটিই default না হলে প্রথম block। তাই সবসময় একটি default block রাখা ভালো — নইলে অচেনা নামে সার্ভারে আসা অনুরোধ ভুল সাইট দেখাতে পারে।

Reverse proxy হেডার

Proxy করলে আসল ক্লায়েন্টের IP হারিয়ে যায় — অ্যাপ দেখে শুধু Nginx-এর IP (127.0.0.1)। তাই এই হেডারগুলো পাঠাতে হয়: `X-Real-IP` (আসল IP), `X-Forwarded-For` (IP-এর শৃঙ্খল), `X-Forwarded-Proto` (http না https)। Express-এ `app.set('trust proxy', 1)` না করলে অ্যাপ ভুল প্রোটোকল ধরে নেবে এবং secure cookie কাজ করবে না — এটি খুব সাধারণ ভুল। `proxy_set_header Host $host;` দিয়ে আসল Host নামও পাঠাতে হয়, নইলে অ্যাপ ভুল নাম দেখে।

TLS handshake — সহজ করে

ব্রাউজার সার্ভারের সার্টিফিকেট চায়, সার্টিফিকেটের স্বাক্ষর যাচাই করে (Let's Encrypt-এর মতো বিশ্বস্ত CA স্বাক্ষর করেছে কি না), তারপর একটি গোপন session key বিনিময় হয়। এরপর সব ডেটা সেই key দিয়ে এনক্রিপ্ট হয়। কেউ মাঝপথে শুনলেও শুধু এলোমেলো বাইট দেখবে। সার্টিফিকেটের মেয়াদ থাকে — Let's Encrypt দেয় ৯০ দিনের জন্য, তাই স্বয়ংক্রিয় নবায়ন জরুরি।

Certbot কী করে

Certbot একটি সরঞ্জাম যা Let's Encrypt-এর সঙ্গে কথা বলে প্রমাণ করে আপনি ডোমেইনের মালিক (সাধারণত HTTP-01 challenge: সার্ভারে একটি বিশেষ ফাইল রাখা হয়, Let's Encrypt তা পড়ে যাচাই করে), সার্টিফিকেট নেয়, Nginx কনফিগে বসায় এবং নবায়নের timer বসিয়ে দেয়। `--nginx` প্লাগইন কনফিগ নিজে সম্পাদনা করে — সুবিধাজনক, তবে কী বদলালো তা `nginx -T` দিয়ে দেখে নেওয়া উচিত।

Certbot নবায়ন — গভীরে

Let's Encrypt সার্টিফিকেটের মেয়াদ ৯০ দিন — এই ছোট মেয়াদ ইচ্ছাকৃত, যাতে সার্টিফিকেটের অপব্যবহারের সময়সীমা কম থাকে। তাই নবায়নকে স্বয়ংক্রিয় করতেই হয়, হাতে মনে রাখার ওপর ছেড়ে দেওয়া যায় না। Certbot ইনস্টলের সময়

`certbot.timer` নামে একটি systemd timer বসিয়ে দেয়, যা দিনে দুবার নবায়ন যাচাই করে এবং মেয়াদ শেষের কাছাকাছি (সাধারণত ৩০ দিনের মধ্যে) এলে নবায়ন করে। `sudo systemctl status certbot.timer` দিয়ে timer সক্রিয় আছে কি না দেখুন — timer বন্ধ থাকলে নবায়ন কখনোই হবে না, আর সাইট একদিন হঠাৎ ভাঙবে।

নবায়ন পরীক্ষার সঠিক উপায় `sudo certbot renew --dry-run` — এটি আসল সার্টিফিকেট বদলায় না, শুধু যাচাই প্রক্রিয়াটি পরীক্ষা করে। নবায়ন ব্যর্থ হওয়ার সবচেয়ে সাধারণ কারণগুলো: DNS-এ ডোমেইন আর সার্ভারের IP-তে নির্দেশিত নেই, পোর্ট ৮০ ফায়ারওয়ালে বন্ধ (HTTP-01 challenge-এর জন্য পোর্ট ৮০ খোলা থাকতে হয়), অথবা Nginx কনফিগে ভুলের কারণে challenge ফাইল পরিবেশন হচ্ছে না। নবায়নের সময় সার্ভিস পুনরায় লোড করা দরকার হলে Certbot-এর deploy-hook ব্যবহার করা যায় — যেমন নবায়নের পর `nginx -t && systemctl reload nginx` চালানো — তবে hook যোগ করার আগে ল্যাবে পরীক্ষা করুন। একাধিক ডোমেইনের সার্টিফিকেট একসঙ্গে নিতে চাইলে একটি Certbot কমান্ডে একাধিক `-d` ফ্ল্যাগ দিন; প্রতিটি ডোমেইনের DNS যাচাই আলাদাভাবে হয়, তাই একটি ডোমেইনের DNS ভুল থাকলে পুরো কমান্ড ব্যর্থ হতে পারে।

> **WARNING:** `certbot renew --force-renewal` পরীক্ষার জন্য ব্যবহার করবেন না — এটি আসল নবায়ন করে এবং বারবার চালালে সার্টিফিকেট দেওয়ার সীমায় পৌঁছে যেতে পারেন। পরীক্ষার জন্য সবসময় `--dry-run` ব্যবহার করুন।

৯.৭ গুরুত্বপূর্ণ কমান্ড ও বিশ্লেষণ

```
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d app.technova.example -d api.technova.example
sudo certbot renew --dry-run     # নবায়ন পরীক্ষা — আসল নবায়ন নয়, শুধু যাচাই
sudo systemctl status certbot.timer   # স্বয়ংক্রিয় নবায়ন timer চলছে কি না
curl -I https://app.technova.example/  # HTTPS উত্তর ও রিডাইরেক্ট যাচাই
curl -v https://app.technova.example/  # TLS handshake বিস্তারিত দেখা (ডিবাগে)
openssl s_client -connect app.technova.example:443 -servername app.technova.example
# সার্টিফিকেটের মেয়াদ ও চেইন দেখার জন্য (নমুনা ডোমেইন)
```

Certbot চালানোর আগে DNS-এ ডোমেইনটি সার্ভারের IP-তে (নমুনা: 203.0.113.10) নির্দেশিত থাকতে হবে, নইলে যাচাই ব্যর্থ হবে।

```
sudo certbot certificates       # কোন সার্টিফিকেট আছে, মেয়াদ কবে শেষ হবে
sudo systemctl list-timers | grep certbot   # নবায়ন timer কবে চলবে
sudo certbot renew --dry-run --cert-name app.technova.example   # একটি সার্টিফিকেটের নবায়ন পরীক্ষা
echo | openssl s_client -connect app.technova.example:443 2>/dev/null | openssl x509 -noout -
dates
# সার্টিফিকেটের শুরুর ও শেষের তারিখ দেখা (নমুনা ডোমেইন)
```

`certbot certificates` দিয়ে মেয়াদের তালিকা দেখা অভ্যাস করুন — ড্যাশবোর্ডে না গিয়েই বোঝা যায় কোন সার্টিফিকেট শীঘ্রই শেষ হচ্ছে।

৯.৮ কনফিগারেশন উদাহরণ

```
# /etc/nginx/sites-available/api-technova — Reverse proxy + TLS (Certbot-পরবর্তী রূপ)
server {
    listen 443 ssl;
```

```
    server_name api.technova.example;

    ssl_certificate /etc/letsencrypt/live/api.technova.example/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.technova.example/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:4000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}

server {
    listen 80;
    server_name api.technova.example;
    return 301 https://$host$request_uri;   # HTTP → HTTPS স্থায়ী রিডাইরেক্ট
}
```

`Upgrade` / `Connection` হেডার দুটি WebSocket-এর জন্য দরকার — না থাকলে Socket.IO সংযোগ বারবার ভাঙবে।

> **WARNING:** Certbot চালানোর আগে বর্তমান Nginx কনফিগের ব্যাকআপ নিন (`sudo cp -r /etc/nginx /etc/nginx.bak`) এবং `nginx -t` চালান। Certbot কনফিগ বদলায়; কিছু ভুল হলে ব্যাকআপ থেকে ফেরত যাওয়া যায়।

৯.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১: এক সার্ভারে মার্কেটিং সাইট (static) ও API (Node) — দুটি server block, দুটি ডোমেইন, একটিই Nginx। পরিস্থিতি ২: সার্টিফিকেটের মেয়াদ শেষ — স্বয়ংক্রিয় নবায়ন timer বন্ধ ছিল বা পোর্ট ৮০ বন্ধ থাকায় challenge ব্যর্থ হয়েছে; ব্রাউজার "Your connection is not private" দেখায়। পরিস্থিতি ৩: লগইন কাজ করছে না — অ্যাপ `trust proxy` সেট করেনি, তাই secure cookie HTTP মনে করে পাঠাচ্ছে না।

> **REAL WORLD — TechNova:** TechNova-র API সাইটে (api.technova.example, 203.0.113.10) এক শনিবার সকালে ব্রাউজার "Your connection is not private" দেখাতে শুরু করে। কারণ অনুসন্ধানে দেখা গেল `certbot.timer` সক্রিয় ছিল, কিন্তু নবায়ন ব্যর্থ হচ্ছিল — কেউ ফায়ারওয়ালে পোর্ট ৮০ বন্ধ করে দিয়েছিল, ফলে HTTP-01 challenge-এর যাচাই ফাইল Let's Encrypt পড়তে পারছিল না। দলটি পোর্ট ৮০ খুলে `sudo certbot renew --dry-run` দিয়ে যাচাই করে, তারপর আসল নবায়ন করে Nginx reload দেয়। শিক্ষা: মেয়াদ দেখার পাশাপাশি নবায়নের পথটিও (পোর্ট ৮০, DNS, timer) নিয়মিত পরীক্ষা করুন।

৯.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

১. ল্যাবে `/etc/hosts`-এ `127.0.0.1 app.technova.example` যোগ করুন (শুধু ল্যাব)। ২. পোর্ট ৩০০০-এ একটি সাধারণ Node সার্ভার চালান যা `req.headers` দেখায়। ৩. উপরের মতো reverse proxy server block লিখুন (TLS ছাড়া প্রথমে)। ৪. `curl -H "Host: app.technova.example" http://127.0.0.1/` দিয়ে যাচাই — অ্যাপ কি আসল Host

ও X-Forwarded-For পাচ্ছে? ৫. তারপর Certbot ধারণা অনুশীলন: আসল ডোমেইন থাকলে `--dry-run` দিয়ে যাচাই করুন; না থাকলে TLS অংশটি শুধু পড়ে বুঝুন, জাল সার্টিফিকেট দিয়ে পরীক্ষা করবেন না।

৯.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল ১: ভাবা যে TLS মানে সার্ভার সম্পূর্ণ নিরাপদ — TLS শুধু পথের গোপনীয়তা দেয়, অ্যাপের দুর্বলতা (যেমন SQL injection) ঠেকায় না। ভুল ২: `trust proxy` না সেট করা। ভুল ৩: সার্টিফিকেট নবায়ন পরীক্ষা না করা — ৯০ দিন পর হঠাৎ সাইট ভাঙে। ভুল ৪: HTTP ও HTTPS দুটোতেই সাইট চালু রাখা — সবসময় HTTPS-এ রিডাইরেক্ট করুন। ভুল ৫: `proxy_pass http://127.0.0.1:4000;`-এ শেষে `/` দেওয়া-না-দেওয়ার পার্থক্য না বোঝা — URI কীভাবে জোড়া লাগে তা বদলে যায়।

৯.১২ নিরাপত্তা বিবেচনা

শুধু আধুনিক TLS সংস্করণ অনুমোদন করুন (TLS 1.2+; Certbot-এর ডিফল্ট সাধারণত ঠিক থাকে), পুরনো দুর্বল cipher বন্ধ রাখুন। HSTS হেডার (`Strict-Transport-Security`) যোগ করলে ব্রাউজার ভবিষ্যতে সবসময় HTTPS ব্যবহার করে — তবে এটি চালু করার আগে নিশ্চিত হোন যে HTTPS নির্ভরযোগ্যভাবে কাজ করছে, কারণ HSTS ভুল করলে ফেরা কঠিন। সার্টিফিকেটের private key (`privkey.pem`) কখনো শেয়ার করবেন না, Git-এ তুলবেন না।

৯.১৩ সমস্যা নির্ণয় (Troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
502 Bad Gateway	অ্যাপ চলছে না / ভুল পোর্ট	`ss -tlnp | grep 4000` ; অ্যাপের লগ দেখুন
ভুল সাইট দেখাচ্ছে	server_name মিলছে না / default block	`nginx -T` ; Host হেডার যাচাই
সার্টিফিকেট সতর্কতা	মেয়াদ শেষ / নামের অমিল	`openssl s_client` ; `certbot renew`
Certbot ব্যর্থ	DNS ভুল / পোর্ট ৮০ বন্ধ	DNS যাচাই; ফায়ারওয়ালে ৮০ খোলা কি না দেখুন
WebSocket ভাঙছে	Upgrade হেডার নেই	location-এ Upgrade/Connection হেডার যোগ করুন
নবায়ন ব্যর্থ হচ্ছে	timer বন্ধ / পোর্ট ৮০ বন্ধ / DNS ভুল	`systemctl status certbot.timer` ; `certbot renew --dry-run` আউটপুট পড়ুন
সার্টিফিকেট বদলালেও পুরনো মেয়াদ দেখাচ্ছে	Nginx reload হয়নি	`nginx -t && systemctl reload nginx` ; তারপর openssl দিয়ে মেয়াদ যাচাই
একটি ডোমেইনে HTTPS চলে, অন্যটিতে নয়	সার্টিফিকেটে সেই ডোমেইন নেই	`certbot certificates` দিয়ে ডোমেইনের তালিকা দেখুন; প্রয়োজনে `-d` যোগ করে নবায়ন করুন
ERR_TOO_MANY_REDIRECTS	HTTP → HTTPS লুপ (অ্যাপও রিডাইরেক্ট করছে)	Nginx না অ্যাপ — এক জায়গায় রিডাইরেক্ট রাখুন; X-Forwarded-Proto যাচাই করুন

৯.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: প্রতি ডোমেইনে আলাদা server block; HTTP → HTTPS 301; নবায়ন timer সক্রিয় ও `--dry-run` দিয়ে পরীক্ষিত; proxy হেডার সম্পূর্ণ; পরিবর্তনের আগে ব্যাকআপ ও `nginx -t`। অ্যান্টি-প্যাটার্ন: Node অ্যাপ সরাসরি ৪৪৩ পোর্টে চালানো; TLS ছাড়া লগইন ফর্ম; সার্টিফিকেটের মেয়াদ হাতে-কলমে মনে রাখার ওপর নির্ভর করা; সব ডোমেইন একটি বিশাল server block-এ গাদাগাদি করা।

৯.১৫ অধ্যায় সারাংশ

Virtual host মানে Host হেডার পড়ে এক সার্ভারে অনেক সাইট চালানো। Reverse proxy মানে Nginx মাঝখানে বসে অনুরোধ পেছনের অ্যাপে পাঠানো — সঙ্গে সঠিক হেডার। TLS মানে এনক্রিপ্টেড পথ ও যাচাইকৃত পরিচয়; Let's Encrypt + Certbot তা বিনামূল্যে ও স্বয়ংক্রিয়ভাবে দেয়। এই তিনটি একসঙ্গে না থাকলে প্রোডাকশন ডেপ্লয়মেন্ট অসম্পূর্ণ।

৯.১৬ মূল প্রযুক্তিগত পরিভাষা

শব্দ	অর্থ
Virtual Host	এক সার্ভারে একাধিক ডোমেইনের সাইট চালানোর ব্যবস্থা।
Reverse Proxy	ক্লায়েন্ট ও পেছনের সার্ভারের মাঝখানে বসা প্রক্সি।
TLS / SSL	যোগাযোগ এনক্রিপ্ট করার প্রোটোকল; HTTPS = HTTP + TLS।
Certificate Authority (CA)	সার্টিফিকেটে স্বাক্ষর করে পরিচয় নিশ্চিত করা বিশ্বস্ত সংস্থা।
Let's Encrypt / Certbot	বিনামূল্যে সার্টিফিকেট দেওয়া CA ও তা নেওয়ার সরঞ্জাম।
HSTS	ব্রাউজারকে সবসময় HTTPS ব্যবহার করতে বলা হেডার।

৯.১৭ জ্ঞান যাচাই প্রশ্ন

1. একটি IP-তে দুটি ডোমেইন থাকলে Nginx কীভাবে বোঝে কোন সাইট চাওয়া হয়েছে?
2. Reverse proxy-তে `X-Forwarded-For` কেন দরকার?
3. Express-এ `trust proxy` না সেট করলে কী সমস্যা হয়?
4. TLS handshake সহজ ভাষায় ব্যাখ্যা করুন — সার্টিফিকেটের ভূমিকা কী?
5. Let's Encrypt সার্টিফিকেটের মেয়াদ কত দিন এবং নবায়ন স্বয়ংক্রিয় করা কেন জরুরি?
6. HTTP → HTTPS রিডাইরেক্টে 301 কেন ব্যবহার করা হয়?
7. `certbot renew --dry-run` ও আসল `certbot renew`-এর মধ্যে পার্থক্য কী, এবং পরীক্ষায় কোনটি ব্যবহার করবেন?
8. নবায়ন ব্যর্থ হলে কোন তিনটি জিনিস প্রথমে যাচাই করবেন — timer, পোর্ট ৮০ ও DNS-এর ভূমিকা ব্যাখ্যা করুন।
9. WebSocket-এর জন্য কোন দুটি proxy হেডার অপরিহার্য?

৯.১৮ ব্যবহারিক অনুশীলন

1. ল্যাবে দুটি ভিন্ন server_name-এর virtual host তৈরি করুন এবং Host হেডার বদলে curl দিয়ে যাচাই করুন যে সঠিক সাইট আসে।
2. একটি Node অ্যাপের সামনে reverse proxy বসান; অ্যাপে লগ করে দেখান যে X-Real-IP আসছে কি না।
3. Certbot-এর `renew --dry-run` ধারণা পড়ে ব্যাখ্যা লিখুন: নবায়ন ব্যর্থ হলে কী কী কারণ হতে পারে।
4. ল্যাবে `certbot certificates` ও `systemctl status certbot.timer` আউটপুট পড়ে একটি ছোট নবায়ন-পরীক্ষা চেকলিস্ট লিখুন: timer সক্রিয় কি না, পোর্ট ৮০ খোলা কি না, dry-run সফল কি না।
5. HTTP অনুরোধে 301 রিডাইরেক্ট হচ্ছে কি না `curl -I` দিয়ে যাচাই করুন।

অধ্যায় ১০

Deploying Node.js Applications on Linux

১০.১ শেখার উদ্দেশ্য (Learning Objectives)

এই অধ্যায় শেষে আপনি পারবেন: একটি Express/Next.js অ্যাপকে ল্যাপটপ থেকে Ubuntu সার্ভারে নিরাপদে ডেপ্লয় করতে; Node.js-এর সঠিক সংস্করণ ইনস্টল ও পরিচালনা করতে; production ও development পরিবেশের পার্থক্য ব্যাখ্যা করতে; পরিবেশ চলক (environment variable) নিরাপদে পরিচালনা করতে; এবং একটি পুনরাবৃত্তিযোগ্য (repeatable) ডেপ্লয় প্রক্রিয়া — হাতে-কলমে ধাপগুলো — লিখতে।

১০.২ প্রোডাকশনে কেন এটি গুরুত্বপূর্ণ

আপনার কোড ল্যাপটপে চললেই প্রোডাকশনে চলবে — এই ধারণা সবচেয়ে বেশি ডেপ্লয় ব্যর্থতার কারণ। ল্যাপটপে Node ২০, সার্ভারে Node ১৮; ল্যাপটপে `.env` ফাইল আছে, সার্ভারে নেই; ল্যাপটপে পোর্ট ৩০০০ ফাঁকা, সার্ভারে ব্যবহৃত। ডেপ্লয় মানে শুধু ফাইল কপি করা নয় — মানে সঠিক রানটাইম, সঠিক নির্ভরতা, সঠিক পরিবেশ চলক, সঠিক অনুমতি ও সঠিক প্রক্রিয়া ব্যবস্থাপনা একসঙ্গে নিশ্চিত করা। একটি ভালো ডেপ্লয় প্রক্রিয়া থাকলে নতুন সংস্করণ আসে নির্ভয়ে; না থাকলে প্রতিটি ডেপ্লয় হয় আতঙ্ক।

১০.৩ পূর্বশর্ত জ্ঞান (Prerequisite Knowledge)

Node.js ও npm-এর ধারণা, Git-এর মৌলিক ব্যবহার (clone, pull), SSH দিয়ে সার্ভারে ঢোকা, অধ্যায় ৭ (systemd) ও অধ্যায় ৮–৯ (Nginx) জানা প্রয়োজন। Next.js অ্যাপ হলে `next build` ও `next start`-এর পার্থক্য জানা থাকা ভালো।

১০.৪ প্রথম নীতি থেকে ব্যাখ্যা (First Principles)

ডেপ্লয়মেন্টের প্রথম নীতি: **প্রোডাকশন পরিবেশটি পুনরাবৃত্তিযোগ্য হতে হবে** — অর্থাৎ একই ধাপ আবার চালালে একই ফল আসবে। দ্বিতীয় নীতি: **কোড, কনফিগ ও গোপন তথ্য আলাদা** — কোড Git-এ থাকে, কনফিগ পরিবেশ চলকে থাকে, গোপন তথ্য (secret) কখনো কোডে থাকে না। তৃতীয় নীতি: **সার্ভারে কিছু তৈরি (build) করার চেয়ে আগে থেকে তৈরি করা ভালো** — তবে ছোট প্রকল্পে সার্ভারে build করাও গ্রহণযোগ্য, যদি ধাপগুলো লিখিত থাকে। এই তিন নীতি মাথায় রাখলে ডেপ্লয় সিদ্ধান্তগুলো নিজে থেকেই পরিষ্কার হয়: কেন `node_modules` Git-এ তোলা হয় না, কেন `.env` সার্ভারে আলাদাভাবে রাখা হয়, কেন `NODE_ENV=production` সেট করা হয়।

১০.৫ আর্কিটেকচার / ধারণাগত চিত্র

ডেপ্লয় প্রবাহ:

ল্যাপটপ (Git push) → Git সার্ভার → সার্ভারে `git pull` → `npm ci` (নির্ভরতা) → `npm run build` (Next.js হলে) → পরিবেশ চলক লোড → systemd/PM2 দিয়ে চালু → Nginx (443) → ব্যবহারকারী

কোড ──→ /opt/technova/app | গোপন তথ্য ──→ /etc/technova/app.env (600 অনুমতি) | লগ ──→ journald

রোলব্যাক: আগের Git commit-এ ফিরে গিয়ে একই ধাপ আবার চালানো।

১০.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Node.js ইনস্টল: কোন পদ্ধতি

Ubuntu-র apt-এ Node থাকলেও তা প্রায়ই পুরনো হয়। প্রোডাকশনে সাধারণ পদ্ধতি: NodeSource রিপোজিটরি থেকে নির্দিষ্ট LTS সংস্করণ ইনস্টল করা, অথবা `nvm`-এর মতো সংস্করণ ব্যবস্থাপক ব্যবহার করা। সার্ভারে একটি মাত্র সংস্করণ স্থির রাখাই ভালো — `node --version` লিখে রাখুন ডেপ্লয় নোটে।

npm ci বনাম npm install

`npm ci` (clean install) `package-lock.json` অনুযায়ী হুবহু একই নির্ভরতা বসায় এবং আগের `node_modules` মুছে ফেলে — প্রোডাকশনে এটিই সঠিক। `npm install` lock ফাইল বদলে দিতে পারে, ফলে ল্যাপটপ ও সার্ভারে ভিন্ন সংস্করণ বসে যেতে পারে।

পরিবেশ চলক (Environment Variables)

`NODE_ENV=production` সেট করলে Express ও অনেক লাইব্রেরি দ্রুত ও নিরাপদ আচরণ করে (বিস্তারিত ভুল বার্তা লুকায়, ক্যাশ করে)। `PORT`, `DATABASE_URL`, `JWT_SECRET`-এর মতো মান পরিবেশ চলকে রাখুন — কোডে নয়। systemd-তে `EnvironmentFile` দিয়ে এগুলো লোড করা হয়; PM2-তে ecosystem ফাইলে (অধ্যায় ১১)।

Build ধাপ

Next.js-এ `npm run build` ছাড়া `next start` চলবে না। Express অ্যাপে সাধারণত build লাগে না, তবে TypeScript হলে `tsc` চালাতে হয়। Build ব্যর্থ হলে পুরনো অ্যাপ চালু রাখুন — ভাঙা build ডেপ্লয় করে সাইট ফেলে দেবেন না।

ফাইল অনুমতি ও মালিকানা

অ্যাপের ডিরেক্টরি `/opt/technova/app`-এর মালিক হোক সার্ভিস ব্যবহারকারী (`technova`), root নয়। ডেপ্লয়কারী ব্যবহারকারীর লেখার অধিকার থাকতে হবে, কিন্তু অ্যাপ চলার সময় লেখার অধিকার সীমিত রাখাই নিরাপদ।

ডেপ্লয় ধাপ গভীরে ও পরিবেশ পরিচালনা

একটি নির্ভরযোগ্য ডেপ্লয় প্রক্রিয়ায় প্রতিটি ধাপের একটি করে যাচাই থাকে। `git pull`-এর পর `git log --oneline -5` দিয়ে দেখুন কোন commit এলো — ভুল শাখা (branch) থেকে টানা হলে এখানেই ধরা পড়ে। Build-এর আগে সার্ভারে যথেষ্ট ডিস্ক ও মেমোরি আছে কি না দেখুন — Next.js build মেমোরি বেশি খায়, আর ডিস্ক পূর্ণ থাকলে build মাঝপথে ব্যর্থ হয়। পরিবেশ চলক পরিচালনার সবচেয়ে নিরাপদ পদ্ধতি হলো একটি `.env.example` ফাইল Git-এ রাখা, যেখানে শুধু চলকের নাম ও নমুনা মান থাকে (আসল মান নয়) — নতুন সার্ভারে বসানোর সময় এই ফাইল দেখে আসল `.env` তৈরি করা যায়, আর কোনো চলক বাদ পড়লে শুরুতেই ধরা পড়ে।

পরিবেশের নামও স্পষ্ট রাখুন: development, staging ও production — তিনটি আলাদা জিনিস। Staging হলো প্রোডাকশনের মতো একটি পরীক্ষার সার্ভার, যেখানে আসল ডেটা থাকে না — নতুন কোড প্রথমে staging-এ যাচাই করে তারপর প্রোডাকশনে নেওয়া হয়। `NODE_ENV` ছাড়াও অ্যাপের নিজস্ব চলক (যেমন `APP_ENV=staging`) ব্যবহার করা যায়, তবে নামের অর্থ সবাইকে জানিয়ে রাখুন। রোলব্যাকের জন্য Git tag ব্যবহার করুন: প্রতিটি সফল ডেপ্লয়ে একটি tag দিন (যেমন `v1.4.2`), তাহলে ফেরত যেতে শুধু সেই tag-এ checkout করে একই স্ক্রিপ্ট চালালেই হয় — কোন commit ভালো ছিল তা খুঁজতে হয় না। ডেপ্লয়ের সময় health যাচাইয়ের আগে পুরনো প্রক্রিয়া থামাবেন না — নতুন কোড চালু হয়েছে নিশ্চিত করে তারপর পুরনোটি থামান, অথবা অধ্যায় ১১-এর PM2 reload ব্যবহার করুন।

১০.৭ গুরুত্বপূর্ণ কমান্ড ও বিশ্লেষণ

```
node --version && npm --version   # সংস্করণ যাচাই — ডেপ্লয় নোটে লিখে রাখুন
git clone https://github.com/example/technova-app.git /opt/technova/app
cd /opt/technova/app && npm ci --omit=dev   # শুধু production নির্ভরতা
npm run build                       # Next.js / TypeScript হলে
NODE_ENV=production node server.js  # হাতে চালিয়ে পরীক্ষা (systemd-এর আগে)
sudo systemctl restart app && systemctl status app
curl http://127.0.0.1:3000/health   # অ্যাপ সাড়া দিচ্ছে কি না (health endpoint)
```

`--omit=dev` মানে devDependencies বাদ — প্রোডাকশনে পরীক্ষার সরঞ্জাম দরকার নেই, এতে ইনস্টল দ্রুত হয় ও আক্রমণের পথ কমে।

```
git log --oneline -5            # কোন commit ডেপ্লয় হচ্ছে দেখা
git tag v1.4.2 && git push origin v1.4.2   # সফল ডেপ্লয়ে tag দেওয়া (রোলব্যাকের জন্য)
npm ls --omit=dev               # production নির্ভরতা ঠিকমতো বসেছে কি না
df -h /opt && free -h            # build-এর আগে ডিস্ক ও মেমোরি যাচাই
diff .env.example .env 2>/dev/null | head -20   # কোন চলক বাদ পড়েছে কি না দেখা (নমুনা)
```

`git tag` দিয়ে সংস্করণ চিহ্নিত রাখলে রোলব্যাক আর অনুমান নয় — tag-এর নাম বললেই সবাই জানে কোন সংস্করণে ফিরতে হবে। `df -h` ও `free -h` build-এর আগে চালানো একটি ছোট অভ্যাস, যা "build কেন মাঝপথে মরল" — এই রহস্যের অনেক সমাধান দেয়।

১০.৮ কনফিগারেশন উদাহরণ

```
# /etc/technova/app.env — অনুমতি 600, মালিক technova (নমুনা মান, আসল গোপন তথ্য নয়)
NODE_ENV=production
PORT=3000
DATABASE_URL=postgresql://technova:CHANGE-ME-2026@127.0.0.1:5432/technova
JWT_SECRET=CHANGE-ME-2026
```

```
# ডেপ্লয় স্ক্রিপ্টের খসড়া — deploy.sh (ল্যাবে যাচাই করে ব্যবহার করুন)
#!/bin/bash
set -e
cd /opt/technova/app
git pull origin main
npm ci --omit=dev
npm run build
sudo systemctl restart app
sleep 3
curl -f http://127.0.0.1:3000/health
```

`set -e` মানে কোনো ধাপ ব্যর্থ হলে স্ক্রিপ্ট থেমে যাবে — ভাঙা অবস্থায় এগিয়ে যাবে না। `curl -f` মানে HTTP ভুল হলে ব্যর্থতা জানাবে।

১০.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১: ডেপ্লয়ের পর সাইট ৫০২ দিচ্ছে — build হয়নি, অথবা `NODE_ENV` সেট হয়নি, অথবা নতুন কোডে পরিবেশ চলক লাগে যা সার্ভারে নেই। পরিস্থিতি ২: ডেপ্লয় আধাঘণ্টা ধরে চলছে — `npm install` নেটওয়ার্ক থেকে প্রতিবার সব

নামাচ্ছে; সমাধান: `npm ci` + ক্যাশ। পরিস্থিতি ৩: রোলব্যাক দরকার — আগের commit-এ `git checkout` করে একই স্ক্রিপ্ট চালান; ডেটাবেস migration ফেরত নেওয়া কঠিন, তাই migration-এর আগে ব্যাকআপ নিন।

> **REAL WORLD — TechNova:** TechNova দল একবার শুক্রবার রাতে প্রোডাকশনে ডেপ্লয় দিল, কিন্তু নতুন কোডে একটি পরিবেশ চলক (`PAYMENT_API_KEY`) লাগত যা সার্ভারের `/etc/technova/app.env`-এ ছিল না — অ্যাপ চালু হয়ে সঙ্গে সঙ্গে বন্ধ হয়ে যাচ্ছিল। ভাগ্য ভালো, তাদের deploy স্ক্রিপ্টে health যাচাই ছিল, তাই ভাঙা অবস্থা ধরা পড়ে এবং দলটি আগের tag (`v1.4.1`)-এ ফিরে গিয়ে একই স্ক্রিপ্ট চালিয়ে দশ মিনিটে সাইট ফিরিয়ে আনে। শিক্ষা: ডেপ্লয় ব্যর্থতা লজ্জার নয় — health যাচাই ছাড়া ডেপ্লয় আর tag ছাড়া রোলব্যাকই আসল ভুল।

১০.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

> **LAB:** শুধু disposable VM-এ অনুশীলন করুন। আসল গোপন তথ্য ব্যবহার করবেন না।

১. VM-এ Node LTS ইনস্টল করুন ও সংস্করণ লিখে রাখুন। ২. একটি ছোট Express অ্যাপ তৈরি করুন যার `/health` endpoint `{"ok": true}` দেয়। ৩. Git রিপোজিটরি থেকে clone করার ভান করে /opt-এ বসান (ল্যাবে সরাসরি কপি করা যায়)। ৪. `npm ci` চালান, `.env` ফাইল তৈরি করুন (CHANGE-ME-2026 মান দিয়ে)। ৫. অধ্যায় ৭-এর unit ফাইল দিয়ে systemd সার্ভিস চালু করুন। ৬. Nginx reverse proxy (অধ্যায় ৯) বসিয়ে ব্রাউজার থেকে যাচাই করুন। ৭. কোডে একটি ছোট পরিবর্তন করে deploy.sh চালিয়ে দেখুন পরিবর্তন আসে কি না।

১০.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল ১: `.env` Git-এ তুলে ফেলা — একবার Git-এ উঠলে ইতিহাস থেকে মোছা কঠিন। ভুল ২: `npm install` প্রোডাকশনে ব্যবহার করা। ভুল ৩: root হিসেবে অ্যাপ চালানো ও ফাইল রাখা। ভুল ৪: `node server.js` টার্মিনালে চালিয়ে SSH বন্ধ করা — প্রক্রিয়া মরে যায় (এজন্যই systemd/PM2)। ভুল ৫: build ছাড়া Next.js চালানোর চেষ্টা। ভুল ৬: ডেপ্লয়ের আগে ব্যাকআপ না নেওয়া।

১০.১২ নিরাপত্তা বিবেচনা

> **WARNING:** কখনো `.env`, private key বা পাসওয়ার্ড Git-এ তুলবেন না। `.gitignore`-এ `.env` আছে কি না ডেপ্লয়ের আগে যাচাই করুন। সন্দেহ হলে ধরে নিন গোপন তথ্য ফাঁস হয়েছে এবং তা বদলে ফেলুন।

অ্যাপ শুধু localhost (127.0.0.1)-এ শুনুক, সরাসরি ইন্টারনেটে নয় — বাইরের ট্র্যাফিক Nginx দিয়ে আসবে। ফায়ারওয়ালে শুধু ২২ (SSH), ৮০ ও ৪৪৩ খোলা রাখুন; ৩০০০-এর মতো অ্যাপ পোর্ট বাইরে থেকে বন্ধ রাখুন। নির্ভরতায় দুর্বলতা আছে কি না `npm audit` দিয়ে দেখুন।

১০.১৩ সমস্যা নির্ণয় (Troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
Cannot find module	npm ci হয়নি / ভুল ডিরেক্টরি	WorkingDirectory ও node_modules যাচাই

PORT already in use	পুরনো প্রক্রিয়া আটকে আছে	`ss -tlnp | grep 3000`; পুরনো প্রক্রিয়া থামান
DATABASE_URL undefined	EnvironmentFile পথ ভুল	unit ফাইলে পথ ও ফাইলের অনুমতি যাচাই
Build ব্যর্থ	মেমোরি কম / Node সংস্করণ অমিল	`node --version`; build লগ পড়ুন
ডেপ্লয়ের পর পুরনো কোড	restart হয়নি / ক্যাশ	systemctl restart; Nginx ক্যাশ যাচাই
git pull-এ ভুল শাখা এলো	সার্ভারে ভুল branch checkout করা	`git branch --show-current` ও `git log --oneline -3` দিয়ে যাচাই
Build মাঝপথে ব্যর্থ	ডিস্ক বা মেমোরি শেষ	`df -h` ও `free -h`; পুরনো build ও লগ পরিষ্কার করুন
নতুন চলক undefined	.env.example-এ আছে, সার্ভারের .env-এ নেই	`diff .env.example .env`; সার্ভারের ফাইলে চলক যোগ করুন
ডেপ্লয়ের পর health ব্যর্থ, কিন্তু অ্যাপ চলছে	/health endpoint-এর পথ বা পোর্ট ভুল	`curl http://127.0.0.1:3000/health` সরাসরি চালিয়ে যাচাই করুন

১০.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: লিখিত ডেপ্লয় চেকলিস্ট; `npm ci`; `/health` endpoint দিয়ে যাচাই; ডেপ্লয়ের আগে ব্যাকআপ; ছোট ছোট ডেপ্লয়, বড় একবারে নয়। অ্যান্টি-প্যাটার্ন: সার্ভারে সরাসরি কোড সম্পাদনা করা (SSH-এ nano দিয়ে ঠিক করা); গোপন তথ্য কোডে লেখা; "আমার ল্যাপটপে চলে" বলে সার্ভারের পরিবেশ না মেলানো; রোলব্যাক পরিকল্পনা ছাড়া ডেপ্লয়।

১০.১৫ অধ্যায় সারাংশ

ডেপ্লয় মানে পুনরাবৃত্তিযোগ্য ধাপের সমষ্টি: সঠিক Node সংস্করণ, `npm ci`, build, পরিবেশ চলক, systemd দিয়ে চালু, Nginx দিয়ে প্রকাশ, health যাচাই। কোড, কনফিগ ও গোপন তথ্য আলাদা রাখুন। প্রতিটি ডেপ্লয়ের আগে ব্যাকআপ ও পরে যাচাই — এই দুটি অভ্যাসই প্রোডাকশনকে শান্ত রাখে।

১০.১৬ মূল প্রযুক্তিগত পরিভাষা

শব্দ	অর্থ
Deployment	কোডকে সার্ভারে নিয়ে চালু করার পুরো প্রক্রিয়া।
Environment Variable	কোডের বাইরে রাখা কনফিগ মান; যেমন PORT, DATABASE_URL।
npm ci	lock ফাইল অনুযায়ী হুবহু নির্ভরতা বসানো (প্রোডাকশনের জন্য)।
Build	সোর্স কোড থেকে চালানোর উপযোগী ফাইল তৈরি করা।
Health Endpoint	অ্যাপ বেঁচে আছে কি না জানার ছোট URL, যেমন /health।
Rollback	ভাঙা ডেপ্লয়ের পর আগের ভালো সংস্করণে ফেরা।

১০.১৭ জ্ঞান যাচাই প্রশ্ন

1. `npm ci` ও `npm install`-এর পার্থক্য কী এবং প্রোডাকশনে কোনটি কেন?
2. `NODE_ENV=production` সেট করলে কী বদলায়?
3. গোপন তথ্য কোডে না রেখে কোথায় রাখবেন এবং কেন?
4. ডেপ্লয়ের পর অ্যাপ চালু হয়েছে কি না কীভাবে যাচাই করবেন?
5. রোলব্যাক পরিকল্পনা কেন ডেপ্লয়ের অংশ হওয়া উচিত?
6. অ্যাপ কেন শুধু localhost-এ শুনবে, সরাসরি ইন্টারনেটে নয়?
7. `.env.example` ফাইল কেন Git-এ রাখা হয়, অথচ আসল `.env` রাখা হয় না?
8. ডেপ্লয়ে Git tag দেওয়ার সুবিধা কী? রোলব্যাকের সময় tag কীভাবে সাহায্য করে?
9. সার্ভারে সরাসরি কোড সম্পাদনা করা কেন অ্যান্টি-প্যাটার্ন?

১০.১৮ ব্যবহারিক অনুশীলন

1. ল্যাব VM-এ একটি Express অ্যাপ ডেপ্লয় করুন: clone → npm ci → systemd সার্ভিস → Nginx proxy → /health যাচাই। প্রতিটি ধাপ লিখে রাখুন।
2. ইচ্ছাকৃতভাবে `.env` ছাড়া অ্যাপ চালু করে ভুল বার্তা পড়ুন; তারপর EnvironmentFile ঠিক করে আবার চালান।
3. একটি deploy.sh স্ক্রিপ্ট লিখুন যাতে set -e ও health যাচাই থাকে; দুইবার চালিয়ে দেখুন দ্বিতীয়বারও কাজ করে কি না (idempotent কি না)।
4. ল্যাবে একটি `.env.example` ফাইল তৈরি করুন (শুধু নাম ও CHANGE-ME-2026 নমুনা মান দিয়ে), তারপর ইচ্ছাকৃতভাবে একটি চলক সার্ভারের .env থেকে বাদ দিয়ে অ্যাপ চালিয়ে ভুলটি পড়ুন — তারপর চলক যোগ করে আবার চালান।
5. আগের commit-এ ফিরে রোলব্যাক অনুশীলন করুন এবং সময় মাপুন — কত মিনিটে ফেরা যায়?

অধ্যায় ১১

Process Management with PM2 and systemd

১১.১ শেখার উদ্দেশ্য (Learning Objectives)

এই অধ্যায় শেষে আপনি পারবেন: PM2 ও systemd-এর মধ্যে গভীর পার্থক্য ব্যাখ্যা করতে এবং কখন কোনটি — বা দুটি একসঙ্গে — ব্যবহার করবেন তা সিদ্ধান্ত নিতে; PM2 দিয়ে Node ক্লাস্টার চালাতে, zero-downtime reload করতে ও লগ পরিচালনা করতে; ecosystem.config.js লিখতে; PM2-কে systemd-এর অধীনে চালিয়ে বুটে চালু করতে; এবং দুটি সরঞ্জামের সীমা জেনে ভুল পছন্দ এড়াতে।

১১.২ প্রোডাকশনে কেন এটি গুরুত্বপূর্ণ

অধ্যায় ৭ ও ১০-এ আপনি systemd দিয়ে অ্যাপ চালাতে শিখেছেন। তাহলে PM2 কেন দরকার? কারণ Node.js একক থ্রেডে চলে — একটি CPU কোর ব্যবহার করে, বাকিগুলো বসে থাকে। PM2 একই অ্যাপের একাধিক কপি (cluster) চালিয়ে সব কোর ব্যবহার করতে পারে, কোড বদলালে সংযোগ না ভেঙে নতুন কোডে যেতে পারে (zero-downtime reload), লগ ঘোরাতে (rotate) পারে ও মনিটরিং দেয়। কিন্তু PM2 সবকিছুর সমাধান নয় — ডেটাবেস, Nginx, Redis — এগুলো PM2 দিয়ে চালানো যায় না। ভুল সরঞ্জাম বেছে নিলে হয় ক্লাস্টারিং হারাবেন, নয়তো বুটে অ্যাপ উঠবে না। এই অধ্যায়ের মূল শিক্ষা: **PM2 ও systemd প্রতিদ্বন্দ্বী নয়, ভিন্ন স্তরের সরঞ্জাম — এবং তারা একসঙ্গে সবচেয়ে ভালো কাজ করে।**

১১.৩ পূর্বশর্ত জ্ঞান (Prerequisite Knowledge)

অধ্যায় ৭ (systemd unit, systemctl, journalctl), অধ্যায় ১০ (Node ডেপ্লয়, পরিবেশ চলক) এবং Node.js-এর একক-থ্রেড মডেলের ধারণা প্রয়োজন। Cluster মানে একই অ্যাপের একাধিক প্রক্রিয়া — এই ধারণাটি এখানেই পরিষ্কার হবে।

১১.৪ প্রথম নীতি থেকে ব্যাখ্যা (First Principles)

একটি প্রক্রিয়া ব্যবস্থাপকের (process manager) কাজ তিনটি: প্রক্রিয়া চালু রাখা, ব্যর্থ হলে ফিরিয়ে আনা, এবং তার অবস্থা জানানো। systemd এই কাজটি করে অপারেটিং সিস্টেমের স্তরে — সে PID 1, তাই বুটের সময় থেকে সবকিছু তার অধীনে। PM2 এই কাজটি করে অ্যাপ্লিকেশনের স্তরে — সে Node.js-কে গভীরভাবে বোঝে: Node-এর cluster মডিউল, ESM/CJS পার্থক্য, মেমোরি ব্যবহার, লগ ধারা। প্রথম নীতি থেকে ভাবুন: যে সরঞ্জাম সমস্যাটিকে গভীরভাবে বোঝে, সে সেই সমস্যায় ভালো। systemd সব সার্ভিসকে সমানভাবে দেখে — এটাই তার শক্তি (সাধারণ) ও সীমা (Node-বিশেষ কিছু পারে না)। PM2 শুধু Node বোঝে — এটাই তার শক্তি (গভীর) ও সীমা (সাধারণ নয়)। তাই প্রশ্ন "কোনটি ভালো?" ভুল প্রশ্ন; সঠিক প্রশ্ন: "কোন স্তরে কোন কাজ?"

১১.৫ আর্কিটেকচার / ধারণাগত চিত্র

দুটি স্তর, একটি সমাধান (সুপারিশকৃত):

systemd (PID 1) ──→ pm2.service (বুটে চালু, root/systemd-এর অধীনে)

└── PM2 Daemon (technova ব্যবহারকারী হিসেবে)

├── Node অ্যাপ instance 1 (CPU কোর ১)

├── Node অ্যাপ instance 2 (CPU কোর ২)

├── Node অ্যাপ instance 3 (CPU কোর ৩)

└── Node অ্যাপ instance 4 (CPU কোর ৪)

Load Balancer (PM2-এর ভেতরে) ──→ অনুরোধ ভাগ করে দেয়; একটি instance reload হলে বাকিগুলো সাড়া দেয়।

বিকল্প: systemd সরাসরি একটিমাত্র Node প্রক্রিয়া চালায় — সহজ, কিন্তু ক্লাস্টারিং নেই।

১১.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

PM2-এর শক্তি

ক্লাস্টারিং: `instances: 'max'` দিলে PM2 CPU কোর সংখ্যা অনুযায়ী অ্যাপের কপি চালায় এবং অনুরোধ ভাগ করে দেয় — একটি কোরের সীমা ভেঙে যায়। **Zero-downtime reload:** `pm2 reload app` দিলে PM2 একবারে একটি instance বদলায় — পুরনো instance নতুন অনুরোধ নেওয়া বন্ধ করে, চলমান অনুরোধ শেষ করে, তারপর নতুন কোডে ওঠে। ব্যবহারকারী কিছু টের পায় না। **Ecosystem ফাইল:** সব অ্যাপ, পরিবেশ চলক, লগ পথ, restart নীতি একটি `ecosystem.config.js` ফাইলে ঘোষণা করা যায় — Git-এ রাখা যায়, পর্যালোচনা করা যায়। **লগ ব্যবস্থাপনা:** PM2 প্রতিটি অ্যাপের stdout/stderr আলাদা ফাইলে লেখে, `pm2-logrotate` দিয়ে ফাইল ঘোরায়। **মনিটরিং:** `pm2 monit` ও `pm2 status` দিয়ে CPU, মেমোরি, restart সংখ্যা দেখা যায়।

PM2-এর সীমা

PM2 একটি ব্যবহারকারী-স্তরের প্রক্রিয়া (user process) — সে PID 1 নয়। সার্ভার রিবুট হলে PM2 নিজে থেকে উঠবে না, যদি না তাকে systemd-এর সঙ্গে যুক্ত করা হয় (`pm2 startup`)। PM2 মূলত Node.js-কেন্দ্রিক — Python, Go বা PostgreSQL চালাতে systemd-ই সঠিক জায়গা। PM2-এর daemon মারা গেলে তার অধীন অ্যাপগুলোর অবস্থা অনিশ্চিত হতে পারে। আর PM2 দিয়ে চালানো অ্যাপের লগ journald-এ যায় না — দুই জায়গায় লগ দেখতে হয়।

systemd-এর শক্তি

systemd PID 1 — বুটের অবিচ্ছেদ্য অংশ, কোনো অতিরিক্ত সংযোগ ছাড়াই চলে। যেকোনো সার্ভিস চালাতে পারে: Nginx, PostgreSQL, Redis, আপনার অ্যাপ, টাইমার। journald-এ এক জায়গায় লগ। Resource limit (`MemoryMax`, `CPUQuota`) ও sandboxing (`ProtectSystem`, `PrivateTmp`, `NoNewPrivileges`) সরাসরি unit ফাইলে লেখা যায় — PM2-এ এগুলো নেই। নির্ভরতা (After/Requires) দিয়ে সার্ভিসের ক্রম নিশ্চিত করা যায়।

systemd-এর সীমা

systemd Node ক্লাস্টারিং বোঝে না — একটি unit একটি প্রক্রিয়া (বা একটি প্রক্রিয়া গাছ) চালায়। Zero-downtime reload-এর ধারণা systemd-এ নেই; restart মানে পুরনো প্রক্রিয়া মেরে নতুন চালু — মাঝখানে সংযোগ ভাঙে। একাধিক instance চাইলে একাধিক unit ফাইল বা template unit লিখতে হয়, যা PM2-এর তুলনায় জটিল। Node-বিশেষ মনিটরিং (মেমোরি বৃদ্ধি দেখে restart) systemd-এ নেই।

সিদ্ধান্ত টেবিল

বিষয়	PM2	systemd
কী চালায়	প্রধানত Node.js অ্যাপ	যেকোনো সার্ভিস (Nginx, DB, অ্যাপ)
বুটে চালু	শুধু `pm2 startup` সংযোগের পর	সহজাত — PID 1
ক্লাস্টারিং	হ্যাঁ, সহজে (instances: max)	না — হাতে একাধিক unit লাগে
Zero-downtime reload	হ্যাঁ (`pm2 reload`)	না — restart মানে বিরতি
লগ	নিজস্ব ফাইল + logrotate	journald-এ এক জায়গায়
Resource limit / sandboxing	সীমিত	শক্তিশালী (MemoryMax, ProtectSystem)
নির্ভরতা (DB-এর পরে চালু)	দুর্বল	শক্তিশালী (After/Requires)

কখন কোনটি বেছে নেবেন

একটি মাত্র Node অ্যাপ, ছোট সার্ভার, সহজতা চাইলে — শুধু systemd যথেষ্ট। একাধিক CPU কোর ব্যবহার করতে চাইলে, zero-downtime ডেপ্লয় চাইলে, একাধিক Node অ্যাপ একসঙ্গে পরিচালনা করতে চাইলে — PM2, তবে অবশ্যই systemd-এর অধীনে (`pm2 startup`) যাতে বুটে ওঠে। সিস্টেম সার্ভিস (Nginx, PostgreSQL, Redis) সবসময় systemd-এ। সোনালি নিয়ম: **systemd সিস্টেমের জন্য, PM2 Node অ্যাপের জন্য — আর PM2 নিজেও systemd-এর অধীনে।**

PM2 বনাম systemd — কিনারা পরিস্থিতি (edge cases)

দুটি সরঞ্জাম একসঙ্গে চালানোর সময় কয়েকটি কিনারা পরিস্থিতি বুঝে রাখা জরুরি। প্রথমটি হলো graceful shutdown: অ্যাপ বন্ধ করার সময় PM2 প্রথমে `SIGINT` সংকেত পাঠায়, কিছুক্ষণ অপেক্ষা করে, তারপর `SIGKILL` দেয়। আপনার অ্যাপ যদি সংকেত পেয়ে চলমান অনুরোধ শেষ করে, ডেটাবেস সংযোগ বন্ধ করে তারপর থামে, তাহলে reload-এর সময় কোনো অনুরোধ মাঝপথে ভাঙে না — এই আচরণকে graceful shutdown বলে। দ্বিতীয় পরিস্থিতি হলো fork বনাম cluster: `exec_mode: 'fork'`-এ একটি instance চলে — ছোট অ্যাপ বা একটিমাত্র ব্যাকগ্রাউন্ড কাজের (worker) জন্য এটিই ঠিক; `exec_mode: 'cluster'`-এ একাধিক instance চলে, কিন্তু তাতে in-memory অবস্থা (session, ক্যাশ) ভাগ হয় না — cluster-এ অ্যাপকে stateless লিখতে হয়।

তৃতীয় পরিস্থিতি হলো ব্যবহারকারীর অমিল: PM2 যে ব্যবহারকারী হিসেবে চলে, `pm2 startup` সেই ব্যবহারকারীর জন্যই unit তৈরি করে। root হিসেবে `pm2 startup` চালিয়ে technova ব্যবহারকারীর অ্যাপ চালানোর চেষ্টা করলে বুটে অ্যাপ উঠবে না — startup কমান্ডটি অ্যাপের ব্যবহারকারী হিসেবেই চালাতে হয়। চতুর্থ পরিস্থিতি হলো পরিবেশ চলক: ecosystem ফাইলে `env_file` দিয়ে `.env` লোড করা যায়, কিন্তু সেই ফাইলের অনুমতি 600 রাখতে হয় এবং Git-এ তোলা যাবে না। পঞ্চম পরিস্থিতি হলো PM2 daemon-এর নিজের ব্যর্থতা: daemon মারা গেলে `pm2 list` ফাঁকা দেখাতে পারে, যদিও অ্যাপ প্রক্রিয়াগুলো বেঁচে আছে — তখন `pm2 resurrect` দিয়ে অবস্থা ফিরিয়ে আনতে হয়।

পরিস্থিতি	PM2 আচরণ	systemd আচরণ
অ্যাপ বন্ধ করা	SIGINT, তারপর অপেক্ষা, তারপর SIGKILL	SIGTERM, তারপর TimeoutStopSec-এর পর SIGKILL
একটি instance ক্র্যাশ	সেই instance পুনরায় চালু হয়	পুরো সার্ভিস পুনরায় চালু হয়
বুটে চালু	শুধু startup + save থাকলে	enable থাকলেই
মেমোরি সীমা ছাড়ানো	max_memory_restart-এ instance restart	MemoryMax-এ প্রক্রিয়া থামতে পারে

১১.৭ গুরুত্বপূর্ণ কমান্ড ও বিশ্লেষণ

```
npm install -g pm2              # PM2 ইনস্টল (ব্যবহারকারী হিসেবে, root এড়িয়ে চলুন)
pm2 start server.js --name technova-api -i max   # ক্লাস্টার মোডে চালু
pm2 start ecosystem.config.js   # ecosystem ফাইল থেকে চালু (সুপারিশকৃত)
pm2 list                        # সব অ্যাপের অবস্থা, restart সংখ্যা, মেমোরি
pm2 monit                       # লাইভ CPU/মেমোরি মনিটর
pm2 logs technova-api           # লগ দেখা (লাইন সংখ্যা সীমিত রাখুন)
pm2 reload technova-api         # zero-downtime reload — একবারে একটি instance
pm2 restart technova-api        # সব instance একসঙ্গে পুনরায় চালু (বিরতি হবে)
pm2 stop technova-api && pm2 delete technova-api
pm2 startup                     # বুটে চালুর নির্দেশনা দেয় — আউটপুটের কমান্ডটি চালাতেই হবে
pm2 save                        # বর্তমান অ্যাপ তালিকা সংরক্ষণ (বুটে ফিরিয়ে আনতে)
```

`reload` বনাম `restart` গুলিয়ে ফেলবেন না: reload ধীরে ধীরে বদলায়, restart সব একসঙ্গে থামায়।

```
pm2 describe technova-api       # একটি অ্যাপের বিস্তারিত: restart সংখ্যা, মেমোরি, পরিবেশ
pm2 flush technova-api          # লগ ফাইল খালি করা (ডিস্ক পূর্ণ হলে, ল্যাবে)
pm2 reset technova-api          # restart সংখ্যার হিসাব শূন্য করা
pm2 resurrect                   # সংরক্ষিত অ্যাপ তালিকা ফিরিয়ে আনা
pm2 update                      # PM2 নিজেকে হালনাগাদ করা (ডেপ্লয়ের আগে নয়, আলাদা সময়ে)
```

`pm2 describe` হলো একটি অ্যাপের স্বাস্থ্য পরীক্ষা — restart সংখ্যা হঠাৎ বেড়ে গেলে বুঝবেন অ্যাপটি বারবার মরছে।

১১.৮ কনফিগারেশন উদাহরণ

```
// ecosystem.config.js — /opt/technova/app/ecosystem.config.js
module.exports = {
  apps: [{
    name: 'technova-api',
    script: './server.js',
    instances: 'max',
    exec_mode: 'cluster',
    env: {
      NODE_ENV: 'production',
      PORT: 3000
    },
    env_production: {
      NODE_ENV: 'production',
      PORT: 3000
```

```
    },
    error_file: './logs/err.log',
    out_file: './logs/out.log',
    log_date_format: 'YYYY-MM-DD HH:mm:ss',
    max_memory_restart: '500M',   // মেমোরি 500MB ছাড়ালে instance পুনরায় চালু
    autorestart: true,
    watch: false,                 // প্রোডাকশনে watch বন্ধ রাখুন
    max_restarts: 10
  }]
};
```

আর systemd-এর অধীনে PM2 চালাতে `pm2 startup` যে unit তৈরি করতে বলে, সেটি যাচাই করুন — মূল ধারণা: systemd PM2 daemon চালায়, PM2 অ্যাপগুলো চালায়। `watch: false` মনে রাখুন: প্রোডাকশনে ফাইল বদলালে অ্যাপ নিজে থেকে restart হওয়া বিপজ্জনক।

১১.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১: ট্র্যাফিক বাড়ায় একটি কোর ১০০% ব্যবহার হচ্ছে, বাকি তিনটি বসে আছে — ক্লাস্টার মোডে না চালানোর ফল। পরিস্থিতি ২: ডেপ্লয়ের সময় ৩০ সেকেন্ড সাইট ডাউন — systemd restart ব্যবহার করা হয়েছে, PM2 reload নয়। পরিস্থিতি ৩: সার্ভার রিবুটের পর অ্যাপ ওঠেনি — `pm2 startup` করা হয়নি বা `pm2 save` করা হয়নি; দুটোই দরকার। পরিস্থিতি ৪: মেমোরি লিক — একটি instance ধীরে ধীরে মেমোরি খাচ্ছে; `max_memory_restart` তাকে স্বয়ংক্রিয়ভাবে ফিরিয়ে আনে, কিন্তু লিকের কারণ খুঁজে ঠিক করাই আসল সমাধান।

> **REAL WORLD — TechNova:** TechNova-র API cluster মোডে চারটি instance নিয়ে চলছিল, কিন্তু ব্যবহারকারীরা মাঝে মাঝে লগইন করে পরের পাতায় গিয়ে আবার লগআউট হয়ে যাচ্ছিল। কারণ ছিল in-memory session: একজন ব্যবহারকারী instance ১-এ লগইন করলে তার session শুধু সেই instance-এই থাকত; পরের অনুরোধ instance ৩-এ গেলে সে session চিনত না। দলটি session Redis-এ সরিয়ে দেয়। শিক্ষা: cluster মোডে যাওয়ার আগে ভাগ করা অবস্থা (session, ক্যাশ, আপলোড) কোথায় থাকে তা তালিকা করুন।

১১.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

> **LAB:** শুধু disposable VM-এ অনুশীলন করুন।

১. PM2 ইনস্টল করুন। ২. উপরের ecosystem ফাইল দিয়ে অ্যাপ চালু করুন, `pm2 list`-এ কয়টি instance দেখুন (কোর সংখ্যা অনুযায়ী)। ৩. `curl http://127.0.0.1:3000/health` দিয়ে যাচাই। ৪. কোডে ছোট পরিবর্তন করে `pm2 reload technova-api` দিন — reload-এর সময় বারবার curl চালিয়ে দেখুন উত্তর বন্ধ হয় কি না (হওয়া উচিত নয়)। ৫. এবার `pm2 restart` দিয়ে পার্থক্য অনুভব করুন। ৬. `pm2 startup`-এর নির্দেশনা অনুসরণ করে `pm2 save` করুন, তারপর VM রিবুট দিয়ে যাচাই করুন অ্যাপ ফিরে আসে কি না।

১১.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল ধারণা ১: "PM2 আর systemd একই জিনিস, যেকোনো একটি নিলেই হয়" — না, তারা ভিন্ন স্তরের; PM2 Node-বিশেষ, systemd সিস্টেম-ব্যাপী। ভুল ২: `pm2 startup` না করে ভাবা যে বুটে উঠবে। ভুল ৩: `pm2 save` না করা — startup থাকলেও সংরক্ষিত তালিকা ছাড়া কিছু ফিরবে না। ভুল ৪: প্রোডাকশনে `watch: true` রাখা। ভুল ৫: cluster মোডে

in-memory session রাখা — একটি instance-এ লগইন করলে অন্য instance চিনবে না; session Redis বা ডেটাবেসে রাখুন, অথবা sticky session ব্যবহার করুন। ভুল ৬: PM2 দিয়ে PostgreSQL/Nginx চালানোর চেষ্টা।

> **WARNING:** Cluster মোডে অ্যাপ যদি ফাইলে লেখে (যেমন আপলোড), প্রতিটি instance আলাদা জায়গায় লিখতে পারে বা সংঘর্ষ হতে পারে — আপলোডের জন্য shared storage বা object storage ব্যবহার করুন।

১১.১২ নিরাপত্তা বিবেচনা

PM2 root হিসেবে চালাবেন না — অ্যাপ ব্যবহারকারী (technova) হিসেবে চালান। Ecosystem ফাইলে গোপন তথ্য লিখবেন না; পরিবেশ চলক বা EnvironmentFile ব্যবহার করুন। PM2-এর মনিটরিং ড্যাশবোর্ড (PM2 Plus) ব্যবহার করলে তার সংযোগ তথ্য গোপন রাখুন। লগ ফাইলে পাসওয়ার্ড বা টোকেন ছাপা হচ্ছে কি না যাচাই করুন — লগে গোপন তথ্য থাকা মানে তা ডিস্কে পড়ে থাকা।

১১.১৩ সমস্যা নির্ণয় (Troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
বুটে PM2 অ্যাপ ওঠে না	startup/save হয়নি	`pm2 startup` নির্দেশনা ও `pm2 save` যাচাই; systemctl status দেখুন
reload-এও ডাউনটাইম	অ্যাপ চালু হতে দেরি / health check নেই	অ্যাপের startup সময় মাপুন; listen-এর আগে দীর্ঘ কাজ এড়ান
এক instance বারবার restart	মেমোরি লিক / ক্র্যাশ	`pm2 logs` ; max_memory_restart যাচাই; লিক খুঁজুন
Session হারিয়ে যাচ্ছে	in-memory session + cluster	Redis/DB-তে session সরান
লগ ফাইল বিশাল	logrotate নেই	pm2-logrotate সেট করুন; লগ স্তর কমান
pm2 list ফাঁকা, কিন্তু অ্যাপ চলছে	PM2 daemon মারা গেছে	`pm2 resurrect` ; daemon লগ দেখুন; `pm2 save` করা ছিল কি না যাচাই করুন
reload-এর সময় কিছু অনুরোধ ভাঙছে	অ্যাপ graceful shutdown করছে না	সংকেত পেয়ে সংযোগ বন্ধ করার কোড যোগ করুন; kill_timeout যাচাই করুন
startup-এর পরও বুটে অ্যাপ ওঠে না	ভুল ব্যবহারকারী হিসেবে startup চালানো	অ্যাপের ব্যবহারকারী হিসেবে `pm2 startup` আবার চালান; unit ফাইলের User দেখুন
এক instance-এ লগইন, অন্য instance-এ অচেনা	in-memory session + cluster	session Redis বা ডেটাবেসে সরান; অথবা sticky session ব্যবহার করুন

১১.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: ecosystem ফাইল Git-এ রাখা; PM2-কে systemd-এর অধীনে চালানো; `pm2 reload` দিয়ে ডেপ্লয়; `max_memory_restart` সেট করা; লগ rotate করা; cluster-এ stateless অ্যাপ লেখা (session বাইরে)। অ্যান্টি-প্যাটার্ন:

PM2-কে systemd-এর বিকল্প ভাবা; root হিসেবে PM2 চালানো; `pm2 restart` দিয়ে প্রতিদিন ডেপ্লয়; in-memory session নিয়ে cluster চালানো; লগ না দেখে বারবার restart দেওয়া।

১১.১৫ অধ্যায় সারাংশ

PM2 Node.js-এর জন্য গভীর সরঞ্জাম — ক্লাস্টারিং, zero-downtime reload, ecosystem ফাইল, লগ ও মনিটরিং। systemd সিস্টেমের জন্য গভীর সরঞ্জাম — PID 1, যেকোনো সার্ভিস, বুট, journald, resource limit, sandboxing। তারা পরস্পরের বিকল্প নয়। সেরা কাঠামো: systemd PM2-কে চালায়, PM2 Node অ্যাপগুলোকে চালায়, আর সিস্টেম সার্ভিসগুলো সরাসরি systemd-এ থাকে।

১১.১৬ মূল প্রযুক্তিগত পরিভাষা

শব্দ	অর্থ
PM2	Node.js অ্যাপের প্রক্রিয়া ব্যবস্থাপক — ক্লাস্টার, reload, লগ, মনিটরিং।
Cluster Mode	একই অ্যাপের একাধিক instance চালিয়ে সব CPU কোর ব্যবহার।
Zero-Downtime Reload	সংযোগ না ভেঙে একবারে একটি instance বদলে নতুন কোডে যাওয়া।
Ecosystem File	PM2 অ্যাপের ঘোষণামূলক কনফিগ (ecosystem.config.js)।
pm2 startup / pm2 save	বুটে PM2 চালু করার সংযোগ ও অ্যাপ তালিকা সংরক্ষণ।
Sticky Session	একই ব্যবহারকারীকে সবসময় একই instance-এ পাঠানো।

১১.১৭ জ্ঞান যাচাই প্রশ্ন

1. PM2 ও systemd কেন পরস্পরের বিকল্প নয় — দুটি করে শক্তি ও সীমা দিয়ে ব্যাখ্যা করুন।
2. `pm2 reload` ও `pm2 restart`-এর পার্থক্য কী? ডেপ্লয়ে কোনটি ব্যবহার করবেন?
3. Cluster মোডে in-memory session কেন সমস্যা করে? সমাধান কী?
4. `pm2 startup` ও `pm2 save` — দুটোরই কেন দরকার?
5. systemd-এ Node ক্লাস্টারিং করতে চাইলে কী করতে হয়, এবং PM2 কেন সহজ?
6. কখন শুধু systemd যথেষ্ট, কখন PM2 যোগ করা উচিত?
7. Graceful shutdown বলতে কী বোঝায়? reload-এর সময় এটি না থাকলে ব্যবহারকারী কী অনুভব করবে?
8. Cluster মোডে অ্যাপকে stateless হতে হয় কেন? একটি উদাহরণ দিয়ে ব্যাখ্যা করুন (session, ক্যাশ বা আপলোড)।
9. `max_memory_restart` কী করে এবং এটি কি মেমোরি লিকের সমাধান?

১১.১৮ ব্যবহারিক অনুশীলন

1. ল্যাবে একটি Node অ্যাপ PM2 cluster মোডে চালান; `pm2 list` ও `pm2 monit` দিয়ে instance সংখ্যা ও মেমোরি পর্যবেক্ষণ করুন।
2. reload বনাম restart-এর ডাউনটাইম মাপুন: reload-এর সময় প্রতি সেকেন্ডে curl চালিয়ে কোনো ব্যর্থতা আসে কি না লিখুন।

3. PM2-কে systemd-এর অধীনে বসান (startup + save), VM রিবুট দিয়ে যাচাই করুন।
4. ল্যাবে একটি অ্যাপ fork মোডে ও একটি অ্যাপ cluster মোডে চালিয়ে `pm2 list`-এর instance সংখ্যা তুলনা করুন; তারপর cluster অ্যাপে in-memory session রেখে দুটি অনুরোধে ভিন্ন instance-এর আচরণ পর্যবেক্ষণ করুন।
5. ইচ্ছাকৃতভাবে একটি instance-এ মেমোরি লিক তৈরি করে `max_memory_restart` আচরণ পর্যবেক্ষণ করুন (শুধু ল্যাবে)।

অধ্যায় ১২

Database Server Operations: PostgreSQL and MongoDB Fundamentals

১২.১ শেখার উদ্দেশ্য (Learning Objectives)

এই অধ্যায় শেষে আপনি পারবেন: PostgreSQL (relational) ও MongoDB (document) — দুই ধরনের ডেটাবেসের মূল পার্থক্য ব্যাখ্যা করতে; Ubuntu সার্ভারে দুটোই ইনস্টল, চালু ও নিরাপদ করতে; ব্যবহারকারী, ডেটাবেস ও অনুমতি তৈরি করতে; `psql` ও `mongosh` দিয়ে মৌলিক কাজ করতে; ব্যাকআপ ও পুনরুদ্ধারের (restore) ধাপ জানতে; এবং সংযোগ নিরাপত্তা (`pg_hba.conf`, bind address) সঠিকভাবে সেট করতে।

১২.২ প্রোডাকশনে কেন এটি গুরুত্বপূর্ণ

অ্যাপ মারা গেলে আবার চালু করা যায়; ডেটা হারালে ফিরিয়ে আনা যায় না। ডেটাবেস হলো প্রোডাকশনের সবচেয়ে মূল্যবান অংশ — ব্যবহারকারীর অ্যাকাউন্ট, অর্ডার, পেমেন্ট সব এখানে। ভুলভাবে খোলা ডেটাবেস (ইন্টারনেটে উন্মুক্ত পোর্ট, দুর্বল পাসওয়ার্ড, ব্যাকআপ ছাড়া) হলো সবচেয়ে সাধারণ ও সবচেয়ে ক্ষতিকর ভুলগুলোর একটি। এই অধ্যায়ে আপনি শিখবেন কীভাবে ডেটাবেস চালু করতে হয়, তার চেয়েও বেশি — কীভাবে তাকে নিরাপদ ও ফিরিয়ে আনার যোগ্য রাখতে হয়।

১২.৩ পূর্বশর্ত জ্ঞান (Prerequisite Knowledge)

SQL-এর মৌলিক ধারণা (SELECT, INSERT), JSON-এর ধারণা, অধ্যায় ৭-এর systemd, অধ্যায় ১০-এর পরিবেশ চলক (DATABASE_URL) জানা প্রয়োজন। MongoDB-এর সঙ্গে আপনার আগের অভিজ্ঞতা থাকলে ভালো হয়; PostgreSQL নতুন হলেও চলবে — পার্থক্যগুলো এখানেই ব্যাখ্যা করা হয়েছে।

১২.৪ প্রথম নীতি থেকে ব্যাখ্যা (First Principles)

ডেটাবেসের মূল কাজ: ডেটা এমনভাবে রাখা যাতে তা হারায় না, ভুল হয় না এবং প্রয়োজনে দ্রুত পাওয়া যায়। Relational ডেটাবেস (PostgreSQL) ডেটাকে টেবিলে ভাগ করে — সারি (row) ও কলাম (column) — এবং টেবিলগুলোর মধ্যে সম্পর্ক (relation) বজায় রাখে। এটি ACID নিশ্চয়তা দেয়: একটি লেনদেন (transaction) হয় সম্পূর্ণ হবে, নয়তো একেবারেই হবে না — মাঝামাঝি নয়। Document ডেটাবেস (MongoDB) ডেটাকে JSON-এর মতো নথি (document) হিসেবে রাখে — একটি অর্ডার ও তার সব আইটেম একটিই নথিতে থাকতে পারে। এটি নমনীয় (schema নমনীয়) ও দ্রুত বিকাশে সুবিধাজনক, কিন্তু জটিল সম্পর্কের ক্ষেত্রে সাবধানে নকশা করতে হয়। কোনটি "ভালো" — তা নির্ভর করে ডেটার আকৃতির ওপর, প্রচলনের ওপর নয়। আপনার পরিচিত MongoDB যেমন আছে, তেমনি PostgreSQL শেখা মানে দুই ধরনের চিন্তা জানা।

১২.৫ আর্কিটেকচার / ধারণাগত চিত্র

দুই ডেটাবেস, এক সার্ভার (ধারণাগত):

Node অ্যাপ (127.0.0.1:3000) ──→ PostgreSQL :5432 (localhost) — টেবিল: users, orders, order_items

Node অ্যাপ ──→ MongoDB :27017 (localhost) — Collection: products, sessions, logs

দুটোই systemd সার্ভিস; দুটোই শুধু localhost-এ শোনে; বাইরে থেকে সরাসরি পৌঁছানো যায় না।

ব্যাকআপ: pg_dump (PostgreSQL) / mongodump (MongoDB) → সংরক্ষিত ফাইল → অন্য স্থানে কপি।

PostgreSQL-এ সম্পর্ক টেবিলে; MongoDB-তে সম্পর্ক নথির ভেতরে বা reference-এ।

১২.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

PostgreSQL: ইনস্টল ও প্রথম ধাপ

Ubuntu 24.04-এ `sudo apt install postgresql` দিলে সার্ভার বসে ও systemd সার্ভিস হিসেবে চালু হয়। ইনস্টলের সময় `postgres` নামে একটি সিস্টেম ব্যবহারকারী ও ডেটাবেস সুপারইউজার তৈরি হয়। `sudo -u postgres psql` দিয়ে তার শেলে ঢোকা যায়। সেখানে `CREATE DATABASE technova;` ও `CREATE USER technova WITH PASSWORD 'CHANGE-ME-2026';` দিয়ে অ্যাপের জন্য আলাদা ব্যবহারকারী তৈরি করুন, তারপর `GRANT ALL PRIVILEGES ON DATABASE technova TO technova;` দিন। অ্যাপ যেন কখনো superuser হিসেবে না চলে — এটিই নিরাপত্তার মূল নিয়ম।

pg_hba.conf — কে, কোথা থেকে, কীভাবে ঢুকবে

PostgreSQL-এ সংযোগের অনুমতি নিয়ন্ত্রণ করে `pg_hba.conf` (host-based authentication)। প্রতিটি লাইন বলে: কোন ধরনের সংযোগ (local/host), কোন ডেটাবেস, কোন ব্যবহারকারী, কোন ঠিকানা থেকে, কোন প্রমাণীকরণ পদ্ধতিতে (যেমন `scram-sha-256` — পাসওয়ার্ড যাচাইয়ের নিরাপদ পদ্ধতি)। `trust` মানে পাসওয়ার্ড ছাড়াই ঢুকতে দেওয়া — প্রোডাকশনে কখনো ব্যবহার করবেন না। ফাইল বদলালে `SELECT pg_reload_conf();` বা সার্ভিস reload দিতে হয়।

MongoDB: ইনস্টল ও প্রথম ধাপ

MongoDB Ubuntu-র ডিফল্ট রিপোজিটরিতে সবসময় থাকে না; সাধারণত MongoDB-এর নিজস্ব রিপোজিটরি যোগ করে ইনস্টল করা হয় (সরকারি নির্দেশনা অনুসরণ করুন)। ইনস্টলের পর `mongod` সার্ভিস হিসেবে চলে। গুরুত্বপূর্ণ: ডিফল্টভাবে পুরনো সংস্করণে প্রমাণীকরণ (authentication) বন্ধ থাকতে পারে — প্রথম কাজ `mongosh`-এ admin ব্যবহারকারী তৈরি করে `mongod.conf`-এ `security.authorization: enabled` চালু করা। `bindIp: 127.0.0.1` রাখুন — MongoDB যেন শুধু localhost-এ শোনে।

ব্যাকআপের নীতি

ব্যাকআপ না থাকলে ডেটাবেস প্রোডাকশনে নেওয়া যায় না। PostgreSQL-এ `pg_dump` একটি ডেটাবেসের SQL ডাম্প দেয়; MongoDB-তে `mongodump` BSON ডাম্প দেয়। ব্যাকআপ নেওয়া শুধু অর্ধেক কাজ — পুনরুদ্ধার (restore) পরীক্ষা না করলে ব্যাকআপ আছে কি না জানা যায় না। নিয়মিত restore পরীক্ষা করুন — ল্যাবে, আসল ডেটা মুছে নয়।

ব্যাকআপ, পুনরুদ্ধার ও ব্যবহারকারী-ভূমিকা — গভীরে

ব্যাকআপের ক্ষেত্রে দুটি প্রশ্ন সবচেয়ে গুরুত্বপূর্ণ: কী ব্যাকআপ নিচ্ছেন, আর তা ফিরিয়ে আনা যায় কি না। PostgreSQL-এ `pg_dump` ডিফল্টভাবে একটি SQL টেক্সট ফাইল দেয় — এটি পড়া যায় ও ছোট ডেটাবেসে ভালো। বড় ডেটাবেসের জন্য `pg_dump -Fc` (custom format) ব্যবহার করা হয়, যা সংকুচিত হয় এবং `pg_restore` দিয়ে ফিরিয়ে আনা যায়; আর শুধু ডেটা ছাড়া কাঠামো (schema) চাইলে `pg_dump -s` ব্যবহার করা হয়। MongoDB-তে `mongodump`-এর পর `mongorestore` দিয়ে ফিরিয়ে আনা হয় — তবে `mongorestore` ডিফল্টভাবে থাকা নথি মুছে না, যোগ করে — পুরোপুরি ফিরিয়ে আনতে আগে ডেটাবেস মুছতে হয়, শুধু ল্যাবে। পুনরুদ্ধার পরীক্ষা করুন একটি আলাদা ল্যাব ডেটাবেসে, সারি গুনে মিলিয়ে — প্রোডাকশনে পরীক্ষামূলক restore কখনো নয়।

ব্যবহারকারী ও ভূমিকা (role) পরিচালনায় ন্যূনতম অনুমতির নীতি মেনে চলুন। PostgreSQL-এ `GRANT SELECT ON users TO reporter;` দিয়ে শুধু পড়ার অনুমতি দেওয়া যায়, আর `REVOKE DELETE ON orders FROM app_user;` দিয়ে একটি নির্দিষ্ট কাজের অনুমতি কেড়ে নেওয়া যায় — অ্যাপের ব্যবহারকারীকে কখনো `DROP` বা `ALTER`-এর অনুমতি দেবেন না, যদি না সত্যিই দরকার হয়। MongoDB-তে ভূমিকা আরও স্পষ্ট: `readWrite` ভূমিকা একটি ডেটাবেসে পড়া ও লেখার অনুমতি দেয়, `read` শুধু পড়ার, আর `dbAdmin` কাঠামো পরিচালনার — অ্যাপকে সাধারণত `readWrite` দেওয়াই যথেষ্ট, admin ভূমিকা নয়। ব্যাকআপ ফাইল নিজেও সংবেদনশীল — এতে ব্যবহারকারীর আসল ডেটা থাকে — তাই তার অনুমতি সীমিত রাখুন এবং সার্ভারের বাইরে অন্য স্থানে একটি কপি রাখুন।

> **WARNING:** `DROP DATABASE`, `dropDatabase()` বা `mongorestore --drop` চালানোর আগে তিনবার যাচাই করুন — কোন সার্ভারে আছেন, কোন ডেটাবেস, ব্যাকআপ আছে কি না এবং সেই ব্যাকআপ ফিরিয়ে আনা পরীক্ষা করা হয়েছে কি না। এই কমান্ডগুলো শুধু disposable ল্যাবে অনুশীলন করুন।

১২.৭ গুরুত্বপূর্ণ কমান্ড ও বিশ্লেষণ

```
sudo systemctl status postgresql mongod   # দুই সার্ভিসের অবস্থা
sudo -u postgres psql -c "\l"             # ডেটাবেসের তালিকা (\l = list)
sudo -u postgres psql -c "\du"            # ব্যবহারকারীর তালিকা
psql "postgresql://technova:CHANGE-ME-2026@127.0.0.1:5432/technova" -c "SELECT 1;"
mongosh --help                            # mongosh ব্যবহার
mongosh -u technova -p --authenticationDatabase admin
# mongosh-এর ভেতরে:
# use technova; db.products.insertOne({name: "Keyboard", price: 2500}); db.products.find();
pg_dump -U technova -h 127.0.0.1 technova > /tmp/technova-backup.sql
mongodump --uri="mongodb://technova:CHANGE-ME-2026@127.0.0.1:27017/technova" --out=/tmp/mongo-
backup
```

নমুনা পাসওয়ার্ড CHANGE-ME-2026 শুধু উদাহরণ — আসল পাসওয়ার্ড শক্তিশালী ও গোপন রাখুন।

```
pg_dump -Fc -U technova -h 127.0.0.1 technova > /tmp/technova-backup.dump   # সংকুচিত ব্যাকআপ
pg_restore -l /tmp/technova-backup.dump   # ব্যাকআপে কী কী আছে তালিকা দেখা (ফিরিয়ে না এনে)
psql -U technova -h 127.0.0.1 -c "\dt" technova   # টেবিলের তালিকা
mongosh -u technova -p --authenticationDatabase admin --eval 'db.getUsers()' technova
# MongoDB-তে ব্যবহারকারী ও ভূমিকার তালিকা দেখা
```

১২.৮ কনফিগারেশন উদাহরণ

```
# pg_hba.conf — নমুনা (আসল ফাইলে সাবধানে সম্পাদনা করুন; বদলানোর আগে ব্যাকআপ নিন)
# TYPE  DATABASE  USER      ADDRESS       METHOD
local   all       postgres                peer
local   technova  technova                scram-sha-256
host    technova  technova  127.0.0.1/32   scram-sha-256
host    all       all       0.0.0.0/0      reject
```

```
# /etc/mongod.conf — নমুনা অংশ
net:
  port: 27017
  bindIp: 127.0.0.1
security:
  authorization: enabled
storage:
  dbPath: /var/lib/mongodb
```

`bindIp: 127.0.0.1` মানে শুধু এই সার্ভার থেকে সংযোগ — ইন্টারনেট থেকে MongoDB পোর্ট খোলা থাকলে তা বিপর্যয় ডেকে আনে।

> **WARNING:** ডেটাবেস কনফিগ বা ব্যবহারকারী বদলানোর আগে ব্যাকআপ নিন। `DROP DATABASE` বা `dropDatabase()` চালানোর আগে তিনবার যাচাই করুন — কোন সার্ভারে আছেন, কোন ডেটাবেস, ব্যাকআপ আছে কি না। এই কমান্ডগুলো শুধু disposable ল্যাবে অনুশীলন করুন।

১২.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১: অ্যাপ সংযোগ পাচ্ছে না — `pg_hba.conf`-এ নিয়ম নেই, অথবা MongoDB-তে authorization চালু করার পর ব্যবহারকারী তৈরি করা হয়নি। পরিস্থিতি ২: ডিস্ক পূর্ণ — লগ ও ব্যাকআপে ডিস্ক ভরে ডেটাবেস লেখা বন্ধ করে দিয়েছে; ডিস্ক পর্যবেক্ষণ (monitoring) জরুরি। পরিস্থিতি ৩: হঠাৎ ধীরগতি — ইনডেক্স ছাড়া বড় টেবিলে query চলছে; `EXPLAIN` (PostgreSQL) বা `explain()` (MongoDB) দিয়ে query পরিকল্পনা দেখুন।

> **REAL WORLD — TechNova:** TechNova-র একবার একটি ভুল migration-এ `orders` টেবিলের একটি কলাম মুছে যায়। দলটি আতঙ্কিত হয়ে প্রোডাকশনে সরাসরি কিছু না করে প্রথমে ল্যাব সার্ভারে গত রাতের `pg_dump` ব্যাকআপ ফিরিয়ে আনে এবং টেবিলের সারি গুনে দেখে ব্যাকআপটি সম্পূর্ণ আছে। তারপর তারা প্রোডাকশনে বর্তমান অবস্থার একটি নতুন ব্যাকআপ নেয় (যাতে ভুল অবস্থাও সংরক্ষিত থাকে), এবং শুধু হারানো কলামটির ডেটা ল্যাব থেকে ফিরিয়ে আনে — পুরো ডেটাবেস মুছে নয়। শিক্ষা: পুনরুদ্ধারের আগে বর্তমান অবস্থার ব্যাকআপ নিন, যাতে ভুলের ওপর ভুল না হয়।

১২.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

> **LAB:** শুধু disposable VM-এ করুন। আসল ডেটা নিয়ে পরীক্ষা নয়।

১. PostgreSQL ইনস্টল করুন, `technova` ডেটাবেস ও ব্যবহারকারী তৈরি করুন। ২. একটি টেবিল তৈরি করুন (`CREATE TABLE users (id SERIAL PRIMARY KEY, email TEXT UNIQUE, created_at TIMESTAMPTZ DEFAULT now());`) ও দুটি সারি ঢোকান। ৩. `pg_dump` দিয়ে ব্যাকআপ নিন, তারপর টেবিল মুছে (`DROP TABLE users;` — ল্যাবে) ব্যাকআপ থেকে ফিরিয়ে আনুন। ৪. MongoDB-তে একইভাবে একটি collection-এ নথি ঢোকান, `mongodump` ও `mongorestore` অনুশীলন করুন। ৫. `bindIp` ও `pg_hba.conf` যাচাই করুন — বাইরের IP থেকে সংযোগ চেষ্টা করে দেখুন তা আটকায় কি না।

১২.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল ১: ডেটাবেস পোর্ট (5432/27017) ফায়ারওয়ালে ইন্টারনেটের জন্য খুলে দেওয়া। ভুল ২: `trust` প্রমাণীকরণ ব্যবহার করা। ভুল ৩: অ্যাপকে superuser/admin হিসেবে চালানো। ভুল ৪: ব্যাকআপ নেওয়া কিন্তু restore কখনো পরীক্ষা না করা। ভুল ৫: MongoDB-কে "schema নেই" ভেবে যাচ্ছেতাই নথি লেখা — নমনীয়তা মানে অগোছালো নয়; validation নিয়ম ব্যবহার করুন। ভুল ৬: PostgreSQL ও MongoDB-এর মধ্যে "কোনটি ভালো" নিয়ে তর্ক — প্রশ্ন হওয়া উচিত "কোন ডেটার জন্য কোনটি উপযুক্ত"।

১২.১২ নিরাপত্তা বিবেচনা

শুধু localhost-এ শোনা (bindIp / listen_addresses), শক্তিশালী পাসওয়ার্ড, প্রতিটি অ্যাপের জন্য আলাদা ব্যবহারকারী ও ন্যূনতম অনুমতি (least privilege), `scram-sha-256` প্রমাণীকরণ, ফায়ারওয়ালে ডেটাবেস পোর্ট বন্ধ, নিয়মিত আপডেট। ডেটাবেস সার্ভারে সরাসরি SSH ছাড়া অন্য কোনো প্রবেশপথ রাখবেন না। ব্যাকআপ ফাইলে গোপন ডেটা থাকে — ব্যাকআপও এনক্রিপ্ট করে ও নিরাপদ স্থানে রাখুন।

১২.১৩ সমস্যা নির্ণয় (Troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
Connection refused	সার্ভিস চলছে না / ভুল পোর্ট	`systemctl status postgresql mongod`; `ss -tlnp`
password authentication failed	ভুল পাসওয়ার্ড / pg_hba নিয়ম	pg_hba.conf যাচাই; পাসওয়ার্ড পুনরায় সেট
MongoDB: auth failed	authorization চালু, ব্যবহারকারী নেই	admin DB-তে ব্যবহারকারী তৈরি হয়েছে কি না দেখুন
ডিস্ক পূর্ণ	লগ / ব্যাকআপ / WAL বেড়েছে	`df -h`; পুরনো লগ ও ব্যাকআপ পরিষ্কার
Query ধীর	ইনডেক্স নেই	EXPLAIN / explain() দিয়ে পরিকল্পনা দেখুন
pg_restore-এ ভুল আসছে	ব্যাকআপ ফরম্যাট ও restore সরঞ্জামের অমিল	টেক্সট ডাম্প `psql` দিয়ে, custom ডাম্প `pg_restore` দিয়ে ফিরিয়ে আনুন
mongorestore-এ পুরনো নথি থেকে যাচ্ছে	ডিফল্টভাবে mongorestore মুছে না, যোগ করে	ল্যাবে `--drop` ব্যবহার করুন; প্রোডাকশনে আগে ব্যাকআপ নিন
reporter ব্যবহারকারী লিখতে পারছে	অতিরিক্ত অনুমতি দেওয়া হয়েছে	`REVOKE` দিয়ে অনুমতি কমান; `\dp` দিয়ে অনুমতি তালিকা দেখুন

ব্যাকআপ ফাইল পাওয়া যাচ্ছে না	ব্যাকআপ শুধু একই সার্ভারে রাখা ছিল	ব্যাকআপের একটি কপি অন্য স্থানে রাখুন; ব্যাকআপ স্ক্রিপ্টের লগ যাচাই করুন

১২.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: আলাদা ব্যবহারকারী ও ন্যূনতম অনুমতি; শুধু localhost; নিয়মিত ব্যাকআপ + restore পরীক্ষা; মনিটরিং (ডিস্ক, সংযোগ সংখ্যা); migration লিখিতভাবে করা। অ্যান্টি-প্যাটার্ন: ডেটাবেস ইন্টারনেটে খোলা; superuser দিয়ে অ্যাপ চালানো; ব্যাকআপ ছাড়া migration; প্রোডাকশন ডেটা ল্যাপটপে কপি করে রাখা; পাসওয়ার্ড কোডে লেখা।

১২.১৫ অধ্যায় সারাংশ

PostgreSQL সম্পর্কভিত্তিক, কঠোর ও লেনদেন-নিরাপদ; MongoDB নথিভিত্তিক, নমনীয় ও দ্রুত বিকাশে সুবিধাজনক। দুটোই systemd সার্ভিস হিসেবে চলে, দুটোই শুধু localhost-এ শোনা উচিত, দুটোতেই আলাদা ব্যবহারকারী ও শক্তিশালী প্রমাণীকরণ দরকার। আর সবচেয়ে গুরুত্বপূর্ণ: ব্যাকআপ নেওয়া ও তা ফিরিয়ে আনা পরীক্ষা করা — এ ছাড়া কোনো ডেটাবেসই প্রোডাকশনের যোগ্য নয়।

১২.১৬ মূল প্রযুক্তিগত পরিভাষা

শব্দ	অর্থ
PostgreSQL	ওপেন সোর্স relational ডেটাবেস; টেবিল ও ACID লেনদেন।
MongoDB	Document ডেটাবেস; JSON-সদৃশ নথি collection-এ রাখে।
ACID	লেনদেনের নিশ্চয়তা: Atomicity, Consistency, Isolation, Durability।
pg_hba.conf	PostgreSQL-এ কে কোথা থেকে কীভাবে ঢুকবে তার নিয়ম ফাইল।
mongod.conf	MongoDB সার্ভারের কনফিগ ফাইল।
pg_dump / mongodump	ব্যাকআপ নেওয়ার সরঞ্জাম (PostgreSQL / MongoDB)।

১২.১৭ জ্ঞান যাচাই প্রশ্ন

1. Relational ও document ডেটাবেসের মূল পার্থক্য কী? একটি উদাহরণে কোনটি বেছে নেবেন?
2. ACID বলতে কী বোঝায় এবং কেন এটি গুরুত্বপূর্ণ?
3. `pg_hba.conf`-এর একটি লাইন কী কী তথ্য দেয়?
4. MongoDB-তে `bindIp: 127.0.0.1` কেন গুরুত্বপূর্ণ?
5. অ্যাপকে ডেটাবেস superuser হিসেবে চালানো কেন ভুল?
6. ব্যাকআপ নেওয়ার পরও কেন restore পরীক্ষা করা জরুরি?
7. `pg_dump`-এর টেক্সট ফরম্যাট ও custom ফরম্যাট (-Fc)-এর মধ্যে পার্থক্য কী? কোনটি কখন ব্যবহার করবেন?
8. অ্যাপের ডেটাবেস ব্যবহারকারীকে শুধু `readWrite` বা সীমিত GRANT দেওয়া কেন জরুরি? অতিরিক্ত অনুমতির একটি ঝুঁকির উদাহরণ দিন।
9. PostgreSQL-এ `psql` ও MongoDB-তে `mongosh`-এর ভূমিকা কী?

১২.১৮ ব্যবহারিক অনুশীলন

1. ল্যাবে PostgreSQL-এ ডেটাবেস, ব্যবহারকারী ও একটি টেবিল তৈরি করুন; `pg_dump` দিয়ে ব্যাকআপ নিয়ে restore অনুশীলন করুন।
2. MongoDB-তে authentication চালু করে একটি ব্যবহারকারী তৈরি করুন; প্রমাণীকরণ ছাড়া সংযোগ ব্যর্থ হয় কি না যাচাই করুন।
3. `pg_hba.conf`-এ একটি ভুল নিয়ম যোগ করে সংযোগ ব্যর্থতা পর্যবেক্ষণ করুন, তারপর ঠিক করুন (ব্যাকআপ নিয়ে)।
4. ল্যাবে `pg_dump -Fc` দিয়ে ব্যাকআপ নিয়ে `pg_restore -l` দিয়ে তার তালিকা দেখুন, তারপর একটি আলাদা ল্যাব ডেটাবেসে ফিরিয়ে এনে টেবিলের সারি গুনে মেলান — প্রোডাকশনে নয়।
5. একটি ছোট Node স্ক্রিপ্ট লিখুন যা পরিবেশ চলক থেকে DATABASE_URL পড়ে PostgreSQL-এ সংযোগ করে — পাসওয়ার্ড কোডে না লিখে।

PART III • DEVOPS ENGINEERING MASTERY

Production Operations and Security

ডিস্ক ও পারফরম্যান্স বিশ্লেষণ, নেটওয়ার্কিং, ফায়ারওয়াল, নিরাপত্তা, ব্যাকআপ, cron ও লগ।

অধ্যায় ১৩

ডিস্ক ব্যবস্থাপনা, স্টোরেজ স্বাস্থ্য এবং ইনোড সমস্যা সমাধান

১৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) ডিস্ক, পার্টিশন, ফাইলসিস্টেম ও মাউন্টের সম্পর্ক ব্যাখ্যা করতে; (২) `df`, `du`, `lsblk`, `findmnt` দিয়ে জায়গা ও ব্যবহার নির্ণয় করতে; (৩) ইনোড (inode) কী এবং কেন ডিস্ক খালি থাকলেও ফাইল তৈরি বন্ধ হয়ে যেতে পারে তা বুঝতে; (৪) লগ, ক্যাশ ও অস্থায়ী ফাইল নিরাপদে পরিষ্কার করতে; (৫) ডিস্ক স্বাস্থ্য পরীক্ষা ও পূর্ণ ডিস্কের জরুরি প্রতিকার করতে।

১৩.২ প্রোডাকশনে কেন এটি গুরুত্বপূর্ণ

প্রোডাকশন সার্ভারের সবচেয়ে সাধারণ ও সবচেয়ে বিপজ্জনক ব্যর্থতাগুলোর একটি হলো ডিস্ক পূর্ণ হয়ে যাওয়া। ডাটাবেস লেখা বন্ধ হয়ে যায়, অ্যাপ্লিকেশন ক্র্যাশ করে, লগ লেখা যায় না বলে সমস্যার কারণই আর ধরা যায় না, আপডেট ব্যর্থ হয়। আরও খারাপ — ডিস্ক ১০০% পূর্ণ হলে অনেক সময় SSH লগইন পর্যন্ত সমস্যা করে। অর্থাৎ ডিস্ক ব্যবস্থাপনা কোনো “পরিষ্কার-পরিচ্ছন্নতার” কাজ নয়; এটি সার্ভার চালু রাখার মৌলিক শর্ত।

> **REAL WORLD:** একটি Next.js অ্যাপের সার্ভারে প্রতিদিনের লগ রোটেট হয়নি। ছয় মাসে `/var/log` ৩০ গিগাবাইট দখল করল। এক সকালে MongoDB লেখা বন্ধ করে দিল — ডিস্ক পূর্ণ। সমাধান ছিল পাঁচ মিনিটের, কিন্তু ডাউনটাইম ছিল দুই ঘণ্টার, কারণ কেউ আগে থেকে ডিস্ক মনিটরিং বসায়নি।

১৩.৩ পূর্বশর্ত জ্ঞান

লিনাক্সে ফাইল ও ডিরেক্টরি, `cd`/`ls`, `sudo`-এর ধারণা এবং টার্মিনালে কমান্ড চালানোর অভিজ্ঞতা থাকলেই চলবে। পার্টিশন বা ফাইলসিস্টেম আগে না জানলেও চলবে — এই অধ্যায়েই ব্যাখ্যা করা হয়েছে।

১৩.৪ প্রথম নীতি থেকে ব্যাখ্যা

ডিস্ক হলো স্থায়ী স্মৃতি — বিদ্যুৎ চলে গেলেও তথ্য থাকে। কিন্তু অপারেটিং সিস্টেম সরাসরি ডিস্কের “কাঁচা” জায়গা ব্যবহার করে না। প্রথমে ডিস্ককে ভাগ করা হয় **পার্টিশন** (partition — ডিস্কের আলাদা আলাদা অংশ) হিসেবে, তারপর প্রতিটি পার্টিশনে বসানো হয় **ফাইলসিস্টেম** (filesystem — ফাইল সাজিয়ে রাখার নিয়ম, যেমন ext4)। ফাইলসিস্টেম প্রতিটি ফাইলের জন্য একটি **ইনোড** (inode — ফাইলের পরিচয়পত্র: মালিক, অনুমতি, আকার, ডিস্কে অবস্থান; নাম নয়) বরাদ্দ করে। ফাইলের নাম থাকে ডিরেক্টরিতে, আর আসল তথ্য থাকে ইনোডের নির্দেশিত ব্লকে। তাই দুটি সম্পদই আলাদাভাবে শেষ হতে পারে: ব্লক (জায়গা) অথবা ইনোড (ফাইলের সংখ্যা)।

১৩.৫ স্থাপত্য / ধারণাগত চিত্র

ফিজিক্যাল ডিস্ক (SSD/HDD) → পার্টিশন (/dev/sda1, /dev/sda2) → ফাইলসিস্টেম (ext4) → মাউন্ট পয়েন্ট (/, /var, /home)

ফাইলসিস্টেমের ভেতরে: [ডিরেক্টরি: নাম → ইনোড নম্বর] → [ইনোড: মেটাডেটা + ডেটা ব্লকের ঠিকানা] → [ডেটা ব্লক: আসল তথ্য]

দুটি আলাদা সীমা: ব্লক ব্যবহার (df) এবং ইনোড ব্যবহার (df -i)

১৩.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

মাউন্ট (mount) মানে ফাইলসিস্টেমকে ডিরেক্টরি গাছের কোনো জায়গায় যুক্ত করা। `/` হলো রুট ফাইলসিস্টেম; `/var` আলাদা পার্টিশন হলে লগ ভরে গেলেও রুট বাঁচে — এটাই প্রোডাকশনে আলাদা পার্টিশনের যুক্তি। **ext4** হলো উবুন্টুর ডিফল্ট ফাইলসিস্টেম; এটি **জার্নালিং** (journaling — লেখার আগে পরিবর্তনের হিসাব রাখা, যাতে বিদ্যুৎ গেলে ফাইলসিস্টেম মেরামত করা যায়) করে। ডিস্ক পূর্ণ হলে প্রথমে দেখুন কোন পার্টিশন পূর্ণ, তারপর কোন ডিরেক্টরি বড়, তারপর কোন ফাইল বড় — এই ক্রমেই তদন্ত করতে হয়, উল্টো নয়।

ইনোড শেষ হওয়ার ঘটনা

লক্ষ লক্ষ ছোট ফাইল (সেশন ফাইল, ক্যাশ, মেইল কিউ) তৈরি হলে ইনোড শেষ হয়ে যায়, যদিও গিগাবাইট জায়গা খালি থাকে। তখন `No space left on device` ত্রুটি আসে — বার্তাটি বিভ্রান্তিকর, কারণ জায়গা আছে, ইনোড নেই। `df -i` দিয়ে এটি ধরা যায়।

ডিলিট করা কিন্তু খোলা ফাইল

কোনো চলমান প্রসেস যে ফাইল খুলে রেখেছে, সেটি ডিলিট করলেও ডিস্ক জায়গা ফেরত আসে না — প্রসেস বন্ধ না হওয়া পর্যন্ত। `lsof | grep deleted` দিয়ে এগুলো দেখা যায়। তাই লগ ফাইল ডিলিট করার চেয়ে সার্ভিস রিস্টার্ট বা `truncate` করা নিরাপদ।

কাজের উদাহরণ — TechNova-র আপলোড ডিরেক্টরি কেন ভরল

TechNova-র সার্ভার 203.0.113.10-এ এক সকালে অ্যালার্ট এল: রুট পার্টিশন ৯২% পূর্ণ। প্রথমে `df -h` দেখা গেল `/` পূর্ণ, কিন্তু `/var/log` মাত্র ২ গিগাবাইট। এরপর `du -sh /var/www/*` চালিয়ে দেখা গেল `/var/www/technova.example/uploads` একাই ৪০ গিগাবাইট — ব্যবহারকারীরা বড় ছবি আপলোড করছিলেন, আর পুরোনো অস্থায়ী থাম্বনেইল কখনো মোছা হয়নি। সমস্যাটি ডিস্কের নয়, নীতির: আপলোডের জন্য কোনো আকার-সীমা ছিল না, পুরোনো ফাইল মোছার নিয়ম ছিল না, আর আপলোড রুট পার্টিশনে ছিল। অস্থায়ী সমাধান ছিল পুরোনো থাম্বনেইল পরীক্ষা করে মোছা; স্থায়ী সমাধান ছিল আপলোড আলাদা পার্টিশনে নেওয়া, আপলোডের সর্বোচ্চ আকার বেঁধে দেওয়া এবং প্রতি রাতে ৩০ দিনের পুরোনো অস্থায়ী ফাইল পরিষ্কারের নির্ধারিত কাজ বসানো। এই উদাহরণের শিক্ষা: `df` বলে কোথায় ভরেছে, `du` বলে কে ভরেছে, আর স্থায়ী সমাধান সবসময় নীতি ও কাঠামোতে — শুধু ফাইল মোছায় নয়। একটি ডিরেক্টরি বড় হলে তার ভেতরে ঢুকে আবার `du` চালান; একবারে পুরো সার্ভার স্ক্যান করলে ধীর হয় এবং অপ্রয়োজনীয় ফাইলসিস্টেমও স্ক্যান হতে পারে, তাই `-xdev` দিয়ে একই ফাইলসিস্টেমের ভেতরে থাকুন।

টুল	কী দেখায়	সীমা
`df -h`	পার্টিশনভিত্তিক খালি জায়গা	কোন ফাইল দায়ী তা বলে না

`du -sh`	ডিরেক্টরির মোট আকার	বড় গাছে ধীর হতে পারে
`ncdu`	ইন্টার‍্যাক্টিভভাবে বড় ডিরেক্টরি ঘোরা	আলাদাভাবে ইনস্টল করতে হয়
`find -size`	আকার অনুযায়ী নির্দিষ্ট ফাইল খোঁজা	পুরো গাছ স্ক্যানে সময় লাগে

PRO TIP: বড় ফাইল খোঁজার সময় `du -ah /var | sort -rh | head -20` চালান — সবচেয়ে বড় ২০টি পথ এক নজরে দেখা যায়। তবে প্রোডাকশনে ব্যস্ত সময়ে পুরো `/` স্ক্যান করবেন না; ব্যস্ত ডিস্ক আরও ধীর হয়ে যাবে।

১৩.৭ গুরুত্বপূর্ণ কমান্ড ও বিশ্লেষণ

```
df -h              # প্রতিটি মাউন্ট করা ফাইলসিস্টেমের জায়গা (human-readable)
df -i              # ইনোড ব্যবহার — IUse% দেখুন
lsblk -f           # ডিস্ক, পার্টিশন ও ফাইলসিস্টেমের গাছ
du -sh /var/*      # /var-এর প্রতিটি ডিরেক্টরির মোট আকার
du -sh /var/log/* | sort -rh | head -20   # সবচেয়ে বড় ২০টি
find /var -xdev -type f -size +100M -exec ls -lh {} \;  # ১০০MB-র বড় ফাইল
lsof | grep deleted   # ডিলিট করা কিন্তু খোলা ফাইল
sudo smartctl -a /dev/sda   # ডিস্ক স্বাস্থ্য (smartmontools ইনস্টল থাকলে)
```

কমান্ড	কী দেখায়	কখন ব্যবহার করবেন
`df -h`	পার্টিশনভিত্তিক খালি জায়গা	“ডিস্ক পূর্ণ” সন্দেহ হলে প্রথমেই
`df -i`	ইনোড ব্যবহার	জায়গা থাকলেও ফাইল তৈরি না হলে
`du -sh`	ডিরেক্টরির মোট আকার	কোন ডিরেক্টরি দায়ী তা খুঁজতে
`lsblk -f`	ডিস্ক–পার্টিশন–ফাইলসিস্টেম মানচিত্র	নতুন ডিস্ক যুক্ত করলে

আরও কমান্ড

```
sudo ncdu /var                 # ইন্টার‍্যাক্টিভভাবে /var-এর বড় ডিরেক্টরি ঘোরা (q দিয়ে বের হন)
df -hT                         # ফাইলসিস্টেমের ধরনসহ (ext4 কি না) জায়গা দেখা
du -ah /var/log | sort -rh | head -20   # /var/log-এর সবচেয়ে বড় ২০টি পথ
find /var -xdev -type f -mtime +90 -size +10M -ls   # ৯০ দিনের পুরোনো, ১০MB-র বড় ফাইল
sudo truncate -s 0 /var/log/app.log   # চলমান সার্ভিসের লগ খালি করা (ডিলিট নয়)
sudo tune2fs -l /dev/sda1 | grep -i inode   # ফাইলসিস্টেমের ইনোড তথ্য দেখা
```

ব্যাখ্যা: `ncdu` নতুনদের জন্য সবচেয়ে সহজ — তীর-চিহ্ন দিয়ে ভেতরে ঢোকা যায়। `df -hT` বলে কোন পার্টিশন কোন ফাইলসিস্টেম, যা নতুন ডিস্ক যুক্ত করলে কাজে লাগে। `find -mtime +90` শুধু পুরোনো ফাইল দেখায়, তাই ভুলে নতুন ফাইল মোছার ঝুঁকি কমে — তবু মোছার আগে তালিকাটি চোখে দেখে নিন। `truncate` মনে রাখুন: এটি ফাইলের নাম ও ইনোড অক্ষত রেখে শুধু ভেতরের তথ্য খালি করে, তাই চলমান প্রসেস লেখা চালিয়ে যেতে পারে।

১৩.৮ কনফিগারেশন উদাহরণ

নতুন ডিস্ক যুক্ত করে মাউন্ট করার নিরাপদ ক্রম (ল্যাবে):

```
# সতর্কতা: mkfs ধ্বংসাত্মক — ভুল ডিভাইসে চালালে সব তথ্য মুছে যাবে।
# ল্যাবে, শুধু নতুন/খালি ডিস্কে, ডিভাইস নাম দুবার যাচাই করে চালান।
lsblk
sudo mkfs.ext4 /dev/sdb1
sudo mkdir -p /data
sudo mount /dev/sdb1 /data
# স্থায়ী মাউন্টের জন্য /etc/fstab-এ UUID ব্যবহার করুন (ডিভাইস নাম বদলাতে পারে):
UUID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx /data ext4 defaults 0 2
```

> **WARNING:** `mkfs` এবং `rm -rf` ধ্বংসাত্মক কমান্ড। প্রোডাকশনে চালানোর আগে ব্যাকআপ/স্ন্যাপশট নিন, ডিভাইস নাম `lsblk` দিয়ে দুবার যাচাই করুন এবং শুধু ডিসপোজেবল VM বা আইসোলেটেড ল্যাবে অনুশীলন করুন।

১৩.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — লগে ডিস্ক ভরা: `/var/log` বেড়ে গেছে। `journalctl --vacuum-size=500M` ও logrotate (অধ্যায় ২০) দিয়ে নিয়ন্ত্রণ করুন। **পরিস্থিতি ২ — আপলোড ডিরেক্টরি:** ব্যবহারকারীর আপলোড `/var/www`-এ জমে রুট পূর্ণ করছে — আপলোড আলাদা পার্টিশন বা অবজেক্ট স্টোরেজে নিন। **পরিস্থিতি ৩ — ডকার:** `docker system df` দেখায় অব্যবহৃত ইমেজ/ভলিউম গিগাবাইট দখল করেছে; `docker system prune` ল্যাবে যাচাই করে চালান।

পরিস্থিতি ৪ — TechNova-র /boot পূর্ণ, আপডেট ব্যর্থ: TechNova-র সার্ভার 203.0.113.10-এ কার্নেল আপডেট ব্যর্থ হলো — `/boot` ১০০% পূর্ণ, কারণ পুরোনো কার্নেল কখনো মোছা হয়নি। `df -h /boot` ও `dpkg --list | grep linux-image` দিয়ে দেখা গেল ছয়টি পুরোনো কার্নেল জমে আছে। সমাধান: বর্তমানে চলমান কার্নেলটি (`uname -r` দিয়ে যাচাই করে) রেখে `sudo apt autoremove --purge` চালানো হলো — হাতে কার্নেল ফাইল মুছবেন না, প্যাকেজ ব্যবস্থা দিয়ে মুছুন। এরপর আপডেট সফল হলো এবং ভবিষ্যতের জন্য স্বয়ংক্রিয় পুরোনো-কার্নেল পরিষ্কার চালু রাখা হলো। শিক্ষা: ছোট পার্টিশনগুলোও নজরে রাখুন; `/boot` পূর্ণ হলে আপডেট আটকে যায়, আর নিরাপত্তা প্যাচ বসানো যায় না।

১৩.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

ডিস্ক পূর্ণ — জরুরি তদন্ত: (১) `df -h` — কোন পার্টিশন ১০০%? (২) `df -i` — ইনোড কি শেষ? (৩) `du -sh /*` দিয়ে বড় ডিরেক্টরি খুঁজুন, তারপর ভেতরে ঢুকে `du -sh * | sort -rh | head`। (৪) বড় লগ পেলে ডিলিট না করে খালি করুন: `sudo truncate -s 0 /var/log/app.log` (চলমান সার্ভিসের ফাইল এভাবে খালি করা নিরাপদ)। (৫) `lsof | grep deleted` — খোলা-ডিলিট ফাইল থাকলে সংশ্লিষ্ট সার্ভিস রিস্টার্ট করুন। (৬) স্থায়ী সমাধান হিসেবে logrotate ও মনিটরিং বসান।

১৩.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল ১: "`df` ৫০% দেখাচ্ছে, তাই জায়গা আছে" — ইনোড আলাদাভাবে শেষ হতে পারে। ভুল ২: চলমান লগ ডিলিট করা — জায়গা ফেরত আসে না, সার্ভিস রিস্টার্ট লাগে। ভুল ৩: `rm -rf /tmp/*` অন্ধভাবে চালানো — কোনো অ্যাপ `/tmp`-এ গুরুত্বপূর্ণ ফাইল রাখতে পারে। ভুল ৪: রুট পার্টিশনে সবকিছু রাখা — লগ, ডাটাবেস ও আপলোড আলাদা না করলে একটিই অন্যটিকে মারবে। ভুল ৫: ডিস্ক ৯৫% পূর্ণ হওয়া পর্যন্ত অপেক্ষা করা — ৮০%-এ অ্যালার্ট বসানো উচিত।

১৩.১২ নিরাপত্তা বিবেচনা

ডিস্ক পূর্ণ হওয়া নিজেই একটি নিরাপত্তা ঝুঁকি — লগ লেখা বন্ধ হলে আক্রমণের প্রমাণ মুছে যায়। `/tmp` ও `/var/tmp` `noexec,nosuid,nodev` অপশনে মাউন্ট করলে সেখানে ম্যালওয়্যার চালানো কঠিন হয়। সংবেদনশীল ডেটা আছে এমন ডিস্কের ব্যাকআপ এনক্রিপ্ট করে রাখুন। অপরিচিত USB ডিস্ক সার্ভারে মাউন্ট করবেন না।

১৩.১৩ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
`No space left on device`, জায়গা আছে	ইনোড শেষ	`df -i`; ছোট ফাইল (সেশন/ক্যাশ) পরিষ্কার করুন
ডিলিট করেও জায়গা ফেরে না	খোলা ফাইল	`lsof | grep deleted`; সার্ভিস রিস্টার্ট
হঠাৎ ডিস্ক ধীর	ডিস্ক ব্যর্থতার শুরু / I/O চাপ	`smartctl -a`, `iotop`; ব্যাকআপ নিন
বুটে মাউন্ট ব্যর্থ	`/etc/fstab` ভুল	UUID যাচাই করুন; `mount -a` দিয়ে পরীক্ষা করুন

আরও লক্ষণ

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
`df` বলে জায়গা আছে, `du` বলে অনেক ব্যবহৃত	ডিলিট করা কিন্তু খোলা ফাইল, বা মাউন্টের নিচে লুকোনো ফাইল	`lsof | grep deleted`; মাউন্ট পয়েন্ট খালি অবস্থায় ভেতরে ফাইল আছে কি না দেখুন
`/boot` ১০০% পূর্ণ, আপডেট ব্যর্থ	পুরোনো কার্নেল জমেছে	`uname -r` দিয়ে চলমান কার্নেল রেখে `sudo apt autoremove --purge`
`/tmp` হঠাৎ ভরে যাচ্ছে	অ্যাপ অস্থায়ী ফাইল মুছছে না	`du -sh /tmp/* | sort -rh | head`; কোন অ্যাপ লিখছে, লগে দেখুন
SMART সতর্কবার্তা, ডিস্কে খারাপ সেক্টর	ডিস্ক ব্যর্থতার শুরু	সঙ্গে সঙ্গে ব্যাকআপ নিন (অধ্যায় ১৮), ডিস্ক বদলানোর পরিকল্পনা করুন

১৩.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: ৮০%-এ অ্যালার্ট, `/var/log` আলাদা পার্টিশন, logrotate সক্রিয়, নিয়মিত `smartctl` পরীক্ষা, UUID দিয়ে fstab। অ্যান্টি-প্যাটার্ন: ডিস্ক পূর্ণ হলে “আগে ডিলিট, পরে ভাবা”; প্রোডাকশনে `rm -rf` দিয়ে আন্দাজে পরিষ্কার; মনিটরিং ছাড়া বড় আপলোড চালু করা।

১৩.১৫ অধ্যায়ের সারাংশ

ডিস্ক মানে শুধু জায়গা নয় — পার্টিশন, ফাইলসিস্টেম, ইনোড ও মাউন্ট একসাথে কাজ করে। `df -h` ও `df -i` দুটোই দেখুন, বড় ডিরেক্টরি `du` দিয়ে খুঁজুন, চলমান ফাইল ডিলিট না করে truncate করুন, আর ডিস্ক পূর্ণ হওয়ার আগেই অ্যালার্ট বসান।

১৩.১৬ মূল প্রযুক্তিগত পরিভাষা

পার্টিশন — ডিস্কের ভাগ; **ফাইলসিস্টেম** — ফাইল সাজানোর নিয়ম (ext4); **ইনোড** — ফাইলের মেটাডেটা রেকর্ড; **মাউন্ট** — ফাইলসিস্টেমকে ডিরেক্টরি গাছে যুক্ত করা; **জার্নালিং** — লেখার হিসাব রাখা; **ব্লক** — ডিস্কে তথ্য রাখার একক; **SMART** — ডিস্কের নিজস্ব স্বাস্থ্য প্রতিবেদন।

১৩.১৭ জ্ঞান যাচাই প্রশ্ন

1. `df -h` ও `df -i`-এর পার্থক্য কী?
2. ডিস্কে জায়গা খালি থাকলেও কেন `No space left on device` আসতে পারে?
3. চলমান প্রসেসের খোলা ফাইল ডিলিট করলে জায়গা ফেরত আসে না কেন?
4. `/var/log` আলাদা পার্টিশনে রাখার সুবিধা কী?
5. ইনোড শেষ হলে কোন ধরনের ফাইলগুলো প্রথমে পরীক্ষা করবেন?
6. fstab-এ ডিভাইস নামের বদলে UUID ব্যবহার করা হয় কেন?
7. `du` ও `df`-এর ফলাফল না মিললে প্রথমে কী সন্দেহ করবেন?
8. `/boot` পূর্ণ হলে আপডেট কেন ব্যর্থ হয়?

১৩.১৮ ব্যবহারিক অনুশীলন

> **LAB:** সব অনুশীলন ডিসপোজেবল VM বা আইসোলেটেড ল্যাবে করুন, প্রোডাকশনে নয়।

1. ল্যাব VM-এ `df -h`, `df -i`, `lsblk -f` চালিয়ে নিজের সার্ভারের ডিস্ক মানচিত্র লিখুন।
2. ছোট ফাইল তৈরি করে ইনোড ব্যবহার বাড়ান, তারপর `df -i` দিয়ে পরিবর্তন পর্যবেক্ষণ করুন ও ফাইলগুলো মুছে ফেলুন।
3. একটি বড় লগ ফাইল বানিয়ে `truncate -s 0` দিয়ে খালি করুন এবং `df`-এ পার্থক্য দেখুন।
4. একটি স্ক্রিপ্ট লিখুন যা ডিস্ক ৮০% ছাড়ালে সতর্কবার্তা ছাপে।
5. ল্যাব VM-এ `ncdu` ইনস্টল করে সবচেয়ে বড় তিনটি ডিরেক্টরি খুঁজে বের করুন ও পরিষ্কারের পরিকল্পনা লিখুন।

অধ্যায় ১৪

সিপিইউ, মেমরি ও প্রসেস পারফরম্যান্স বিশ্লেষণ

১৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) লোড অ্যাভারেজ, CPU ব্যবহার ও মেমরি ব্যবহারের অর্থ ব্যাখ্যা করতে; (২) `top`/`htop`, `free`, `vmstat`, `ps` দিয়ে পারফরম্যান্স নির্ণয় করতে; (৩) CPU-বাউন্ড, মেমরি-বাউন্ড ও I/O-বাউন্ড সমস্যা আলাদা করতে; (৪) মেমরি লিক ও সোয়াপের লক্ষণ চিনতে; (৫) প্রসেসের অগ্রাধিকার ও সীমা বুঝতে।

১৪.২ প্রোডাকশনে কেন এটি গুরুত্বপূর্ণ

“সার্ভার ধীর” — এই অভিযোগের পেছনে সাধারণত তিনটি কারণের একটি থাকে: CPU শেষ, মেমরি শেষ, অথবা ডিস্ক I/O আটকে আছে। কারণ না জেনে RAM বাড়ানো বা CPU বাড়ানো টাকা নষ্ট করা। সঠিক নির্ণয় মানে সঠিক সমাধান — কখনো কোডের বাগ, কখনো ভুল কনফিগ, কখনো সত্যিই হার্ডওয়্যার সীমা। প্রোডাকশনে ধীর সার্ভার মানে ধীর ওয়েবসাইট, ব্যর্থ অনুরোধ, আর হারানো ব্যবহারকারী।

১৪.৩ পূর্বশর্ত জ্ঞান

প্রসেস (process — চলমান প্রোগ্রাম) ও থ্রেডের প্রাথমিক ধারণা, `ps` ও `top` চালানোর অভিজ্ঞতা থাকলে ভালো। RAM, CPU ও সোয়াপ সম্পর্কে বিস্তারিত এখানেই ব্যাখ্যা করা হয়েছে।

১৪.৪ প্রথম নীতি থেকে ব্যাখ্যা

CPU হলো হিসাবের কারখানা — এক সময়ে একটি কোর একটি কাজ করে। মেমরি (RAM) হলো কাজের টেবিল — চলমান প্রোগ্রাম ও তথ্য এখানে থাকে, বিদ্যুৎ গেলে মুছে যায়। ডিস্ক হলো গুদাম — স্থায়ী কিন্তু ধীর। কোনো প্রোগ্রাম চালাতে হলে তার তথ্য গুদাম (ডিস্ক) থেকে টেবিলে (RAM) আনতে হয়, তারপর CPU হিসাব করে। টেবিল ছোট হলে বারবার গুদামে যাতায়াত করতে হয় — এটাই **সোয়াপ** (swap — RAM পূর্ণ হলে তথ্য ডিস্কে সরিয়ে রাখা), যা অত্যন্ত ধীর। তাই পারফরম্যান্স বিশ্লেষণ মানে এই তিনটি সম্পদের মধ্যে কোনটি বাধা (bottleneck) তা খুঁজে বের করা।

১৪.৫ স্থাপত্য / ধারণাগত চিত্র

[ব্যবহারকারীর অনুরোধ] → [Node.js / Nginx প্রসেস] → CPU (হিসাব) + RAM (কাজের টেবিল) + ডিস্ক I/O (গুদাম)

RAM পূর্ণ → সোয়াপ (ডিস্ক) → সবকিছু ধীর | CPU ১০০% → অনুরোধ লাইনে দাঁড়ায় | I/O অপেক্ষা → প্রসেস “D” অবস্থায় আটকে থাকে

লোড অ্যাভারেজ: ১, ৫ ও ১৫ মিনিটে গড়ে কত প্রসেস CPU বা I/O-র জন্য অপেক্ষা করছে

১৪.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

লোড অ্যাভারেজ (load average) হলো চলমান বা অপেক্ষমাণ প্রসেসের গড় সংখ্যা। ৪ কোরের সার্ভারে লোড ৪.০ মানে CPU পুরোপুরি ব্যবহৃত হচ্ছে; ৮.০ মানে লাইন জমছে। এক কোরের সার্ভারে লোড ১.০-ই পূর্ণ। তাই লোডকে সবসময় কোর সংখ্যার সাথে তুলনা করুন (`nproc` দিয়ে কোর দেখুন)। **CPU ব্যবহারের** দুটি ভাগ: user (অ্যাপের হিসাব) ও system/

kernel (সিস্টেম কল)। **মেমরি** দেখতে `free -h`-এ "available" কলামটিই আসল — "used"-এ ক্যাশও ধরা থাকে, লিনাক্স খালি RAM ক্যাশ হিসেবে ব্যবহার করে, সেটা খারাপ নয়। সোয়াপ ব্যবহার শূন্যের কাছাকাছি থাকা উচিত; নিয়মিত সোয়াপ মানে RAM কম।

প্রসেসের অবস্থা

`ps aux`-এ `R` মানে চলছে, `S` মানে ঘুমাচ্ছে (অপেক্ষা করছে), `D` মানে ডিস্ক I/O-র জন্য আটকে আছে (খারাপ লক্ষণ), `Z` মানে জম্বি (zombie — মৃত কিন্তু বাবা-প্রসেস এখনো খবর নেয়নি)। অনেক `D` অবস্থা মানে ডিস্কই বাধা, CPU নয়।

মেমরি লিক

কোনো প্রোগ্রাম মেমরি নিয়ে আর ফেরত না দিলে ব্যবহার বাড়তেই থাকে — এটাই **মেমরি লিক** (memory leak)। লক্ষণ: রিস্টার্টের পর ধীরে ধীরে RAM বাড়ে, শেষে OOM-killer (আউট-অফ-মেমরি কিলার — RAM শেষ হলে কার্নেল যে প্রসেস মেরে ফেলে) কোনো প্রসেস মেরে দেয়। `dmesg | grep -i oom` দিয়ে ধরা যায়।

কাজের উদাহরণ — vmstat পড়ে বাধা চেনা

TechNova-র সার্ভার 203.0.113.10-এ দুপুরে সাইট ধীর হলো। `uptime` দেখাল লোড ৯.৫, কোর ৪টি — অর্থাৎ লাইন জমছে। কিন্তু কারণ কী? `vmstat 1 5`-এর আউটপুট পড়া হলো কলাম ধরে: `us` (user CPU) ৮৫% মানে অ্যাপ নিজেই হিসাব করছে; `sy` বেশি হলে কার্নেলের কাজ; `wa` (I/O wait) বেশি হলে ডিস্কের অপেক্ষা; `id` বেশি হলে CPU আসলে ফাঁকা। এই ক্ষেত্রে `us` ৮৫% ও `wa` মাত্র ২% — তাই বাধা CPU, ডিস্ক নয়। `top`-এ দেখা গেল একটি রিপোর্ট তৈরির প্রসেস একাই ৩২০% CPU খাচ্ছে: সেটি ডাটাবেসের সব রেকর্ড মেমরিতে এনে সাজাচ্ছিল। সাময়িক সমাধান ছিল প্রসেসটির অগ্রাধিকার কমানো (`renice`) যাতে ওয়েব অনুরোধ আগে চলে; স্থায়ী সমাধান ছিল রিপোর্টটি পাতায় পাতায় (pagination) ভাগ করা ও কম ট্রাফিকের সময়ে চালানো। বিপরীত উদাহরণটিও মনে রাখুন: `wa` ৬০% হলে CPU বাড়িয়ে লাভ নেই — তখন ডিস্ক বা ক্যাশ দেখতে হয়। তাই একটিমাত্র সংখ্যা (লোড বা CPU শতাংশ) দেখে সিদ্ধান্ত নেবেন না; `vmstat`-এর চারটি কলাম একসাথে পড়ুন, তারপর `top` দিয়ে দায়ী প্রসেস শনাক্ত করুন।

vmstat কলাম	বেশি হলে অর্থ	পরের ধাপ
`us` বেশি	অ্যাপের হিসাব ভারী	`top` — কোন প্রসেস, কোড দেখুন
`sy` বেশি	কার্নেল/সিস্টেম কলের চাপ	অনেক ছোট I/O বা সংযোগ হচ্ছে কি না দেখুন
`wa` বেশি	ডিস্কের অপেক্ষা	`iotop`, অধ্যায় ১৩ — ডিস্ক দেখুন
`si/so` (swap in/out) বেশি	RAM কম, সোয়াপ চলছে	`free -h`, মেমরি-ভারী প্রসেস খুঁজুন

১৪.৭ গুরুত্বপূর্ণ কমান্ড ও বিশ্লেষণ

```
uptime                    # লোড অ্যাভারেজ (১, ৫, ১৫ মিনিট)
nproc                     # CPU কোর সংখ্যা
top                       # সরাসরি CPU/মেমরি — q দিয়ে বের হন
htop                      # রঙিন, সহজ সংস্করণ (ইনস্টল: sudo apt install htop)
free -h                   # RAM ও সোয়াপ — available কলাম দেখুন
vmstat 1 5                # CPU, মেমরি, I/O, সোয়াপের চলমান চিত্র
ps aux --sort=-%mem | head -15   # সবচেয়ে বেশি মেমরি খাওয়া প্রসেস
ps aux --sort=-%cpu | head -15   # সবচেয়ে বেশি CPU খাওয়া প্রসেস
```

```
pidstat -u 1 3            # নির্দিষ্ট প্রসেসের CPU (sysstat প্যাকেজ)
dmesg | grep -i oom       # OOM-killer কোনো প্রসেস মেরেছে কি না
```

নমুনা আউটপুট — চিত্রণমূলক:

```
নমুনা আউটপুট — চিত্রণমূলক
$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.8Gi       1.2Gi       0.4Gi       0.1Gi       2.2Gi       2.3Gi
Swap:          2.0Gi          0B       2.0Gi
```

এখানে available ২.৩ গিগাবাইট — সার্ভার সুস্থ, যদিও “used” ১.২ গিগাবাইটের সাথে ক্যাশ মিশে আছে।

আরও কমান্ড

```
mpstat -P ALL 1 3        # প্রতি কোরের CPU ব্যবহার (sysstat প্যাকেজ)
pidstat -r 1 3            # প্রসেসভিত্তিক মেমরি ব্যবহারের চলমান চিত্র
sar -r 1 3                # মেমরি ও সোয়াপের ইতিহাসভিত্তিক চিত্র
sudo renice -n 10 -p 1234   # চলমান প্রসেসের অগ্রাধিকার কমানো (PID যাচাই করে)
nice -n 10 ./report.sh    # কম অগ্রাধিকারে নতুন কাজ চালানো
cat /proc/loadavg         # লোড অ্যাভারেজের কাঁচা মান দেখা
```

ব্যাখ্যা: `mpstat -P ALL` দেখায় একটি কোর ১০০% আর বাকিগুলো ফাঁকা কি না — একক-থ্রেডের কাজ এমনই হয়, তখন কোর বাড়িয়ে লাভ নেই। `sar` অতীতের তথ্য দেয়, তাই “গত রাতে কখন ধীর হয়েছিল” প্রশ্নের উত্তর এতে মেলে। `nice`-এর মান বেশি মানে অগ্রাধিকার কম — নামটি বিভ্রান্তিকর, মনে রাখুন: nice মান ১০ মানে “ভদ্রভাবে” পেছনে থাকা। `renice` চালানোর আগে PID `ps` দিয়ে দুবার যাচাই করুন, যাতে ভুল প্রসেসের অগ্রাধিকার না বদলায়।

১৪.৮ কনফিগারেশন উদাহরণ

Node.js অ্যাপের মেমরি সীমা বেঁধে দেওয়া (systemd সার্ভিসে):

```
[Service]
ExecStart=/usr/bin/node /var/www/technova.example/app.js
MemoryMax=1G
CPUQuota=150%
Restart=on-failure
```

`MemoryMax` একটি প্রসেসকে পুরো সার্ভারের RAM খেয়ে ফেলতে বাধা দেয়; `CPUQuota` ১৫০% মানে দেড় কোর পর্যন্ত ব্যবহার করতে পারবে। সীমা বসানোর আগে অ্যাপের স্বাভাবিক ব্যবহার মেপে নিন — খুব কম সীমা অ্যাপকে বারবার ক্র্যাশ করাবে।

১৪.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — হঠাৎ ধীর: লোড ১২, কোর ৪। `top`-এ একটি Node প্রসেস ৩০০% CPU খাচ্ছে — অসীম লুপ বা ভারী JSON পার্সিং সন্দেহ করুন, কোড দেখুন। **পরিস্থিতি ২ — ধীরে ধীরে মেমরি বাড়া:** সপ্তাহে RAM ৯০% ছোঁয় — মেমরি লিক; রিস্টার্ট সাময়িক, প্রোফাইলিং স্থায়ী সমাধান। **পরিস্থিতি ৩ — রাতে ব্যাকআপের সময় ধীর:** ব্যাকআপ I/O চাপ তৈরি করে — `ionice` দিয়ে ব্যাকআপের অগ্রাধিকার কমান, বা কম ট্রাফিকের সময়ে চালান।

পরিস্থিতি ৪ — TechNova-র রিপোর্টের সময় সাইট ধীর: TechNova-র অ্যাডমিন প্যানেলে প্রতি সোমবার সকাল ১০টায় মাসিক রিপোর্ট তৈরি হতো, আর ঠিক তখনই গ্রাহকদের সাইট ধীর হয়ে যেত। তদন্তে দেখা গেল রিপোর্টের প্রসেসটি একসাথে ৪ কোরের ৩টি দখল করে নিচ্ছে, আর মেমরি ৩ গিগাবাইট ছুঁয়ে সোয়াপ শুরু হচ্ছে। সমাধান ছিল তিনটি: রিপোর্ট রাত ২টায় সরিয়ে নেওয়া, systemd-তে `CPUQuota=100%` ও `MemoryMax=1G` বেঁধে দেওয়া, এবং রিপোর্টের প্রশ্নটি (query) পাতায় ভাগ করা। এরপর সোমবার সকালেও সাইট স্বাভাবিক থাকল। শিক্ষা: ভারী কাজের সময় ও সীমা — দুটোই পরিকল্পনার অংশ; শুধু "কাজটি চালানো" যথেষ্ট নয়, কখন ও কতটুকু সম্পদ নিয়ে চালাবেন, তা-ও ঠিক করতে হয়।

১৪.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

"সার্ভার ধীর" নির্ণয়: (১) `uptime` — লোড কত, কোর কত? (২) `free -h` — available কম কি? সোয়াপ ব্যবহৃত হচ্ছে? (৩) `top` — কোন প্রসেস CPU/মেমরি খাচ্ছে? (৪) `vmstat 1 5` — `wa` (I/O অপেক্ষা) বেশি হলে ডিস্কের সমস্যা, `us` বেশি হলে CPU-র। (৫) `dmesg | grep -i oom` — কিছু মারা গেছে কি? (৬) কারণ অনুযায়ী ব্যবস্থা: কোড ঠিক করা, সীমা বসানো, অথবা রিসোর্স বাড়ানো।

১৪.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল ১: "RAM ৯০% ব্যবহৃত, তাই RAM কম" — ক্যাশসহ হিসাব; available দেখুন। ভুল ২: লোড ২.০ দেখে আতঙ্ক — ৪ কোরে সেটা স্বাভাবিক। ভুল ৩: `kill -9` প্রথমেই চালানো — প্রসেসকে স্বাভাবিকভাবে থামার সুযোগ না দিয়ে তথ্য নষ্ট করা। ভুল ৪: সোয়াপ পুরো বন্ধ করে দেওয়া — RAM শেষ হলে তখন OOM-killer সরাসরি প্রসেস মারবে, সতর্ক সংকেত ছাড়াই। ভুল ৫: এক মুহূর্তের `top` দেখে সিদ্ধান্ত — কয়েক মিনিট পর্যবেক্ষণ করুন।

১৪.১২ নিরাপত্তা বিবেচনা

অস্বাভাবিক CPU ব্যবহার (হঠাৎ ১০০%, অজানা প্রসেস) ক্রিপ্টোমাইনিং ম্যালওয়্যারের লক্ষণ হতে পারে — অজানা প্রসেসের উৎস যাচাই করুন। `nice` ও সীমা দিয়ে একটি অ্যাপ পুরো সার্ভার দখল করতে পারবে না — এটি সেবা-ব্যাহত (DoS) আক্রমণের বিরুদ্ধেও সুরক্ষা। প্রসেস তালিকায় সংবেদনশীল তথ্য (পাসওয়ার্ড আর্গুমেন্টে) দেখা যেতে পারে — কমান্ড লাইনে গোপন তথ্য দেবেন না।

১৪.১৩ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
লোড বেশি, CPU বেশি	ভারী হিসাব / অসীম লুপ	`top`, প্রসেস শনাক্ত, কোড পরীক্ষা
লোড বেশি, CPU কম, wa বেশি	ডিস্ক I/O বাধা	`vmstat`, `iotop`; অধ্যায় ১৩ দেখুন
RAM বাড়তেই থাকে	মেমরি লিক	`ps --sort=-%mem`, OOM লগ, প্রোফাইলিং
অ্যাপ হঠাৎ বন্ধ	OOM-killer	`dmesg | grep -i oom`; সীমা ও RAM বাড়ান

আরও লক্ষণ

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
লোড বেশি, কিন্তু `top`-এ কোনো প্রসেস বেশি CPU খাচ্ছে না	I/O অপেক্ষা (D অবস্থা) বা অনেক ছোট প্রসেস	`vmstat`-এ `wa` দেখুন; `ps aux`-এ D অবস্থার প্রসেস গুনুন

জম্বি (Z) প্রসেস অনেক	বাবা-প্রসেস মৃত সন্তানের খবর নিচ্ছে না	বাবা-প্রসেস শনাক্ত করুন; প্রয়োজনে বাবা-প্রসেস রিস্টার্ট (ল্যাবে যাচাই করে)
সোয়াপ ব্যবহৃত হচ্ছে, যদিও RAM খালি দেখায়	সোয়াপিনেস (swappiness) বেশি, বা আগের চাপের অবশেষ	`free -h` ও `cat /proc/sys/vm/swappiness`; স্বাভাবিক হলে আতঙ্ক নয়
একটি প্রসেস ১০০% CPU, বাকি কোর ফাঁকা	একক-থ্রেডের কাজ	কোডে সমান্তরাল (parallel) করা যায় কি না দেখুন; কোর বাড়িয়ে লাভ নেই

১৪.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: লোড/CPU/RAM-এর বেসলাইন (স্বাভাবিক মান) জেনে রাখুন, অ্যালার্ট বসান, প্রতি সার্ভিসে মেমরি সীমা দিন, `kill -9`-এর আগে `SIGTERM` দিন। অ্যান্টি-প্যাটার্ন: কারণ না জেনে রিসোর্স বাড়ানো; সোয়াপ বন্ধ করে সমস্যা ঢাকা; একটিমাত্র `top` স্ন্যাপশটে সিদ্ধান্ত নেওয়া।

১৪.১৫ অধ্যায়ের সারাংশ

পারফরম্যান্সের তিনটি সম্পদ — CPU, RAM, I/O — আলাদাভাবে মাপুন। লোডকে কোরের সাথে, RAM-কে available দিয়ে, আর সোয়াপকে সতর্ক সংকেত হিসেবে পড়ুন। `top`, `free`, `vmstat` ও `ps` দিয়ে বাধা শনাক্ত করুন, তারপর কারণ অনুযায়ী ব্যবস্থা নিন।

১৪.১৬ মূল প্রযুক্তিগত পরিভাষা

লোড অ্যাভারেজ — অপেক্ষমাণ প্রসেসের গড়; **সোয়াপ** — RAM-এর বিকল্প হিসেবে ডিস্ক ব্যবহার; **বটলনেক** — সবচেয়ে ধীর অংশ; **OOM-killer** — RAM শেষ হলে প্রসেস মেরে ফেলা কার্নেল ব্যবস্থা; **মেমরি লিক** — ফেরত না দেওয়া মেমরি; **nice** — প্রসেসের অগ্রাধিকার মান; **জম্বি প্রসেস** — মৃত কিন্তু এখনো তালিকায় থাকা প্রসেস।

১৪.১৭ জ্ঞান যাচাই প্রশ্ন

1. ৪ কোরের সার্ভারে লোড অ্যাভারেজ ৮.০ মানে কী?
2. `free -h`-এ “available” কেন “free”-এর চেয়ে গুরুত্বপূর্ণ?
3. সোয়াপ ব্যবহার বাড়লে কী বোঝায়?
4. CPU-বাউন্ড ও I/O-বাউন্ড সমস্যা কীভাবে আলাদা করবেন?
5. OOM-killer কী এবং কখন সক্রিয় হয়?
6. `kill -9`-এর আগে সাধারণত কোন সিগন্যাল দেওয়া উচিত?
7. `vmstat`-এ `wa` ৬০% হলে CPU বাড়িয়ে লাভ হবে কি? কেন?
8. একটি প্রসেস ১০০% CPU খেলেও বাকি কোর ফাঁকা থাকতে পারে কেন?

১৪.১৮ ব্যবহারিক অনুশীলন

1. ল্যাব VM-এ `uptime`, `free -h`, `vmstat 1 5` চালিয়ে বেসলাইন মান লিখে রাখুন।
2. ইচ্ছাকৃতভাবে CPU-ভারী কমান্ড (যেমন `yes > /dev/null &`) চালিয়ে `top`-এ পর্যবেক্ষণ করুন, তারপর প্রসেসটি স্বাভাবিকভাবে থামান।

3. `ps aux --sort=-%mem | head` দিয়ে সবচেয়ে বেশি মেমরি খাওয়া ৫টি প্রসেস চিহ্নিত করুন।
4. একটি systemd সার্ভিসে `MemoryMax` সীমা বসিয়ে পরীক্ষা করুন।
5. `mpstat -P ALL 1 3` চালিয়ে প্রতি কোরের ব্যবহার লিখুন এবং একটি কোর বেশি ব্যবহৃত হলে তার কারণ অনুমান করুন।

অধ্যায় ১৫

লিনাক্স সার্ভার নেটওয়ার্কিং ও নেটওয়ার্ক সমস্যা সমাধান

১৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) IP ঠিকানা, সাবনেট, গেটওয়ে ও DNS-এর সম্পর্ক ব্যাখ্যা করতে; (২) `ip`, `ss`, `ping`, `curl`, `dig`, `traceroute` দিয়ে নেটওয়ার্ক পরীক্ষা করতে; (৩) কোন পোর্টে কোন সার্ভিস শুনছে তা নির্ণয় করতে; (৪) DNS, রাউটিং ও ফায়ারওয়াল সমস্যা আলাদা করতে; (৫) নেটওয়ার্ক ইন্টারফেস ও রুট কনফিগার করতে।

১৫.২ প্রোডাকশনে কেন এটি গুরুত্বপূর্ণ

সার্ভার মানেই নেটওয়ার্ক। ওয়েবসাইট খুলছে না, API সাড়া দিচ্ছে না, ডাটাবেসে সংযোগ হচ্ছে না — এই অভিযোগগুলোর অর্ধেকের বেশি নেটওয়ার্ক স্তরের সমস্যা: ভুল DNS, বন্ধ পোর্ট, ভুল রুট, অথবা সার্ভিস ভুল ঠিকানায় শুনছে। নেটওয়ার্ক না বুঝলে প্রতিটি বিভ্রাটই "সার্ভার নষ্ট" মনে হয়; বুঝলে পাঁচ মিনিটে স্তর ভাগ করে কারণ ধরা যায়।

১৫.৩ পূর্বশর্ত জ্ঞান

ইন্টারনেট, ডোমেইন নাম ও পোর্টের সাধারণ ধারণা থাকলেই চলবে। IP ঠিকানা কীভাবে কাজ করে, তা এই অধ্যায়েই ধাপে ধাপে ব্যাখ্যা করা হয়েছে।

১৫.৪ প্রথম নীতি থেকে ব্যাখ্যা

নেটওয়ার্ক মানে ঠিকানা ও পথ। প্রতিটি সার্ভারের একটি **IP ঠিকানা** (IP address — নেটওয়ার্কে যন্ত্রের সংখ্যাসূচক ঠিকানা, যেমন 203.0.113.10) থাকে। মানুষ নাম মনে রাখে, যন্ত্র সংখ্যা — নামকে সংখ্যায় বদলায় **DNS** (Domain Name System — ডোমেইন নামকে IP ঠিকানায় অনুবাদ করার ব্যবস্থা)। **পোর্ট** (port — একই সার্ভারে আলাদা সার্ভিস চেনার নম্বর; ৮০/৪৪৩ ওয়েব, ২২ SSH, ৩০০০ Node অ্যাপ) হলো দরজার নম্বর: ঠিকানা বলে কোন বাড়ি, পোর্ট বলে কোন দরজা। **গেটওয়ে** (gateway — স্থানীয় নেটওয়ার্কের বাইরে যাওয়ার দরজা) না জানলে সার্ভার বাইরের জগতে পৌঁছাতে পারে না। অনুরোধের পথ: DNS → রাউটিং → ফায়ারওয়াল → সার্ভিস — এই স্তরগুলো একে একে পরীক্ষা করাই নেটওয়ার্ক ট্রাবলশুটিং।

১৫.৫ স্থাপত্য / ধারণাগত চিত্র

[ব্যবহারকারী: app.technova.example] → DNS: নাম → 203.0.113.10 → ইন্টারনেট → সার্ভারের গেটওয়ে → ফায়ারওয়াল (পোর্ট ৪৪৩ খোলা?) → Nginx (৪৪৩) → Node.js (৩০০০, শুধু localhost) → MongoDB (২৭০১৭, শুধু localhost)

পরীক্ষার ক্রম: ping (সার্ভার জীবিত?) → DNS (নাম মিলছে?) → পোর্ট (দরজা খোলা?) → অ্যাপ (সার্ভিস সাড়া দিচ্ছে?)

১৫.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

ইন্টারফেস হলো নেটওয়ার্ক কার্ডের সফটওয়্যার রূপ (`eth0`, `ens3` ইত্যাদি)। `ip addr` দিয়ে ঠিকানা দেখা যায়। ঠিকানার সাথে **সাবনেট মাস্ক** (/২৪ মানে প্রথম ২৪ বিট নেটওয়ার্ক অংশ) বলে কোন ঠিকানাগুলো "পাশের বাড়ি" আর কোনগুলো দূরের। **রাউটিং টেবিল** (`ip route`) বলে প্রতিটি গন্তব্যে কোন পথে যেতে হবে; ডিফল্ট রুট সাধারণত গেটওয়ের দিকে যায়। **শোনা (listening)** মানে সার্ভিস কোনো পোর্টে সংযোগের অপেক্ষায় আছে। `127.0.0.1` (localhost) শুধু একই সার্ভারের

ভেতর থেকে পৌঁছানো যায়; `0.0.0.0` মানে সব ইন্টারফেসে শোনা — বাইরে থেকেও। MongoDB-কে `0.0.0.0`-তে শোনানো বিপজ্জনক; Node অ্যাপকে Nginx-এর পেছনে `127.0.0.1`-তে রাখাই নিরাপদ নকশা।

TCP বনাম UDP

TCP (ওয়েব, SSH, ডাটাবেস) সংযোগ স্থাপন করে, তথ্য পৌঁছানো নিশ্চিত করে; **UDP** (DNS, ভিডিও স্ট্রিম) দ্রুত কিন্তু নিশ্চয়তা দেয় না। বেশিরভাগ সার্ভার সমস্যা TCP স্তরে ধরা পড়ে।

কাজের উদাহরণ — DNS বদলের পর আধা ব্যবহারকারী পুরোনো সাইট দেখছে

TechNova ডোমেইন app.technova.example-কে নতুন সার্ভার 203.0.113.20-তে সরাল। কিছু ব্যবহারকারী নতুন সাইট দেখছেন, কিছু পুরোনো — কারণ DNS রেকর্ড বদলালেও পুরোনো মান বিভিন্ন জায়গায় ক্যাশে (cache) থেকে যায়, যার মেয়াদ হলো **TTL** (Time To Live — রেকর্ডটি কতক্ষণ মনে রাখা হবে)। TTL যদি ৩৬০০ সেকেন্ড (এক ঘণ্টা) হয়, তবে এক ঘণ্টা পর্যন্ত পুরোনো ঠিকানা ফিরতে পারে। `dig app.technova.example`-এর উত্তরে TTL দেখা যায়; `dig @8.8.8.8` দিয়ে গুগলের DNS দিয়ে পরীক্ষা করলে বোঝা যায় সমস্যাটি আপনার সার্ভারের নাকি সারা বিশ্বের ক্যাশের। শিক্ষা: বড় পরিবর্তনের অন্তত একদিন আগে TTL কমিয়ে (যেমন ৩০০ সেকেন্ড) রাখুন, পরিবর্তন করুন, তারপর আবার বাড়ান। আর “DNS ছড়িয়ে পড়তে ২৪ ঘণ্টা লাগে” কথাটি আংশিক সত্য — আসলে ছড়িয়ে পড়ার সময় নির্ভর করে TTL-এর ওপর, কোনো নির্দিষ্ট ঘণ্টার ওপর নয়। স্থানীয় পরীক্ষায় `getent hosts` দিয়ে দেখুন সার্ভার নিজে নামটি কী ঠিকানায় মেলাচ্ছে; সার্ভারের `/etc/hosts`-এ পুরোনো ঠিকানা লেখা থাকলে DNS ঠিক থাকলেও সার্ভার ভুল ঠিকানায় যাবে।

পরীক্ষা	কী জানা যায়	কোথায় চালাবেন
`dig নাম`	DNS উত্তর ও TTL	নিজের কম্পিউটার ও সার্ভার — দুজায়গায়
`dig @8.8.8.8 নাম`	বাইরের DNS কী বলছে	সার্ভার থেকে
`getent hosts নাম`	সার্ভার নিজে কী মেলাচ্ছে	সার্ভার থেকে
`cat /etc/hosts`	স্থানীয়ভাবে ঠিকানা জোর করে বসানো আছে কি না	সার্ভার থেকে

১৫.৭ গুরুত্বপূর্ণ কমান্ড ও বিশ্লেষণ

```
ip addr show           # ইন্টারফেস ও IP ঠিকানা
ip route show          # রাউটিং টেবিল ও ডিফল্ট গেটওয়ে
ss -tlnp               # কোন TCP পোর্টে কোন প্রসেস শুনছে (-p-তে sudo লাগতে পারে)
ss -tulpn              # TCP + UDP
ping -c 4 203.0.113.10          # সার্ভার পৌঁছানো যাচ্ছে কি না
ping -c 4 8.8.8.8               # ইন্টারনেট পৌঁছানো যাচ্ছে কি না
dig app.technova.example        # DNS অনুবাদ পরীক্ষা
curl -I https://app.technova.example   # HTTP সাড়া পরীক্ষা
curl -v http://127.0.0.1:3000/  # স্থানীয়ভাবে অ্যাপ পরীক্ষা
traceroute 203.0.113.10         # পথের কোন ধাপে আটকাচ্ছে
cat /etc/resolv.conf            # কোন DNS সার্ভার ব্যবহৃত হচ্ছে
```

কমান্ড	কী পরীক্ষা করে	ব্যর্থ হলে অর্থ
`ping`	সার্ভার জীবিত ও পৌঁছানো যায়	নেটওয়ার্ক/ফায়ারওয়াল/সার্ভার বন্ধ
`dig`	DNS অনুবাদ	DNS ভুল বা ডোমেইন কনফিগ ভুল

`ss -tlnp`	সার্ভিস শুনছে কি না	সার্ভিস বন্ধ বা ভুল ঠিকানায় বাঁধা
`curl -v`	অ্যাপ স্তরের সাড়া	অ্যাপ ক্র্যাশ বা ভুল প্রক্সি

আরও কমান্ড

```
ss -tnp                    # TCP সংযোগ: কে কার সাথে যুক্ত, কোন প্রসেস
getent hosts app.technova.example    # সার্ভার নামটি কী ঠিকানায় মেলাচ্ছে
host app.technova.example          # সংক্ষিপ্ত DNS অনুবাদ পরীক্ষা
mtr --report 203.0.113.10          # ping + traceroute একসাথে, প্যাকেট হারানোসহ
ip -s link show eth0       # ইন্টারফেসে কত প্যাকেট গেল/এল, ত্রুটি কত
sudo ss -tlnp | grep 443   # ৪৪৩ পোর্টে কে শুনছে, নিশ্চিতভাবে দেখা
```

ব্যাখ্যা: `ss -tnp` চলমান সংযোগ দেখায় — কোন ক্লায়েন্ট আটকে আছে, তা এতে ধরা পড়ে। `mtr` একবারে পথের প্রতিটি ধাপে প্যাকেট হারানোর হার দেখায়, যা `traceroute`-এর চেয়ে তথ্যবহুল। `ip -s link`-এ ত্রুটি (errors/dropped) বাড়তে থাকলে কেবল, ভার্চুয়াল নেটওয়ার্ক বা ড্রাইভার সন্দেহ করুন। `host` হলো `dig`-এর সংক্ষিপ্ত বিকল্প — দ্রুত পরীক্ষায় সুবিধা, বিস্তারিত TTL দেখতে `dig` ব্যবহার করুন।

১৫.৮ কনফিগারেশন উদাহরণ

উবুন্টু ২৪.০৪-এ নেটওয়ার্ক কনফিগারেশন **Netplan** দিয়ে হয় (`/etc/netplan/`)। উদাহরণ:

```
# /etc/netplan/50-cloud-init.yaml — নমুনা (পরিবর্তনের আগে ব্যাকআপ নিন)
network:
  version: 2
  ethernets:
    eth0:
      dhcp4: true
# স্থির IP চাইলে:
#      addresses: [203.0.113.10/24]
#      routes:
#        - to: default
#          via: 203.0.113.1
#      nameservers:
#        addresses: [8.8.8.8, 1.1.1.1]
# প্রয়োগ: sudo netplan apply   (SSH সেশন খোলা রেখে পরীক্ষা করুন)
```

> **WARNING:** নেটওয়ার্ক কনফিগারেশন বদলালে SSH সংযোগ ছিন্ন হতে পারে। পরিবর্তনের আগে বর্তমান ফাইলের ব্যাকআপ নিন, একটি দ্বিতীয় SSH সেশন খোলা রাখুন এবং সম্ভব হলে কনসোল অ্যাক্সেস হাতে রাখুন।

১৫.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — "সাইট ডাউন" কিন্তু সার্ভার চলছে: `curl 127.0.0.1:3000` কাজ করে, বাইরে থেকে নয় — Nginx বন্ধ অথবা ফায়ারওয়ালে ৪৪৩ বন্ধ। **পরিস্থিতি ২ — হঠাৎ DNS বিভ্রাট:** IP-তে সাইট খোলে, নামে নয় — DNS রেকর্ড বা `resolv.conf` পরীক্ষা করুন। **পরিস্থিতি ৩ — ডাটাবেস সংযোগ ব্যর্থ:** অ্যাপ থেকে MongoDB-তে `curl`/টেলনেট নয়, `ss` দিয়ে দেখুন MongoDB `127.0.0.1:27017`-এ শুনছে কি না।

পরিস্থিতি ৪ — TechNova-র API হঠাৎ ধীর, কিন্তু সাইট ঠিক: TechNova-র ওয়েবসাইট খুলছিল, কিন্তু মোবাইল অ্যাপের API অনুরোধ মাঝে মাঝে সময় শেষ (timeout) হয়ে যাচ্ছিল। সার্ভারে `ss -tnp` চালিয়ে দেখা গেল API সার্ভিসের দিকে অনেক সংযোগ `SYN_SENT` বা `TIME_WAIT` অবস্থায় জমে আছে — অ্যাপ সার্ভার সংযোগ নিচ্ছে, কিন্তু ডাটাবেসের উত্তর দেরিতে আসছে। আরও দেখা গেল API সার্ভিসটি ডাটাবেসের পাবলিক ঠিকানায় সংযোগ করছিল, localhost-এর বদলে — ফলে প্রতিটি অনুরোধ বাইরের পথ ঘুরে আসছিল। ডাটাবেসের ঠিকানা `127.0.0.1` করে দেওয়ার পর সময় স্বাভাবিক হলো। শিক্ষা: "নেটওয়ার্ক ধীর" মানেই ইন্টারনেট ধীর নয় — অনেক সময় সার্ভিসগুলো নিজেদের মধ্যে ভুল পথে কথা বলছে। অভ্যন্তরীণ সংযোগ সবসময় localhost বা ব্যক্তিগত নেটওয়ার্কে রাখুন।

১৫.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

ওয়েবসাইট খুলছে না — স্তরে স্তরে নির্ণয়: (১) সার্ভারে SSH হয়? না হলে সার্ভার/নেটওয়ার্ক স্তর। (২) `ping` ডোমেইনে — DNS কাজ করছে? (৩) `ss -tlnp` — Nginx ৪৪৩-এ শুনছে? (৪) `curl -I http://127.0.0.1` — স্থানীয়ভাবে Nginx সাড়া দেয়? (৫) `curl http://127.0.0.1:3000` — Node অ্যাপ জীবিত? (৬) `systemctl status nginx` ও লগ দেখুন। যে স্তরে প্রথম ব্যর্থতা, সমস্যা সেখানেই।

১৫.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল ১: "ping হয় না, তাই সার্ভার ডাউন" — অনেক ফায়ারওয়াল ping (ICMP) বন্ধ রাখে, সার্ভিস চললেও। ভুল ২: অ্যাপকে `0.0.0.0`-তে শোনানো যখন Nginx-এর পেছনে `127.0.0.1`-ই যথেষ্ট ছিল। ভুল ৩: DNS পরিবর্তনের পরপরই পরীক্ষা — DNS ক্যাশ (TTL) পুরোনো মান ধরে রাখতে পারে। ভুল ৪: `localhost` আর সার্ভারের পাবলিক IP-কে এক ভাবা। ভুল ৫: ফায়ারওয়াল না দেখে অ্যাপের কোডে দোষ খোঁজা।

১৫.১২ নিরাপত্তা বিবেচনা

শুধু প্রয়োজনীয় পোর্ট খোলা রাখুন — প্রতিটি খোলা পোর্ট একটি দরজা। ডাটাবেস পোর্ট (২৭০১৭, ৫৪৩২, ৩৩০৬) কখনোই ইন্টারনেটে খুলবেন না; শুধু localhost বা ব্যক্তিগত নেটওয়ার্কে রাখুন। `ss -tlnp` দিয়ে নিয়মিত অপ্রত্যাশিত শোনা-পোর্ট খুঁজুন — অজানা পোর্ট মানে অজানা সার্ভিস, সম্ভবত আপসের লক্ষণ।

১৫.১৩ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
ডোমেইনে খোলে না, IP-তে খোলে	DNS সমস্যা	`dig`, DNS রেকর্ড, TTL অপেক্ষা
Connection refused	সার্ভিস বন্ধ / পোর্টে কিছু শুনছে না	`ss -tlnp`, `systemctl status`
Connection timed out	ফায়ারওয়াল / রাউটিং	ফায়ারওয়াল নিয়ম, `traceroute`
স্থানীয়ভাবে চলে, বাইরে নয়	localhost-এ বাঁধা বা ফায়ারওয়াল	বাইন্ড ঠিকানা ও UFW নিয়ম দেখুন

আরও লক্ষণ

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
DNS ঠিক, কিন্তু `curl` ব্যর্থ	প্রক্সি সেটিং বা ভুল পোর্ট	`env | grep -i proxy`; `curl -v` দিয়ে কোন ঠিকানায় যাচ্ছে দেখুন

ছোট প্যাকেট যায়, বড় ফাইল আটকে যায়	MTU সমস্যা	`ping -M do -s 1400` দিয়ে MTU পরীক্ষা; নেটওয়ার্ক প্রদানকারীকে জানান
IPv6-এ চেষ্টা করে ব্যর্থ, IPv4-এ চলে	IPv6 রাউটিং নেই	`curl -4` দিয়ে পরীক্ষা; প্রয়োজনে সার্ভিস IPv4-এ বাঁধুন
কিছুক্ষণ চলে, তারপর সংযোগ ছিন্ন	ফায়ারওয়ালের সময়সীমা (timeout) বা NAT	`ss -tnp`-এ সংযোগের অবস্থা দেখুন; keepalive সেটিং পরীক্ষা করুন

১৫.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: স্তরে স্তরে নির্ণয় (DNS → পোর্ট → অ্যাপ), অভ্যন্তরীণ সার্ভিস localhost-এ, Netplan পরিবর্তনের আগে ব্যাকআপ, নিয়মিত পোর্ট অডিট। অ্যান্টি-প্যাটার্ন: সব পোর্ট খুলে "সমস্যা মেটানো"; DNS TTL না বুঝে বারবার রেকর্ড বদলানো; `ifconfig`-এর মতো পুরোনো টুলের ওপর নির্ভর করা (উবুন্টু ২৪.০৪-এ `ip` ও `ss`-ই মান)।

১৫.১৫ অধ্যায়ের সারাংশ

নেটওয়ার্ক সমস্যা স্তরে স্তরে ভাগ করুন: DNS, রাউটিং, পোর্ট, অ্যাপ। `ip`, `ss`, `ping`, `dig` ও `curl` — এই পাঁচটি টুলেই বেশিরভাগ সমস্যার কারণ ধরা যায়। অভ্যন্তরীণ সার্ভিস localhost-এ রাখুন, শুধু প্রয়োজনীয় পোর্ট খুলুন।

১৫.১৬ মূল প্রযুক্তিগত পরিভাষা

IP ঠিকানা — যন্ত্রের নেটওয়ার্ক ঠিকানা; **DNS** — নাম থেকে IP অনুবাদ; **পোর্ট** — সার্ভিস চেনার নম্বর; **গেটওয়ে** — বাইরে যাওয়ার দরজা; **সাবনেট** — IP ঠিকানার ভাগ; **localhost** — নিজের সার্ভার (127.0.0.1); **TCP/UDP** — পরিবহন প্রোটোকল; **Netplan** — উবুন্টুর নেটওয়ার্ক কনফিগ ব্যবস্থা।

১৫.১৭ জ্ঞান যাচাই প্রশ্ন

1. DNS ব্যর্থ হলে IP ঠিকানায় সাইট খুলতে পারে কেন?
2. `Connection refused` ও `Connection timed out`-এর পার্থক্য কী?
3. Node অ্যাপকে `127.0.0.1`-তে বাঁধা নিরাপদ কেন?
4. `ss -tlnp` কী দেখায়?
5. ping ব্যর্থ হলেই কি সার্ভার ডাউন বলা যায়?
6. ডিফল্ট গেটওয়ের কাজ কী?
7. DNS রেকর্ড বদলের আগে TTL কমিয়ে রাখা হয় কেন?
8. `/etc/hosts`-এ ভুল ঠিকানা থাকলে DNS ঠিক থাকলেও কী হবে?

১৫.১৮ ব্যবহারিক অনুশীলন

1. ল্যাব VM-এ `ip addr`, `ip route`, `ss -tlnp` চালিয়ে নিজের নেটওয়ার্ক মানচিত্র আঁকুন।
2. একটি Node সার্ভার ৩০০০ পোর্টে চালিয়ে `curl 127.0.0.1:3000` ও বাইরের IP দিয়ে পরীক্ষা করে পার্থক্য দেখুন।
3. `dig` দিয়ে তিনটি পরিচিত ডোমেইনের DNS অনুবাদ পরীক্ষা করুন।
4. ইচ্ছাকৃতভাবে ভুল DNS সার্ভার বসিয়ে (ল্যাবে) কী হয় দেখুন, তারপর ফিরিয়ে আনুন।

5. একটি ডোমেইনের জন্য `dig` ও `dig @8.8.8.8`-এর উত্তর তুলনা করে TTL ও ঠিকানা লিখুন।

অধ্যায় ১৬

ফায়ারওয়াল: UFW, nftables ও নেটওয়ার্ক অ্যাক্সেস নিয়ন্ত্রণ

১৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) ফায়ারওয়াল কী ও কেন দরকার তা ব্যাখ্যা করতে; (২) UFW দিয়ে নিরাপদে পোর্ট খোলা/বন্ধ করতে; (৩) nftables-এর মৌলিক ধারণা ও নিয়ম লিখতে; (৪) ডিফল্ট-ডিনাই নীতি প্রয়োগ করতে; (৫) SSH অ্যাক্সেস না হারিয়ে ফায়ারওয়াল পরীক্ষা করতে।

১৬.২ প্রোডাকশনে কেন এটি গুরুত্বপূর্ণ

ইন্টারনেটে সার্ভার চালু করার মুহূর্তেই স্বয়ংক্রিয় স্ক্যানার সেটি খুঁজে পায় — কয়েক মিনিটের মধ্যে। ফায়ারওয়াল ছাড়া সার্ভার মানে খোলা বাড়ি: ডাটাবেস পোর্ট, অ্যাডমিন প্যানেল, পরীক্ষামূলক সার্ভিস — সবই দৃশ্যমান। ফায়ারওয়াল হলো সেই দরজার পাহারাদার, যে শুধু অনুমোদিত দরজা দিয়ে ঢুকতে দেয়। একটি ভুল নিয়মে হয় সাইট বন্ধ, নয় সার্ভার উন্মুক্ত — তাই ফায়ারওয়াল শেখা মানে নিরাপত্তা ও উপলভ্যতা দুটোই রক্ষা করা।

১৬.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১৫-এর পোর্ট, IP ও localhost ধারণা জানা থাকতে হবে। UFW ও nftables আগে না জানলেও চলবে।

১৬.৪ প্রথম নীতি থেকে ব্যাখ্যা

ফায়ারওয়াল একটি ফিল্টার: প্রতিটি নেটওয়ার্ক প্যাকেটের গায়ে লেখা থাকে উৎস, গন্তব্য, পোর্ট ও প্রোটোকল। ফায়ারওয়াল নিয়মের তালিকা মিলিয়ে সিদ্ধান্ত নেয় — ঢুকতে দেবে, নাকি ফেলে দেবে। দুটি নীতি সম্ভব: **ডিফল্ট-অ্যালাউ** (সব খোলা, শুধু নিষিদ্ধটি বন্ধ — বিপজ্জনক) ও **ডিফল্ট-ডিনাই** (সব বন্ধ, শুধু অনুমোদিতটি খোলা — নিরাপদ)। প্রোডাকশনে সবসময় ডিফল্ট-ডিনাই। লিনাক্স কার্নেলের ভেতরে প্যাকেট ফিল্টার করে **netfilter**, আর তার আধুনিক নিয়ম-ভাষা হলো **nftables**। **UFW** (Uncomplicated Firewall) হলো nftables/iptables-এর ওপরে সহজ মুখোশ — মানুষের ভাষায় নিয়ম লিখলে সে ভেতরে অনুবাদ করে দেয়।

১৬.৫ স্থাপত্য / ধারণাগত চিত্র

[ইন্টারনেট] → [nftables / netfilter: INPUT চেইন] → নিয়ম মেলানো → { SSH ২২: অনুমোদিত | HTTP ৮০: অনুমোদিত | HTTPS ৪৪৩: অনুমোদিত | বাকি সব: বাতিল }

[সার্ভার থেকে বাইরে] → OUTPUT চেইন (সাধারণত অনুমোদিত) | UFW = সহজ কমান্ড → nftables নিয়মে অনুবাদ

নীতি: ডিফল্ট-ডিনাই — যা স্পষ্ট অনুমোদিত নয়, তা বন্ধ।

১৬.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

UFW-এর নিয়মগুলো ক্রমানুসারে মেলে; প্রথম মিলেই সিদ্ধান্ত। `ufw status numbered` দিয়ে ক্রম দেখা যায়। **ইনকামিং** (incoming — বাইরে থেকে আসা) নিয়মই মূলত গুরুত্বপূর্ণ; **আউটগোয়িং** (outgoing — সার্ভার থেকে বাইরে যাওয়া) সাধারণত খোলা রাখা হয়, যাতে apt আপডেট ও API কল চলে। nftables-এ নিয়ম লেখা হয় **টেবিল → চেইন → নিয়ম** কাঠামোয়। একটি চেইনের **পলিসি** (policy — কোনো নিয়ম না মিললে কী হবে) `drop` (চুপচাপ ফেলে দেওয়া) বা `accept` হতে পারে; `drop` নিরাপদ, কারণ আক্রমণকারী জানতেও পারে না পোর্টটি আছে কি না। `reject` উত্তর পাঠায় — ডিবাগে সুবিধা, কিন্তু তথ্য ফাঁস করে।

UFW বনাম nftables — কখন কোনটি

একক সার্ভারে UFW-ই যথেষ্ট ও নিরাপদ। nftables দরকার হয় জটিল ক্ষেত্রে: একাধিক নেটওয়ার্ক, রেট লিমিট, নির্দিষ্ট IP থেকে অনুমতি, বা কাস্টম চেইন। UFW শিখে nftables বোঝা সহজ, কারণ নীতি একই — শুধু ভাষা বিস্তারিত।

কাজের উদাহরণ — SSH-তে রেট লিমিট: এক IP থেকে ঝড় ঠেকানো

TechNova-র সার্ভার 203.0.113.10-এর `auth.log`-এ দেখা গেল একটি IP থেকে প্রতি মিনিটে শত শত SSH চেষ্টা আসছে। UFW-তে শুধু পোর্ট খোলা থাকলে এই চেষ্টা চলতেই থাকে — পোর্ট খোলা মানে দরজা খোলা, ভেতরে কে ঢুকতে চাইছে তা ফায়ারওয়াল যাচাই করে না। সমাধান দুই স্তরে: প্রথমে UFW-তে `ufw limit 22/tcp` — এটি একই IP থেকে অল্প সময়ে অনেক সংযোগ এলে বাকিগুলো আটকে দেয় (rate limiting)। দ্বিতীয়ত, fail2ban (অধ্যায় ১৭) বারবার ব্যর্থ লগইন করা IP-কে এক ঘণ্টা বন্ধ করে দেয়। দুটোর পার্থক্য মনে রাখুন: রেট লিমিট সংযোগের গতি কমায়, fail2ban ব্যর্থ চেষ্টার শাস্তি দেয় — একসাথে ব্যবহার করলে সবচেয়ে ভালো ফল মেলে। তবে সাবধান: অফিসের সবাই একই IP (NAT-এর পেছনে) থেকে ঢুকলে রেট লিমিট আসল ব্যবহারকারীকেও আটকাতে পারে — তাই অফিস IP-কে সীমা থেকে ছাড় দেওয়ার নিয়ম আগে যোগ করুন, অথবা অফিস IP থেকে SSH-এর আলাদা অনুমোদন রাখুন। নিয়ম বদলের পর সবসময় দ্বিতীয় SSH সেশন দিয়ে লগইন পরীক্ষা করুন, যাতে নিজেই আটকে না যান।

ব্যবস্থা	কী ঠেকায়	সীমা
`ufw limit 22/tcp`	এক IP থেকে সংযোগের ঝড়	আসল ব্যবহারকারীও আটকাতে পারে (NAT হলে)
fail2ban	বারবার ব্যর্থ লগইন	IP বদলে আক্রমণকারী ফিরতে পারে
শুধু নির্দিষ্ট IP থেকে SSH	অচেনা IP থেকে চেষ্টাই বন্ধ	IP বদলালে নিজে আটকে যাবেন — কনসোল রাখুন

> **WARNING:** SSH-সংক্রান্ত কোনো নিয়ম বদলানোর আগে একটি SSH সেশন খোলা রাখুন এবং কনসোল অ্যাক্সেস হাতে রাখুন। রেট লিমিট বা IP-সীমা ভুলভাবে বসালে আপনিও ঢুকতে পারবেন না।

১৬.৭ গুরুত্বপূর্ণ কমান্ড ও বিশ্লেষণ

```
sudo ufw status verbose     # বর্তমান নিয়ম ও ডিফল্ট নীতি
sudo ufw status numbered    # নম্বরসহ নিয়ম (মুছতে সুবিধা)
sudo ufw allow 22/tcp       # SSH অনুমোদন
sudo ufw allow 80,443/tcp   # ওয়েব পোর্ট অনুমোদন
sudo ufw allow from 203.0.113.50 to any port 22   # শুধু একটি IP থেকে SSH
sudo ufw deny 3306/tcp      # MySQL পোর্ট স্পষ্টভাবে বন্ধ
sudo ufw delete 3           # ৩ নম্বর নিয়ম মোছা
sudo ufw --force enable     # ফায়ারওয়াল চালু (সতর্কতা নিচে দেখুন)
```

```
sudo ufw reload             # নিয়ম পুনরায় লোড
sudo nft list ruleset       # বর্তমান nftables নিয়ম দেখা
sudo nft -c -f /etc/nftables.conf    # কনফিগ যাচাই (চালু না করে)
```

> **WARNING:** `ufw --force enable` চালানোর আগে অবশ্যই SSH নিয়ম (`ufw allow 22/tcp` বা `ufw allow OpenSSH`) যোগ করুন এবং একটি দ্বিতীয় SSH সেশন খোলা রাখুন। SSH নিয়ম ছাড়া ফায়ারওয়াল চালু করলে আপনি নিজেই সার্ভার থেকে বেরিয়ে যাবেন। ভুল হলে কনসোল অ্যাক্সেস ছাড়া ফেরা যায় না।

আরও কমান্ড

```
sudo ufw limit 22/tcp        # এক IP থেকে SSH সংযোগের ঝড় সীমিত করা
sudo ufw logging on           # ফায়ারওয়ালে বাতিল প্যাকেটের লগ চালু
sudo ufw show added           # যোগ করা নিয়মগুলো কমান্ড আকারে দেখা
sudo ufw allow from 203.0.113.0/24 to any port 5432    # শুধু একটি সাবনেট থেকে ডাটাবেস পোর্ট
sudo nft list table inet filter   # nftables-এর নির্দিষ্ট টেবিলের নিয়ম দেখা
```

ব্যাখ্যা: `ufw limit` ছয়টির বেশি সংযোগ ৩০ সেকেন্ডে এলে বাকি আটকে দেয় — ব্রুট-ফোর্সের গতি কমানোর সহজ উপায়। `ufw logging on` চালু করলে বাতিল প্যাকেট `/var/log/ufw.log`-এ লেখা হয়, যা আক্রমণ বোঝার জন্য দরকারি। `ufw show added` নিয়মগুলো পুনরায় চালানোর মতো করে দেখায়, তাই নতুন সার্ভারে একই নিয়ম বসাতে সুবিধা। ডাটাবেস পোর্টের নিয়মে `/24` সাবনেট লেখার আগে নিশ্চিত হোন সেই সাবনেটটি আসলেই আপনার বিশ্বস্ত নেটওয়ার্ক — ভুল সাবনেট মানে ভুল লোকেরা ঢুকতে পারবে।

১৬.৮ কনফিগারেশন উদাহরণ

UFW — নিরাপদ শুরুর সেট (ক্রমটি গুরুত্বপূর্ণ):

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw --force enable
sudo ufw status verbose
```

nftables — মৌলিক নিয়মসেট (`/etc/nftables.conf`):

```
#!/usr/sbin/nft -f
flush ruleset

table inet filter {
    chain input {
        type filter hook input priority filter; policy drop;
        iif "lo" accept
        ct state established,related accept
        tcp dport 22 accept
        tcp dport { 80, 443 } accept
        icmp type echo-request limit rate 5/second accept
        log prefix "nft-drop: " drop
    }
    chain forward {
```

```
        type filter hook forward priority filter; policy drop;
    }
    chain output {
        type filter hook output priority filter; policy accept;
    }
}
```

ব্যাখ্যা: `lo` (লুপব্যাক) সবসময় অনুমোদিত; `established,related` মানে ইতিমধ্যে চলমান সংযোগের উত্তর আসতে দেওয়া; তারপর SSH ও ওয়েব; বাকি সব লগসহ বাতিল। প্রয়োগের আগে `nft -c -f` দিয়ে যাচাই করুন, আর SSH সেশন খোলা রাখুন। রোলব্যাক: `sudo ufw disable` অথবা nftables-এর ক্ষেত্রে আগের কনফিগ ফাইলের ব্যাকআপ থেকে `nft -f` দিয়ে ফিরিয়ে আনুন।

১৬.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — নিজেকে লকআউট: SSH নিয়ম ছাড়া UFW চালু — কনসোল থেকে `ufw disable` করতে হয়। এজন্যই দ্বিতীয় সেশন ও কনসোল অ্যাক্সেস আগাম নিশ্চিত করুন। **পরিস্থিতি ২ — SSH ব্রুট-ফোর্স:** লগে হাজারো ব্যর্থ লগইন — UFW-তে শুধু নিজের অফিস IP থেকে SSH অনুমোদন, বাকিদের জন্য fail2ban (অধ্যায় ১৭)। **পরিস্থিতি ৩ — ডাটাবেস উন্মুক্ত:** `ss`-এ দেখা গেল MongoDB পোর্ট বাইরে শোনা যাচ্ছে — ফায়ারওয়ালে বন্ধ করুন ও বাইন্ড ঠিকানা localhost করুন।

পরিস্থিতি ৪ — TechNova-র অফিস IP বদলে সবাই লকআউট: TechNova অফিসের ইন্টারনেট সংযোগ বদলাল, নতুন IP হলো 203.0.113.80। কিন্তু সার্ভারের UFW-তে SSH শুধু পুরোনো অফিস IP 203.0.113.50 থেকে অনুমোদিত ছিল — ফলে অফিসের কেউ SSH করতে পারল না। ভাগ্য ভালো, কনসোল অ্যাক্সেস ছিল; সেখান থেকে পুরোনো নিয়ম মুছে নতুন IP-র নিয়ম যোগ করা হলো। শিক্ষা: IP-ভিত্তিক সীমা শক্তিশালী, কিন্তু IP বদলালে নিজেই আটকে যাবেন — তাই কনসোল অ্যাক্সেস সবসময় হাতে রাখুন, আর জরুরি পথ হিসেবে একটি দ্বিতীয় বিশ্বস্ত IP (যেমন দলের প্রধানের বাড়ির IP) অনুমোদন করে রাখুন। নিয়মের তালিকা একটি ফাইলে লিখে রাখুন, যাতে কনসোল থেকে দ্রুত ফিরিয়ে আনা যায়।

১৬.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

নতুন সার্ভারে নিরাপদ ফায়ারওয়াল (ল্যাবে অনুশীলন): (১) দ্বিতীয় SSH সেশন খুলুন। (২) `ufw default deny incoming` ও `allow outgoing`। (৩) `ufw allow 22/tcp` — যাচাই করুন নিয়ম যোগ হয়েছে। (৪) `ufw allow 80,443/tcp`। (৫) `ufw --force enable` — দ্বিতীয় সেশন দিয়ে সংযোগ পরীক্ষা করুন। (৬) অন্য পোর্ট (যেমন ৩০০০) বাইরে থেকে বন্ধ কি না পরীক্ষা করুন। (৭) `ufw status verbose` দিয়ে চূড়ান্ত অবস্থা লিখে রাখুন।

১৬.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল ১: SSH নিয়ম ছাড়া ফায়ারওয়াল চালু করা — সবচেয়ে সাধারণ লকআউট। ভুল ২: সমস্যা হলেই ফায়ারওয়াল বন্ধ করে দেওয়া — নিরাপত্তা বিসর্জন দিয়ে ডিবাগ করা; বরং নির্দিষ্ট নিয়ম পরীক্ষা করুন। ভুল ৩: “UFW চালু আছে, তাই নিরাপদ” — UFW শুধু নেটওয়ার্ক স্তর; দুর্বল SSH পাসওয়ার্ড বা পুরোনো অ্যাপের দুর্বলতা সে ঠেকায় না। ভুল ৪: `allow 3000` স্থায়ীভাবে খুলে রাখা — Node অ্যাপ Nginx-এর পেছনে থাকলে ৩০০০ বাইরে খোলার দরকার নেই। ভুল ৫: নিয়মের ক্রম না দেখে `deny` যোগ করা — আগের `allow` জিতে যেতে পারে।

১৬.১২ নিরাপত্তা বিবেচনা

ফায়ারওয়াল কখনো বন্ধ করবেন না “সাময়িক” অজুহাতে — সমস্যার কারণ খুঁজুন। SSH শুধু নির্দিষ্ট IP থেকে অনুমোদন করুন, যদি সম্ভব হয়। প্রতিটি খোলা পোর্টের যুক্তি লিখে রাখুন। nftables-এ `log` দিয়ে বাতিল প্যাকেটের হিসাব রাখলে

আক্রমণের চেষ্টা ধরা পড়ে। মনে রাখুন: ফায়ারওয়াল একটি স্তর মাত্র — SSH কী, আপডেট ও অ্যাপ নিরাপত্তা (অধ্যায় ১৭) ছাড়া সম্পূর্ণ নয়।

১৬.১৩ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
ফায়ারওয়াল চালুর পর SSH বন্ধ	SSH নিয়ম নেই	কনসোল থেকে `ufw disable`, নিয়ম যোগ করে আবার চালু
পোর্ট খোলার পরও সংযোগ হয় না	সার্ভিস শুনছে না / ভুল ঠিকানা	`ss -tlnp` দিয়ে বাইন্ড যাচাই
UFW ও nftables দ্বন্দ্ব	দুটোই নিয়ম লিখছে	একটি ব্যবস্থাই ব্যবহার করুন; `nft list ruleset` দেখুন
নিয়ম যোগ করেও কাজ করে না	ক্রম / রিলোড হয়নি	`ufw status numbered`, `ufw reload`

আরও লক্ষণ

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
রিবুটের পর UFW বন্ধ দেখায়	UFW সক্রিয় (enabled) করা হয়নি	`sudo ufw status`; `sudo systemctl is-enabled ufw` — প্রয়োজনে `ufw enable` (SSH নিয়ম যাচাই করে)
IPv4-এ নিয়ম কাজ করে, IPv6-এ নয়	UFW-তে IPv6 বন্ধ	`/etc/default/ufw`-এ `IPV6=yes` আছে কি না দেখুন
ডকার চালু করলে ফায়ারওয়াল নিয়ম এড়িয়ে যায়	ডকার নিজের iptables/nft নিয়ম যোগ করে	ডকারের পোর্ট প্রকাশের (publish) নিয়ম দেখুন; ডকার কনফিগে পোর্ট শুধু localhost-এ বাঁধুন
`allow` করেও পোর্ট বন্ধ	নিয়মের ক্রমে আগের `deny` জিতছে	`ufw status numbered` দিয়ে ক্রম দেখে নিয়ম সরিয়ে সঠিক ক্রমে আবার যোগ করুন

১৬.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: ডিফল্ট-ডিনাই, SSH আগে, দ্বিতীয় সেশন খোলা রেখে পরীক্ষা, নিয়মের তালিকা ডকুমেন্ট করা, নিয়মিত অডিট। অ্যান্টি-প্যাটার্ন: ফায়ারওয়াল নিষ্ক্রিয় করে সমস্যা “সমাধান”; `allow all` নিয়ম; একই সার্ভারে UFW ও হাতে লেখা nftables নিয়মের মিশ্রণ।

১৬.১৫ অধ্যায়ের সারাংশ

ফায়ারওয়াল মানে ডিফল্ট-ডিনাই: সব বন্ধ, শুধু প্রয়োজনীয় পোর্ট খোলা। UFW সহজ পথ, nftables বিস্তারিত নিয়ন্ত্রণ — নীতি একই। SSH নিয়ম আগে, দ্বিতীয় সেশন খোলা রেখে চালু করুন, আর কখনোই নিরাপত্তা বন্ধ করে ডিবাগ করবেন না।

১৬.১৬ মূল প্রযুক্তিগত পরিভাষা

UFW — সহজ ফায়ারওয়াল মুখোশ; **nftables** — আধুনিক কার্নেল প্যাকেট ফিল্টার ভাষা; **netfilter** — কার্নেলের ফিল্টার কাঠামো; **ডিফল্ট-ডিনাই** — সব বন্ধ রেখে শুধু অনুমোদিতটি খোলা; **চেইন** — নিয়মের ক্রম; **পলিসি** — নিয়ম না মিললে ডিফল্ট সিদ্ধান্ত; **drop/reject** — চুপচাপ ফেলা / উত্তর দিয়ে ফেরত।

১৬.১৭ জ্ঞান যাচাই প্রশ্ন

1. ডিফল্ট-ডিনাই কেন ডিফল্ট-অ্যালাউয়ের চেয়ে নিরাপদ?
2. `ufw --force enable`-এর আগে কী করতে হবে এবং কেন?
3. UFW ও nftables-এর সম্পর্ক কী?
4. `drop` ও `reject`-এর পার্থক্য কী?
5. Node অ্যাপ ৩০০০ পোর্টে চললে ফায়ারওয়ালে ৩০০০ খোলা কি দরকার? কেন?
6. SSH-তে শুধু একটি IP থেকে অনুমতি দেওয়ার নিয়মটি লিখুন।
7. `ufw limit` ও fail2ban-এর মধ্যে পার্থক্য কী?
8. অফিসের IP বদলে গেলে শুধু সেই IP থেকে SSH অনুমোদিত থাকলে কী হবে?

১৬.১৮ ব্যবহারিক অনুশীলন

> **LAB:** ফায়ারওয়াল অনুশীলন শুধু ডিসপোজেবল VM-এ করুন; প্রোডাকশনে করার আগে ব্যাকআপ ও কনসোল অ্যাক্সেস নিশ্চিত করুন।

1. ল্যাব VM-এ ক্রম মেনে UFW সেট করুন (SSH আগে, তারপর ৮০/৪৪৩) ও দ্বিতীয় সেশন দিয়ে সংযোগ পরীক্ষা করুন।
2. `ufw status numbered` দেখে একটি নিয়ম মুছে আবার যোগ করুন।
3. nftables-এর নমুনা নিয়মসেট `nft -c -f` দিয়ে যাচাই করুন (চালু না করে)।
4. ৩০০০ পোর্ট খোলা ও বন্ধ অবস্থায় বাইরে থেকে সংযোগ পরীক্ষা করে পার্থক্য লিখুন।
5. ল্যাব VM-এ `ufw limit 22/tcp` বসিয়ে `ufw status verbose`-এ নিয়মটি যাচাই করুন ও দ্বিতীয় সেশন দিয়ে লগইন পরীক্ষা করুন।

অধ্যায় ১৭

লিনাক্স নিরাপত্তা কঠোরকরণ ও SSH সুরক্ষা

১৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) SSH কী-ভিত্তিক লগইন সেট আপ করতে; (২) `sshd_config` নিরাপদে কঠোর করতে; (৩) fail2ban দিয়ে ব্রুট-ফোর্স ঠেকাতে; (৪) ব্যবহারকারী, অনুমতি ও আপডেট ব্যবস্থাপনা ব্যাখ্যা করতে; (৫) নিজেকে লকআউট না করে নিরাপত্তা পরিবর্তন পরীক্ষা করতে।

১৭.২ প্রোডাকশনে কেন এটি গুরুত্বপূর্ণ

ইন্টারনেটে উন্মুক্ত SSH পোর্টে প্রতিদিন হাজারো স্বয়ংক্রিয় লগইন চেষ্টা আসে — দুর্বল পাসওয়ার্ড থাকলে সার্ভার দখল সময়ের ব্যাপার মাত্র। একবার দখল হলে আক্রমণকারী আপনার সার্ভার দিয়ে স্প্যাম পাঠায়, মাইনিং চালায়, গ্রাহকের তথ্য চুরি করে। নিরাপত্তা কঠোরকরণ কোনো "অতিরিক্ত" কাজ নয় — সার্ভার ইন্টারনেটে তোলার আগেই শেষ করতে হয়।

১৭.৩ পূর্বশর্ত জ্ঞান

SSH দিয়ে সার্ভারে ঢোকার অভিজ্ঞতা, `sudo` ও ফাইল অনুমতির ধারণা এবং অধ্যায় ১৬-এর ফায়ারওয়াল জ্ঞান থাকতে হবে।

১৭.৪ প্রথম নীতি থেকে ব্যাখ্যা

নিরাপত্তার মূলনীতি দুটি: **সর্বনিম্ন সুযোগ** (least privilege — যার যতটুকু দরকার, ততটুকুই অধিকার) ও **গভীরতায় প্রতিরক্ষা** (defense in depth — একটি স্তর ভাঙলেও পরের স্তর আটকায়)। পাসওয়ার্ড হলো "জানা কথা" — আন্দাজ করা যায়, ফাঁস হয়। **SSH কী** হলো গাণিতিক জোড়া: ব্যক্তিগত কী (private key — শুধু আপনার কাছে থাকে, কাউকে দেবেন না) ও পাবলিক কী (public key — সার্ভারে থাকে)। লগইনের সময় সার্ভার একটি চ্যালেঞ্জ দেয়, শুধু ব্যক্তিগত কী-ধারীই তার উত্তর দিতে পারে — পাসওয়ার্ড আন্দাজ করে ঢোকা অসম্ভব। তাই কী-ভিত্তিক লগইন পাসওয়ার্ডের চেয়ে মৌলিকভাবে নিরাপদ।

১৭.৫ স্থাপত্য / ধারণাগত চিত্র

[আক্রমণকারী: পাসওয়ার্ড আন্দাজ] → SSH → fail2ban (বারবার ব্যর্থ হলে IP বন্ধ) → পাসওয়ার্ড লগইন নিষিদ্ধ → শুধু কী-ধারী ঢুকতে পারে

[আপনি: ব্যক্তিগত কী] → SSH (পোর্ট ২২, ফায়ারওয়ালে সীমিত) → সার্ভার: পাবলিক কী মেলানো → লগইন

স্তর: ফায়ারওয়াল → SSH কনফিগ → কী → fail2ban → আপডেট → অনুমতি

১৭.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

SSH কী তৈরি: নিজের কম্পিউটারে `ssh-keygen -t ed25519` চালান (ed25519 আধুনিক ও নিরাপদ)। পাবলিক কী সার্ভারের `~/.ssh/authorized_keys`-এ যায়, ব্যক্তিগত কী কখনো সার্ভারে পাঠাবেন না। **sshd_config কঠোরকরণ:** `PasswordAuthentication no` (পাসওয়ার্ড লগইন বন্ধ), `PermitRootLogin no` (রুট সরাসরি লগইন বন্ধ — আগে

`sudo` ব্যবহারকারী দিয়ে ঢুকে `sudo` করুন), `MaxAuthTries 3`। **fail2ban** লগ দেখে বারবার ব্যর্থ লগইন করা IP-কে সাময়িকভাবে ফায়ারওয়ালে বন্ধ করে দেয়। **আপডেট:** `unattended-upgrades` নিরাপত্তা প্যাচ স্বয়ংক্রিয়ভাবে বসায় — পুরোনো সফটওয়্যারের জানা দুর্বলতাই সবচেয়ে সহজ আক্রমণপথ।

কাজের উদাহরণ — চাবি হারালে কী করবেন: পুনরুদ্ধারের পরিকল্পনা

ধরুন আপনার ল্যাপটপ চুরি গেল — ব্যক্তিগত কী-ও গেল। যদি সার্ভারে শুধু কী-লগইন চালু থাকে, তবে আপনি ঢুকতে পারবেন না — এটাই কী-ভিত্তিক লগইনের সবচেয়ে বড় ঝুঁকি। তাই পুনরুদ্ধারের পরিকল্পনা আগেই করে রাখতে হয়: প্রথমত, ব্যক্তিগত কী-র একটি এনক্রিপ্ট করা ব্যাকআপ নিরাপদ জায়গায় রাখুন (পাসফ্রেজসহ — পাসফ্রেজ ছাড়া ব্যাকআপও বিপজ্জনক)। দ্বিতীয়ত, দলের অন্তত দুজনের কী সার্ভারে রাখুন, যাতে একজন আটকে গেলে অন্যজন ঢুকে তার নতুন কী যোগ করতে পারে। তৃতীয়ত, হোস্টিং প্রদানকারীর কনসোল অ্যাক্সেসের তথ্য হাতে রাখুন — কনসোল দিয়ে সার্ভারে ঢুকে `authorized_keys` থেকে পুরোনো পাবলিক কী মুছে নতুনটি যোগ করা যায়। ল্যাপটপ হারানোর পর প্রথম কাজই হলো পুরোনো কী মুছে ফেলা — কারণ চোরের হাতে কী থাকলে সে-ও ঢুকতে পারে। নিচের সারণিতে তিনটি প্রমাণীকরণ পদ্ধতির তুলনা দেওয়া হলো; মনে রাখুন, কোনো পদ্ধতিই একা সম্পূর্ণ নয় — কী + পাসফ্রেজ + fail2ban + ফায়ারওয়াল একসাথে স্তরে স্তরে সুরক্ষা দেয়।

পদ্ধতি	সুবিধা	ঝুঁকি
পাসওয়ার্ড	সহজ, কিছু মনে রাখতে হয় না (সরঞ্জাম ছাড়া)	আন্দাজ করা যায়, ফাঁস হয়, বারবার ব্যবহার হয়
SSH কী	আন্দাজ করা যায় না, চুরি ছাড়া ভাঙা কঠিন	চাবি হারালে লকআউট; ব্যক্তিগত কী ফাঁস হলে বিপদ
কী + পাসফ্রেজ	চাবি চুরি গেলেও পাসফ্রেজ ছাড়া অচল	পাসফ্রেজ ভুলে গেলে একই লকআউট

১৭.৭ গুরুত্বপূর্ণ কমান্ড ও বিশ্লেষণ

```
ssh-keygen -t ed25519 -C "nahid@technova.example"   # নিজের কম্পিউটারে কী তৈরি
ssh-copy-id nahid@203.0.113.10                      # পাবলিক কী সার্ভারে পাঠানো
ssh -i ~/.ssh/id_ed25519 nahid@203.0.113.10         # কী দিয়ে লগইন পরীক্ষা
sudo systemctl status ssh                           # SSH সার্ভিস অবস্থা
sudo sshd -t                                        # sshd_config যাচাই (রিস্টার্টের আগে)
sudo systemctl reload ssh                           # কনফিগ পুনরায় লোড
sudo apt install fail2ban -y
sudo fail2ban-client status sshd                    # কোন IP বন্ধ আছে দেখা
sudo apt update && sudo apt upgrade -y              # আপডেট
lastb | head -20                                    # ব্যর্থ লগইন চেষ্টা
```

আরও কমান্ড

```
ssh-add ~/.ssh/id_ed25519     # কী এজেন্টে যোগ করা (পাসফ্রেজ একবার দিয়ে বারবার ব্যবহার)
ssh -v nahid@203.0.113.10     # লগইন ব্যর্থ হলে বিস্তারিত (verbose) কারণ দেখা
ssh-keygen -y -f ~/.ssh/id_ed25519   # ব্যক্তিগত কী থেকে পাবলিক কী বের করা
sudo grep "Failed password" /var/log/auth.log | tail -20   # ব্যর্থ পাসওয়ার্ড চেষ্টা দেখা
sudo journalctl -u ssh --since "1 hour ago"   # SSH সার্ভিসের গত ঘণ্টার লগ
sudo fail2ban-client status sshd   # বর্তমানে কোন IP ব্যান আছে দেখা
```

ব্যাখ্যা: `ssh -v` সবচেয়ে দরকারি ডিবাগ টুল — এটি দেখায় কোন কী পাঠানো হলো, সার্ভার কী উত্তর দিল, কোথায় আটকাল। `ssh-keygen -y` দিয়ে হারানো পাবলিক কী আবার বের করা যায়, ব্যক্তিগত কী থাকলে। `auth.log`-এ “Failed

password” গুনে বোঝা যায় আক্রমণ চলছে কি না। fail2ban-এর স্ট্যাটাস দেখে নিশ্চিত হোন আপনার নিজের IP ভুলে ব্যান হয়নি — হলে কনসোল থেকে আনব্যান করুন।

> **WARNING:** ব্যক্তিগত কী কখনো চ্যাট, ইমেইল বা Git-এ পাঠাবেন না। পাসফ্রেজ ছাড়া কী ফাঁস হলে যে কেউ সার্ভারে ঢুকতে পারবে — ফাঁসের সন্দেহ হলেই সঙ্গে সঙ্গে সার্ভার থেকে পাবলিক কী মুছে নতুন কী জোড়া বানান।

১৭.৮ কনফিগারেশন উদাহরণ

নিরাপদ `/etc/ssh/sshd_config` অংশ (পরিবর্তনের আগে ব্যাকআপ নিন):

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
LoginGraceTime 30
AllowUsers nahid deploy
# রোলব্যাক: sudo cp /etc/ssh/sshd_config.bak /etc/ssh/sshd_config && sudo systemctl reload ssh
```

> **WARNING:** `PasswordAuthentication no` করার আগে অবশ্যই কী দিয়ে লগইন পরীক্ষা করুন — নতুন একটি SSH সেশনে কী-লগইন সফল হচ্ছে কি না দেখুন, তারপরই পাসওয়ার্ড বন্ধ করুন। কী ছাড়া পাসওয়ার্ড বন্ধ করলে আপনি নিজেই তালা লাগিয়ে চাবি হারাবেন। দ্বিতীয় SSH সেশন খোলা রাখুন।

fail2ban মৌলিক কনফিগ (`/etc/fail2ban/jail.local`):

```
[sshd]
enabled = true
maxretry = 5
findtime = 10m
bantime = 1h
```

অর্থ: ১০ মিনিটে ৫ বার ব্যর্থ হলে IP এক ঘণ্টা বন্ধ। নিজের IP ভুলে বন্ধ হয়ে গেলে কনসোল থেকে `fail2ban-client unban <IP>` করুন।

১৭.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — লগে ব্রুট-ফোর্স ঝড়: `lastb` -তে একই IP থেকে শত শত চেষ্টা — fail2ban চালু করুন ও SSH শুধু নির্দিষ্ট IP থেকে অনুমোদন করুন। **পরিস্থিতি ২ — দলের নতুন সদস্য:** পাসওয়ার্ড না দিয়ে তার পাবলিক কী `authorized_keys` -এ যোগ করুন; চলে গেলে কী মুছে দিন — পাসওয়ার্ড বদলানোর ঝামেলা নেই। **পরিস্থিতি ৩ — ব্যক্তিগত কী ফাঁস:** সঙ্গে সঙ্গে সার্ভার থেকে পাবলিক কী মুছুন ও নতুন কী জোড়া বানান — ফাঁস হওয়া কী আর ব্যবহার করবেন না।

পরিস্থিতি ৪ — TechNova-র ঠিকাদার চলে গেল, তার কী রয়ে গেল: TechNova-র এক ঠিকাদার তিন মাস কাজ করে চলে গেলেন, কিন্তু তার পাবলিক কী সার্ভারের `authorized_keys` -এ রয়ে গেল — কেউ মুছতে ভুলে গেল। দুই মাস পর লগে দেখা গেল তার পুরোনো কী দিয়ে রাত ৩টায় লগইন চেষ্টা হয়েছে। সৌভাগ্যবশত fail2ban ও IP-সীমা থাকায় ঢোকা যায়নি, তবু শিক্ষাটি স্পষ্ট: চলে যাওয়া সদস্যের কী সঙ্গে সঙ্গে মুছতে হয়, ঠিক যেমন অফিসের চাবি ফেরত নেওয়া হয়। TechNova এখন প্রতিটি সদস্যের জন্য আলাদা ব্যবহারকারী রাখে, আর দল ছাড়ার চেকলিস্টে “কী মুছুন” প্রথম ধাপে লেখা থাকে।

`authorized_keys`-এর প্রতিটি লাইনে মন্তব্য (comment) লিখে রাখুন — কার কী, তা না জানলে মুছতে দ্বিধা হয়, আর দ্বিধা থেকেই পুরোনো কী জমে থাকে।

১৭.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

কী-লগইনে রূপান্তর (নিরাপদ ক্রম): (১) নিজের কম্পিউটারে `ssh-keygen -t ed25519`। (২) `ssh-copy-id` দিয়ে পাবলিক কী পাঠান। (৩) নতুন টার্মিনালে কী দিয়ে লগইন পরীক্ষা করুন — পুরোনো সেশন বন্ধ করবেন না। (৪) সফল হলে `sshd_config`-এর ব্যাকআপ নিয়ে `PasswordAuthentication no` ও `PermitRootLogin no` বসান। (৫) `sshd -t` দিয়ে যাচাই, তারপর `systemctl reload ssh`। (৬) আবার নতুন সেশনে লগইন পরীক্ষা করুন। (৭) fail2ban ইনস্টল ও চালু করুন।

১৭.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল ১: কী পরীক্ষা না করে পাসওয়ার্ড বন্ধ করা — সবচেয়ে ভয়ংকর লকআউট। ভুল ২: ব্যক্তিগত কী Git-এ বা চ্যাটে পাঠানো — এটি পাসওয়ার্ডের চেয়েও সংবেদনশীল। ভুল ৩: "SSH পোর্ট বদলালেই নিরাপদ" — পোর্ট বদলানো শুধু শব্দ কমায়, নিরাপত্তা দেয় না; কী-ই আসল সুরক্ষা। ভুল ৪: রুট দিয়ে সরাসরি লগইন — কে কী করল, তার হিসাব থাকে না। ভুল ৫: `chmod 777` দিয়ে অনুমতি সমস্যা "মেটানো" — সবাইকে সব অধিকার দেওয়া নিরাপত্তা ভঙ্গ।

১৭.১২ নিরাপত্তা বিবেচনা

ব্যক্তিগত কীতে পাসফ্রেজ (passphrase — কী খুলতে বাড়তি পাসওয়ার্ড) দিন, যাতে ল্যাপটপ চুরি গেলেও কী ব্যবহার করা না যায়। `~/.ssh` ডিরেক্টরি ৭০০ ও `authorized_keys` ৬০০ অনুমতিতে রাখুন — ভুল অনুমতি হলে SSH কী উপেক্ষা করে। নিয়মিত `lastb` ও `auth.log` দেখুন। সার্ভারে অপ্রয়োজনীয় ব্যবহারকারী রাখবেন না; চলে যাওয়া সদস্যের কী সঙ্গে সঙ্গে মুছুন।

১৭.১৩ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
কী দিয়ে লগইন হয় না, পাসওয়ার্ড চায়	অনুমতি ভুল / কী যোগ হয়নি	`chmod 700 ~/.ssh; chmod 600 ~/.ssh/authorized_keys`
SSH রিলোডের পর সংযোগ বন্ধ	কনফিগ ভুল	`sshd -t`, ব্যাকআপ থেকে ফেরান, কনসোল ব্যবহার
নিজের IP বন্ধ (banned)	fail2ban	কনসোল থেকে `fail2ban-client unban <IP>`
Permission denied (publickey)	সার্ভারে পাবলিক কী নেই	`ssh-copy-id` আবার চালান

আরও লক্ষণ

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
`ssh -v`-তে "no such identity" বা কী পাওয়া যাচ্ছে না	কী ফাইলের পথ ভুল, বা এজেন্টে কী নেই	`ls ~/.ssh/`; `ssh-add -l` দিয়ে এজেন্টে কী আছে দেখুন

“WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED”	সার্ভার নতুন করে বানানো হয়েছে, হোস্ট কী বদলেছে	সত্যিই নতুন সার্ভার কি না নিশ্চিত হয়ে `ssh-keygen -R 203.0.113.10` দিয়ে পুরোনো রেকর্ড মুছুন
`AllowUsers` -এ নাম নেই বলে লগইন ব্যর্থ	নতুন ব্যবহারকারী অনুমোদিত তালিকায় নেই	`sshd_config` -এ `AllowUsers` -এ নাম যোগ করে `sshd -t` যাচাই ও রিলোড
কী দিয়ে লগইন হয়, কিন্তু `sudo` পাসওয়ার্ড চায়	ব্যবহারকারীর পাসওয়ার্ড আলাদা, বা sudo নিয়ম নেই	এটি স্বাভাবিক — sudo-র জন্য ব্যবহারকারীর পাসওয়ার্ড লাগে; `sudo -l` দিয়ে অধিকার দেখুন

১৭.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: কী + পাসফ্রেজ, রুট লগইন বন্ধ, পাসওয়ার্ড লগইন বন্ধ (কী যাচাইয়ের পর), fail2ban, স্বয়ংক্রিয় নিরাপত্তা আপডেট, নিয়মিত লগ পর্যালোচনা। অ্যান্টি-প্যাটার্ন: পাসওয়ার্ড-মাত্র লগইন, `PermitRootLogin yes`, ব্যক্তিগত কী ভাগাভাগি, `chmod 777`, আপডেট মাসের পর মাস ফেলে রাখা।

১৭.১৫ অধ্যায়ের সারাংশ

SSH নিরাপত্তার কেন্দ্র হলো কী-ভিত্তিক লগইন: পাসওয়ার্ড আন্দাজ করা যায়, কী আন্দাজ করা যায় না। কী যাচাই করে তারপর পাসওয়ার্ড বন্ধ করুন, রুট লগইন বন্ধ করুন, fail2ban বসান, আপডেট চালু রাখুন — আর প্রতিটি পরিবর্তনের আগে ব্যাকআপ ও দ্বিতীয় সেশন রাখুন।

১৭.১৬ মূল প্রযুক্তিগত পরিভাষা

SSH কী জোড়া — ব্যক্তিগত + পাবলিক কী; **পাসফ্রেজ** — কী খুলতে বাড়তি পাসওয়ার্ড; **ব্রুট-ফোর্স** — বারবার আন্দাজ করে ঢোকার চেষ্টা; **fail2ban** — ব্যর্থ চেষ্টাকারী IP বন্ধ করার টুল; **সর্বনিম্ন সুযোগ** — যতটুকু দরকার ততটুকু অধিকার; **sshd_config** — SSH সার্ভারের কনফিগ ফাইল।

১৭.১৭ জ্ঞান যাচাই প্রশ্ন

1. SSH কী পাসওয়ার্ডের চেয়ে নিরাপদ কেন?
2. `PasswordAuthentication no` করার আগে কী যাচাই করতে হবে?
3. `PermitRootLogin no` -এর যুক্তি কী?
4. fail2ban কীভাবে ব্রুট-ফোর্স ঠেকায়?
5. ব্যক্তিগত কী ও পাবলিক কী কোথায় থাকে?
6. `chmod 777` কেন বিপজ্জনক?
7. ব্যক্তিগত কী চুরি গেলে প্রথমে কী করবেন এবং কেন?
8. “REMOTE HOST IDENTIFICATION HAS CHANGED” সতর্কবার্তা দেখলে কী যাচাই করবেন?

১৭.১৮ ব্যবহারিক অনুশীলন

1. ল্যাব VM-এ ed25519 কী জোড়া বানিয়ে `ssh-copy-id` দিয়ে কী-লগইন পরীক্ষা করুন।

2. কী-লগইন সফল হওয়ার পর `sshd_config`-এ `PasswordAuthentication no` বসিয়ে `sshd -t` ও রিলোড করুন — নতুন সেশনে পরীক্ষা করুন।
3. fail2ban ইনস্টল করে ইচ্ছাকৃত ভুল লগইন দিয়ে ব্যান পরীক্ষা করুন, তারপর আনব্যান করুন।
4. `lastb` ও `/var/log/auth.log` দেখে সাম্প্রতিক লগইন চেষ্টার চিত্র লিখুন।
5. নিজের ব্যক্তিগত কী থেকে `ssh-keygen -y` দিয়ে পাবলিক কী বের করুন এবং `authorized_keys`-এর লাইনের সাথে মিলিয়ে দেখুন।

অধ্যায় ১৮

ব্যাকআপ, রিস্টোর ও দুর্যোগ পুনরুদ্ধার মৌলিক বিষয়

১৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) ৩-২-১ ব্যাকআপ নীতি ব্যাখ্যা করতে; (২) RPO ও RTO-র অর্থ ও পার্থক্য বলতে; (৩) ফাইল, ডাটাবেস ও কনফিগের ব্যাকআপ স্ক্রিপ্ট লিখতে; (৪) রিস্টোর পরীক্ষা করতে; (৫) দুর্যোগের পর পুনরুদ্ধারের পরিকল্পনা লিখতে।

১৮.২ প্রোডাকশনে কেন এটি গুরুত্বপূর্ণ

ব্যাকআপ নেই মানে তথ্য হারানো সময়ের ব্যাপার — ডিস্ক নষ্ট, ভুল কমান্ডে ডিলিট, র‍্যানসমওয়্যার, অথবা ডেটা সেন্টারের সমস্যা। সবচেয়ে কঠিন সত্য: ব্যাকআপ থাকলেই হবে না, রিস্টোর করতে পারতে হবে। বহু দল আবিষ্কার করে — দুর্যোগের দিনে — যে তাদের ব্যাকআপ ভাঙা, অসম্পূর্ণ, অথবা রিস্টোরের নিয়ম কেউ জানে না। ব্যাকআপের মূল্য রিস্টোরের সময়েই প্রমাণিত হয়।

১৮.৩ পূর্বশর্ত জ্ঞান

ফাইল, ডিরেক্টরি, ডাটাবেস ও ক্রন (অধ্যায় ১৯) সম্পর্কে প্রাথমিক ধারণা থাকলে ভালো। `tar` ও `rsync` এখানেই ব্যাখ্যা করা হয়েছে।

১৮.৪ প্রথম নীতি থেকে ব্যাখ্যা

ব্যাকআপ মানে তথ্যের অতিরিক্ত কপি, এমন জায়গায় যা মূল তথ্যের সাথে একসাথে নষ্ট হবে না। **৩-২-১ নীতি:** তথ্যের **৩**টি কপি রাখুন, **২**টি ভিন্ন মাধ্যমে (যেমন সার্ভার + বাইরের ডিস্ক), **১**টি ভিন্ন স্থানে (অন্য শহর/ক্লাউড)। যুক্তি: একই ডিস্কের দুটি কপি একই আগুনে পোড়ে; একই ভবনের দুটি কপি একই বন্যায় ডোবে। **RPO** (Recovery Point Objective — কতটুকু তথ্য হারানো মেনে নেওয়া যায়; যেমন ১ ঘণ্টা মানে সর্বোচ্চ ১ ঘণ্টার তথ্য হারাবে) ঠিক করে ব্যাকআপ কত ঘন ঘন নিতে হবে। **RTO** (Recovery Time Objective — কত সময়ে সেবা ফেরাতে হবে; যেমন ৪ ঘণ্টা) ঠিক করে পুনরুদ্ধার কত দ্রুত হতে হবে। RPO ছোট মানে ঘন ঘন ব্যাকআপ; RTO ছোট মানে আগে থেকে প্রস্তুত রিস্টোর প্রক্রিয়া।

১৮.৫ স্থাপত্য / ধারণাগত চিত্র

[প্রোডাকশন সার্ভার: অ্যাপ + MongoDB + কনফিগ] → দৈনিক ব্যাকআপ → [কপি ১: একই সার্ভারের আলাদা ডিস্ক] + [কপি ২: বাইরের স্টোরেজ] + [কপি ৩: ভিন্ন স্থান/ক্লাউড]

সময়রেখা: ব্যাকআপ (প্রতি ৬ ঘণ্টা → RPO ৬ ঘণ্টা) ..... দুর্যোগ ..... রিস্টোর পরীক্ষা (লক্ষ্য: RTO ৪ ঘণ্টার মধ্যে সেবা ফেরানো)

১৮.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

ব্যাকআপের তিনটি স্তর: (১) **ফাইল ব্যাকআপ** — অ্যাপ কোড, আপলোড, কনফিগ (`tar` বা `rsync`); (২) **ডাটাবেস ব্যাকআপ** — চলমান ডাটাবেসের ফাইল কপি করবেন না, ডাটাবেসের নিজস্ব টুলে ডাম্প নিন (`mongodump`, `pg_dump`); (৩) **কনফিগ ব্যাকআপ** — Nginx, systemd, পরিবেশ ভ্যারিয়েবলের তালিকা (গোপন তথ্য ছাড়া)। **পূর্ণ ব্যাকআপ** (full —

সবকিছু) সপ্তাহে একবার, **ক্রমবর্ধমান** (incremental — শুধু পরিবর্তন) প্রতিদিন — এতে সময় ও জায়গা বাঁচে। ব্যাকআপ এনক্রিপ্ট করুন: ব্যাকআপে গ্রাহকের তথ্য থাকে, চুরি গেলে মূল তথ্য ফাঁসের সমান ক্ষতি। ব্যাকআপের লগ রাখুন — কখন, কী, সফল না ব্যর্থ — আর ব্যর্থতায় অ্যালার্ট বসান। নীরবে ব্যর্থ হওয়া ব্যাকআপ না থাকার চেয়ে খারাপ, কারণ মিথ্যা নিরাপত্তা দেয়।

কাজের উদাহরণ — পূর্ণ, ক্রমবর্ধমান ও পার্থক্যমূলক ব্যাকআপের হিসাব

TechNova-র অ্যাপ ও ডাটাবেস মিলিয়ে মোট ২০ গিগাবাইট। প্রতিদিন পূর্ণ ব্যাকআপ নিলে সপ্তাহে ১৪০ গিগাবাইট লাগে — অযথা। তাই নীতি হলো: রবিবার পূর্ণ ব্যাকআপ (২০ গিগাবাইট), সোম–শনি শুধু পরিবর্তন (ক্রমবর্ধমান, incremental — শেষ ব্যাকআপের পর যা বদলেছে)। প্রতিদিন গড়ে ১ গিগাবাইট বদলালে সপ্তাহের মোট হয় ২০ + ৬ = ২৬ গিগাবাইট — প্রায় এক-পঞ্চমাংশ। তবে রিস্টোরের সময় ক্রমবর্ধমান ব্যাকআপের সব কপি একে একে লাগে, তাই রিস্টোর ধীর হয়; পূর্ণ ব্যাকআপ একটিতেই ফেরানো যায়। মাঝের পথ হলো **পার্থক্যমূলক** (differential — শেষ পূর্ণ ব্যাকআপের পর যা বদলেছে): রিস্টোরে শুধু পূর্ণ + সর্বশেষ পার্থক্যমূলক কপি লাগে। ছোট দলের জন্য সহজ নিয়ম: সপ্তাহে একবার পূর্ণ, প্রতিদিন ক্রমবর্ধমান, আর মাসে একবার রিস্টোর পরীক্ষা। RPO মনে রাখুন: ব্যাকআপ প্রতি ৬ ঘণ্টায় নিলে সর্বোচ্চ ৬ ঘণ্টার তথ্য হারাতে পারে — ব্যবসা যদি ১ ঘণ্টার বেশি হারাতে না পারে, তবে ব্যাকআপও প্রতি ঘণ্টায় নিতে হবে। হিসাবটি লিখে রাখুন, যাতে সবাই জানে কী হারানো মেনে নেওয়া হয়েছে।

ধরন	জায়গা	রিস্টোরের গতি	কখন ভালো
পূর্ণ (full)	সবচেয়ে বেশি	সবচেয়ে দ্রুত	সাপ্তাহিক ভিত্তি
ক্রমবর্ধমান (incremental)	সবচেয়ে কম	ধীর — সব কপি লাগে	দৈনিক, বড় তথ্যে
পার্থক্যমূলক (differential)	মাঝামাঝি	মাঝামাঝি — দুটি কপি	দৈনিক, দ্রুত রিস্টোর চাইলে

১৮.৭ গুরুত্বপূর্ণ কমান্ড ও বিশ্লেষণ

```
tar -czf /backup/app-2026-11-08.tar.gz /var/www/technova.example
# -c তৈরি, -z gzip সংকোচন, -f ফাইলের নাম
tar -tzf /backup/app-2026-11-08.tar.gz | head   # ভেতরে কী আছে দেখা (না খুলে)
rsync -av --delete /var/www/technova.example/ /backup/www/
# -a সংরক্ষণ মোড, -v বিস্তারিত; --delete সতর্কতায় ব্যবহার করুন
mongodump --uri="mongodb://127.0.0.1:27017/technova" --out=/backup/mongo-2026-11-08
pg_dump -U appuser technova > /backup/db-2026-11-08.sql
ls -lh /backup/            # ব্যাকআপ ফাইলের আকার ও তারিখ যাচাই
sha256sum /backup/*.tar.gz # অখণ্ডতা যাচাই (চেকসাম)
```

আরও কমান্ড

```
rsync -av --dry-run /var/www/technova.example/ /backup/www/   # আসলে না কপি করে কী হবে দেখা
tar -tvf /backup/app-2026-11-08.tar.gz | head -20   # আর্কাইভের ভেতরের তালিকা দেখা
gpg --symmetric --cipher-algo AES256 /backup/app-2026-11-08.tar.gz   # ব্যাকআপ এনক্রিপ্ট করা
du -sh /backup/* | sort -rh   # কোন ব্যাকআপ কত বড়, সাজিয়ে দেখা
mongorestore --dryRun --uri="mongodb://127.0.0.1:27017/testdb" /backup/mongo-2026-11-08   # রিস্টোর না করে যাচাই
sha256sum -c /backup/checksums.txt   # সংরক্ষিত চেকসামের সাথে মিলিয়ে অখণ্ডতা যাচাই
```

ব্যাখ্যা: `--dry-run` হলো ব্যাকআপের সবচেয়ে নিরাপদ অভ্যাস — কী কপি হবে, কী মুছবে, তা আগেই দেখা যায়। `gpg --symmetric` ব্যাকআপ এনক্রিপ্ট করে; উদাহরণের পাসফ্রেজ হিসেবে শুধু ফিকশনাল `CHANGE-ME-2026` ব্যবহার করুন — আসল পাসফ্রেজ কখনো বইয়ে বা স্ক্রিপ্টে লিখবেন না, নিরাপদ জায়গায় আলাদাভাবে রাখুন। `mongorestore --dryRun` রিস্টোরের আগে ডাম্পটি পড়া যায় কি না যাচাই করে, কিন্তু তথ্য বদলায় না। চেকসাম ফাইলটি ব্যাকআপের সাথে রাখুন, যাতে মাস পরে বোঝা যায় ফাইলটি অক্ষত আছে কি না।

> **WARNING:** `rsync --delete` চালানোর আগে সবসময় `--dry-run` দিয়ে দেখুন কী মুছবে। ভুল গন্তব্য পথে `--delete` মানে আসল ফাইল মুছে ফেলা — আর ব্যাকআপই তখন একমাত্র ভরসা।

১৮.৮ কনফিগারেশন উদাহরণ

দৈনিক ব্যাকআপ স্ক্রিপ্ট (`/usr/local/bin/backup.sh`):

```
#!/bin/bash
set -euo pipefail
DATE=$(date +%F)
DEST="/backup/$DATE"
mkdir -p "$DEST"
tar -czf "$DEST/app.tar.gz" /var/www/technova.example
mongodump --uri="mongodb://127.0.0.1:27017/technova" --out="$DEST/mongo" >/dev/null
cp /etc/nginx/sites-available/technova.example "$DEST/" 2>/dev/null || true
find /backup -maxdepth 1 -type d -mtime +14 -exec rm -rf {} \;   # ১৪ দিনের পুরোনো মুছুন
echo "Backup $DATE OK" | tee -a /var/log/backup.log
# সতর্কতা: rm -rf যুক্ত লাইনটি ল্যাবে পরীক্ষা করে, পথ যাচাই করে তবেই চালান।
# আসল পাসওয়ার্ড/URI স্ক্রিপ্টে লিখবেন না — পরিবেশ ভ্যারিয়েবল বা সুরক্ষিত ফাইল ব্যবহার করুন।
```

> **WARNING:** ব্যাকআপ স্ক্রিপ্টে `rm -rf` দিয়ে পুরোনো ব্যাকআপ মোছা বিপজ্জনক — ভুল পথে চললে আসল তথ্য মুছে যাবে। প্রথমে `echo` দিয়ে কী মুছবে দেখুন, ল্যাবে পরীক্ষা করুন, তারপরই স্বয়ংক্রিয় করুন।

১৮.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — ভুলে ডিলিট: ডেভেলপার ভুল ডিরেক্টরি মুছে ফেলল — গত রাতের ব্যাকআপ থেকে `tar -xzf` দিয়ে ফেরানো হলো, ক্ষতি একদিনের কাজ নয়, কয়েক ঘণ্টা। **পরিস্থিতি ২ — ডিস্ক নষ্ট:** SSD হঠাৎ মারা গেল — ভিন্ন স্থানের কপি থেকে নতুন সার্ভারে রিস্টোর, RTO-র মধ্যে সেবা ফিরল। **পরিস্থিতি ৩ — র‍্যানসমওয়্যার:** সার্ভারের ফাইল এনক্রিপ্ট হয়ে গেল — কিন্তু ব্যাকআপ আলাদা মাধ্যমে থাকায় মুক্তিপণ না দিয়ে পুনরুদ্ধার সম্ভব হলো।

পরিস্থিতি ৪ — TechNova-র ডাটাবেস আপগ্রেডের আগে রোলব্যাক পরিকল্পনা: TechNova MongoDB আপগ্রেড করার আগে `mongodump` দিয়ে পূর্ণ ডাম্প নিল, চেকসাম মেলাল, আর ল্যাব সার্ভারে `mongorestore` দিয়ে পরীক্ষামূলকভাবে ফিরিয়ে দেখল — রিস্টোর সফল হলো ১২ মিনিটে। আপগ্রেডের পর অ্যাপে ত্রুটি দেখা দিলে দল আতঙ্কিত হলো না: পুরোনো সংস্করণে ফিরে ডাম্প থেকে রিস্টোর করা হলো, মোট ডাউনটাইম ২৫ মিনিট — যা RTO (৪ ঘণ্টা)-র অনেক ভেতরে। শিক্ষা: বড় পরিবর্তনের আগে ব্যাকআপ নেওয়া যথেষ্ট নয় — ব্যাকআপটি কাজ করে কি না, তা আগেই পরীক্ষা করতে হয়। রোলব্যাক পরিকল্পনা লিখে রাখুন: কোন ডাম্প, কোন কমান্ড, কে চালাবে, কত সময় লাগবে — দুর্যোগের সময়ে মনে করে করার সময় থাকে না।

১৮.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

রিস্টোর পরীক্ষা (সবচেয়ে গুরুত্বপূর্ণ অনুশীলন): (১) ল্যাব VM-এ খালি ডিরেক্টরি বানান। (২) ব্যাকআপ ফাইল কপি করুন। (৩) `tar -tzf` দিয়ে ভেতর দেখুন, তারপর `tar -xzf -C /tmp/restore-test` দিয়ে খুলুন। (৪) ফাইলগুলো আসলটির সাথে মিলিয়ে দেখুন (`diff -r`)। (৫) MongoDB ডাম্প `mongorestore` দিয়ে পরীক্ষামূলক ডাটাবেসে ফেরান ও কয়েকটি রেকর্ড গুনে দেখুন। (৬) কত সময় লাগল, লিখে রাখুন — এটাই আপনার আসল RTO। মাসে অন্তত একবার এই পরীক্ষা করুন।

১৮.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল ১: “ব্যাকআপ নেওয়া হয়েছে” — কিন্তু রিস্টোর কখনো পরীক্ষা করা হয়নি। ভুল ২: ব্যাকআপ একই ডিস্কে রাখা — ডিস্ক নষ্ট হলে দুটোই যায়। ভুল ৩: চলমান ডাটাবেসের ফাইল সরাসরি কপি করা — অসম্পূর্ণ/ভাঙা ডাম্প হয়। ভুল ৪: ব্যাকআপ ব্যর্থতার অ্যালার্ট না থাকা — মাস ধরে ব্যর্থ হচ্ছে, কেউ জানে না। ভুল ৫: গোপন তথ্য (পাসওয়ার্ড, কী) ব্যাকআপে খোলা অবস্থায় রাখা।

১৮.১২ নিরাপত্তা বিবেচনা

ব্যাকআপ এনক্রিপ্ট করুন ও অ্যাক্সেস সীমিত রাখুন (`chmod 700 /backup`) — ব্যাকআপ মানেই তথ্যের দ্বিতীয় দরজা। ব্যাকআপে গোপন তথ্য থাকলে তা আলাদাভাবে সুরক্ষিত রাখুন, Git-এ তুলবেন না। পুরোনো ব্যাকআপ মুছে ফেলার আগে নিশ্চিত হোন নতুন ব্যাকআপ সফল হয়েছে। রিস্টোর পরীক্ষা আইসোলেটেড পরিবেশে করুন — পরীক্ষামূলক রিস্টোর যেন প্রোডাকশন তথ্য না ছুঁয়ে ফেলে।

১৮.১৩ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
ব্যাকআপ ফাইল ০ বাইট	ডিস্ক পূর্ণ / অনুমতি	`df -h`, স্ক্রিপ্টের লগ, হাতে চালিয়ে দেখুন
`mongodump` ব্যর্থ	সংযোগ / প্রমাণীকরণ	URI ও MongoDB চলছে কি না যাচাই
রিস্টোরে ফাইল ভাঙা	ব্যাকআপ অসম্পূর্ণ	`tar -tzf` ও চেকসাম; পূর্ণ ব্যাকআপ নিন
ব্যাকআপে জায়গা শেষ	পুরোনো কপি জমেছে	রিটেনশন নীতি (১৪ দিন) ও সংকোচন

আরও লক্ষণ

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
`rsync` গন্তব্যের ফাইল মুছে ফেলছে	`--delete` ভুল পথে চলছে	তৎক্ষণাৎ থামান; `--dry-run` দিয়ে পথ যাচাই করে আবার চালান
ব্যাকআপ সফল দেখায়, কিন্তু রিস্টোরে তথ্য পুরোনো	ভুল ডাটাবেস/ভুল তারিখের ডাম্প	ডাম্পের তারিখ ও ডাটাবেসের নাম যাচাই; স্ক্রিপ্টে তারিখ স্পষ্ট লিখুন
এনক্রিপ্ট করা ব্যাকআপ খোলা যাচ্ছে না	পাসফ্রেজ হারিয়েছে	পাসফ্রেজ নিরাপদ জায়গায় আলাদাভাবে সংরক্ষণ করুন — হারালে ব্যাকআপ অকেজো

ক্রনে ব্যাকআপ চলছে না, হাতে চলে	ক্রনের পরিবেশে পথ/ভ্যারিয়েবল নেই	স্ক্রিপ্টে সম্পূর্ণ পথ লিখুন; অধ্যায় ১৯ দেখুন

১৮.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: ৩-২-১ নীতি, RPO/RTO লিখিতভাবে ঠিক করা, স্বয়ংক্রিয় ব্যাকআপ + ব্যর্থতায় অ্যালার্ট, মাসিক রিস্টোর পরীক্ষা, এনক্রিপ্টেড ব্যাকআপ, রিটেনশন নীতি। অ্যান্টি-প্যাটার্ন: হাতে মনে করে ব্যাকআপ নেওয়া; একই ডিস্কে ব্যাকআপ; রিস্টোর না পরীক্ষা করা; “ব্যাকআপ আছে” বলে নিশ্চিন্ত থাকা।

১৮.১৫ অধ্যায়ের সারাংশ

ব্যাকআপের মূল্য রিস্টোরে। ৩-২-১ নীতি মেনে তিনটি কপি, দুটি মাধ্যম, একটি ভিন্ন স্থানে রাখুন। RPO ঠিক করে ব্যাকআপের ঘনত্ব, RTO ঠিক করে পুনরুদ্ধারের গতি। স্বয়ংক্রিয় করুন, ব্যর্থতায় অ্যালার্ট বসান, আর মাসে একবার ল্যাবে রিস্টোর পরীক্ষা করুন।

১৮.১৬ মূল প্রযুক্তিগত পরিভাষা

৩-২-১ নীতি — ৩ কপি, ২ মাধ্যম, ১ ভিন্ন স্থান; **RPO** — কতটুকু তথ্য হারানো চলবে; **RTO** — কত সময়ে সেবা ফেরাতে হবে; **পূর্ণ ব্যাকআপ** — সবকিছুর কপি; **ক্রমবর্ধমান ব্যাকআপ** — শুধু পরিবর্তনের কপি; **ডাম্প** — ডাটাবেসের রপ্তানি করা কপি; **রিটেনশন** — ব্যাকআপ কতদিন রাখা হবে।

১৮.১৭ জ্ঞান যাচাই প্রশ্ন

1. ৩-২-১ নীতির প্রতিটি সংখ্যার যুক্তি কী?
2. RPO ১ ঘণ্টা ও RTO ৪ ঘণ্টা মানে কী?
3. চলমান MongoDB-র ফাইল সরাসরি কপি করা উচিত নয় কেন?
4. ব্যাকআপ থাকলেও রিস্টোর পরীক্ষা কেন জরুরি?
5. পূর্ণ ও ক্রমবর্ধমান ব্যাকআপের পার্থক্য কী?
6. ব্যাকআপ ব্যর্থতার অ্যালার্ট না থাকলে কী ঝুঁকি?
7. ক্রমবর্ধমান ব্যাকআপে রিস্টোর ধীর হয় কেন?
8. এনক্রিপ্ট করা ব্যাকআপের পাসফ্রেজ হারালে কী হবে?

১৮.১৮ ব্যবহারিক অনুশীলন

1. ল্যাব VM-এ একটি ডিরেক্টরির `tar` ব্যাকআপ নিয়ে `tar -tzf` দিয়ে যাচাই করুন।
2. নমুনা ব্যাকআপ স্ক্রিপ্ট লিখে হাতে চালান ও `/var/log/backup.log`-এ ফলাফল দেখুন।
3. ব্যাকআপ থেকে `/tmp/restore-test`-এ রিস্টোর করে আসল ফাইলের সাথে `diff` করুন।
4. নিজের অ্যাপের জন্য RPO ও RTO লিখুন এবং ব্যাকআপের সময়সূচি ঠিক করুন।
5. একটি ফাইলের ব্যাকআপ `gpg --symmetric` দিয়ে এনক্রিপ্ট করে আবার ডিক্রিপ্ট করুন এবং চেকসাম মিলিয়ে দেখুন (পাসফ্রেজ নিরাপদে রাখুন)।

অধ্যায় ১৯

ক্রন, নির্ধারিত কাজ ও রক্ষণাবেক্ষণ স্বয়ংক্রিয়করণ

১৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) ক্রন (cron) কী ও কীভাবে কাজ করে ব্যাখ্যা করতে; (২) ক্রনট্যাব (crontab) সিনট্যাক্স পড়তে ও লিখতে; (৩) ব্যাকআপ, লগ পরিষ্কার ও আপডেটের নির্ধারিত কাজ বানাতে; (৪) systemd টাইমার (timer) কী তা বুঝতে; (৫) ব্যর্থ নির্ধারিত কাজের লগ দেখে সমস্যা ধরতে।

১৯.২ প্রোডাকশনে কেন এটি গুরুত্বপূর্ণ

সার্ভার রক্ষণাবেক্ষণের বেশিরভাগ কাজই পুনরাবৃত্তিমূলক: রাতে ব্যাকআপ, সপ্তাহে লগ পরিষ্কার, মাসে সার্টিফিকেট নবায়ন পরীক্ষা। মানুষ মনে করে করলে ভুলে যাবে, আর ভুলে গেলে ডিস্ক ভরবে, ব্যাকআপ থামবে, সার্টিফিকেট মেয়াদোত্তীর্ণ হবে। ক্রন সেই পুনরাবৃত্তি যন্ত্রের হাতে দেয় — নির্ভরযোগ্যভাবে, নির্দিষ্ট সময়ে, লগসহ।

১৯.৩ পূর্বশর্ত জ্ঞান

টার্মিনাল, স্ক্রিপ্ট ফাইল ও `chmod +x`-এর ধারণা থাকলে ভালো। সময়সূচির পাঁচটি ক্ষেত্র এখানেই ব্যাখ্যা করা হয়েছে।

১৯.৪ প্রথম নীতি থেকে ব্যাখ্যা

ক্রন (cron — লিনাক্সের সময়-ভিত্তিক কাজ চালানোর ডেমন) একটি ঘড়ি-পাহারাদার: প্রতি মিনিটে সে **ক্রনট্যাব** (crontab — কাজের সময়সূচির তালিকা) দেখে, কোন কাজের সময় হয়েছে কি না। সময় লেখা হয় পাঁচটি ক্ষেত্রে: মিনিট ঘণ্টা দিন-মাস মাস সপ্তাহের-দিন। যেমন `0 2 * * *` মানে প্রতিদিন রাত ২টায়। তারকা (`*`) মানে "প্রতিটি"। ক্রন কাজ চালায় ব্যাকগ্রাউন্ডে, টার্মিনাল ছাড়া — তাই কাজের আউটপুট কোথায় যাবে, তা আগেই ঠিক করতে হয়, নইলে ব্যর্থতা নীরবে হারিয়ে যায়। **systemd টাইমার** হলো আধুনিক বিকল্প: ক্রনের মতো সময়সূচি, কিন্তু systemd সার্ভিসের সাথে যুক্ত, লগ journald-এ যায়, আর "মিস হওয়া কাজ পরে চালানো" (persistent) সুবিধা আছে।

১৯.৫ স্থাপত্য / ধারণাগত চিত্র

[ক্রন ডেমন: প্রতি মিনিটে পরীক্ষা] → সময় মিললে → [স্ক্রিপ্ট চালানো: /usr/local/bin/backup.sh] → আউটপুট → লগ ফাইল / ইমেইল

ক্রনট্যাবের পাঁচ ক্ষেত্র: মিনিট (0-59) | ঘণ্টা (0-23) | মাসের দিন (1-31) | মাস (1-12) | সপ্তাহের দিন (0-7, 0 ও 7 = রবিবার)

উদাহরণ: 0 2 * * * = প্রতিদিন ০২:০০ | */15 * * * * = প্রতি ১৫ মিনিট | 0 0 * * 0 = প্রতি রবিবার মধ্যরাত

১৯.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

প্রতিটি ব্যবহারকারীর নিজের ক্রনট্যাব থাকে (`crontab -e` দিয়ে সম্পাদনা), আর সিস্টেম-ব্যাপী কাজ `/etc/crontab` ও `/etc/cron.d/`-এ থাকে। ক্রন কাজ চালায় ন্যূনতম পরিবেশে (environment) — আপনার টার্মিনালের `PATH` বা পরিবেশ ভ্যারিয়েবল সেখানে থাকে না। তাই স্ক্রিপ্টে সম্পূর্ণ পথ (`/usr/bin/node`, `/usr/local/bin/backup.sh`) লিখুন, আপেক্ষিক পথ নয়। কাজের আউটপুট লগ ফাইলে পাঠান (`>> /var/log/backup.log 2>&1`), যাতে ব্যর্থতা দেখা যায়।

ক্রন কাজ একই সময়ে দুবার চললে (আগেরটি শেষ হয়নি) তথ্য নষ্ট হতে পারে — দীর্ঘ কাজে `flock` দিয়ে তালা (lock) ব্যবহার করুন।

কাজের উদাহরণ — cron বনাম systemd টাইমার: কোনটি কখন

TechNova-র দৈনিক ব্যাকআপ প্রথমে ক্রনে ছিল: `0 2 * * * /usr/local/bin/backup.sh`। এক রাতে সার্ভার রক্ষণাবেক্ষণের জন্য বন্ধ ছিল রাত ২টায় — ক্রন সেই ব্যাকআপটি আর চালাল না, কারণ সময় পেরিয়ে গেছে। পরদিন কেউ টেরও পেল না। systemd টাইমারে `Persistent=true` থাকলে মিস হওয়া কাজটি সার্ভার চালু হওয়ার পর চালানো হতো। তবে টাইমারের দাম আছে: এর জন্য আলাদা সার্ভিস ফাইল ও টাইমার ফাইল লিখতে হয়, ক্রনের এক লাইনের চেয়ে বেশি কাজ। নিচের সারণি দেখে সিদ্ধান্ত নিন — ছোট, নিয়মিত, সবসময়-চালু সার্ভারের কাজে ক্রন যথেষ্ট; মিস হওয়া চলবে না এমন কাজে, বা লগ ও নির্ভরতা (dependency) দরকার হলে টাইমার ভালো। আরেকটি পার্থক্য: ক্রন কাজের আউটপুট লগ ফাইলে পাঠাতে হয়, টাইমারের আউটপুট journald-এ যায় — `journalctl -u backup.service` দিয়ে দেখা যায়। দ্বিতীয় দৃশ্য: স্বাস্থ্য পরীক্ষা প্রতি ১৫ মিনিটে ক্রনে ভালো চলে, কারণ একবার মিস হলে পরের ১৫ মিনিটে আবার চলবে — এখানে টাইমারের বাড়তি জটিলতা দরকার নেই।

বৈশিষ্ট্য	cron	systemd টাইমার
লেখা	এক লাইন	দুটি ফাইল (সার্ভিস + টাইমার)
মিস হওয়া কাজ	আর চলে না	`Persistent=true`-তে পরে চলে
লগ	নিজে ফাইলে পাঠাতে হয়	journald-এ যায়
নির্ভরতা	নেই	সার্ভিসের আগে-পরে শর্ত দেওয়া যায়

১৯.৭ গুরুত্বপূর্ণ কমান্ড ও বিশ্লেষণ

```
crontab -e            # নিজের ক্রনট্যাব সম্পাদনা
crontab -l            # বর্তমান ক্রনট্যাব দেখা
crontab -r            # ক্রনট্যাব মোছা (সতর্কতা: ব্যাকআপ ছাড়া করবেন না)
sudo systemctl status cron
sudo journalctl -u cron --since "today"   # ক্রনের আজকের লগ
grep CRON /var/log/syslog                 # ক্রন কী চালিয়েছে দেখা
flock -n /tmp/backup.lock /usr/local/bin/backup.sh   # একসাথে দুবার চলা ঠেকানো
```

আরও কমান্ড

```
crontab -l                      # বর্তমান ক্রনট্যাব দেখা (মোছার আগে ব্যাকআপ নিন)
crontab -l > ~/crontab.bak      # ক্রনট্যাবের ব্যাকআপ নেওয়া
systemctl list-timers --all     # সব systemd টাইমার ও পরের চালনার সময় দেখা
sudo journalctl -u backup.service --since "1 day ago"   # টাইমারের সার্ভিসের লগ দেখা
sudo systemctl status backup.timer   # টাইমার সক্রিয় কি না দেখা
run-parts --test /etc/cron.daily    # দৈনিক ক্রন ডিরেক্টরিতে কী চলবে, পরীক্ষা করা
```

ব্যাখ্যা: `crontab -l > ~/crontab.bak` পাঁচ সেকেন্ডের কাজ, কিন্তু ভুলে `crontab -r` চালিয়ে ফেললে এটাই উদ্ধার করে। `systemctl list-timers` দেখায় প্রতিটি টাইমার পরেরবার কখন চলবে — "কাজটি কি আদৌ সময়সূচিতে আছে" প্রশ্নের সরাসরি উত্তর। `run-parts --test` কিছু না চালিয়ে শুধু তালিকা দেয়, তাই সিস্টেমের দৈনিক কাজগুলো পরীক্ষা

করতে নিরাপদ। টাইমারের লগ journald-এ থাকায় আলাদা লগ ফাইল বানাতে হয় না — তবে লগের আকারের দিকে নজর রাখুন (অধ্যায় ২০)।

১৯.৮ কনফিগারেশন উদাহরণ

নমুনা ক্রনট্যাব:

```
# প্রতিদিন রাত ২টায় ব্যাকআপ
0 2 * * * /usr/local/bin/backup.sh >> /var/log/backup.log 2>&1
# প্রতি ১৫ মিনিটে স্বাস্থ্য পরীক্ষা
*/15 * * * * /usr/local/bin/healthcheck.sh >> /var/log/health.log 2>&1
# প্রতি রবিবার রাত ৩টায় পুরোনো লগ পরিষ্কার (সতর্কতায় — নিচে দেখুন)
0 3 * * 0 find /var/log/app -name "*.log" -mtime +30 -delete
# প্রতি মাসের ১ তারিখে ডিস্ক প্রতিবেদন
0 6 1 * * df -h > /var/log/disk-report.txt 2>&1
```

> **WARNING:** ফাইল মোছার ক্রন কাজ (`find ... -delete`) অত্যন্ত বিপজ্জনক। প্রথমে `-delete`-এর বদলে `-print` দিয়ে কী মুছবে দেখুন, ল্যাবে পরীক্ষা করুন, পথ দুবার যাচাই করুন। ভুল পথে একটি ক্রন কাজ প্রতি রাতে তথ্য মুছে ফেলবে — কেউ টের পাওয়ার আগেই।

systemd টাইমার উদাহরণ: সার্ভিস `backup.service` (ExecStart-এ স্ক্রিপ্ট) ও টাইমার `backup.timer` (`OnCalendar=daily`, `Persistent=true`) — সার্ভার বন্ধ থাকলে মিস হওয়া কাজ পরে চালায়।

১৯.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — ব্যাকআপ থেমে গেছে: ক্রন লগে দেখা গেল স্ক্রিপ্টের পথ ভুল ছিল — ব্যর্থতা লগ না দেখায় তিন সপ্তাহ ব্যাকআপ হয়নি। **পরিস্থিতি ২ — সার্টিফিকেট মেয়াদোত্তীর্ণ:** নবায়ন পরীক্ষার ক্রন কাজ ছিল না — সাইটে "নিরাপদ নয়" সতর্কতা। **পরিস্থিতি ৩ — ডিস্ক ভরা:** অস্থায়ী ফাইল পরিষ্কারের ক্রন কাজ ভুল ডিরেক্টরি দেখছিল — লগে দেখা গেল কাজ চলছে কিন্তু কিছু মুছছে না।

পরিস্থিতি ৪ — TechNova-র রাতের রিপোর্ট দুবার চলল: TechNova-র মাসিক রিপোর্টের স্ক্রিপ্টটি বড় ডাটাবেসে ৭০ মিনিট ধরে চলত, কিন্তু ক্রনে সেটি প্রতি ঘণ্টায় চালানোর নিয়ম ছিল — ফলে আগেরটি শেষ হওয়ার আগেই নতুনটি শুরু হয়ে দুটি প্রসেস একসাথে ডাটাবেসে লিখতে গিয়ে তথ্য জট পাকাল। সমাধান ছিল `flock -n /tmp/report.lock` দিয়ে তালা দেওয়া: আগেরটি চলতে থাকলে নতুনটি সঙ্গে সঙ্গে বেরিয়ে যায়, আর লগে লেখা থাকে "আগের কাজ চলছে"। আরও ভালো সমাধান ছিল সময়সূচি বদলে রিপোর্টটি দিনে একবার চালানো। শিক্ষা: নির্ধারিত কাজের সময়কাল জানুন — কাজটি যতক্ষণ চলে, তার চেয়ে কম বিরতিতে সময়সূচি বসাবেন না। লম্বা কাজে সবসময় `flock` ব্যবহার করুন, যাতে একই কাজ দুবার না চলে।

১৯.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

নতুন নির্ধারিত কাজ যোগ করা: (১) স্ক্রিপ্ট লিখে হাতে চালিয়ে পরীক্ষা করুন (`bash /usr/local/bin/backup.sh`)। (২) স্ক্রিপ্টে সম্পূর্ণ পথ ব্যবহার করুন। (৩) `crontab -e`-তে লাইন যোগ করুন, আউটপুট লগ ফাইলে পাঠান। (৪) পরের নির্ধারিত সময়ের পর লগ দেখে নিশ্চিত হোন কাজটি চলেছে। (৫) ইচ্ছাকৃতভাবে একটি পরীক্ষামূলক কাজ (`* * * * * date >> /tmp/cron-test.log`) এক মিনিট চালিয়ে ক্রন কাজ করছে কি না যাচাই করুন, তারপর মুছে দিন।

১৯.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল ১: আপেক্ষিক পথ বা পরিবেশ ভ্যারিয়েবলের ওপর নির্ভর করা — ক্রনে সেগুলো থাকে না। ভুল ২: আউটপুট কোথাও না পাঠানো — ব্যর্থতা অদৃশ্য থাকে। ভুল ৩: `crontab -r` ভুলে চালিয়ে সব কাজ মুছে ফেলা। ভুল ৪: সময়ক্ষেত্রের ক্রম গুলিয়ে ফেলা (মিনিট আগে, ঘণ্টা পরে)। ভুল ৫: প্রতি মিনিটে ভারী কাজ চালানো — সার্ভারকে ব্যস্ত রাখা। ভুল ৬: মোছার ক্রন কাজ পরীক্ষা ছাড়া চালু করা।

১৯.১২ নিরাপত্তা বিবেচনা

ক্রন স্ক্রিপ্টে আসল পাসওয়ার্ড বা API কী লিখবেন না — সুরক্ষিত ফাইল বা পরিবেশ ভ্যারিয়েবল ব্যবহার করুন। স্ক্রিপ্টের মালিকানা ও অনুমতি সীমিত রাখুন (`chmod 700`), যাতে অন্য ব্যবহারকারী বদলাতে না পারে — ক্রন রুট হিসেবে চললে বদলে যাওয়া স্ক্রিপ্ট রুট অধিকারে চলবে। `/etc/cron.d/` -তে কাজ যোগ করার সময় ফাইলের অনুমতি যাচাই করুন।

১৯.১৩ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
ক্রন কাজ চলছে না	সিনট্যাক্স ভুল / PATH সমস্যা	`journalctl -u cron` , হাতে চালিয়ে দেখুন, সম্পূর্ণ পথ ব্যবহার
হাতে চলে, ক্রনে চলে না	পরিবেশ ভ্যারিয়েবল নেই	স্ক্রিপ্টে ভ্যারিয়েবল ও PATH স্পষ্ট লিখুন
কাজ দুবার চলছে	আগেরটি শেষ হয়নি	`flock` দিয়ে তালা ব্যবহার
লগে কিছু নেই	আউটপুট পাঠানো হয়নি	`>> /var/log/x.log 2>&1` যোগ করুন

আরও লক্ষণ

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
ক্রন কাজ ভুল সময়ে চলে	সার্ভারের সময়ক্ষেত্র (timezone) ভুল	`timedatectl` দিয়ে সময়ক্ষেত্র যাচাই; ক্রন সিস্টেম সময় মেনে চলে
ক্রনট্যাবে `%` থাকলে কাজ ভেঙে যায়	ক্রনে `%` মানে নতুন লাইন	`%` এড়িয়ে চলুন বা `\%` লিখুন; জটিল কমান্ড স্ক্রিপ্ট ফাইলে রাখুন
টাইমার সক্রিয়, কিন্তু সার্ভিস চলে না	টাইমার চালু, সার্ভিস ফাইল ভুল	`systemctl status backup.service` ও `journalctl -u backup.service` দেখুন
ক্রন কাজ চলছে, কিন্তু ইমেইল/ সতর্কতা আসছে না	সার্ভারে মেইল ব্যবস্থা নেই	আউটপুট লগ ফাইলে পাঠান ও লগে অ্যালার্ট বসান — মেইলের ওপর নির্ভর করবেন না

১৯.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: স্ক্রিপ্ট হাতে পরীক্ষা করে তারপর ক্রনে দেওয়া, সম্পূর্ণ পথ, আউটপুট লগে পাঠানো, ব্যর্থতায় অ্যালার্ট, `flock`, ক্রনট্যাবের ব্যাকআপ রাখা। অ্যান্টি-প্যাটার্ন: পরীক্ষা ছাড়া মোছার কাজ; প্রতি মিনিটে ভারী কাজ; লগ ছাড়া ক্রন; গোপন তথ্য স্ক্রিপ্টে লেখা।

১৯.১৫ অধ্যায়ের সারাংশ

ক্রন মানে নির্দিষ্ট সময়ে নির্ভরযোগ্য পুনরাবৃত্তি। পাঁচটি সময়ক্ষেত্র শিখুন, স্ক্রিপ্টে সম্পূর্ণ পথ লিখুন, আউটপুট লগে পাঠান, মোছার কাজ পরীক্ষা করে চালু করুন। systemd টাইমার আধুনিক বিকল্প — মিস হওয়া কাজ পরে চালাতে পারে।

১৯.১৬ মূল প্রযুক্তিগত পরিভাষা

ক্রন — সময়-ভিত্তিক কাজ চালানোর ডেমন; **ক্রনট্যাব** — কাজের সময়সূচির তালিকা; **ডেমন** — ব্যাকগ্রাউন্ডে চলা সার্ভিস; **systemd টাইমার** — systemd-এর সময়সূচি ব্যবস্থা; **flock** — একসাথে দুবার চলা ঠেকানোর তালা; **পরিবেশ ভ্যারিয়েবল** — প্রোগ্রামের বাইরে থেকে দেওয়া সেটিংস।

১৯.১৭ জ্ঞান যাচাই প্রশ্ন

1. `0 2 * * *` মানে কী?
2. হাতে চলা স্ক্রিপ্ট ক্রনে ব্যর্থ হতে পারে কেন?
3. ক্রন কাজের আউটপুট লগে পাঠানো জরুরি কেন?
4. `flock`-এর কাজ কী?
5. systemd টাইমারের একটি সুবিধা বলুন।
6. মোছার ক্রন কাজ চালুর আগে কী পরীক্ষা করবেন?
7. সার্ভার বন্ধ থাকায় মিস হওয়া ব্যাকআপ ক্রনে আর চলবে না কেন?
8. ক্রনট্যাবে `%` চিহ্ন ব্যবহার করলে কী সমস্যা হয়?

১৯.১৮ ব্যবহারিক অনুশীলন

1. ল্যাব VM-এ প্রতি মিনিটে সময় লেখার পরীক্ষামূলক ক্রন কাজ বানিয়ে লগ দেখুন, তারপর মুছে দিন।
2. ডিস্ক ব্যবহারের প্রতিবেদন দেওয়ার একটি স্ক্রিপ্ট লিখে দৈনিক ক্রনে বসান।
3. ইচ্ছাকৃতভাবে আপেক্ষিক পথের স্ক্রিপ্ট ক্রনে চালিয়ে ব্যর্থতা দেখুন ও সম্পূর্ণ পথে ঠিক করুন।
4. একটি systemd টাইমার বানিয়ে দৈনিক কাজ চালানোর পরীক্ষা করুন।
5. `systemctl list-timers --all` চালিয়ে কোন টাইমার পরেরবার কখন চলবে, তা লিখুন।

অধ্যায় ২০

সার্ভার লগ, জার্নালড, লগরোটেট ও স্বাস্থ্য পরীক্ষা

২০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) লগ কেন সার্ভারের “কালো বাক্স” তা ব্যাখ্যা করতে; (২) journald ও `journalctl` দিয়ে সিস্টেম লগ পড়তে; (৩) logrotate দিয়ে লগের আকার নিয়ন্ত্রণ করতে; (৪) অ্যাপ, Nginx ও সিস্টেম লগের মধ্যে পার্থক্য করতে; (৫) স্বয়ংক্রিয় স্বাস্থ্য পরীক্ষার স্ক্রিপ্ট লিখতে।

২০.২ প্রোডাকশনে কেন এটি গুরুত্বপূর্ণ

সার্ভারে কিছু ভুল হলে প্রথম প্রশ্ন: “লগে কী লেখা আছে?” লগ ছাড়া সমস্যা সমাধান মানে অন্ধকারে হাতড়ানো। আবার লগই যদি অনিয়ন্ত্রিতভাবে বেড়ে ডিস্ক ভরে ফেলে, তবে সেই লগই সার্ভারকে মারবে (অধ্যায় ১৩)। তাই লগ পড়া, লগের আকার নিয়ন্ত্রণ করা ও নিয়মিত স্বাস্থ্য পরীক্ষা — তিনটি একসাথে শিখতে হয়।

২০.৩ পূর্বশর্ত জ্ঞান

ফাইল পড়া (`cat`, `less`), systemd সার্ভিসের ধারণা ও অধ্যায় ১৩-এর ডিস্ক জ্ঞান থাকলে ভালো। journald ও logrotate এখানেই ব্যাখ্যা করা হয়েছে।

২০.৪ প্রথম নীতি থেকে ব্যাখ্যা

লগ (log — ঘটনার সময়-ক্রমিক রেকর্ড) হলো সার্ভারের ডায়েরি: কে ঢুকল, কী ভুল হলো, কোন সার্ভিস কখন থামল। **journald** হলো systemd-এর লগ ব্যবস্থা — সিস্টেম ও সার্ভিসের বার্তা এক জায়গায় জমা করে, সময় ও সার্ভিস অনুযায়ী খোঁজা যায়। পুরোনো পদ্ধতিতে লগ থাকত `/var/log/`-এর টেক্সট ফাইলে (যেমন `syslog`, `auth.log`)। **logrotate** হলো লগের “পরিচ্ছন্নতাকর্মী”: পুরোনো লগ সংকুচিত করে, নির্দিষ্ট সংখ্যক কপি রেখে বাকি মুছে দেয়, যাতে লগ ডিস্ক না ভরে। **স্বাস্থ্য পরীক্ষা** (health check) মানে নিয়মিত প্রশ্ন করা: সার্ভিস চলছে? ডিস্ক কত? লোড কত? উত্তর অস্বাভাবিক হলে সতর্ক হওয়া।

২০.৫ স্থাপত্য / ধারণাগত চিত্র

[Nginx / Node.js / SSH / কার্নেল] → বার্তা → [journald (সিস্টেম লগ) + /var/log/*.log (ফাইল লগ)] → [logrotate: দৈনিক ঘোরানো + সংকোচন + পুরোনো মোছা]

[স্বাস্থ্য পরীক্ষা স্ক্রিপ্ট] → ডিস্ক + লোড + সার্ভিস + HTTP → স্বাভাবিক? → হ্যাঁ: লগে লেখা | না: সতর্কবার্তা

২০.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

`journalctl` দিয়ে লগ দেখা যায়: `-u nginx` নির্দিষ্ট সার্ভিস, `--since "1 hour ago"` সময়সীমা, `-f` সরাসরি অনুসরণ (follow), `-p err` শুধু ত্রুটি। journald-এর আকার সীমিত রাখতে `journalctl --vacuum-size=500M` চালান। Nginx-এর দুটি লগ: **অ্যাক্সেস লগ** (access log — কে কোন পাতা চাইল) ও **এরর লগ** (error log — কী ভুল

হলো)। অ্যাপ লগে সংবেদনশীল তথ্য (পাসওয়ার্ড, টোকেন) লিখবেন না — লগ অনেকের হাতে যায়। লগের **স্তর** (level — debug, info, warning, error) বুঝুন: প্রোডাকশনে debug চালু রাখলে লগের বন্যা বয়ে যায়।

কাজের উদাহরণ — লগের স্তর ও কাঠামোগত লগ পড়া

TechNova-র অ্যাপে একদিন হাজারো 502 ত্রুটি এল। Nginx এরর লগে শুধু "upstream prematurely closed connection" লেখা — কারণ অস্পষ্ট। তদন্তে দেখা গেল অ্যাপ লগে `debug` স্তর চালু ছিল, তাই আসল ত্রুটির বার্তা লাখো লাইনের ভেতরে হারিয়ে গেছে। শিক্ষা: প্রোডাকশনে `debug` বন্ধ রাখুন, `info` সাধারণ ঘটনা, `warning` অস্বাভাবিক কিন্তু চলমান অবস্থা, `error` ব্যর্থতার জন্য। দ্বিতীয় শিক্ষা: **কাঠামোগত লগ** (structured log — প্রতিটি লাইনে সময়, স্তর, সার্ভিস, অনুরোধের আইডি একই বিন্যাসে) থাকলে `grep` ও `awk` দিয়ে গোনা যায়। উদাহরণ: `grep -c " 502 " /var/log/nginx/access.log` দিয়ে 502-এর সংখ্যা, আর `awk '{print $1}' access.log | sort | uniq -c | sort -rn | head` দিয়ে সবচেয়ে বেশি অনুরোধ করা IP জানা যায়। লগ পড়ার ক্রম মনে রাখুন: প্রথমে সময়সীমা ঠিক করুন (`--since`), তারপর স্তর দিয়ে ছাঁকুন (`-p err`), তারপর নির্দিষ্ট শব্দ খুঁজুন — উল্টো ক্রমে লাখো লাইন পড়তে হয়। আর লগে কখনো পাসওয়ার্ড, টোকেন বা ব্যক্তিগত তথ্য লিখবেন না; একবার লগে লিখলে তা ব্যাকআপেও ছড়িয়ে পড়ে।

লগ স্তর	অর্থ	প্রোডাকশনে
debug	বিস্তারিত অনুসন্ধান তথ্য	বন্ধ রাখুন (লগের বন্যা হয়)
info	সাধারণ ঘটনা (সার্ভিস শুরু, অনুরোধ)	চালু রাখুন
warning	অস্বাভাবিক, কিন্তু সেবা চলছে	নজরে রাখুন, অ্যালার্ট বিবেচনা করুন
error	ব্যর্থতা	সবসময় পর্যালোচনা করুন

২০.৭ গুরুত্বপূর্ণ কমান্ড ও বিশ্লেষণ

```
sudo journalctl -u nginx --since "1 hour ago"   # Nginx-এর গত ঘণ্টার লগ
sudo journalctl -u app -f                        # অ্যাপের লগ সরাসরি অনুসরণ
sudo journalctl -p err --since today             # আজকের শুধু ত্রুটি
sudo journalctl --disk-usage                     # journald কত জায়গা নিয়েছে
sudo journalctl --vacuum-size=500M               # ৫০০MB-এ সীমিত করা
sudo tail -f /var/log/nginx/error.log            # Nginx এরর লগ অনুসরণ
sudo tail -n 100 /var/log/auth.log               # SSH লগইন রেকর্ড
sudo grep -i "error" /var/log/nginx/error.log | tail -20
sudo logrotate -d /etc/logrotate.conf            # কী হবে দেখা (ড্রাই-রান)
```

নমুনা আউটপুট — চিত্রণমূলক:

```
নমুনা আউটপুট — চিত্রণমূলক
$ sudo journalctl -u nginx --since "1 hour ago" | tail -3
Nov 08 22:10:01 server systemd[1]: Started nginx.service
Nov 08 22:15:44 server nginx[1234]: 203.0.113.50 - - [08/Nov/2026] "GET / HTTP/1.1" 200
```

আরও কমান্ড

```
sudo journalctl --since yesterday --until today   # গতকালের সব লগ দেখা
sudo journalctl -u nginx -p err --since "2 hours ago"   # Nginx-এর গত ২ ঘণ্টার শুধু ত্রুটি
sudo journalctl --vacuum-time=7d   # ৭ দিনের পুরোনো জার্নাল মুছে জায়গা খালি করা
```

```
sudo grep -c " 502 " /var/log/nginx/access.log   # অ্যাক্সেস লগে 502 ত্রুটির সংখ্যা গোনা
sudo awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head -10   # সবচেয়ে বেশি অনুরোধ করা ১০টি IP
sudo logrotate -f /etc/logrotate.d/technova-app   # নির্দিষ্ট কনফিগ জোর করে ঘোরানো (ল্যাবে পরীক্ষা করে)
```

ব্যাখ্যা: `--since yesterday --until today` দিয়ে নির্দিষ্ট দিনের লগ আলাদা করা যায় — "গতকাল রাতে কী হয়েছিল" প্রশ্নের জন্য এটাই সঠিক পথ। `--vacuum-time=7d` পুরোনো জার্নাল মুছে দেয়; মুছার আগে জায়গা দেখুন (`--disk-usage`) এবং নিশ্চিত হোন পুরোনো লগ আর দরকার নেই। `awk` দিয়ে IP গোনা বট ট্রাফিক ধরার দ্রুত উপায় — এক IP থেকে হাজারো অনুরোধ মানেই সন্দেহ। `logrotate -f` জোর করে ঘোরায়, তাই প্রোডাকশনে চালানোর আগে ল্যাবে বা `-d` ড্রাই-রান দিয়ে যাচাই করুন।

২০.৮ কনফিগারেশন উদাহরণ

অ্যাপ লগের logrotate কনফিগ (`/etc/logrotate.d/technova-app`):

```
/var/log/app/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 www-data adm
    sharedscripts
    postrotate
        systemctl reload nginx > /dev/null 2>&1 || true
    endscript
}
```

অর্থ: প্রতিদিন ঘোরানো, ১৪টি কপি রাখা, সংকুচিত করা, খালি ফাইল ঘোরানো নয়, ঘোরানোর পর Nginx রিলোড করা। `logrotate -d` দিয়ে ড্রাই-রান করে যাচাই করুন।

স্বাস্থ্য পরীক্ষার স্ক্রিপ্ট (`/usr/local/bin/healthcheck.sh`):

```
#!/bin/bash
# স্বাস্থ্য পরীক্ষা — ফলাফল /var/log/health.log-এ লেখে
DISK=$(df / | awk 'NR==2 {print $5}' | tr -d '%')
LOAD=$(uptime | awk -F'load average:' '{print $2}' | cut -d, -f1 | xargs)
echo "$(date '+%F %T') disk=${DISK}% load=${LOAD}" >> /var/log/health.log
if [ "$DISK" -gt 80 ]; then echo "WARNING: disk ${DISK}% full" >> /var/log/health.log; fi
systemctl is-active --quiet nginx || echo "ALERT: nginx down" >> /var/log/health.log
curl -fsS -o /dev/null http://127.0.0.1:3000/ || echo "ALERT: app not responding" >> /var/log/health.log
```

২০.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — সাইট ডাউন, কারণ অজানা: `journalctl -u app --since "30 min ago"`-তে দেখা গেল MongoDB সংযোগ ব্যর্থ — ডাটাবেস সার্ভিস থেমে গেছে। **পরিস্থিতি ২ — ধীর সাইট:** Nginx অ্যাক্সেস লগে একই IP থেকে হাজারো অনুরোধ — বট ট্রাফিক, ফায়ারওয়ালে সীমিত করা হলো। **পরিস্থিতি ৩ — সন্দেহজনক লগইন:** `auth.log`-এ অচেনা দেশের IP থেকে সফল লগইন — কী বদলানো ও তদন্ত জরুরি।

পরিস্থিতি ৪ — TechNova-র চেকআউটে 502, কারণ লগের তিন জায়গায়: TechNova-র চেকআউট পাতায় গ্রাহকরা 502 ত্রুটি পাচ্ছিলেন, কিন্তু অ্যাপ লগে কিছু লেখা ছিল না। তদন্ত তিন ধাপে হলো: প্রথমে Nginx এরর লগে দেখা গেল “upstream timed out” — Node অ্যাপ ৩০ সেকেন্ডে উত্তর দিচ্ছে না। তারপর `journalctl -u app`-তে দেখা গেল অ্যাপটি MongoDB সংযোগের অপেক্ষায় আটকে আছে। শেষে ডাটাবেস সার্ভারের লগে দেখা গেল সংযোগের সংখ্যা সীমা ছাড়িয়ে গেছে — অনেক সংযোগ খোলা রয়ে গেছে। সমাধান ছিল সংযোগের সংখ্যা সীমিত করা ও অলস সংযোগ বন্ধ করা। শিক্ষা: একটি ত্রুটির কারণ তিনটি লগে ভাগ হয়ে থাকতে পারে — Nginx বলে কী ব্যর্থ হলো, অ্যাপ বলে কোথায় আটকাল, ডাটাবেস বলে কেন। শুধু একটি লগ দেখে সিদ্ধান্ত নেবেন না।

২০.১০ ধাপে ধাপে ব্যবহারিক উদাহরণ

“অ্যাপ সাড়া দিচ্ছে না” — লগ দিয়ে নির্ণয়: (১) `systemctl status app` — সার্ভিস চলছে? (২) `journalctl -u app --since "1 hour ago"` — শেষ ত্রুটি কী? (৩) `tail -n 50 /var/log/nginx/error.log` — Nginx কী বলছে (502? সংযোগ ব্যর্থ?)। (৪) `df -h` ও `free -h` — ডিস্ক/RAM কি কারণ? (৫) কারণ ঠিক করে সার্ভিস রিস্টার্ট করুন, তারপর লগে নিশ্চিত হোন সার্ভিস উঠেছে। প্রতিটি ধাপের ফলাফল লিখে রাখুন — অনুমানে নয়, প্রমাণে এগোন।

২০.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল ১: শুধু অ্যাপ লগ দেখা — সিস্টেম লগে (OOM, ডিস্ক) আসল কারণ থাকতে পারে। ভুল ২: logrotate না বসিয়ে লগ জমতে দেওয়া — অধ্যায় ১৩-এর ডিস্ক বিপর্যয়। ভুল ৩: লগে পাসওয়ার্ড/টোকেন লেখা — লগ ফাঁস মানেই গোপন তথ্য ফাঁস। ভুল ৪: `tail` দিয়ে শুধু শেষ কয়েক লাইন দেখে সিদ্ধান্ত — ত্রুটির শুরুটা আগের লাইনে থাকতে পারে। ভুল ৫: স্বাস্থ্য পরীক্ষা ছাড়া “সব ঠিক আছে” ধরে নেওয়া — নীরব ব্যর্থতা সবচেয়ে বিপজ্জনক।

২০.১২ নিরাপত্তা বিবেচনা

লগে সংবেদনশীল তথ্য লিখবেন না; লগ ফাইলের অনুমতি সীমিত রাখুন (`/var/log`-এ সাধারণ ব্যবহারকারীর প্রবেশ সীমিত)। গুরুত্বপূর্ণ লগ (auth.log) নিয়মিত পর্যালোচনা করুন — অস্বাভাবিক লগইন প্রায়ই আপসের প্রথম সংকেত। সম্ভব হলে গুরুত্বপূর্ণ লগ আলাদা সার্ভারে পাঠান (centralized logging) — সার্ভার দখল হলেও প্রমাণ থেকে যায়। লগ মুছে ফেলা নিজেই সন্দেহজনক — কে, কখন, কেন মুছল, তার হিসাব রাখুন।

২০.১৩ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
journald বিশাল জায়গা নিয়েছে	ভ্যাকুয়াম হয়নি	`journalctl --vacuum-size=500M`, `SystemMaxUse` বসান
লগ ঘুরছে না	logrotate কনফিগ ভুল	`logrotate -d` দিয়ে ড্রাই-রান, পথ যাচাই
লগে কিছু পাওয়া যাচ্ছে না	ভুল সার্ভিস/সময়সীমা	`-u` ও `--since` যাচাই, `tail -f` ব্যবহার
স্বাস্থ্য পরীক্ষা সতর্ক করছে না	স্ক্রিপ্ট চলছে না / লগ পথ ভুল	ক্রন লগ ও স্ক্রিপ্ট হাতে চালিয়ে দেখুন

আরও লক্ষণ

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
জার্নাল ফাইল নষ্ট (corrupt) বলে ত্রুটি	হঠাৎ বিদ্যুৎ যাওয়া বা ডিস্ক সমস্যা	`journalctl --verify` দিয়ে যাচাই; নষ্ট ফাইল সরিয়ে (ব্যাকআপ রেখে) জার্নাল পুনর্নির্মাণ করুন
logrotate চলছে, কিন্তু ফাইল ছোট হচ্ছে না	অ্যাপ পুরোনো ফাইল হাতলে (handle) ধরে আছে	`lsof | grep deleted`; কনফিগে `copytruncate` বিবেচনা করুন বা সার্ভিস রিলোড করুন
লগের সময় ভুল দেখায়	সময়ক্ষেত্র (timezone) বা সিস্টেম সময় ভুল	`timedatectl` দিয়ে সময় ও সময়ক্ষেত্র যাচাই করুন
হঠাৎ লগের বন্যা, ডিস্ক ভরছে	debug চালু, বা একই ত্রুটি বারবার লেখা হচ্ছে	`journalctl -p err --since "10 min ago" | head` — কোন বার্তা বারবার আসছে দেখুন ও মূল কারণ ঠিক করুন

২০.১৪ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: প্রতিটি সার্ভিসের লগ জানা, logrotate সব লগে, journald আকার সীমিত, দৈনিক স্বাস্থ্য পরীক্ষা ক্রনে, লগে গোপন তথ্য নয়, ত্রুটির সময়রেখা ধরে তদন্ত। অ্যান্টি-প্যাটার্ন: লগ না পড়ে রিস্টার্ট করা; লগ ডিলিট করে “সমস্যা মেটানো”; debug লগ প্রোডাকশনে চালু রাখা; স্বাস্থ্য পরীক্ষা ছাড়া সার্ভার চালানো।

২০.১৫ অধ্যায়ের সারাংশ

লগ হলো সার্ভারের স্মৃতি — `journalctl` দিয়ে সিস্টেম লগ, ফাইল লগ দিয়ে অ্যাপ ও Nginx পড়ুন। logrotate দিয়ে লগের আকার নিয়ন্ত্রণ করুন, journald ভ্যাকুয়াম করুন, আর স্বাস্থ্য পরীক্ষার স্ক্রিপ্ট ক্রনে বসিয়ে সমস্যা ব্যবহারকারীর আগে ধরুন।

২০.১৬ মূল প্রযুক্তিগত পরিভাষা

লগ — ঘটনার সময়-ক্রমিক রেকর্ড; **journald** — systemd-এর লগ ব্যবস্থা; **journalctl** — journald পড়ার টুল; **logrotate** — লগ ঘোরানো ও সংকোচনের টুল; **অ্যাক্সেস লগ** — কে কী চাইল; **এরর লগ** — কী ভুল হলো; **স্বাস্থ্য পরীক্ষা** — নিয়মিত সার্ভিস/রিসোর্স যাচাই।

২০.১৭ জ্ঞান যাচাই প্রশ্ন

1. journald ও `/var/log`-এর ফাইল লগের পার্থক্য কী?
2. logrotate না থাকলে কী সমস্যা হয়?
3. `journalctl -u nginx --since "1 hour ago"` কী দেখায়?
4. Nginx-এর অ্যাক্সেস লগ ও এরর লগের পার্থক্য কী?
5. লগে পাসওয়ার্ড লেখা উচিত নয় কেন?
6. স্বাস্থ্য পরীক্ষায় কোন তিনটি জিনিস অন্তত যাচাই করা উচিত?
7. প্রোডাকশনে `debug` লগ চালু রাখলে কী সমস্যা হয়?
8. 502 ত্রুটির কারণ খুঁজতে কেন তিনটি লগ (Nginx, অ্যাপ, ডাটাবেস) দেখতে হতে পারে?

২০.১৮ ব্যবহারিক অনুশীলন

1. ল্যাব VM-এ `journalctl -u ssh --since today` দিয়ে আজকের SSH লগ পড়ুন।
2. একটি নমুনা অ্যাপ লগ বানিয়ে logrotate কনফিগ লিখুন ও `logrotate -d` দিয়ে ড্রাই-রান করুন।
3. স্বাস্থ্য পরীক্ষার স্ক্রিপ্টটি ল্যাবে চালিয়ে `/var/log/health.log`-এ ফলাফল দেখুন।
4. `journalctl --vacuum-size=100M` চালানোর আগে ও পরে `--disk-usage` দিয়ে পার্থক্য মাপুন।
5. অ্যাক্সেস লগে `awk` দিয়ে সবচেয়ে বেশি অনুরোধ করা ৫টি IP বের করুন এবং সংখ্যাসহ লিখুন।

PART IV • DEVOPS ENGINEERING MASTERY

Deployment, Reliability and Operations

MERN প্রোডাকশন ডেপ্লয়মেন্ট, Secrets ব্যবস্থাপনা, ডেপ্লয়মেন্ট কৌশল, Incident Response ও Runbook।

অধ্যায় ২১

Production Deployment of a MERN Application

MERN — MongoDB, Express, React, Node.js — দিয়ে তৈরি অ্যাপ্লিকেশন লোকাল মেশিনে চলে, কিন্তু প্রোডাকশন মানে সম্পূর্ণ ভিন্ন জগৎ। লোকালে `npm run dev` দিলেই হয়; প্রোডাকশনে দরকার স্থায়ী প্রসেস, রিভার্স প্রক্সি, ডোমেইন, HTTPS, লগ, মনিটরিং আর পুনরায় চালু হওয়ার নিশ্চয়তা। এই অধ্যায়ে আমরা একটি সম্পূর্ণ MERN অ্যাপ Ubuntu Server 24.04 LTS-এ ধাপে ধাপে ডিপ্লয় করব।

শেখার উদ্দেশ্য

- Ubuntu Server 24.04 LTS-এ Node.js, MongoDB ও Nginx ইনস্টল ও কনফিগার করতে পারবেন।
- PM2 দিয়ে Node অ্যাপকে প্রোডাকশন প্রসেস হিসেবে চালাতে ও সার্ভার রিবুটের পর স্বয়ংক্রিয়ভাবে ফিরিয়ে আনতে পারবেন।
- Nginx রিভার্স প্রক্সি দিয়ে React ফ্রন্টএন্ড ও Express API একই ডোমেইনে পরিবেশন করতে পারবেন।
- AWS EC2, Azure VM, Google Compute Engine ও IBM Cloud Virtual Servers-এ Linux VM চালানোর মূল পার্থক্য ব্যাখ্যা করতে পারবেন।
- ডিপ্লয়মেন্টের পর স্বাস্থ্য পরীক্ষা (health check) ও লগ দেখে সমস্যা নির্ণয় করতে পারবেন।

প্রোডাকশনে এটি কেন গুরুত্বপূর্ণ

ডেভেলপমেন্ট সার্ভার (যেমন Vite dev server বা `nodemon`) তৈরি হয়েছে দ্রুত পরিবর্তন দেখার জন্য, স্থায়িত্বের জন্য নয়। সেটি ক্র্যাশ করলে নিজে থেকে ওঠে না, একটিমাত্র অনুরোধ একসাথে সামলায়, HTTPS দেয় না, আর লগ ঘোরায় (rotate) না — ফলে ডিস্ক ভরে যায়। প্রোডাকশনে একটি ক্র্যাশ মানে ব্যবহারকারী সেবা পাচ্ছেন না, আর প্রতিটি মিনিট ডাউনটাইম মানে আস্থা ও আয়ের ক্ষতি। তাই প্রোডাকশন ডিপ্লয়মেন্ট কোনো একক কমান্ড নয়; এটি একটি ব্যবস্থা — প্রসেস ম্যানেজার, ওয়েব সার্ভার, ডাটাবেস, ফায়ারওয়াল ও ব্যাকআপের সমন্বয়।

> **বাস্তব ঘটনা:** একটি ছোট ই-কমার্স দল সরাসরি `node server.js &` দিয়ে অ্যাপ চালাত। এক রাতে সার্ভার রিবুট হলে অ্যাপ আর ওঠেনি; সকালে তারা জানতে পারে আট ঘণ্টা ধরে সাইট ডাউন ছিল। PM2-এর startup সুবিধা থাকলে এই ঘটনা ঘটত না।

প্রয়োজনীয় পূর্বজ্ঞান

- JavaScript, Node.js ও Express-এর মৌলিক ধারণা; `package.json` ও `npm install` জানা।
- React অ্যাপ বিল্ড করা (`npm run build`) ও MongoDB-তে CRUD।
- Linux কমান্ড লাইনের মৌলিক জ্ঞান: ফাইল দেখা, সম্পাদনা, `sudo` ব্যবহার (এই বইয়ের আগের অধ্যায়গুলো)।
- ডোমেইন ও DNS-এর প্রাথমিক ধারণা: A রেকর্ড একটি নামকে IP-তে মেলায়।

প্রথম নীতি: প্রোডাকশন ডিপ্লয়মেন্ট আসলে কী

প্রথম নীতি থেকে ভাবুন: একটি ওয়েব অ্যাপ শেষ পর্যন্ত কিছু প্রসেস (process) যা পোর্টে শুনছে (listening), কিছু ফাইল যা ডিস্কে আছে, আর একটি ডাটাবেস যা তথ্য ধরে রাখে। ডিপ্লয়মেন্ট মানে এই তিনটি জিনিসকে এমনভাবে সাজানো যাতে (ক)

প্রসেস মারা গেলে আবার ওঠে, (খ) বাইরের দুনিয়া শুধু নিরাপদ পথে (পোর্ট ৮০/৪৪৩) ঢুকতে পারে, (গ) কোড বদলালেও পুরোনো সংস্করণে ফেরা যায়, (ঘ) কী ঘটছে তা লগে দেখা যায়। এই চারটি শর্তই PM2, Nginx, UFW ও লগ ব্যবস্থাপনার কাজ। বাকি সব — ডোমেইন, HTTPS, মনিটরিং — এই ভিত্তির ওপর দাঁড়ায়।

আর্কিটেকচার: সম্পূর্ণ চিত্র

ব্যবহারকারী (ব্রাউজার) → HTTPS :443 → Nginx (রিভার্স প্রক্সি, স্ট্যাটিক ফাইল) → দুটি পথ:
(১) /api/* → PM2 পরিচালিত Node/Express প্রসেস :5000 → MongoDB :27017
(২) অন্য সব পথ → React বিল্ডের স্ট্যাটিক ফাইল (/var/www/technova)
পাশে: UFW ফায়ারওয়াল (শুধু 22/80/443 খোলা) • PM2 লগ • systemd (বুটে PM2 ফেরানো)

এই নকশার সৌন্দর্য: Node সরাসরি ইন্টারনেটের মুখোমুখি হয় না। Nginx সামনে দাঁড়িয়ে ধীর সংযোগ, স্ট্যাটিক ফাইল ও HTTPS সামলায়; Node শুধু ব্যবসায়িক যুক্তি (business logic) নিয়ে ব্যস্ত থাকে। একে বলে **রিভার্স প্রক্সি** — প্রক্সি মানে মধ্যস্থতাকারী, আর রিভার্স মানে এটি সার্ভারের পক্ষে কাজ করে, ক্লায়েন্টের পক্ষে নয়।

বিস্তারিত ব্যাখ্যা: ধাপে ধাপে সার্ভার প্রস্তুতি

১. সার্ভার হালনাগাদ ও ব্যবহারকারী

প্রথমে সিস্টেম হালনাগাদ করুন এবং একটি অ-রুট (non-root) sudo ব্যবহারকারী ব্যবহার করুন — রুট দিয়ে সরাসরি কাজ করা নিরাপত্তা ঝুঁকি।

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl git ufw nginx
node --version   # Node 20 LTS আশা করা যায় (Ubuntu 24.04 সংগ্রহস্থল বা NodeSource থেকে)
```

২. Node.js ইনস্টল

Ubuntu 24.04-এর সংগ্রহস্থলে Node আছে, তবে প্রোডাকশনে নির্দিষ্ট LTS সংস্করণ চাইলে NodeSource সংগ্রহস্থল বা `nvm` ব্যবহার করা হয়। প্রোডাকশন সার্ভারে `nvm` এড়িয়ে সিস্টেম-ব্যাপী (system-wide) ইনস্টল ভালো, কারণ PM2 ও systemd সেটিকে সহজে খুঁজে পায়।

৩. MongoDB

MongoDB Community Edition-এর অফিসিয়াল সংগ্রহস্থল যোগ করে ইনস্টল করুন; তারপর সেবা চালু ও বুটে সক্রিয় করুন। ডিফল্টে MongoDB শুধু localhost (`127.0.0.1`) থেকে শোনে — এটিই নিরাপদ ডিফল্ট, বদলাবেন না যতক্ষণ না আলাদা ডাটাবেস সার্ভারের দরকার।

```
sudo systemctl enable --now mongod
sudo systemctl status mongod
mongosh --eval 'db.runCommand({ ping: 1 })'   # নমুনা আউটপুট — চিত্রণমূলক: { ok: 1 }
```

৪. অ্যাপ কোড আনা ও বিল্ড

```
cd /var/www
sudo git clone https://github.com/example-org/technova.git technova
cd technova
```

```
npm ci --omit=dev
npm run build   # React ফ্রন্টএন্ডের প্রোডাকশন বিল্ড তৈরি হয়
```

`npm ci` কেন? `npm install` সংস্করণ বদলে দিতে পারে; `npm ci` `package-lock.json` হুবহু মেনে চলে — প্রোডাকশনে পুনরুৎপাদনযোগ্যতা (reproducibility) জরুরি।

৫. শূন্য-ডাউনটাইম ডিপ্লয় ও স্বাস্থ্য পরীক্ষা

শূন্য-ডাউনটাইম (zero-downtime) ডিপ্লয় মানে নতুন সংস্করণ চালু হওয়ার সময় ব্যবহারকারীর একটিও অনুরোধ ব্যর্থ হবে না। PM2-এর cluster mode-এ এটি সম্ভব হয়, কারণ একাধিক ইনস্ট্যান্সের মধ্যে একবারে একটি বদলানো হয়। কিন্তু শুধু cluster mode থাকলেই হবে না — প্রতিটি ইনস্ট্যান্সকে প্রমাণ করতে হবে যে সে সত্যিই প্রস্তুত। সেই প্রমাণের নাম **health check**। একটি ভালো health endpoint শুধু "সার্ভার চলছে" বলে না; সে ডাটাবেস সংযোগ, প্রয়োজনীয় পরিবেশ ভেরিয়েবল ও ডিস্কের অবস্থাও যাচাই করে। নিচের উদাহরণটি দেখুন — এতে `/api/health` ডাটাবেসে ping পাঠিয়ে তবেই 200 দেয়, ব্যর্থ হলে 503 দেয়।

```
// server.js — health endpoint উদাহরণ (ফিকশনাল)
app.get('/api/health', async (req, res) => {
  try {
    await mongoose.connection.db.admin().ping();
    res.status(200).json({ status: 'ok', uptime: process.uptime() });
  } catch (err) {
    res.status(503).json({ status: 'error', message: 'database unreachable' });
  }
});
```

ডিপ্লয়ের ধাপগুলো এই ক্রমে সাজান: (১) নতুন কোড টেনে বিল্ড করুন; (২) `pm2 reload` দিয়ে একবারে একটি ইনস্ট্যান্স বদলান; (৩) প্রতিটি ইনস্ট্যান্স ওঠার পর health endpoint পরীক্ষা করুন — সফল না হলে পরের ইনস্ট্যান্সে যাবেন না; (৪) সব ইনস্ট্যান্স নতুন হলে বাইরে থেকে `curl` দিয়ে পুরো পথ (Nginx → API → MongoDB) যাচাই করুন; (৫) অন্তত পনেরো মিনিট লগ ও ত্রুটির হার দেখুন। এই ক্রমের মূল্য তখনই বোঝা যায় যখন বিল্ড সফল কিন্তু ডাটাবেস সংযোগ ভাঙা থাকে — তখন health check ব্যর্থ হয়, ডিপ্লয় থেমে যায়, আর পুরোনো ইনস্ট্যান্সগুলো সেবা চালিয়ে যায়। মনে রাখবেন: ডিপ্লয় শেষ হয় reload-এ নয়, যাচাইয়ে।

পদ্ধতি	ডাউনটাইম	কখন ব্যবহার করবেন
`pm2 reload`	প্রায় শূন্য (cluster mode)	সাধারণ কোড বদলে — এটিই ডিফল্ট হওয়া উচিত।
`pm2 restart`	সংক্ষিপ্ত বিরতি	একটিমাত্র ইনস্ট্যান্স থাকলে বা কনফিগ বড় বদলে গেলে।
`pm2 stop` তারপর `start`	স্পষ্ট ডাউনটাইম	শুধু রক্ষণাবেক্ষণের জানানো সময়ে (maintenance window)।

> **পরামর্শ:** ডিপ্লয়ের আগে `curl -f http://127.0.0.1:5000/api/health` দিয়ে পুরোনো সংস্করণের health দেখে নিন — তাহলে ডিপ্লয়ের পরের ফলাফলের সঙ্গে তুলনা করা যায়। Health endpoint-এ কখনো সিক্রেট বা ভেতরের বিস্তারিত ত্রুটি ছাপাবেন না।

PM2: প্রসেস ম্যানেজার

PM2 হলো Node অ্যাপের প্রহরী। অ্যাপ ক্র্যাশ করলে সে আবার চালায়, লগ আলাদা ফাইলে রাখে, CPU অনুযায়ী একাধিক ইনস্ট্যান্স (cluster mode) চালাতে পারে, আর সার্ভার রিবুটের পর নিজে থেকে ফিরে আসে।

```
sudo npm install -g pm2
pm2 start ecosystem.config.js --env production
pm2 save
pm2 startup systemd   # দেওয়া কমান্ডটি চালিয়ে বুটে চালু হওয়া সক্রিয় করুন
```

PM2 ইকোসিস্টেম ফাইল

```
// ecosystem.config.js — ফিকশনাল উদাহরণ
module.exports = {
  apps: [{
    name: 'technova-api',
    script: 'server.js',
    cwd: '/var/www/technova',
    instances: 2,
    exec_mode: 'cluster',
    env_production: {
      NODE_ENV: 'production',
      PORT: 5000,
      MONGO_URI: 'mongodb://127.0.0.1:27017/technova'
    },
    error_file: '/var/log/pm2/technova-error.log',
    out_file: '/var/log/pm2/technova-out.log',
    max_memory_restart: '500M'
  }]
};
```

`max_memory_restart` মানে মেমোরি লিক হলে প্রসেস নিজে থেকে পুনরায় চালু হবে — এটি চিকিৎসা নয়, সাময়িক সুরক্ষা; আসল লিক খুঁজে সারাতে হবে।

Nginx রিভার্স প্রক্সি কনফিগারেশন

```
# /etc/nginx/sites-available/technova
server {
    listen 80;
    server_name app.technova.example;

    location /api/ {
        proxy_pass http://127.0.0.1:5000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location / {
        root /var/www/technova/client/build;
        try_files $uri /index.html;
    }
}
```

```
sudo ln -s /etc/nginx/sites-available/technova /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

`try_files $uri /index.html` React Router-এর জন্য অপরিহার্য: সরাসরি `/dashboard`-এ গেলে Nginx প্রথমে ফাইল খোঁজে, না পেলে `index.html` দেয়, React রাউটার বাকিটা সামলায়। `proxy_set_header` লাইনগুলো ছাড়া Express জানতেই পারবে না আসল ব্যবহারকারীর IP কী — লগ ও রেট-লিমিট ভুল হবে। HTTPS-এর জন্য পরে Certbot (`certbot --nginx`) যোগ করলে এই ফাইলেই 443 ব্লক তৈরি হয়।

ক্লাউড VM-এ Linux সার্ভার চালানো

নিজের ঘরে সার্ভার চালানো আর ক্লাউডে ভার্চুয়াল মেশিন (VM) চালানো এক নয়। ক্লাউডে আপনি একটি ভার্চুয়াল সার্ভার ভাড়া করেন — হার্ডওয়্যার, নেটওয়ার্ক ও বিদ্যুৎ প্রদানকারীর দায়িত্ব; অপারেটিং সিস্টেম, অ্যাপ ও নিরাপত্তা আপনার। চারটি বড় প্রদানকারীর উচ্চ-স্তরের তুলনা নিচে দেওয়া হলো। **গুরুত্বপূর্ণ:** বিস্তারিত বৈশিষ্ট্য, সীমা ও শর্ত অঞ্চল (region) ও পরিকল্পনা (plan) ভেদে বদলায় — সিদ্ধান্তের আগে সংশ্লিষ্ট প্রদানকারীর অফিসিয়াল ডকুমেন্টেশন দেখুন। এই অধ্যায়ে কোনো মূল্য দেওয়া হয়নি, কারণ মূল্য পরিবর্তনশীল।

বিষয়	AWS EC2	Azure Virtual Machines	Google Compute Engine	IBM Cloud Virtual Servers
VM তৈরি (provisioning)	AMI (মেশিন ইমেজ) বেছে ইনস্ট্যান্স চালু; ইনস্ট্যান্স টাইপে CPU/RAM নির্ধারণ।	Marketplace ইমেজ থেকে VM তৈরি; VM সাইজে CPU/RAM নির্ধারণ।	পাবলিক বা কাস্টম ইমেজ থেকে VM ইনস্ট্যান্স তৈরি; মেশিন টাইপে CPU/RAM নির্ধারণ।	ক্যাটালগ থেকে Virtual Server ইনস্ট্যান্স তৈরি; প্রোফাইলে CPU/RAM নির্ধারণ।
SSH প্রবেশ	তৈরির সময় কী পেয়ার (key pair); `ssh -i key.pem ubuntu@IP`।	SSH কী বা পাসওয়ার্ড; পোর্টালে কী আপলোড করা যায়।	প্রকল্প-ব্যাপী বা ইনস্ট্যান্স-স্তরের SSH কী; ব্রাউজার থেকেও SSH।	তৈরির সময় SSH কী দেওয়া হয়; কী দিয়ে লগইন।
ফায়ারওয়াল	Security Group — VM-এর বাইরে, নেটওয়ার্ক স্তরে নিয়ম।	Network Security Group (NSG) — সাবনেট বা NIC স্তরে নিয়ম।	ফায়ারওয়াল নিয়ম — নেটওয়ার্ক/ট্যাগভিত্তিক।	Security Group — VPC নেটওয়ার্কে নিয়ম।
স্থায়ী/পাবলিক IP	Elastic IP — রিবুটেও একই থাকে (বরাদ্দ রাখার শর্ত প্রযোজ্য)।	স্ট্যাটিক পাবলিক IP বরাদ্দ করা যায়।	স্ট্যাটিক বহিঃস্থ IP সংরক্ষণ (reserve) করা যায়।	Floating IP সংযুক্ত করা যায়।
ডিস্ক ও স্ন্যাপশট	EBS ভলিউম সংযুক্তি; স্ন্যাপশট থেকে ভলিউম পুনরুদ্ধার।	Managed Disk; স্ন্যাপশট ও ব্যাকআপ সেবা।	Persistent Disk; স্ন্যাপশট ও মেশিন ইমেজ।	Block Storage ভলিউম; স্ন্যাপশট সুবিধা।
বিলিং ও বন্ধ করা	চালু সময়ের ভিত্তিতে বিল; বন্ধ (stop) করলে বেশিরভাগ কম্পিউট চার্জ থামে, ডিস্ক/IP	চালু সময়ের ভিত্তিতে বিল; deallocate করলে কম্পিউট চার্জ থামে।	চালু সময়ের ভিত্তিতে বিল; বন্ধ করলে কম্পিউট চার্জ থামে।	বিলিং মডেল পরিকল্পনাভেদে ভিন্ন; বন্ধ করলে কী চার্জ হয় ডক দেখে নিশ্চিত হোন।

	চার্জ থাকতে পারে — অফিসিয়াল ডক দেখুন।			

চারটিতেই সাধারণ নীতি এক: (ক) SSH পোর্ট ২২ সবার জন্য খোলা রাখবেন না — নিজের IP-তে সীমিত করুন; (খ) ক্লাউড ফায়ারওয়ালের পাশাপাশি সার্ভারের UFW-ও চালু রাখুন — দুই স্তরের প্রতিরক্ষা; (গ) স্ন্যাপশট নেওয়ার পরীক্ষা করুন — ব্যাকআপ যা ফেরানো যায়নি, তা ব্যাকআপ নয়; (ঘ) ব্যবহার না করা VM বন্ধ/মুছে ফেলুন, নইলে বিল চলতেই থাকবে। প্রোডাকশনে যাওয়ার আগে একটি ছোট, ফেলে দেওয়া যায় এমন (disposable) VM-এ পুরো ডিপ্লয়মেন্ট একবার অনুশীলন করুন।

> **সতর্কতা:** ক্লাউড কনসোলে "Terminate/Delete" মানে স্থায়ী মুছে ফেলা। বড় কোনো পরিবর্তন, ডিস্ক অপারেশন বা রোলব্যাকের আগে অবশ্যই স্ন্যাপশট/ব্যাকআপ নিন।

গুরুত্বপূর্ণ কমান্ড ও বিশ্লেষণ

কমান্ড	কী করে	বিশ্লেষণ
`pm2 status`	সব প্রসেসের অবস্থা দেখায়	online/errored/stopped এক নজরে; প্রথম ডায়াগনসিস।
`pm2 logs technova-api --lines 100`	সাম্প্রতিক লগ দেখায়	ক্র্যাশের কারণ সাধারণত লগের শেষ ২০ লাইনে থাকে।
`pm2 reload technova-api`	শূন্য-ডাউনটাইমে পুনরায় চালু (cluster)	একটি করে ইনস্ট্যান্স বদলায়; একসাথে সব বন্ধ হয় না।
`sudo nginx -t`	কনফিগের সিনট্যাক্স যাচাই	ভুল কনফিগ নিয়ে reload করলে Nginx মারা যেতে পারে — আগেই পরীক্ষা করুন।
`sudo systemctl reload nginx`	সংযোগ না কেটে কনফিগ পুনরায় লোড	restart নয় — reload চলমান সংযোগ বাঁচায়।
`curl -I https://app.technova.example/api/health`	বাইরে থেকে স্বাস্থ্য পরীক্ষা	HTTP স্ট্যাটাস দেখে ডিপ্লয় সফল কি না যাচাই।
`sudo ufw status verbose`	ফায়ারওয়াল নিয়ম দেখায়	শুধু 22/80/443 খোলা আছে কি না নিশ্চিত হোন।
`pm2 monit`	CPU ও মেমোরি সরাসরি পর্যবেক্ষণ	কোন ইনস্ট্যান্স মেমোরি খাচ্ছে, তা চোখে দেখা যায়।
`pm2 install pm2-logrotate`	লগ ঘূর্ণন চালু করা	লগ GB-তে না গিয়ে নিয়মিত ঘোরে — ডিস্ক বাঁচে।
`curl -f http://127.0.0.1:5000/api/health`	স্থানীয় health check	ব্যর্থ হলে exit code শূন্য নয় — ডিপ্লয় স্ক্রিপ্টে ব্যবহার করুন।
`sudo certbot renew --dry-run`	সার্টিফিকেট নবায়ন পরীক্ষা	আসল নবায়নের আগে শুকনো মহড়া — মেয়াদোত্তীর্ণ সার্টিফিকেটের ঝুঁকি কমে।

`ss -tlnp`	কোন পোর্টে কী শুনছে	৫০০০ পোর্টে শুধু Node, ৮০/৪৪৩-এ Nginx থাকা উচিত।

বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — ডিপ্লয়ের পর 502 Bad Gateway: Nginx চলছে কিন্তু Node মারা গেছে। `pm2 logs` দেখে জানা গেল `MONGO_URI` ভুল। সমাধান: পরিবেশ ভেরিয়েবল ঠিক করে `pm2 restart`। শিক্ষা: 502 প্রায়ই মানে প্রক্সির পেছনের অ্যাপ নেই।

পরিস্থিতি ২ — ডিস্ক ভরে যাওয়া: PM2 লগ কয়েক GB হয়ে গেল। সমাধান: `pm2 install pm2-logrotate` ও লগ ঘূর্ণন। শিক্ষা: লগ ব্যবস্থাপনা ডিপ্লয়েরই অংশ।

পরিস্থিতি ৩ — বিল্ড ফাইল পুরোনো: API নতুন কিন্তু ফ্রন্টএন্ড পুরোনো দেখাচ্ছে। কারণ: `npm run build` না চালিয়ে পুরোনো `build/` পরিবেশন করা হয়েছে। সমাধান: ডিপ্লয় স্ক্রিপ্টে বিল্ড ধাপ বাধ্যতামূলক করুন।

পরিস্থিতি ৪ — দুপুরে ডিপ্লয়, health check ব্যর্থ: TechNova-র দল (technova.example, সার্ভার 203.0.113.10) দুপুরে নতুন সংস্করণ ডিপ্লয় করল। `pm2 reload`-এর পর প্রথম ইনস্ট্যান্সের health check 503 দিল — নতুন কোডে MONGO_URI-র একটি অক্ষর ভুল ছিল। ডিপ্লয় স্ক্রিপ্ট সেখানেই থেমে গেল, পুরোনো ইনস্ট্যান্সগুলো সেবা চালিয়ে গেল, ব্যবহারকারী কিছুই টের পেল না। দল URI ঠিক করে আবার ডিপ্লয় করল, এবার সব health check পাস করল। শিক্ষা: health check ছাড়া reload করলে ভুল কোড সব ইনস্ট্যান্সে ছড়িয়ে পড়ত — যাচাইয়ের ধাপই ডাউনটাইম আটকায়।

ধাপে ধাপে ব্যবহারিক উদাহরণ: প্রথম ডিপ্লয়

1. Disposable VM-এ Ubuntu 24.04 চালু করে SSH করুন; `sudo apt update && sudo apt upgrade -y` চালান।
2. UFW চালু করুন: `sudo ufw allow OpenSSH`, `sudo ufw allow 'Nginx Full'`, `sudo ufw enable`।
3. Node, MongoDB, Nginx, PM2 ইনস্টল করুন; `mongod` চালু আছে কি না দেখুন।
4. কোড ক্লোন করে `npm ci --omit=dev && npm run build` চালান।
5. `ecosystem.config.js` লিখে `pm2 start`, `pm2 save`, `pm2 startup` করুন।
6. Nginx সাইট কনফিগ লিখে `nginx -t` দিয়ে যাচাই করে reload করুন।
7. `curl -I http://SERVER-IP/api/health` দিয়ে পরীক্ষা করুন; ব্রাউজারে ডোমেইন খুলুন।
8. সার্ভার রিবুট করে (`sudo reboot`) অ্যাপ নিজে থেকে ওঠে কি না যাচাই করুন — এটিই আসল পরীক্ষা।

সাধারণ ভুল ও ভুল ধারণা

- **ভুল:** `node server.js &` দিয়ে চালানো — রিবুট/ক্র্যাশে আর ওঠে না। **সঠিক:** PM2 বা systemd।
- **ভুল ধারণা:** "Nginx শুধু স্ট্যাটিক সাইটের জন্য" — আসলে এটি রিভার্স প্রক্সি, লোড ব্যালান্সার ও HTTPS সমাপ্তিকারী।
- **ভুল:** প্রোডাকশনে `npm install` দিয়ে dev dependency-সহ ইনস্টল — আকার ও আক্রমণ-তল (attack surface) বাড়ে।
- **ভুল:** MongoDB পোর্ট ২৭০১৭ ইন্টারনেটে খোলা — কয়েক ঘণ্টায় বট এসে ডাটাবেস মুছে মুক্তিপণ চাইতে পারে।
- **ভুল ধারণা:** "ডিপ্লয় মানেই কোড কপি করা" — আসলে বিল্ড, পরিবেশ, প্রসেস, প্রক্সি ও যাচাই সব মিলিয়ে ডিপ্লয়।

নিরাপত্তা বিবেচনা

- SSH-তে পাসওয়ার্ড লগইন বন্ধ করে শুধু কী-ভিত্তিক লগইন রাখুন; রুট লগইন নিষিদ্ধ করুন (`PermitRootLogin no`)।
- UFW + ক্লাউড security group — দুই স্তরেই শুধু প্রয়োজনীয় পোর্ট খুলুন।
- MongoDB শুধু localhost-এ বাঁধুন; প্রমাণীকরণ (authentication) চালু করুন।
- .env ফাইল কখনো git-এ কমিট করবেন না — বিস্তারিত অধ্যায় ২২-এ।
- নিয়মিত `sudo apt upgrade` দিয়ে নিরাপত্তা প্যাচ দিন; পুরোনো Node সংস্করণে জানা দুর্বলতা থাকে।

সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
502 Bad Gateway	Node প্রসেস বন্ধ / ভুল পোর্ট	`pm2 status` , `pm2 logs` ; পোর্ট মিলিয়ে দেখুন।
404-এ React পেজ	`try_files` নেই	Nginx কনফিগে `try_files $uri /index.html;` যোগ করুন।
API থেকে CORS ত্রুটি	ফ্রন্টএন্ড ডোমেইন অনুমোদিত নয়	Express CORS-এ সঠিক origin দিন; `*` প্রোডাকশনে নয়।
সাইট ধীর	একটিমাত্র Node ইনস্ট্যান্স / ডিস্ক পূর্ণ	cluster mode, `df -h` , লগ ঘূর্ণন পরীক্ষা করুন।
ডিপ্লয়ের পর কিছু ব্যবহারকারী পুরোনো সাইট দেখছেন	ব্রাউজার/CDN ক্যাশে পুরোনো ফাইল	সম্পদে (asset) হ্যাশযুক্ত ফাইলনাম ব্যবহার করুন; `index.html` -এ `no-cache` দিন।
Health check 503 দিচ্ছে, কিন্তু অ্যাপ চলছে	ডাটাবেস সংযোগ ব্যর্থ বা MONGO_URI ভুল	`mongosh` দিয়ে ping করুন; URI, ব্যবহারকারী ও পাসওয়ার্ড যাচাই করুন।
Reload-এর সময় কিছু অনুরোধ ব্যর্থ হয়	Graceful shutdown নেই — চলমান অনুরোধ মাঝপথে কাটা পড়ছে	`SIGTERM` ধরে নতুন সংযোগ বন্ধ করে চলমান অনুরোধ শেষ হতে দিন; PM2-এর `wait_ready` ও `listen_timeout` দেখুন।
HTTPS-এ গেলে "সংযোগ নিরাপদ নয়"	সার্টিফিকেট মেয়াদোত্তীর্ণ বা ডোমেইন মিলছে না	`certbot renew --dry-run` দিয়ে নবায়ন পরীক্ষা করুন; `server_name` সার্টিফিকেটের ডোমেইনের সঙ্গে মেলান।
রিবুটের পর অ্যাপ ওঠে না	`pm2 startup` / `save` হয়নি	`pm2 startup systemd` ও `pm2 save` আবার করুন।

সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

- **করুন:** ডিপ্লয় স্ক্রিপ্ট লিখুন (pull → install → build → reload → health check) যাতে প্রতিবার একই ধাপ চলে।
- **করুন:** প্রতিটি ডিপ্লয়ের পর health endpoint পরীক্ষা করুন।
- **করবেন না:** প্রোডাকশন সার্ভারে সরাসরি কোড সম্পাদনা (hot-fix) — পরিবর্তন git দিয়ে আসুক।
- **করবেন না:** `chmod 777` দিয়ে অনুমতি সমস্যা "সমাধান" করা — এটি নিরাপত্তা ছিদ্র।

অধ্যায়ের সারসংক্ষেপ

প্রোডাকশন MERN ডিপ্লয়মেন্ট মানে Node-কে PM2 দিয়ে স্থায়ী করা, MongoDB-কে localhost-এ নিরাপদ রাখা, আর Nginx-কে সামনে রেখে একই ডোমেইনে API ও React পরিবেশন করা। ক্লাউড VM-এ এই একই নকশা চলে — শুধু VM তৈরি, SSH, ফায়ারওয়াল, IP ও স্ন্যাপশটের পদ্ধতি প্রদানকারীভেদে বদলায়, আর সেসবের বিশদ অফিসিয়াল ডকে দেখতে হয়। ডিপ্লয়ের শেষ ধাপ যাচাই: রিবুটের পরও অ্যাপ ওঠে কি না।

মূল প্রযুক্তিগত পরিভাষা

রিভার্স প্রক্সি (Reverse Proxy): সার্ভারের সামনে বসে অনুরোধ গ্রহণ করে ভেতরের সার্ভারে পাঠানো ব্যবস্থা। **প্রসেস ম্যানেজার (Process Manager):** অ্যাপ প্রসেস চালু রাখা, পুনরায় চালু ও লগ রাখার টুল (PM2)। **ক্লাস্টার মোড:** একাধিক CPU কোরে একই অ্যাপের একাধিক ইনস্ট্যান্স চালানো। **Health Check:** অ্যাপ জীবিত ও সুস্থ আছে কি না যাচাইয়ের endpoint। **AMI/ইমেজ:** VM তৈরির ছাঁচ (template)। **Security Group:** ক্লাউডে VM-এর নেটওয়ার্ক ফায়ারওয়াল নিয়মের সেট।

জ্ঞান যাচাই প্রশ্ন

1. প্রোডাকশনে `node server.js &` কেন যথেষ্ট নয়?
2. Nginx রিভার্স প্রক্সি হিসেবে কী কাজ করে? Node সরাসরি ইন্টারনেটে রাখলে কী সমস্যা?
3. `npm ci` ও `npm install`-এর পার্থক্য কী, প্রোডাকশনে কোনটি ভালো?
4. `try_files $uri /index.html;` না থাকলে React Router-এ কী হয়?
5. 502 Bad Gateway দেখলে প্রথম কোন তিনটি জিনিস পরীক্ষা করবেন?
6. ক্লাউডে UFW ছাড়াও security group কেন দরকার?
7. Health endpoint-এ শুধু "সার্ভার চলছে" দেখানো কেন যথেষ্ট নয় — ডাটাবেস যাচাই কেন জরুরি?
8. `pm2 reload` ও `pm2 restart`-এর মধ্যে ডাউনটাইমের পার্থক্য কী, আর কখন কোনটি বেছে নেবেন?
9. PM2-এর `save` ও `startup` না করলে রিবুটের পর কী হবে?

ব্যবহারিক অনুশীলন

1. **ল্যাব ডিপ্লয়:** একটি disposable VM-এ এই অধ্যায়ের ধাপগুলো অনুসরণ করে একটি নমুনা MERN অ্যাপ ডিপ্লয় করুন; রিবুটের পরও চলে কি না যাচাই করুন।
2. **502 তৈরি ও সারানো:** ইচ্ছা করে PM2 প্রসেস থামিয়ে (`pm2 stop`) 502 দেখুন, তারপর লগ পড়ে কারণ বের করে সারান।
3. **ক্লাউড তুলনা:** AWS, Azure, GCP ও IBM Cloud-এর অফিসিয়াল ডক খুলে VM তৈরি, SSH ও ফায়ারওয়ালের ধাপগুলো একটি টেবিলে লিখুন — কোনো মূল্য লিখবেন না।
4. **Health check ডিজাইন:** একটি নমুনা Express অ্যাপে ডাটাবেস ping-সহ `/api/health` লিখুন; ডাটাবেস থামিয়ে (disposable পরিবেশে) 503 আসে কি না পরীক্ষা করুন।
5. **লগ ঘূর্ণন:** `pm2-logrotate` ইনস্টল করে লগের সর্বোচ্চ আকার নির্ধারণ করুন ও পরীক্ষা করুন।

অধ্যায় ২২

Environment Variables, Secrets and Configuration Management

একই কোড ল্যাপটপে, স্টেজিংয়ে ও প্রোডাকশনে চলে — কিন্তু ডাটাবেসের ঠিকানা, API চাবি আর পাসওয়ার্ড প্রতিটি পরিবেশে আলাদা। এই ভিন্নতাগুলোকে কোডের ভেতরে লিখে রাখলে বিপদ: কোড বদলালেই গোপন তথ্য ফাঁস হয়। সমাধান হলো কনফিগারেশনকে কোড থেকে আলাদা করা — এটিই এই অধ্যায়ের মূল কথা।

শেখার উদ্দেশ্য

- পরিবেশ ভেরিয়েবল (environment variable) কী ও কেন কোড থেকে আলাদা রাখতে হয়, ব্যাখ্যা করতে পারবেন।
- `.env` ফাইল নিরাপদে ব্যবহার, `.gitignore` করা ও `.env.example` বানাতে পারবেন।
- PM2 ecosystem, systemd ও Nginx-এ পরিবেশ ভেরিয়েবল দেওয়ার পদ্ধতি জানবেন।
- গোপন তথ্য (secret) ফাঁস হলে ঘোরানো (rotation) ও প্রতিকারের ধাপগুলো বলতে পারবেন।
- কনফিগারেশন যাচাই (validation) করে ভুল মান নিয়ে অ্যাপ চালু হওয়া আটকাতে পারবেন।

প্রোডাকশনে এটি কেন গুরুত্বপূর্ণ

গোপন তথ্য ফাঁস হওয়া প্রোডাকশনের সবচেয়ে সাধারণ ও ব্যয়বহুল দুর্ঘটনাগুলোর একটি। GitHub-এ ভুল করে `.env` পুশ করলে কয়েক মিনিটের মধ্যে স্বয়ংক্রিয় বট সেটি খুঁজে পায় — ডাটাবেস মুছে ফেলা, ক্লাউড অ্যাকাউন্ট দিয়ে ক্রিপ্টো মাইনিং, গ্রাহকের তথ্য চুরি — সবই ঘটেছে বাস্তবে। আরেকটি দিক: পরিবেশভেদে কনফিগ বদলানো না গেলে প্রতিটি পরিবেশের জন্য আলাদা কোড বানাতে হয়, যা ভুলের কারখানা। Twelve-Factor App নীতি বলে: কনফিগ পরিবেশে থাকবে, কোডে নয়।

> **সতর্কতা:** একবার git-এ কমিট হওয়া সিক্রেট "মুছে ফেলা" যায় না — ইতিহাসে (history) থেকে যায়। ফাঁস হলে একমাত্র নিরাপদ পথ হলো সেটি বাতিল (revoke) করে নতুনটি দেওয়া। এই অধ্যায়ের সব সিক্রেট উদাহরণ কাল্পনিক (যেমন CHANGE-ME-SECRET-...) — আসল সিক্রেট কখনো উদাহরণে লিখবেন না।

প্রয়োজনীয় পূর্বজ্ঞান

- অধ্যায় ২১-এর ডিপ্লয়মেন্ট: PM2, Nginx, MongoDB।
- Node.js-এ `process.env` পড়া ও `dotenv` প্যাকেজের ধারণা।
- git-এর মৌলিক জ্ঞান: commit, `.gitignore`, remote।
- JSON ও কী-মান (key-value) কনফিগের ধারণা।

প্রথম নীতি: কোড বনাম কনফিগারেশন

প্রথম নীতি: কোড হলো "কীভাবে" — এটি সব পরিবেশে একই থাকে। কনফিগারেশন হলো "কোথায়, কোন চাবি দিয়ে" — এটি পরিবেশভেদে বদলায়। এই দুটি মেশালে দুটি ক্ষতি হয়: (১) একটি পরিবেশের গোপন তথ্য অন্য পরিবেশে চলে যায়; (২) কনফিগ বদলাতে কোড পুনরায় ডিপ্লয় করতে হয়। তাই কনফিগারেশনকে পরিবেশ ভেরিয়েবল হিসেবে বাইরে রাখা হয় —

অপারেটিং সিস্টেম অ্যাপ চালুর সময় সেগুলো দেয়, অ্যাপ শুধু পড়ে। অ্যাপ কখনো জানতে পারে না মানটি কোথা থেকে এল — ফাইল থেকে, PM2 থেকে, নাকি সিক্রেট ম্যানেজার থেকে — এটিই ভালো নকশা।

আর্কিটেকচার: কনফিগ প্রবাহ

উৎস (একটি বেছে নিন): .env ফাইল (শুধু ল্যাব/ছোট সেটআপ) • PM2 ecosystem ফাইল • systemd EnvironmentFile • ক্লাউড সিক্রেট ম্যানেজার
↓ (অ্যাপ চালুর সময় ইনজেক্ট)
Node প্রসেস — process.env পড়ে → Express/Mongoose কনফিগ (MONGO_URI, JWT_SECRET, PORT)
× কখনো নয়: git সংগ্রহস্থল • ফ্রন্টএন্ড বান্ডল • লগ ফাইল • স্ক্রিনশট/চ্যাট

একটি কঠিন নিয়ম মনে রাখুন: ব্রাউজারে যাওয়া যেকোনো কিছুই পাবলিক। React বান্ডলে `REACT_APP_*` দিয়ে দেওয়া মান সবাই দেখতে পায় — তাই ফ্রন্টএন্ডে শুধু পাবলিক মান (যেমন API-র ঠিকানা) দিন, কখনো সিক্রেট নয়।

বিস্তারিত ব্যাখ্যা

পরিবেশ ভেরিয়েবল কী

পরিবেশ ভেরিয়েবল হলো প্রসেসের কাছে দেওয়া নাম-মান জোড়া: `PORT=5000`, `NODE_ENV=production`। শেলে `export PORT=5000` দিলে সেই শেল থেকে চালু প্রসেস তা পায়। Node-এ পড়া হয় `process.env.PORT` দিয়ে। সুবিধা: কোড না বদলে মান বদলানো যায়; অসুবিধা: ভুল মান দিলে অ্যাপ নীরবে ভুল আচরণ করতে পারে — তাই চালুর সময় যাচাই জরুরি।

কনফিগারেশন উদাহরণ

.env ফাইল: সুবিধা ও সীমা

```
# /var/www/technova/.env — কাল্পনিক উদাহরণ, কখনো git-এ নয়
NODE_ENV=production
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/technova
JWT_SECRET=CHANGE-ME-SECRET-7f3a9c1e-EXAMPLE-ONLY
SESSION_SECRET=CHANGE-ME-SECRET-9b2d4f6a-EXAMPLE-ONLY
STRIPE_KEY=sk_test_CHANGE-ME-EXAMPLE-ONLY
```

`dotenv` প্যাকেজ এই ফাইল পড়ে `process.env`-এ তোলে — তবে শুধু ডেভেলপমেন্টে। প্রোডাকশনে PM2 বা systemd থেকে ভেরিয়েবল দেওয়াই ভালো, কারণ তাতে ফাইলের অনুমতি, মালিকানা ও লগে ফাঁসের ঝুঁকি কমে। .env ফাইলের অনুমতি সবসময় `600` রাখুন (শুধু মালিক পড়তে পারে)।

.env.example — দলের জন্য চুক্তি

```
# .env.example — git-এ কমিট করা হয়, আসল মান নেই
NODE_ENV=development
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/technova
JWT_SECRET=CHANGE-ME-SECRET-HERE
STRIPE_KEY=sk_test_CHANGE-ME-HERE
```

এটি নতুন ডেভেলপারকে বলে কোন ভেরিয়েবল লাগবে, মান না জানিয়ে। ডিপ্লয় চেকলিস্টে রাখুন: ".env.example-এর প্রতিটি কী-র মান প্রোডাকশনে আছে কি?"

PM2 ও systemd-এ ভেরিয়েবল

```
// ecosystem.config.js-এ env_production ব্লকে মান দিন (সিক্রেট নয়, সিক্রেট আলাদা ফাইলে)
env_production: { NODE_ENV: 'production', PORT: 5000 }

// systemd সার্ভিসে:
# /etc/systemd/system/technova.service.d/override.conf
[Service]
EnvironmentFile=/etc/technova/env
# /etc/technova/env ফাইলের অনুমতি 600, মালিক root
```

কনফিগ যাচাই: ভুল মান নিয়ে চালু হতে দেবেন না

```
// config.js — চালুর শুরুতেই যাচাই, না হলে স্পষ্ট বার্তা দিয়ে থামুন
const required = ['MONGO_URI', 'JWT_SECRET', 'PORT'];
for (const key of required) {
  if (!process.env[key]) throw new Error(`Missing required env var: ${key}`);
}
if (process.env.JWT_SECRET.length < 32)
  throw new Error('JWT_SECRET too short — use a long random value');
module.exports = {
  port: Number(process.env.PORT),
  mongoUri: process.env.MONGO_URI,
  jwtSecret: process.env.JWT_SECRET,
};
```

এই যাচাইয়ের মূল্য: ভুল কনফিগে অ্যাপ আধা-চালু হয়ে অদ্ভুত আচরণ করার চেয়ে শুরুতেই স্পষ্টভাবে থেমে যাওয়া হাজার গুণ ভালো —

স্টেজিং বনাম প্রোডাকশন কনফিগ ও সিক্রেট রোটেশন

স্টেজিং আর প্রোডাকশন দেখতে একই হলেও তাদের কনফিগের উদ্দেশ্য আলাদা: স্টেজিংয়ে আপনি ভুল করে শিখবেন, প্রোডাকশনে ভুলের মূল্য দিতে হবে। তাই দুটি পরিবেশের সিক্রেট কখনো এক হবে না — একই JWT সিক্রেট দুই জায়গায় থাকলে স্টেজিং ফাঁস হলে প্রোডাকশনের সব টোকেনও ঝুঁকিতে পড়ে। প্রতিটি পরিবেশের জন্য আলাদা ডাটাবেস ব্যবহারকারী, আলাদা API চাবি ও আলাদা JWT সিক্রেট বানান। নিচের তুলনাটি মনে রাখুন: স্টেজিংয়ে Stripe-এর test key চলে, প্রোডাকশনে শুধু live key — আর live key কখনো চ্যাটে, স্ক্রিনশটে বা টিকিটে পাঠাবেন না।

বিষয়	স্টেজিং	প্রোডাকশন
ডাটাবেস	নমুনা তথ্য, যেকোনো সময় মুছে আবার বানানো যায়	আসল গ্রাহক তথ্য — ব্যাকআপ ও পুনরুদ্ধার পরীক্ষা আবশ্যক
JWT সিক্রেট	আলাদা, দুর্বল হলেও চলে না — তবু প্রোডাকশন থেকে ভিন্ন	লম্বা দৈব মান, শুধু অনুমোদিত সার্ভারে, অনুমতি 600
পেমেন্ট চাবি	Test key — আসল টাকা কাটে না	Live key — সিক্রেট ম্যানেজারে, লগে কখনো নয়
লগের মাত্রা	বিস্তারিত (debug) চলতে পারে	সতর্কতা ও ত্রুটি — সিক্রেট ছাপানো নিষিদ্ধ

সিক্রেট রোটেশন মানে পুরোনো গোপন মান বাতিল করে নতুন মান দেওয়া — দুর্ঘটনার পর নয়, নিয়মিতভাবে। নিরাপদ ঘূর্ণনের ক্রমটি হলো: (১) নতুন সিক্রেট বানান (`openssl rand -hex 32`); (২) প্রোডাকশনে নতুন মানটি পাশাপাশি বসান — JWT-এর ক্ষেত্রে কিছু সময় পুরোনো ও নতুন দুটি সিক্রেটই গ্রহণ করুন, যাতে চলমান সেশন হঠাৎ না ভাঙে; (৩) `pm2 restart --update-env` দিয়ে নতুন মান কার্যকর করুন ও health check করুন; (৪) নিশ্চিত হয়ে পুরোনো সিক্রেটটি বাতিল (revoke) করুন; (৫) কে, কখন, কোন সিক্রেট বদলাল — লিখে রাখুন। এই ক্রম না মানলে ঘূর্ণন নিজেই একটি ছোট ইনসিডেন্ট হয়ে যায় — ব্যবহারকারীরা হঠাৎ লগআউট হয়ে যান।

> **সতর্কতা — ঘূর্ণনের আগে:** সিক্রেট বদলানোর আগে ব্যাকআপ/স্ন্যাপশট নিন ও দলকে জানান; JWT সিক্রেট বদলালে সব চলমান টোকেন বাতিল হয়, ব্যবহারকারীদের আবার লগইন করতে হবে। কাল্পনিক মান (CHANGE-ME-SECRET-...) দিয়ে মহড়া দিন — আসল সিক্রেট কখনো উদাহরণে বা চ্যাটে লিখবেন না।

তখন লগে কারণ লেখা থাকে।

গুরুত্বপূর্ণ কমান্ড ও বিশ্লেষণ

কমান্ড	কী করে	বিশ্লেষণ
`printenv | sort`	সব পরিবেশ ভেরিয়েবল দেখায়	সিক্রেট আছে কি না যাচাই; তবে আউটপুট কোথাও পেস্ট করবেন না — তাতে সিক্রেট ফাঁস হয়।
`chmod 600 .env`	শুধু মালিক পড়তে পারে	অন্য ব্যবহারকারী ও ওয়েব সার্ভার যেন .env পড়তে না পারে।
`git check-ignore -v .env`	.env উপেক্ষিত কি না যাচাই	কমিটের আগে এই পরীক্ষা বাধ্যতামূলক করুন।
`git log --all -- .env`	ইতিহাসে .env ছিল কি না	কিছু পেলে সিক্রেট ফাঁস ধরে নিয়ে ঘোরান।
`grep -r "sk_live" --include="*.js" .`	কোডে সিক্রেট খোঁজা	ডিপ্লয়ের আগে স্ক্যান; CI-তেও চালান।
`openssl rand -hex 32`	৬৪ অক্ষরের দৈব সিক্রেট বানায়	মানুষের বানানো পাসওয়ার্ডের চেয়ে অনেক শক্তিশালী।
`pm2 restart technova-api --update-env`	নতুন ভেরিয়েবল নিয়ে পুনরায় চালু	শুধু restart করলে পুরোনো ভেরিয়েবল থেকে যেতে পারে।
`ls -l .env && stat -c %a .env`	ফাইলের মালিকানা ও অনুমতি দেখা	৬০০ না হলে ঠিক করুন — ৬৪৪ মানে সবাই পড়তে পারে।
`diff .env.example <(cut -d= -f1 .env)`	কোন কী বাদ পড়েছে, তা খোঁজা	example-এর প্রতিটি কী-র মান পরিবেশে আছে কি না মেলান।
`grep -rn "process.env" --include="*.js" server/`	কোডে কোথায় env পড়া হয়	অপ্রত্যাশিত জায়গায় সিক্রেট পড়া হচ্ছে কি না, তা ধরা পড়ে।

`env -i PATH=/usr/bin:/bin node -e "console.log(process.env.PORT)"`	খালি পরিবেশে অ্যাপ পরীক্ষা	শেলের ভেরিয়েবল ছাড়া কী হয়, দেখা যায় — যাচাই কোড কাজ করে কি না বোঝা যায়।
`sudo systemctl daemon-reload`	systemd-কে নতুন কনফিগ জানানো	override.conf বদলের পর আবশ্যক।

বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — GitHub-এ .env ফাঁস: নতুন ডেভেলপার `.gitignore` না দেখে কমিট করল। কয়েক ঘণ্টায় MongoDB Atlas অ্যালার্ট: অজানা IP থেকে লগইন চেষ্টা। প্রতিকার: সব সিক্রেট বাতিল ও নতুন দেওয়া, ডাটাবেস পাসওয়ার্ড বদলানো, IP allowlist শক্ত করা, git ইতিহাস থেকে ফাইল সরানো (তবুও সিক্রেট বাতিলই আসল সমাধান)।

পরিস্থিতি ২ — স্টেজিংয়ের চাবি প্রোডাকশনে: Stripe test key প্রোডাকশনে যাওয়ায় আসল পেমেন্ট নেওয়া যাচ্ছিল না। কারণ: পরিবেশভেদে ভেরিয়েবল আলাদা করা হয়নি। সমাধান: পরিবেশের নাম দিয়ে ভেরিয়েবল সেট যাচাইয়ের চেকলিস্ট।

পরিস্থিতি ৩ — লগে সিক্রেট: ভুল করে `console.log(process.env)` লেখায় লগ ফাইলে JWT সিক্রেট। সমাধান: লগে কখনো পুরো env বা টোকেন ছাপাবেন না; প্রয়োজনে শুধু কী-র নাম ও মানের দৈর্ঘ্য ছাপান।

পরিস্থিতি ৪ — TechNova-তে স্টেজিং চাবি প্রোডাকশনে: TechNova-র (technova.example, সার্ভার 203.0.113.10) এক ডেভেলপার তাড়াহুড়ো করে প্রোডাকশন সার্ভারে স্টেজিংয়ের `.env` কপি করে দিলেন। ফলে প্রোডাকশন অ্যাপ স্টেজিং ডাটাবেসে লিখতে শুরু করল — আসল অর্ডার ভুল জায়গায় যাচ্ছিল, আর Stripe test key থাকায় আসল পেমেন্ট নেওয়া যাচ্ছিল না। ধরা পড়ল health check-এ নয়, গ্রাহকের অভিযোগে। প্রতিকার: সার্ভিস থামিয়ে সঠিক প্রোডাকশন মান বসানো, দুই পরিবেশের সিক্রেটই ঘোরানো, আর ডিপ্লয় চেকলিস্টে "পরিবেশের নাম যাচাই" যোগ করা। শিক্ষা: পরিবেশের নাম (`NODE_ENV`) ও ডাটাবেসের নাম কনফিগ যাচাইয়ে রাখুন — ভুল পরিবেশে অ্যাপ যেন শুরুতেই থেমে যায়।

ধাপে ধাপে ব্যবহারিক উদাহরণ: নিরাপদ কনফিগ সেটআপ

1. প্রকল্পে `.env.example` বানান (মান ছাড়া) ও `.gitignore`-এ `.env` যোগ করুন; `git check-ignore -v .env` দিয়ে যাচাই করুন।
2. `openssl rand -hex 32` দিয়ে JWT সিক্রেট বানান (কাল্পনিক মান দিয়ে অনুশীলন করুন)।
3. সার্ভারে `/etc/technova/env` ফাইল বানিয়ে অনুমতি `600` দিন; PM2 ecosystem থেকে সেটি পড়ার ব্যবস্থা করুন।
4. `config.js`-এর মতো যাচাই কোড যোগ করে ইচ্ছা করে একটি ভেরিয়েবল বাদ দিয়ে দেখুন অ্যাপ স্পষ্ট বার্তা দিয়ে থামে কি না।
5. পুরোনো git ইতিহাসে সিক্রেট আছে কি না `git log --all -- .env` দিয়ে পরীক্ষা করুন।
6. সিক্রেট ঘোরানোর মহড়া: নতুন মান দিয়ে `pm2 restart --update-env` করুন ও health endpoint পরীক্ষা করুন।

সাধারণ ভুল ও ভুল ধারণা

- **ভুল ধারণা:** "প্রাইভেট রিপোজিটরি মানেই নিরাপদ" — দলের সদস্য, ভবিষ্যতে পাবলিক করা, অ্যাক্সেস হারানো — সবই ঝুঁকি। সিক্রেট কখনোই git-এ নয়।
- **ভুল:** ফ্রন্টএন্ড বান্ডলে API সিক্রেট রাখা — ব্রাউজারে সব দেখা যায়।
- **ভুল:** সব পরিবেশে একই JWT সিক্রেট — একটি ফাঁস হলে সব পরিবেশ আক্রান্ত হয়।
- **ভুল:** `.env` ফাইলে `chmod 644` রাখা — সার্ভারের যেকোনো ব্যবহারকারী পড়তে পারে।

- **ভুল ধারণা:** "সিক্রেট মুছে ফেললেই ঠিক" — ইতিহাসে থাকা সিক্রেট বাতিল না করলে ঝুঁকি থেকেই যায়।

নিরাপত্তা বিবেচনা

- সিক্রেটের ন্যূনতম সুযোগ (least privilege): প্রতিটি চাবি শুধু তার কাজটুকু করুক; ডাটাবেস ব্যবহারকারীর শুধু প্রয়োজনীয় অনুমতি থাকুক।
- নিয়মিত ঘূর্ণন (rotation): JWT সিক্রেট, API চাবি, ডাটাবেস পাসওয়ার্ড — সময়সূচি মেনে বদলান, দুর্ঘটনার অপেক্ষা করবেন না।
- সিক্রেট ম্যানেজার (যেমন ক্লাউড প্রদানকারীর সিক্রেট সেবা) বড় দলের জন্য ভালো — ফাইলে সিক্রেট রাখার চেয়ে নিরাপদ ও অডিটযোগ্য।
- লগ, ত্রুটি বার্তা ও API উত্তরে সিক্রেট কখনো ছাপাবেন না; ত্রুটি বার্তায় কনফিগের মানও নয়।
- সার্ভারে .env ফাইলের ব্যাকআপ নিলে সেটিও এনক্রিপ্ট করে রাখুন — প্লেইন ব্যাকআপও ফাঁসের পথ।

সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`Missing required env var`	ভেরিয়েবল দেওয়া হয়নি	PM2/systemd-এ মান আছে কি না দেখুন; `--update-env` দিয়ে restart করুন।
অ্যাপ চলে কিন্তু DB সংযোগ ব্যর্থ	MONGO_URI-তে ভুল হোস্ট/পাসওয়ার্ড	URI-র হোস্ট, পোর্ট, ব্যবহারকারী যাচাই; পাসওয়ার্ডে বিশেষ অক্ষর URL-encode করুন।
নতুন মান কাজ করছে না	পুরোনো env ক্যাশে	`pm2 restart --update-env` বা `pm2 delete && pm2 start` করুন।
ফ্রন্টএন্ডে API ঠিকানা ভুল	বিল্ডের সময়ের ভেরিয়েবল	React-এ env বিল্ড-সময়ে জমে যায় — মান বদলে আবার build করুন।
স্টেজিং ঠিক, প্রোডাকশনে পেমেন্ট ব্যর্থ	প্রোডাকশনে test key রয়ে গেছে	পরিবেশের নাম দিয়ে চাবির ধরন যাচাই করুন; live key শুধু প্রোডাকশনে দিন।
সিক্রেট বদলের পর সবাই লগআউট	JWT সিক্রেট বদলে সব টোকেন বাতিল	এটি প্রত্যাশিত — আগেভাগে জানান; প্রয়োজনে কিছু সময় দুটি সিক্রেটই গ্রহণ করুন।
.env ফাইলের মান পড়া যাচ্ছে না	ফাইলের মালিকানা/অনুমতি ভুল, বা PM2 অন্য ব্যবহারকারীর অধীনে চলছে	`ls -l .env` দিয়ে মালিক ও অনুমতি (600) দেখুন; PM2 কোন ব্যবহারকারীতে চলে, মেলান।
ভেরিয়েবলে বিশেষ অক্ষর (যেমন @ বা #) থাকায় URI ভাঙছে	পাসওয়ার্ড URL-encode করা হয়নি	বিশেষ অক্ষর শতাংশ-এনকোড করুন (যেমন @ → %40); URI-র গঠন যাচাই করুন।
git-এ সিক্রেট ধরা পড়ল	ফাঁস	আতঙ্কিত না হয়ে: সিক্রেট বাতিল → নতুন দিন → ইতিহাস পরিষ্কার → দলকে জানান।

সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

- **করুন:** চালুর সময় সব আবশ্যক ভেরিয়েবল যাচাই করুন; ব্যর্থ হলে স্পষ্ট বার্তায় থামুন।

- **করুন:** `.env.example` সবসময় হালনাগাদ রাখুন — এটি কনফিগের ডকুমেন্টেশন।
- **করবেন না:** কোডে হার্ডকোড করা সিক্রেট, ডিফল্ট সিক্রেট (`secret123`), বা চ্যাট/ইমেইলে সিক্রেট পাঠানো।
- **করবেন না:** সিক্রেট ঘোরানো ছাড়া "সমস্যা মিটে গেছে" ধরে নেওয়া।

অধ্যায়ের সারসংক্ষেপ

কনফিগারেশন কোড নয় — এটি পরিবেশ থেকে আসে। .env, PM2, systemd বা সিক্রেট ম্যানেজার — যে পথেই আসুক, নিয়ম এক: সিক্রেট কখনো git-এ নয়, ফ্রন্টএন্ডে নয়, লগে নয়; অনুমতি শক্ত রাখুন, চালুর সময় যাচাই করুন, আর ফাঁস হলে বাতিল করে ঘোরান। ভালো কনফিগ ব্যবস্থাপনা মানে ডিপ্লয় নির্ভরযোগ্য ও নিরাপদ।

মূল প্রযুক্তিগত পরিভাষা

পরিবেশ ভেরিয়েবল (Environment Variable): প্রসেস চালুর সময় দেওয়া নাম-মান জোড়া। **সিক্রেট (Secret):** পাসওয়ার্ড, API চাবি, টোকেনের মতো গোপন মান। **সিক্রেট রোটেশন (Secret Rotation):** নিয়মিত বা ফাঁসের পর গোপন মান বদলানো। **Twelve-Factor App:** কনফিগ পরিবেশে রাখার নীতিসহ আধুনিক অ্যাপ তৈরির বারোটি নীতি। **Least Privilege:** প্রতিটি চাবি/ব্যবহারকারীকে শুধু প্রয়োজনীয় অনুমতি দেওয়া। **.env.example:** আসল মান ছাড়া প্রয়োজনীয় ভেরিয়েবলের তালিকা।

জ্ঞান যাচাই প্রশ্ন

1. কনফিগারেশন কোড থেকে আলাদা রাখা কেন জরুরি? দুটি কারণ দিন।
2. .env ফাইল git-এ কমিট করলে কী কী ঝুঁকি হয়?
3. প্রাইভেট রিপোজিটরিতেও সিক্রেট রাখা কেন নিরাপদ নয়?
4. React বান্ডলে সিক্রেট রাখলে কী হয়?
5. সিক্রেট ফাঁস হলে প্রথম তিনটি করণীয় কী?
6. `pm2 restart` করেও নতুন ভেরিয়েবল কাজ না করলে কী করবেন?
7. স্টেজিং ও প্রোডাকশনে একই JWT সিক্রেট ব্যবহার করলে কী ঝুঁকি হয়?
8. সিক্রেট ঘোরানোর সময় ব্যবহারকারীদের হঠাৎ লগআউট এড়াতে কী কৌশল নেবেন?
9. চালুর সময় কনফিগ যাচাই কেন ভালো — নীরবে ভুল মান নিয়ে চলার চেয়ে?

ব্যবহারিক অনুশীলন

1. **নিরাপদ সেটআপ:** একটি নমুনা প্রকল্পে .env, .env.example ও .gitignore সাজান; `git check-ignore` দিয়ে যাচাই করুন।
2. **যাচাই কোড:** `config.js`-এর মতো যাচাই লিখে একটি ভেরিয়েবল বাদ দিয়ে অ্যাপ থামার পরীক্ষা করুন (disposable পরিবেশে)।
3. **ঘূর্ণন মহড়া:** একটি কাল্পনিক JWT সিক্রেট বদলে `--update-env` দিয়ে পুনরায় চালু করে health check করুন; ধাপগুলো লিখে রাখুন।
4. **পরিবেশ-তুলনা টেবিল:** একটি নমুনা প্রকল্পের জন্য স্টেজিং ও প্রোডাকশনের কনফিগের তুলনা টেবিল লিখুন — ডাটাবেস, চাবি, লগের মাত্রা ও ব্যাকআপ নীতি আলাদা করে।

5. **স্ক্যান:** নিজের কোনো পুরোনো প্রকল্পে `grep` দিয়ে হার্ডকোড করা সিক্রেট খুঁজুন (পেলে বাতিলের পরিকল্পনা লিখুন, আসল সিক্রেট কোথাও পেস্ট করবেন না)।

অধ্যায় ২৩

Deployment Strategies, Rollback and Downtime Reduction

কোড লেখা শেষ — এবার সেটি জীবন্ত ব্যবহারকারীদের কাছে পৌঁছে দিতে হবে, তাদের কাজ না থামিয়ে। ডিপ্লয়মেন্ট কৌশল (deployment strategy) হলো সেই প্রশ্নের উত্তর: নতুন সংস্করণ কীভাবে চালু করব, কিছু ভুল হলে কীভাবে ফিরব, আর ব্যবহারকারী যেন টেরই না পায়? এই অধ্যায়ে ব্লু-গ্রিন, ক্যানারি ও রোলিং ডিপ্লয়মেন্ট শিখব, সঙ্গে রোলব্যাক পরিকল্পনা।

শেখার উদ্দেশ্য

- ব্লু-গ্রিন, ক্যানারি ও রোলিং ডিপ্লয়মেন্টের পার্থক্য ও উপযুক্ত পরিস্থিতি ব্যাখ্যা করতে পারবেন।
- শূন্য-ডাউনটাইম (zero-downtime) ডিপ্লয়ের নীতিগুলো — health check, ধাপে ধাপে বদল, ডাটাবেস সামঞ্জস্য — প্রয়োগ করতে পারবেন।
- রোলব্যাক পরিকল্পনা লিখে অনুশীলন করতে পারবেন।
- ডাটাবেস মাইগ্রেশন কেন ডিপ্লয়ের সবচেয়ে ঝুঁকিপূর্ণ অংশ, তা ব্যাখ্যা করতে পারবেন।
- PM2 reload, Nginx ও লোড ব্যালান্সিং দিয়ে নিরাপদ ডিপ্লয় সাজাতে পারবেন।

প্রোডাকশনে এটি কেন গুরুত্বপূর্ণ

প্রতিটি ডিপ্লয় একটি ঝুঁকি: নতুন কোডে বাগ থাকতে পারে, ডাটাবেস বদলে যেতে পারে, নির্ভরতা (dependency) ভাঙতে পারে। "সব বন্ধ করে নতুনটা চালাই" পদ্ধতিতে যেকোনো ভুল মানেই সব ব্যবহারকারী ক্ষতিগ্রস্ত। ভালো কৌশল ঝুঁকিকে ছোট ছোট টুকরোয় ভাগ করে: প্রথমে ৫% ব্যবহারকারী, তারপর ৫০%, তারপর সবাই — আর যেকোনো ধাপে ফেরার পথ খোলা থাকে। বড় কোম্পানিগুলো দিনে শতবার ডিপ্লয় করে এই কারণেই — তাদের কৌশল ভুলকে সস্তা করে দেয়।

> **বাস্তব ঘটনা:** একটি দল শুক্রবার রাতে সরাসরি প্রোডাকশনে ডিপ্লয় করে দেখল পেমেন্ট পেজ ভাঙা। রোলব্যাকের কোনো পরিকল্পনা ছিল না, পুরোনো কোড খুঁজে পেতে দুই ঘণ্টা লাগল। তারপর থেকে তাদের নিয়ম: রোলব্যাক পরীক্ষা না করে ডিপ্লয় নয়।

প্রয়োজনীয় পূর্বজ্ঞান

- অধ্যায় ২১–২২: PM2, Nginx, পরিবেশ ভেরিয়েবল।
- git-এ ট্যাগ/রিলিজ ও `git revert`-এর ধারণা।
- ডাটাবেস স্কিমা পরিবর্তন (migration) কী — প্রাথমিক ধারণা।
- HTTP স্ট্যাটাস কোড ও health endpoint-এর ধারণা।

প্রথম নীতি: বদল ছোট করুন, ফেরার পথ রাখুন

প্রথম নীতি থেকে: যেকোনো বদলের ঝুঁকি = বদলের আকার × আক্রান্ত ব্যবহারকারীর সংখ্যা × ফিরতে লাগা সময়। তিনটি কৌশলই এই তিনটি গুণকের একটিকে কমায়। রোলিং বদলকে ছোট টুকরোয় ভাগ করে; ক্যানারি আক্রান্ত ব্যবহারকারী কমায়; ব্লু-গ্রিন ফেরার সময় প্রায় শূন্যে নামায়। আর সব কৌশলের ভিত্তি একটিই: পুরোনো সংস্করণ যতক্ষণ না নতুনটি

প্রমাণিত হচ্ছে, ততক্ষণ হাতের কাছে রাখুন। ডিপ্লয়মেন্ট তাই "নতুনটা চালু করা" নয় — এটি "নতুনটা প্রমাণ করে পুরোনোটা সরানো"।

আর্কিটেকচার: তিনটি কৌশলের চিত্র

রোলিং: [v1] [v1] [v1] → [v2] [v1] [v1] → [v2] [v2] [v1] → [v2] [v2] [v2] — একবারে একটি ইনস্ট্যান্স বদলায়।
ব্লু-গ্রিন: Blue (v1, লাইভ) + Green (v2, প্রস্তুত) → রাউটার সুইচ → Green লাইভ, Blue স্ট্যান্ডবাই (ফেরার পথ)।
ক্যানারি: 95% → v1, 5% → v2 → মেট্রিক ভালো হলে 50% → 100%; খারাপ হলে 0%-এ ফেরত।
সব ক্ষেত্রে সামনে: Nginx/লোড ব্যালান্সার + health check; পেছনে: একটিই ডাটাবেস (সামঞ্জস্য আবশ্যক)।

বিস্তারিত ব্যাখ্যা

রোলিং ডিপ্লয়মেন্ট

একাধিক ইনস্ট্যান্স থাকলে একবারে একটি করে বদলানো হয়। PM2 cluster mode-এ `pm2 reload` ঠিক এটিই করে — একটি ইনস্ট্যান্স নতুন কোডে ওঠে, health check পাস করলে পরেরটি। সুবিধা: অতিরিক্ত সার্ভার লাগে না; অসুবিধা: কিছুক্ষণ দুটি সংস্করণ একসাথে চলে — তাই API-র দুটি সংস্করণকে একই ডাটাবেসের সঙ্গে চলতে হবে। ছোট দলের জন্য এটিই সবচেয়ে বাস্তবসম্মত শুরু।

ব্লু-গ্রিন ডিপ্লয়মেন্ট

দুটি অভিন্ন পরিবেশ: Blue চলছে, Green-এ নতুন সংস্করণ তৈরি। পরীক্ষা শেষে রাউটার (Nginx-এ `proxy_pass` বদলে) ট্রাফিক Green-এ পাঠায়। সমস্যা হলে এক মিনিটে Blue-তে ফেরত। সুবিধা: দ্রুততম রোলব্যাক, ডিপ্লয়ের আগে পূর্ণ পরীক্ষা; অসুবিধা: দ্বিগুণ সম্পদ (দুটি পরিবেশ) ও ডাটাবেস শেয়ার করার জটিলতা।

```
# Nginx-এ সুইচ: upstream বদলে reload (সংযোগ না কেটে)
upstream technova_api {
    server 127.0.0.1:5000;   # Blue — বর্তমান লাইভ
    # server 127.0.0.1:5001; # Green — প্রস্তুত হলে এই লাইন চালু, আগেরটি মন্তব্য করুন
}
# তারপর: sudo nginx -t && sudo systemctl reload nginx
```

ক্যানারি ডিপ্লয়মেন্ট

নতুন সংস্করণ প্রথমে অল্প ব্যবহারকারীর কাছে (যেমন ৫%) পাঠানো হয়; ত্রুটির হার, লেটেন্সি ও ব্যবসায়িক মেট্রিক দেখে ধীরে ধীরে বাড়ানো হয়। Nginx-এ ওজন (weight) দিয়ে বা কুকি/হেডার দিয়ে ভাগ করা যায়। সুবিধা: বাগের আঘাত সীমিত; অসুবিধা: মেট্রিক দেখার ব্যবস্থা ও ট্রাফিক ভাগের কনফিগ লাগে — ছোট দলে হাতে করা কঠিন, তবে ধারণাটি জানা জরুরি।

ডাটাবেস: সবচেয়ে কঠিন অংশ

কোড ফেরানো সহজ, ডাটাবেস ফেরানো কঠিন — কারণ তথ্য বদলে গেছে। নিয়ম: **পেছন-সামঞ্জস্যপূর্ণ (backward-compatible) মাইগ্রেশন** করুন। নতুন কলাম যোগ করা নিরাপদ; কলাম মুছে ফেলা বা নাম বদলানো বিপজ্জনক। ধাপ: (১) ডিপ্লয় ১: নতুন কলাম যোগ (পুরোনো কোডও চলে); (২) ডিপ্লয় ২: নতুন কোড কলামটি ব্যবহার করে; (৩) পরে আলাদা ডিপ্লয়ে পুরোনো কলাম সরান। একে বলে expand-and-contract — প্রথমে প্রসারিত করুন, পরে সংকুচিত করুন।

শূন্য-ডাউনটাইমের চেকলিস্ট

- Health endpoint (`/api/health`) যা DB সংযোগও যাচাই করে; লোড ব্যালান্সার অসুস্থ ইনস্ট্যান্সকে ট্রাফিক দেবে না।
- Graceful shutdown: নতুন সংযোগ নেওয়া বন্ধ করে চলমান অনুরোধ শেষ হতে দিন (`SIGTERM` ধরে)।
- বিল্ড ও টেস্ট ডিপ্লয়ের আগেই — প্রোডাকশন সার্ভারে বিল্ড ব্যর্থ হওয়া মানে ডাউনটাইম।

রোলব্যাক মহড়া: ধাপে ধাপে অনুশীলন

রোলব্যাক পরিকল্পনা কাগজে থাকা আর সত্যিই কাজ করা এক কথা নয় — তাই নিয়মিত মহড়া (drill) দরকার। মহড়ার উদ্দেশ্য শুধু গতি নয়; উদ্দেশ্য হলো প্রতিটি ধাপ, প্রতিটি কমান্ড ও প্রতিটি সিদ্ধান্ত আগে থেকে পরীক্ষা করা, যাতে আসল ঘটনার রাত দুটোয় কেউ আন্দাজে কাজ না করে। মহড়া সবসময় disposable VM বা স্টেজিংয়ে করবেন — কখনো প্রোডাকশনে নয়। শুরুতে একটি ভাঙা পরিবর্তন ইচ্ছা করে ডিপ্লয় করুন (যেমন health endpoint-এ ভুল), তারপর ঘড়ি ধরে নিচের ধাপগুলো অনুসরণ করুন।

1. **ঘোষণা:** দলকে জানান মহড়া চলছে; কে সিদ্ধান্ত নেবে, ঠিক করুন। সময়রেখা লেখা শুরু করুন।
2. **শনাক্তকরণ:** health check ব্যর্থ, ত্রুটির হার বেড়েছে — কোন মেট্রিক দেখে বুঝলেন? লিখুন।
3. **সিদ্ধান্ত:** ঠিক করুন — রোলব্যাক করবেন, নাকি সামনে এগিয়ে ঠিক করবেন (roll forward)? নিয়ম: কারণ অজানা থাকলে রোলব্যাকই নিরাপদ।
4. **রোলব্যাক:** আগের ট্যাগে ফিরুন — `git checkout v1.3.0` , `npm ci --omit=dev` , `npm run build` , `pm2 reload` — তারপর health check। ডাটাবেস মাইগ্রেশন হয়ে থাকলে ব্যাকআপ থেকে ফেরার ধাপটি আগে থেকে লেখা থাকতে হবে।
5. **নিশ্চিতকরণ:** বাইরে থেকে সাইট খুলে, লগইন করে ও একটি লেনদেন করে দেখুন। ত্রুটির হার আগের মানে ফিরেছে কি না, দেখুন।
6. **পর্যালোচনা:** কত মিনিট লাগল? কোন ধাপে আটকে গেলেন? রানবুকে কী বদলাতে হবে? প্রতিটি মহড়া শেষে রোলব্যাক নথিটি হালনাগাদ করুন।

পরিস্থিতি	প্রথম পছন্দ	কেন
নতুন কোডে বাগ, কারণ অজানা	রোলব্যাক	পরিচিত-ভালো সংস্করণে ফেরা দ্রুততম ও নিরাপদ।
ছোট ও স্পষ্ট ভুল (যেমন টাইপো), ঠিক করতে দুই মিনিট	Roll forward (সামনে ঠিক করা)	ছোট ঠিক করা রোলব্যাকের চেয়ে দ্রুত হতে পারে — তবে ঝুঁকি মেপে।
ডাটাবেস মাইগ্রেশনসহ বড় বদল	আগে থেকে লেখা পরিকল্পনা অনুযায়ী রোলব্যাক	শুধু কোড ফেরানো যথেষ্ট নয় — ব্যাকআপ ও মাইগ্রেশন-ফেরতের ধাপ লাগে।

> **সতর্কতা — রোলব্যাক মহড়ার আগে:** মহড়া শুধু disposable পরিবেশে করুন। প্রোডাকশনে রোলব্যাক করার আগে ব্যাকআপ/স্ন্যাপশট নিন, দলকে জানান, আর রোলব্যাকের ধাপগুলো লিখিতভাবে সামনে রাখুন — মুখস্থ ধাপে ভুল হয়। মাইগ্রেশন হয়ে গিয়ে থাকলে শুধু কোড ফেরাবেন না; ডাটাবেস ফেরার পরিকল্পনা ছাড়া রোলব্যাক করবেন না।

- স্ট্যাটিক সম্পদে (asset) হ্যাশযুক্ত ফাইলনাম — পুরোনো HTML নতুন JS চাইবে না।

গুরুত্বপূর্ণ কমান্ড ও বিশ্লেষণ

কমান্ড	কী করে	বিশ্লেষণ
`pm2 reload technova-api`	শূন্য-ডাউনটাইম রোলিং পুনরায় চালু	cluster mode-এ একবারে একটি ইনস্ট্যান্স বদলায়।
`pm2 restart technova-api`	সব ইনস্ট্যান্স একসাথে পুনরায় চালু	সংক্ষিপ্ত ডাউনটাইম হয় — রোলিং চাইলে reload ব্যবহার করুন।
`git tag v1.4.0 && git push --tags`	রিলিজ চিহ্নিত করা	রোলব্যাক মানে নির্দিষ্ট ট্যাগে ফেরা — ট্যাগ ছাড়া "আগেরটা" অস্পষ্ট।
`git revert <commit>`	একটি কমিটের বিপরীত পরিবর্তন	ইতিহাস মুছে না — নিরাপদে ফেরার পথ।
`curl -f http://127.0.0.1:5000/api/health`	health check (ব্যর্থ হলে exit code ≠ 0)	ডিপ্লয় স্ক্রিপ্টে ব্যবহার করুন — ব্যর্থ হলে থামুন।
`sudo nginx -t && sudo systemctl reload nginx`	যাচাই করে কনফিগ পুনরায় লোড	সুইচের আগে সিনট্যাক্স যাচাই বাধ্যতামূলক।
`git log --oneline -10 && git tag --sort=-v:refname | head -5`	সাম্প্রতিক কমিট ও ট্যাগ দেখা	রোলব্যাকে কোন ট্যাগে ফিরবেন, তা এখান থেকে ঠিক করুন।
`git checkout v1.3.0 -- . && npm ci --omit=dev`	আগের ট্যাগের কোডে ফেরা	রোলব্যাকের মূল ধাপ — তবে ডাটাবেসের অবস্থা আলাদা করে দেখুন।
`watch -n 2 'curl -s -o /dev/null -w "%{http_code}" http://127.0.0.1:5000/api/health'`	health স্ট্যাটাস ক্রমাগত দেখা	ডিপ্লয়/রোলব্যাকের সময় প্রতি দুই সেকেন্ডে অবস্থা জানা যায়।
`mongorestore --drop /backup/technova-2026-11-08`	ব্যাকআপ থেকে ডাটাবেস ফেরানো	শুধু পরিকল্পনা অনুযায়ী ও স্ন্যাপশট/ব্যাকআপ নিশ্চিত করে — এটি ধ্বংসাত্মক কাজ।
`mongodump --out /backup/technova-2026-11-08`	MongoDB ব্যাকআপ	মাইগ্রেশনের আগে আবশ্যক — ফেরার শেষ ভরসা।

ডিপ্লয় স্ক্রিপ্ট উদাহরণ (রোলিং)

```
#!/bin/bash
# deploy.sh — ফিকশনাল উদাহরণ; প্রোডাকশনে চালানোর আগে disposable VM-এ পরীক্ষা করুন
set -e
cd /var/www/technova
git pull origin main
npm ci --omit=dev
npm run build
npm test || { echo "Tests failed — aborting deploy"; exit 1; }
pm2 reload ecosystem.config.js --update-env
sleep 5
```

```
curl -f http://127.0.0.1:5000/api/health || { echo "Health check failed — rolling back"; pm2
reload ecosystem.config.js; exit 1; }
echo "Deploy OK"
```

`set -e` মানে যেকোনো ধাপ ব্যর্থ হলে স্ক্রিপ্ট থেমে যাবে — অর্ধেক ডিপ্লয় চলতে দেবে না। health check ব্যর্থ হলে রোলব্যাকের চেষ্টা করা হয়েছে — বাস্তবে রোলব্যাক মানে আগের ট্যাগে `git checkout` করে আবার reload করা।

> **সতর্কতা — রোলব্যাকের আগে:** রোলব্যাক নিজেও একটি ডিপ্লয় — তাতে ডাটাবেস বদলে গিয়ে থাকলে শুধু কোড ফেরানো যথেষ্ট নাও হতে পারে। মাইগ্রেশনসহ যেকোনো ঝুঁকিপূর্ণ ডিপ্লয়ের আগে ব্যাকআপ/স্ন্যাপশট নিন, আর রোলব্যাকের ধাপ আগে থেকে লিখে রাখুন ও অনুশীলন করুন। কখনো প্রোডাকশনে প্রথমবার রোলব্যাক পরীক্ষা করবেন না — disposable পরিবেশে মহড়া দিন।

বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — ডিপ্লয়ের পর ত্রুটির হার বাড়ল: ক্যানারি ৫%-এ ত্রুটি ০.১% থেকে ৪% হলো। সিদ্ধান্ত: সঙ্গে সঙ্গে ট্রাফিক ০%-এ ফেরানো হলো — মাত্র ৫% ব্যবহারকারী ক্ষতিগ্রস্ত। শিক্ষা: ক্যানারি ছাড়া এই বাগ সবাইকে আঘাত করত।

পরিস্থিতি ২ — মাইগ্রেশন আটকে গেল: বড় টেবিলে কলাম যোগ করতে গিয়ে ডাটাবেস লক হয়ে গেল। শিক্ষা: বড় মাইগ্রেশন ব্যস্ত সময়ে নয়, ব্যাকগ্রাউন্ডে ধাপে ধাপে করুন; আগে ব্যাকআপ নিন।

পরিস্থিতি ৩ — পুরোনো ক্যাশ: ডিপ্লয়ের পর ব্যবহারকারীরা পুরোনো ফ্রন্টএন্ড দেখছেন। কারণ: CDN/ব্রাউজার ক্যাশে `index.html` আটকে আছে। সমাধান: `index.html`-এ `no-cache`, সম্পদে (asset) হ্যাশযুক্ত নাম।

পরিস্থিতি ৪ — TechNova-তে রোলব্যাক মহড়া কাজে লাগল: TechNova (technova.example, সার্ভার 203.0.113.10) প্রতি মাসে রোলব্যাক মহড়া করে। এক মঙ্গলবার সত্যিকারের ডিপ্লয়ে পেমেন্ট পেজ ভাঙল — ক্যানারি ৫%-এ ত্রুটির হার লাফিয়ে উঠল। দল আতঙ্কিত না হয়ে মহড়ার ধাপ অনুসরণ করল: ট্রাফিক শূন্যে নামানো, আগের ট্যাগ v1.3.0-এ ফেরা, `npm ci` ও বিল্ড, `pm2 reload`, health check — মোট নয় মিনিটে সেবা ফিরল। পরে দেখা গেল মহড়ায় লেখা একটি ধাপেই সময় বেঁচেছে: কোন ট্যাগে ফিরতে হবে, তা আগে থেকে লেখা ছিল। শিক্ষা: মহড়া না থাকলে সেই নয় মিনিট দুই ঘণ্টা হতো — রোলব্যাক পরিকল্পনা অনুশীলন ছাড়া কাগজের কথাই থাকে।

ধাপে ধাপে ব্যবহারিক উদাহরণ: নিরাপদ রোলিং ডিপ্লয়

1. পরিবর্তনগুলো git-এ কমিট করে `vX.Y.Z` ট্যাগ দিন; কী বদলাচ্ছে লিখে রাখুন।
2. Disposable VM বা স্টেজিংয়ে একই ডিপ্লয় স্ক্রিপ্ট চালিয়ে যাচাই করুন।
3. প্রোডাকশনে ডিপ্লয়ের আগে `mongodump` ব্যাকআপ নিন (মাইগ্রেশন থাকলে আবশ্যক)।
4. `deploy.sh` চালান; health check পাস না হলে স্ক্রিপ্ট থামবে — লগ দেখুন।
5. ডিপ্লয়ের পর ১৫ মিনিট মেট্রিক দেখুন: ত্রুটির হার, লেটেন্সি, লগে ব্যতিক্রম (exception)।
6. সমস্যা হলে রোলব্যাক: আগের ট্যাগে ফিরে `npm ci && npm run build && pm2 reload` করুন ও health check করুন।

সাধারণ ভুল ও ভুল ধারণা

- **ভুল ধারণা:** "রোলব্যাক মানেই git revert" — ডাটাবেস বদলে গেলে শুধু কোড ফেরানো যথেষ্ট নয়।
- **ভুল:** ব্যস্ত সময়ে (peak hour) ডিপ্লয় করা — কম ট্রাফিকের সময় বেছে নিন।

- **ভুল:** health check ছাড়া ডিপ্লয় — অ্যাপ চালু হলেই সুস্থ নয়, DB সংযোগও যাচাই করুন।
- **ভুল ধারণা:** "ব্লু-গ্রিন মানে দ্বিগুণ খরচ সবসময়" — ছোট সেটআপে একই সার্ভারে দুটি পোর্টে দুটি সংস্করণ চালিয়েও নীতি অনুশীলন করা যায়।
- **ভুল:** ডিপ্লয়ের সময় কেউ না দেখা — ডিপ্লয়ের পর অন্তত ১৫–৩০ মিনিট পর্যবেক্ষণ করুন।

নিরাপত্তা বিবেচনা

- ডিপ্লয় শুধু অনুমোদিত কী/ব্যবহারকারী দিয়ে — সার্ভারে সবার SSH নয়; ডিপ্লয় কী আলাদা ও সীমিত রাখুন।
- ডিপ্লয় স্ক্রিপ্টে সিক্রেট লিখবেন না; পরিবেশ ভেরিয়েবল থেকে নিন (অধ্যায় ২২)।
- রোলব্যাকের সময়ও নিরাপত্তা প্যাচ ফিরিয়ে আনবেন না — কোন সংস্করণে ফিরছেন, তাতে জানা দুর্বলতা আছে কি না দেখুন।
- ডিপ্লয় লগ সংরক্ষণ করুন: কে, কখন, কোন সংস্করণ ডিপ্লয় করল — ঘটনার তদন্তে এটিই প্রমাণ।

সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
reload-এর পরও পুরোনো কোড	বিল্ড হয়নি / ভুল ডিরেক্টরি	`npm run build` হয়েছে কি না, cwd ঠিক আছে কি না দেখুন।
নতুন কোডে DB ত্রুটি	অসামঞ্জস্যপূর্ণ মাইগ্রেশন	মাইগ্রেশন লগ দেখুন; expand-and-contract মেনে চলুন; ব্যাকআপ থেকে ফেরার পরিকল্পনা করুন।
একটি ইনস্ট্যান্স অসুস্থ	মেমোরি/নির্ভরতা সমস্যা	`pm2 logs`, `pm2 monit`; অসুস্থ ইনস্ট্যান্স restart করুন।
রোলব্যাকের পরও নতুন বাগ রয়ে গেল	ডাটাবেস মাইগ্রেশন ফেরানো হয়নি	কোডের সঙ্গে মাইগ্রেশনের অবস্থা মেলান; প্রয়োজনে ব্যাকআপ থেকে ডাটাবেস ফেরান (পরিকল্পনা অনুযায়ী)।
Canary-তে ৫% ব্যবহারকারীর ত্রুটি বাড়ল, কিন্তু কেউ দেখল না	মেট্রিক/অ্যালার্ট দেখা হয়নি	ত্রুটির হার ও লেটেন্সির সতর্কতা (alert) আগেই নির্ধারণ করুন; ক্যানারি মানেই পর্যবেক্ষণ।
দুটি সংস্করণ একসাথে চলায় ডাটা অসামঞ্জস্য	API-র দুটি সংস্করণ একই ডাটাবেসে ভিন্নভাবে লিখছে	Backward-compatible মাইগ্রেশন মেনে চলুন; ভাঙা বদল দুই ধাপে করুন (expand-and-contract)।
ডিপ্লয় স্ক্রিপ্ট অর্ধেক পথে থেমে গেল	একটি ধাপ ব্যর্থ, কিন্তু `set -e` নেই	স্ক্রিপ্টে `set -e` রাখুন; প্রতিটি ধাপের পর health check দিন।
ডিপ্লয়ের পর লগইন ভাঙল	JWT সিক্রেট বদলে গেছে	সিক্রেট ঘোরালে সব টোকেন বাতিল হয় — ব্যবহারকারীদের আবার লগইন করতে হবে; পরিকল্পনায় জানান।

সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

- **করুন:** ছোট, ঘন ঘন ডিপ্লয় — বড় "বিগ ব্যাং" ডিপ্লয়ের চেয়ে ঝুঁকি কম।
- **করুন:** প্রতিটি রিলিজে ট্যাগ, পরিবর্তন-তালিকা (changelog) ও রোলব্যাক ধাপ লিখুন।

- **করবেন না:** শুক্রবার রাতে বা ছুটির আগে বড় ডিপ্লয় — "Friday deploy" এড়িয়ে চলুন।
- **করবেন না:** পরীক্ষা ছাড়া সরাসরি প্রোডাকশনে ডিপ্লয়।

অধ্যায়ের সারসংক্ষেপ

ভালো ডিপ্লয়মেন্ট কৌশল ঝুঁকিকে ছোট করে: রোলিং ছোট দলের জন্য বাস্তবসম্মত, ব্লু-গ্রিন দ্রুততম রোলব্যাক দেয়, ক্যানারি বাগের আঘাত সীমিত রাখে। সব কৌশলের প্রাণ হলো health check, পেছন-সামঞ্জস্যপূর্ণ ডাটাবেস বদল, আর আগে থেকে লেখা ও অনুশীলন করা রোলব্যাক পরিকল্পনা। ডিপ্লয় শেষ হয় পর্যবেক্ষণ দিয়ে, reload দিয়ে নয়।

মূল প্রযুক্তিগত পরিভাষা

ব্লু-গ্রিন ডিপ্লয়মেন্ট (Blue-Green): দুটি অভিন্ন পরিবেশের মধ্যে ট্রাফিক সুইচ করে ডিপ্লয়। **ক্যানারি ডিপ্লয়মেন্ট (Canary):** প্রথমে অল্প ব্যবহারকারীর কাছে নতুন সংস্করণ পাঠানো। **রোলিং ডিপ্লয়মেন্ট (Rolling):** একবারে একটি করে ইনস্ট্যান্স বদলানো। **রোলব্যাক (Rollback):** আগের পরিচিত-ভালো সংস্করণে ফেরা। **Expand-and-Contract:** প্রথমে নতুন কাঠামো যোগ, পরে পুরোনোটি সরানোর মাইগ্রেশন নীতি। **Graceful Shutdown:** চলমান অনুরোধ শেষ করে প্রসেস বন্ধ করা।

জ্ঞান যাচাই প্রশ্ন

1. ব্লু-গ্রিন, ক্যানারি ও রোলিং — কোনটি কখন বেছে নেবেন? প্রতিটির একটি করে উপযুক্ত পরিস্থিতি দিন।
2. `pm2 reload` ও `pm2 restart`-এর পার্থক্য কী?
3. ডাটাবেস মাইগ্রেশনে expand-and-contract কেন জরুরি?
4. Health check-এ শুধু "সার্ভার চলছে" দেখলেই কি যথেষ্ট? কেন নয়?
5. রোলব্যাক পরিকল্পনা আগে থেকে লেখা কেন জরুরি — ঘটনার সময় লিখলে কী সমস্যা?
6. শূন্য-ডাউনটাইম ডিপ্লয়ের তিনটি শর্ত লিখুন।
7. Roll forward ও rollback — কখন কোনটি বেছে নেবেন? একটি করে উদাহরণ দিন।
8. রোলব্যাক মহড়া কেন প্রোডাকশনে নয়, disposable পরিবেশে করতে হয়?
9. ক্যানারি ডিপ্লয়ে কোন মেট্রিকগুলো দেখবেন?

ব্যবহারিক অনুশীলন

1. **রোলিং ডিপ্লয়:** disposable VM-এ cluster mode-এ অ্যাপ চালিয়ে `pm2 reload` করুন; ডিপ্লয়ের সময় `curl` লুপ চালিয়ে দেখুন কোনো অনুরোধ ব্যর্থ হয় কি না।
2. **রোলব্যাক মহড়া:** একটি ভাঙা পরিবর্তন ডিপ্লয় করে (ল্যাবে) ট্যাগে ফিরে রোলব্যাক করুন; কত মিনিট লাগল লিখুন।
3. **ব্লু-গ্রিন অনুশীলন:** একই সার্ভারে 5000 ও 5001 পোর্টে দুটি সংস্করণ চালিয়ে Nginx upstream বদলে সুইচ করুন ও ফেরত আনুন।
4. **মহড়ার সময়রেখা:** disposable VM-এ একটি রোলব্যাক মহড়া করে প্রতিটি ধাপের সময় লিখুন; কোন ধাপটি সবচেয়ে ধীর ছিল, তা চিহ্নিত করে রানবুক হালনাগাদ করুন।
5. **মাইগ্রেশন পরিকল্পনা:** একটি ফিল্ডের নাম বদলাতে হলে expand-and-contract-এর তিনটি ধাপ লিখুন।

অধ্যায় ২৪

Production Incident Response, Runbooks and Server Architecture

সব সার্ভারই একদিন ভাঙে — ডিস্ক ভরে, ডাটাবেস থামে, ডিপ্লয় ভুল হয়, ট্রাফিক হঠাৎ বাড়ে। প্রশ্ন নয় "ভাঙবে কি না", প্রশ্ন "ভাঙলে আমরা কত দ্রুত ও শান্তভাবে সারাব?" ইনসিডেন্ট রেসপন্স (incident response) হলো সেই শৃঙ্খলা: আগে থেকে লেখা রানবুক (runbook), স্পষ্ট ভূমিকা, আর ঘটনার পর শিক্ষা নেওয়ার সংস্কৃতি। এই অধ্যায়ে আমরা একটি সম্পূর্ণ ইনসিডেন্ট ব্যবস্থা গড়ব।

শেখার উদ্দেশ্য

- ইনসিডেন্টের তীব্রতা (severity) শ্রেণিবদ্ধ করে অগ্রাধিকার ঠিক করতে পারবেন।
- একটি সম্পূর্ণ রানবুক লিখতে ও অনুসরণ করতে পারবেন।
- সার্ভার আর্কিটেকচারের একক ব্যর্থতার বিন্দু (single point of failure) চিহ্নিত করতে পারবেন।
- ইনসিডেন্টের সময় যোগাযোগ, সমন্বয় ও সিদ্ধান্ত নেওয়ার কাঠামো ব্যাখ্যা করতে পারবেন।
- দোষারোপহীন পোস্ট-মর্টেম (blameless post-mortem) লিখে পুনরাবৃত্তি রোধের ব্যবস্থা নিতে পারবেন।

প্রোডাকশনে এটি কেন গুরুত্বপূর্ণ

ইনসিডেন্টের সময় সবচেয়ে ব্যয়বহুল জিনিস হলো বিভ্রান্তি — "কে কী করছে?", "কোথায় দেখব?", "আগে কী হয়েছিল?" — এই প্রশ্নগুলোর উত্তর খুঁজতে খুঁজতে ডাউনটাইম বাড়ে। গবেষণা ও অভিজ্ঞতা বলে: যেসব দলের রানবুক আছে, তারা গড়ে অনেক দ্রুত সারে, কারণ সিদ্ধান্ত আগে থেকে নেওয়া থাকে — ঘটনার সময় শুধু অনুসরণ করতে হয়। আর পোস্ট-মর্টেম না লিখলে একই ঘটনা বারবার ঘটে — প্রতিবার নতুন করে শেখার মূল্য দিয়ে।

> **বাস্তব ঘটনা:** একটি দলের ডাটাবেস ডিস্ক পূর্ণ হয়ে সাইট থামল। রানবুক ছিল না; একজন লগ মুছতে গিয়ে ভুল করে ডাটাবেস ফাইল মুছে ফেলার উপক্রম করল। ভাগ্যক্রমে ব্যাকআপ ছিল। এরপর তাদের প্রথম রানবুকটি ছিল "ডিস্ক পূর্ণ হলে কী করব" — আর সেটি আর কখনো জরুরি অবস্থায় লেখা হয়নি।

প্রয়োজনীয় পূর্বজ্ঞান

- অধ্যায় ২১–২৩: PM2, Nginx, MongoDB, ডিপ্লয় ও রোলব্যাক।
- Linux লগ পড়া (`journalctl` , `tail`) ও মৌলিক মনিটরিং।
- HTTP স্ট্যাটাস কোড ও DNS-এর প্রাথমিক ধারণা।

প্রথম নীতি: ঘটনার সময় মানুষ ভুল করে — ব্যবস্থাই বাঁচায়

প্রথম নীতি: চাপের মধ্যে মানুষের স্মৃতি ও বিচার দুর্বল হয়। তাই ইনসিডেন্ট ব্যবস্থা মানুষের স্মৃতির ওপর নির্ভর করে না — এটি নির্ভর করে লিখিত ধাপ, স্বয়ংক্রিয় সতর্কতা (alert) ও স্পষ্ট ভূমিকার ওপর। একজন "ঘটনা পরিচালক" (incident commander) সিদ্ধান্ত নেয়, বাকিরা কাজ করে; সবাই একই জায়গায় তথ্য লেখে; আর ঘটনার পর প্রশ্ন হয় "ব্যবস্থায় কী ঘাটতি ছিল?", "কে ভুল করল?" নয়। এই শেষ নীতিটিকে বলে দোষারোপহীন সংস্কৃতি (blameless culture) — মানুষ ভুল লুকালে ব্যবস্থা কখনো শেখে না।

আর্কিটেকচার: স্থিতিস্থাপক সার্ভার নকশা

ব্যবহারকারী → DNS → লোড ব্যালান্সার / Nginx → অ্যাপ সার্ভার ১ (PM2 cluster) + অ্যাপ সার্ভার ২
→ MongoDB (প্রাইমারি + রেপ্লিকা) • স্ট্যাটিক ফাইল / অবজেক্ট স্টোরেজ
পাশে: মনিটরিং ও অ্যালার্ট (স্বাস্থ্য, ডিস্ক, মেমোরি, ত্রুটির হার) • কেন্দ্রীয় লগ • নিয়মিত ব্যাকআপ (পরীক্ষিত পুনরুদ্ধারসহ)
একক ব্যর্থতার বিন্দু খুঁজুন: একটি সার্ভার? একটি ডিস্ক? একজন মানুষ যিনি একাই সব জানেন?

স্থিতিস্থাপকতা (resilience) মানে ভাঙা রোধ করা নয় — ভাঙলেও সেবা চালু রাখা। দুটি অ্যাপ সার্ভার থাকলে একটি মারা গেলেও অন্যটি চলে; ডাটাবেস রেপ্লিকা থাকলে প্রাইমারি মারা গেলে রেপ্লিকা এগিয়ে আসে। ছোট দলের পক্ষে সবকিছু দ্বিগুণ করা সম্ভব নয় — তাই অগ্রাধিকার দিন: প্রথমে ব্যাকআপ ও পুনরুদ্ধার, তারপর মনিটরিং, তারপর অতিরিক্ত সার্ভার।

বিস্তারিত ব্যাখ্যা: ইনসিডেন্টের জীবনচক্র

১. শনাক্তকরণ (Detection)

ব্যবহারকারী জানানোর আগেই মনিটরিং জানাবে — এটিই লক্ষ্য। Health check ব্যর্থ, ত্রুটির হার বাড়া, ডিস্ক ৮০% ছাড়ানো, লেটেন্সি বেড়ে যাওয়া — প্রতিটির জন্য সতর্কতা (alert) নির্ধারণ করুন। সতর্কতা এমন হতে হবে যাতে মানুষ ব্যবস্থা নেয়; অতিরিক্ত সতর্কতা মানুষকে সতর্কতাই উপেক্ষা করতে শেখায় (alert fatigue)।

২. তীব্রতা নির্ধারণ

তীব্রতা	অর্থ	উদাহরণ	প্রতিক্রিয়া
SEV-1 (জটিল)	সেবা সম্পূর্ণ বন্ধ / তথ্য ঝুঁকিতে	সাইট ডাউন, ডাটাবেস হারানো	সব কাজ থামিয়ে এখনই সারান
SEV-2 (উচ্চ)	বড় অংশ ক্ষতিগ্রস্ত	পেমেন্ট ব্যর্থ, লগইন ভাঙা	ঘণ্টার মধ্যে সারান
SEV-3 (মাঝারি)	সীমিত প্রভাব	একটি পেজ ধীর, ছোট ফিচার ভাঙা	কর্মদিবসে সারান
SEV-4 (নিম্ন)	প্রায় প্রভাব নেই	লগে সতর্কবার্তা, কসমেটিক ত্রুটি	পরিকল্পনা করে সারান

৩. প্রতিক্রিয়া: স্থিতিশীল করা আগে, কারণ খোঁজা পরে

প্রথম কাজ সেবা ফেরানো — রোলব্যাক, ট্রাফিক সরানো, অতিরিক্ত সম্পদ দেওয়া। কারণ অনুসন্ধান (root cause) পরে, শান্তভাবে। ভুল ক্রমে কাজ করলে (কারণ খুঁজতে গিয়ে সেবা বন্ধ রেখে) ক্ষতি বাড়ে। ঘটনা পরিচালক সময়রেখা (timeline) লেখেন: কখন কী দেখা গেল, কী করা হলো, কী ফল হলো।

৪. যোগাযোগ

ভেতরে: কে ঘটনা পরিচালক, কে কী করছে, পরের আপডেট কখন — একটি চ্যানেলে। বাইরে: ব্যবহারকারীকে সত্য ও সংক্ষেপে জানান — কী হয়েছে, কী প্রভাব, কখন পরের আপডেট। নীরবতা গুজবের চেয়ে খারাপ।

৫. পোস্ট-মর্টেম ও শিক্ষা

ঘটনার ২৪–৪৮ ঘণ্টার মধ্যে পোস্ট-মর্টেম লিখুন (টেমপ্লেট নিচে)। প্রতিটি শিক্ষা থেকে একটি করে কার্যকরী পদক্ষেপ (action item) — দায়িত্বপ্রাপ্ত ব্যক্তি ও সময়সীমাসহ। পদক্ষেপ ছাড়া পোস্ট-মর্টেম শুধু গল্প।

৬. দোষারোপহীন পোস্ট-মর্টেম ও অন-কল হস্তান্তর

পোস্ট-মর্টেমের সবচেয়ে কঠিন অংশ প্রযুক্তি নয় — সংস্কৃতি। "কে ভুল করল?" জিজ্ঞেস করলে মানুষ ভুল লুকাতে শেখে, আর লুকানো ভুল আবার ঘটে। দোষারোপহীন (blameless) পোস্ট-মর্টেমে প্রশ্নটি বদলে যায়: "ব্যবস্থায় কী ঘাটতি ছিল যে একজন সতর্ক মানুষও এই ভুলটি করতে পারল?" উদাহরণ: কেউ ভুল কমান্ড চালিয়ে ডাটাবেস মুছতে বসেছিল — দোষ তার নয়; দোষ ব্যবস্থার, কারণ ধ্বংসাত্মক কমান্ডে কোনো নিশ্চিতকরণ (confirmation) বা অনুমতি-সীমা ছিল না। ভালো পোস্ট-মর্টেমে তাই তিনটি জিনিস থাকে: সত্য সময়রেখা (কারও নামে দোষ না চাপিয়ে), মূল ও অবদানকারী কারণ, আর প্রতিটি শিক্ষার জন্য একজন দায়িত্বপ্রাপ্ত ব্যক্তিসহ কার্যকরী পদক্ষেপ ও সময়সীমা। পদক্ষেপ ছাড়া পোস্ট-মর্টেম শুধু গল্প — আর গল্পে ব্যবস্থা বদলায় না।

অন-কল হস্তান্তর (on-call handover) হলো দায়িত্ব একজনের কাছ থেকে আরেকজনের কাছে দেওয়ার লিখিত পদ্ধতি। হস্তান্তর ছাড়া ছুটির দিনে নতুন মানুষটি অন্ধকারে পড়ে: কোন সতর্কতা চলছে, কোন সমস্যা আধা-সারা, কোন সার্ভার নজরে রাখতে হবে — কিছুই জানে না। একটি ভালো হস্তান্তর নোটে থাকে: (১) চলমান সমস্যা ও তাদের অবস্থা; (২) সাম্প্রতিক ডিপ্লয় ও পরিচিত ঝুঁকি; (৩) গুরুত্বপূর্ণ যোগাযোগের পথ (কে ঘটনা পরিচালক, কাকে কখন ডাকবেন); (৪) পাসওয়ার্ড বা সিক্রেট নয় — শুধু কোথায় সেগুলো নিরাপদে পাওয়া যায়; (৫) আগের হস্তান্তরের পর কী বদলেছে। হস্তান্তর মৌখিক নয়, লিখিত — কারণ চাপের সময় মানুষ মৌখিক কথা ভুলে যায়। আর অন-কল শেষে একটি ছোট পর্যালোচনা রাখুন: কোন সতর্কতাটি ভুয়া ছিল, কোনটি আসল ছিল, রানবুকের কোন ধাপটি অস্পষ্ট লেগেছে — এভাবেই রানবুক জীবন্ত থাকে।

> **পরামর্শ:** পোস্ট-মর্টেম লেখার সময় "যদি" দিয়ে শুরু করা বাক্য এড়িয়ে চলুন ("যদি সে সতর্ক থাকত...") — বদলে ব্যবস্থার ভাষায় লিখুন ("সতর্কতা আসেনি কারণ...")। হস্তান্তর নোটে কখনো সিক্রেট পেস্ট করবেন না; সিক্রেট ম্যানেজারের পথটি লিখুন।

সম্পূর্ণ রানবুক টেমপ্লেট

```
# রানবুক: [পরিষেবার নাম] — [সমস্যার ধরন, যেমন: সাইট ডাউন / ডিস্ক পূর্ণ]
# শেষ হালনাগাদ: [তারিখ] | মালিক: [নাম] | পর্যালোচনা: প্রতি ৩ মাসে

## ১. সংক্ষিপ্ত বিবরণ
- এই রানবুক কখন ব্যবহার করবেন:
- প্রভাব: কারা ক্ষতিগ্রস্ত হবেন:
- তীব্রতা (SEV): SEV-1 / SEV-2 / SEV-3

## ২. তাৎক্ষণিক পরীক্ষা (প্রথম ৫ মিনিট)
1. সাইট কি বাইরে থেকে ডাউন? curl -I https://app.technova.example
2. সার্ভারে SSH হয় কি? ssh user@203.0.113.10
3. pm2 status — অ্যাপ online কি?
4. sudo systemctl status nginx mongod — সেবাগুলো চলছে কি?
5. df -h — ডিস্ক পূর্ণ কি? free -h — মেমোরি?
6. শেষ ডিপ্লয় কখন? (সাম্প্রতিক বদলই প্রায়শই কারণ)

## ৩. নির্ণয় (পরের ১৫ মিনিট)
- pm2 logs technova-api --lines 200 — শেষ ত্রুটি কী?
- sudo journalctl -u nginx --since "30 min ago"
- sudo tail -n 200 /var/log/nginx/error.log
- mongosh --eval 'db.runCommand({ ping: 1 })'
- ss -tlnp — প্রত্যাশিত পোর্টগুলো শুনছে কি?

## ৪. প্রতিকার (সেবা ফেরানো)
- অ্যাপ মারা গেলে: pm2 restart technova-api, তারপর health check
```

```
- সাম্প্রতিক ডিপ্লয় সন্দেহ হলে: রোলব্যাক (অধ্যায় ২৩-এর ধাপ)
- ডিস্ক পূর্ণ হলে: pm2 logs ঘূর্ণন, পুরোনো লগ/ব্যাকআপ সরান (ডাটাবেস ফাইল কখনো মুছবেন না)
- Nginx সমস্যা হলে: sudo nginx -t, তারপর reload
- প্রতিটি পদক্ষেপের পর health check করুন ও সময়রেখায় লিখুন

## ৫. যোগাযোগ
- ভেতরের চ্যানেল: [যেমন: দলের চ্যাট গ্রুপ]
- বাইরের বার্তা: "আমরা [সমস্যা] সম্পর্কে অবগত; [প্রভাব]; পরের আপডেট [সময়]-এ।"
- কাকে কখন জানাবেন: SEV-1 — অবিলম্বে সবাইকে

## ৬. ঘটনার পর
- পোস্ট-মর্টেম লিখুন (নিচের টেমপ্লেট)
- রানবুক হালনাগাদ করুন: যা কাজ করেনি, তা বদলান
- পুনরাবৃত্তি রোধের পদক্ষেপ নির্ধারণ করুন

## ৭. সতর্কতা
- প্রোডাকশনে ধ্বংসাত্মক কমান্ড (rm, ডাটাবেস মুছে ফেলা) চালানোর আগে দুইবার যাচাই করুন;
 সম্ভব হলে ঘটনা পরিচালকের অনুমোদন নিন। ব্যাকআপ/স্ন্যাপশট ছাড়া ঝুঁকিপূর্ণ কাজ নয়।
```

ইনসিডেন্ট-রিপোর্ট (পোস্ট-মর্টেম) টেমপ্লেট

```
# ইনসিডেন্ট রিপোর্ট / পোস্ট-মর্টেম — [শিরোনাম]
- তারিখ ও সময় (শুরু–শেষ, সময় অঞ্চলসহ):
- তীব্রতা: SEV-1 / SEV-2 / SEV-3
- ঘটনা পরিচালক: [নাম]
- প্রভাব: কতজন ব্যবহারকারী, কতক্ষণ, কী কী সেবা:
- সারসংক্ষেপ (২–৩ লাইন): কী ঘটেছিল?

## সময়রেখা (Timeline)
- HH:MM — প্রথম সতর্কতা / ব্যবহারকারীর অভিযোগ
- HH:MM — কী পরীক্ষা করা হলো, কী পাওয়া গেল
- HH:MM — কী প্রতিকার নেওয়া হলো
- HH:MM — সেবা ফিরল (নিশ্চিতকরণসহ)

## মূল কারণ (Root Cause)
- প্রযুক্তিগত কারণ:
- অবদানকারী কারণ (যেমন: সতর্কতা ছিল না, রানবুক পুরোনো):

## যা ভালো হয়েছে
-

## যা ভালো হয়নি / যা বিভ্রান্ত করেছে
-

## শিক্ষা
-

## কার্যকরী পদক্ষেপ (Action Items)
| পদক্ষেপ | দায়িত্ব | সময়সীমা | অবস্থা |
|---|---|---|---|
| (উদাহরণ) ডিস্ক ৮০%-এ সতর্কতা যোগ করুন | [নাম] | [তারিখ] | বাকি |
| (উদাহরণ) রানবুকের ডিস্ক-পূর্ণ ধাপ হালনাগাদ করুন | [নাম] | [তারিখ] | বাকি |
```

```
## দোষারোপহীন নীতি
এই নথি ব্যক্তিকে দোষ দিতে নয় — ব্যবস্থা শুধরাতে লেখা হয়েছে।
```

গুরুত্বপূর্ণ কমান্ড ও বিশ্লেষণ

কমান্ড	কী করে	বিশ্লেষণ
`sudo journalctl -u nginx --since "1 hour ago"`	সেবার সাম্প্রতিক লগ	সময়-সীমা দিয়ে লগ ছোট করুন — পুরো লগ পড়া সময় নষ্ট।
`sudo tail -n 200 -f /var/log/nginx/error.log`	ত্রুটির লগ সরাসরি দেখা	`-f` মানে নতুন লাইন এলেই দেখাবে।
`df -h && du -sh /var/log/* | sort -h | tail`	ডিস্ক ব্যবহার ও বড় লগ খোঁজা	ডিস্ক পূর্ণের সবচেয়ে দ্রুত নির্ণয়।
`free -h && top -b -n 1 | head -20`	মেমোরি ও CPU-খেকো প্রসেস	কোন প্রসেস মেমোরি খাচ্ছে, তা চিহ্নিত করুন।
`curl -f http://127.0.0.1:5000/api/health`	স্থানীয় health check	বাইরে থেকে ব্যর্থ কিন্তু ভেতরে সফল হলে সমস্যা Nginx/নেটওয়ার্কে।
`sudo ss -tlnp`	শোনা পোর্টের তালিকা	প্রত্যাশিত পোর্ট না থাকলে সেই সেবা মারা গেছে।
`uptime && who`	সার্ভার কতক্ষণ চলছে, কে লগইন	অপ্রত্যাশিত রিবুট বা অজানা লগইন ধরা পড়ে।
`sudo journalctl -u technova --since "30 min ago" --no-pager`	অ্যাপ সার্ভিসের সাম্প্রতিক লগ	সময়-সীমা ও `--no-pager` দিয়ে লগ দ্রুত পড়া যায়।
`sudo du -sh /var/* 2>/dev/null | sort -h | tail -5`	কোন ডিরেক্টরি সবচেয়ে বড়	ডিস্ক পূর্ণের উৎস দ্রুত চিহ্নিত করা যায়।
`ps aux --sort=-%mem | head -10`	সবচেয়ে বেশি মেমোরি খাওয়া প্রসেস	মেমোরি লিকের সন্দেহভাজন প্রসেস চোখে পড়ে।
`sudo systemctl list-units --failed`	ব্যর্থ সার্ভিসের তালিকা	কোন সেবা মারা গেছে, এক নজরে দেখা যায় — ঘটনার শুরুতে চালান।
`mongosh --eval 'db.runCommand({ ping: 1 })'`	ডাটাবেস জীবিত কি না	নমুনা আউটপুট — চিত্রণমূলক: { ok: 1 }।

বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — ডিস্ক পূর্ণ (SEV-1): ভোরে অ্যালার্ট: ডিস্ক ৯৮%। রানবুক অনুযায়ী `du` দিয়ে দেখা গেল PM2 লগ ৪০ GB। লগ ঘোরানো হলো, ডিস্ক খালি হলো, সাইট ফিরল। পোস্ট-মর্টেমের পদক্ষেপ: লগ ঘূর্ণন চালু + ৮০%-এ অ্যালার্ট। শিক্ষা: মনিটরিং থাকলে ঘটনা SEV-1 হওয়ার আগেই ধরা পড়ত।

পরিস্থিতি ২ — ডিপ্লয়ের পর সাইট ধীর (SEV-2): নতুন কোডে N+1 কোয়েরি — প্রতিটি অনুরোধে শত শত DB কোয়েরি। প্রতিকার: রোলব্যাক করে সেবা ফেরানো, পরে কোয়েরি ঠিক করে আবার ডিপ্লয়। শিক্ষা: স্থিতিশীল করা আগে, কারণ পরে।

পরিস্থিতি ৩ — ভুয়া সতর্কতা: প্রতি রাতে অ্যালার্ট আসত, সবাই উপেক্ষা করতে শিখল — তারপর এক রাতে আসল ঘটনা উপেক্ষিত হলো। শিক্ষা: ভুয়া সতর্কতা ঠিক করা ইনসিডেন্ট ব্যবস্থারই অংশ।

পরিস্থিতি ৪ — TechNova-তে দোষারোপহীন পোস্ট-মর্টেম ও হস্তান্তর: TechNova-র (technova.example, সার্ভার 203.0.113.10) এক জুনিয়র ডেভেলপার ভুল করে প্রোডাকশনে একটি পরীক্ষামূলক কমান্ড চালিয়ে অ্যাপ থামিয়ে দিলেন। আগের সংস্কৃতিতে তাকে দোষ দেওয়া হতো; এবার দল পোস্ট-মর্টেমে লিখল: "ধ্বংসাত্মক কমান্ডে কোনো নিশ্চিতকরণ ছিল না, আর প্রোডাকশন ও স্টেজিংয়ের প্রম্পট একই দেখাত।" পদক্ষেপ হলো: প্রোডাকশন শেলে সতর্কবার্তা যোগ করা ও ঝুঁকিপূর্ণ কমান্ডে ঘটনা পরিচালকের অনুমোদন বাধ্যতামূলক করা। একই সপ্তাহে অন-কল হস্তান্তর নোটে লেখা হলো চলমান ডিস্ক-সতর্কতা ও সাম্প্রতিক ডিপ্লয়ের ঝুঁকি — ফলে ছুটিতে যাওয়া সিনিয়রের বদলে নতুন অন-কল ব্যক্তি অন্ধকারে পড়েননি। শিক্ষা: মানুষকে দোষ দিলে ভুল লুকানো হয়; ব্যবস্থা শুধরালে ভুল আর ঘটে না।

ধাপে ধাপে ব্যবহারিক উদাহরণ: ইনসিডেন্ট মহড়া (Game Day)

1. Disposable VM-এ নমুনা অ্যাপ চালান; দলের একজন "ঘটনা পরিচালক" হোন, আরেকজন "প্রতিকারক"।
2. ইচ্ছা করে সমস্যা তৈরি করুন: `pm2 stop technova-api` বা ডিস্ক ভরার ভান (বড় ফাইল তৈরি) — **শুধু disposable VM-এ**, কখনো প্রোডাকশনে নয়।
3. রানবুক অনুসরণ করে সময় মেপে সারান; প্রতিটি পদক্ষেপ সময়রেখায় লিখুন।
4. সারানোর পর পোস্ট-মর্টেম টেমপ্লেট পূরণ করুন: মূল কারণ, শিক্ষা, দুটি কার্যকরী পদক্ষেপ।
5. রানবুকে যা অস্পষ্ট ছিল, তা হালনাগাদ করুন — মহড়ার উদ্দেশ্যই রানবুক শুধরানো।

> **সতর্কতা:** মহড়া কখনো প্রোডাকশন সার্ভারে নয়। ইচ্ছা করে প্রসেস থামানো, ডিস্ক ভরা বা সেবা বন্ধ করা শুধু ফেলে দেওয়া যায় এমন ল্যাব VM-এ করুন। প্রোডাকশনে যেকোনো ঝুঁকিপূর্ণ কাজের আগে ব্যাকআপ/স্ন্যাপশট নিন।

সাধারণ ভুল ও ভুল ধারণা

- **ভুল ধারণা:** "মনিটরিং মানেই ড্যাশবোর্ড" — ড্যাশবোর্ড কেউ না দেখলে মূল্যহীন; অ্যালার্টই আসল।
- **ভুল:** ঘটনার সময় একসাথে পাঁচজন আলাদা জিনিস বদলানো — কোনটি কাজ করল, জানা যায় না; একবারে একটি বদলান।
- **ভুল ধারণা:** "পোস্ট-মর্টেম মানেই দোষ খোঁজা" — দোষারোপ করলে মানুষ ভুল লুকায়, ব্যবস্থা শেখে না।
- **ভুল:** রানবুক লিখে আর কখনো না পড়া — রানবুক জীবন্ত নথি; প্রতি ৩ মাসে পর্যালোচনা করুন।
- **ভুল:** ব্যাকআপ নেওয়া কিন্তু পুনরুদ্ধার কখনো পরীক্ষা না করা — পরীক্ষিত নয় এমন ব্যাকআপ ভরসাযোগ্য নয়।

নিরাপত্তা বিবেচনা

- ইনসিডেন্টের সময়ও নিরাপত্তা ছাড় দেবেন না — ফায়ারওয়াল খুলে "সাময়িক" ঠিক করা স্থায়ী ছিদ্র হয়ে যায়।
- লগে পাসওয়ার্ড/টোকেন ছাপাবেন না; ইনসিডেন্ট চ্যাটে সিক্রেট পেস্ট করবেন না।
- সন্দেহজনক লগইন বা অজানা প্রসেস দেখলে সেটিকে নিরাপত্তা ঘটনা ধরে আলাদা রানবুক অনুসরণ করুন — শুধু restart নয়।
- ব্যাকআপের অ্যাক্সেস সীমিত রাখুন — ব্যাকআপ চুরি হলে পুরো ডাটাবেস চুরি হয়।

- ঘটনার পর অ্যাক্সেস লগ পর্যালোচনা করুন: ঘটনার সময় কে কী করেছিল?

সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
সাইট সম্পূর্ণ ডাউন	Nginx/অ্যাপ/সার্ভার বন্ধ	রানবুকের ৫ মিনিটের পরীক্ষা; স্তরে স্তরে যান (সার্ভার → Nginx → অ্যাপ → DB)।
মাঝে মাঝে 502	একটি ইনস্ট্যান্স অসুস্থ / মেমোরি শেষ	`pm2 status`, মেমোরি দেখুন; অসুস্থ ইনস্ট্যান্স restart করুন।
হঠাৎ ধীরগতি	DB কোয়েরি / ডিস্ক I/O / ট্রাফিক বৃদ্ধি	ধীর কোয়েরি লগ, `top`, অ্যাক্সেস লগের হার দেখুন।
ডিস্ক দ্রুত ভরছে	লগ বিস্ফোরণ / ব্যাকআপ জমা	`du` দিয়ে বড় ফাইল খুঁজুন; লগ ঘূর্ণন চালু করুন।
একই ঘটনা বারবার ঘটছে	পোস্ট-মর্টেমের পদক্ষেপ বাস্তবায়িত হয়নি	প্রতিটি পদক্ষেপের দায়িত্ব ও সময়সীমা দেখুন; পরের সভায় অগ্রগতি পর্যালোচনা করুন।
অন-কলে নতুন ব্যক্তি কিছুই জানে না	হস্তান্তর নোট নেই বা পুরোনো	লিখিত হস্তান্তর টেমপ্লেট চালু করুন; প্রতি হস্তান্তরে চলমান সমস্যা ও ঝুঁকি লিখুন।
ঘটনার সময় দুজন একই কাজ করছে	ভূমিকা স্পষ্ট নয়	একজন ঘটনা পরিচালক ঘোষণা করুন; বাকিরা তার নির্দেশে কাজ করবেন।
ব্যবহারকারী অভিযোগের আগেই জানতে পারল না	মনিটরিং শুধু ড্যাশবোর্ডে, অ্যালার্ট নেই	স্বাস্থ্য, ডিস্ক ও ত্রুটির হারের সতর্কতা (alert) নির্ধারণ করুন; অ্যালার্ট পরীক্ষা করুন।
অ্যালার্ট আসছে না	মনিটরিং নিজেই মারা গেছে	মনিটরিংয়েরও health check রাখুন; "নীরবতা" নিজেই একটি সতর্কতা।

সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

- **করুন:** প্রতিটি সাধারণ ব্যর্থতার জন্য রানবুক লিখুন; নিয়মিত মহড়া (game day) করুন।
- **করুন:** পোস্ট-মর্টেমের প্রতিটি পদক্ষেপের দায়িত্ব ও সময়সীমা দিন; পরের সভায় অগ্রগতি দেখুন।
- **করবেন না:** ঘটনার সময় দোষারোপ বা চিৎকার — শান্ত, লিখিত, ধাপে ধাপে কাজ করুন।
- **করবেন না:** "আর কখনো হবে না" বলে আশ্বস্ত হওয়া — ব্যবস্থা বদলালেই কেবল পুনরাবৃত্তি কমে।

অধ্যায়ের সারসংক্ষেপ

ইনসিডেন্ট অনিবার্য; বিভ্রান্তি নয়। তীব্রতা দিয়ে অগ্রাধিকার ঠিক করুন, রানবুক দিয়ে শান্তভাবে সেবা ফেরান, সময়রেখা লিখুন, আর দোষারোপহীন পোস্ট-মর্টেম দিয়ে ব্যবস্থা শুধরান। স্থিতিস্থাপক আর্কিটেকচার — অতিরিক্ত সার্ভার, রেপ্লিকা, মনিটরিং ও পরীক্ষিত ব্যাকআপ — ঘটনার আঘাত কমায়। শেষ কথা: রানবুক লেখা হয় শান্ত সময়ে, যাতে ঝড়ের সময় শুধু অনুসরণ করতে হয়।

মূল প্রযুক্তিগত পরিভাষা

ইনসিডেন্ট (Incident): সেবা ব্যাহত করা অপ্রত্যাশিত ঘটনা। **রানবুক (Runbook):** নির্দিষ্ট সমস্যা সারানোর লিখিত ধাপ-নির্দেশিকা। **পোস্ট-মর্টেম (Post-mortem):** ঘটনার পর কারণ, শিক্ষা ও পদক্ষেপের লিখিত পর্যালোচনা। **SEV (Severity):** ঘটনার তীব্রতার স্তর। **একক ব্যর্থতার বিন্দু (Single Point of Failure):** যার একক ব্যর্থতায় পুরো সেবা থামে। **স্থিতিস্থাপকতা (Resilience):** ভাঙলেও সেবা চালু রাখার ক্ষমতা। **Game Day:** ইচ্ছা করে ব্যর্থতা তৈরি করে প্রতিক্রিয়া অনুশীলন।

জ্ঞান যাচাই প্রশ্ন

1. SEV-1 ও SEV-3-এর পার্থক্য কী? প্রতিটির একটি উদাহরণ দিন।
2. ইনসিডেন্টের সময় "স্থিতিশীল করা আগে, কারণ খোঁজা পরে" — কেন এই ক্রম?
3. রানবুক না থাকলে ইনসিডেন্টের সময় কী কী সমস্যা হয়?
4. দোষারোপহীন পোস্ট-মর্টেম কেন জরুরি? দোষারোপ করলে কী ক্ষতি?
5. একক ব্যর্থতার বিন্দু কী? নিজের সার্ভারে তিনটি খুঁজে বের করুন।
6. Alert fatigue কী, এবং তা কীভাবে রোধ করবেন?
7. অন-কল হস্তান্তর নোটে কী কী থাকা উচিত, আর কী কখনো থাকা উচিত নয়?
8. পোস্ট-মর্টেমে "যদি সে সতর্ক থাকত" ধরনের বাক্য কেন এড়িয়ে চলবেন?
9. ব্যাকআপ থাকলেই কি যথেষ্ট? পুনরুদ্ধার পরীক্ষা কেন জরুরি?

ব্যবহারিক অনুশীলন

1. **রানবুক লেখা:** "ডিস্ক পূর্ণ" সমস্যার জন্য এই অধ্যায়ের টেমপ্লেটে নিজের রানবুক লিখুন।
2. **Game Day:** disposable VM-এ ইচ্ছা করে অ্যাপ থামিয়ে রানবুক অনুসরণ করে সারান; সময় মাপুন ও সময়রেখা লিখুন।
3. **পোস্ট-মর্টেম:** মহড়ার ঘটনাটির পোস্ট-মর্টেম লিখুন — মূল কারণ, শিক্ষা ও দুটি কার্যকরী পদক্ষেপসহ।
4. **হস্তান্তর নোট লেখা:** একটি কাল্পনিক সপ্তাহের জন্য অন-কল হস্তান্তর নোট লিখুন — চলমান সমস্যা, সাম্প্রতিক ডিপ্লয়, ঝুঁকি ও যোগাযোগের পথসহ (কোনো সিক্রেট লিখবেন না)।
5. **SPOF নিরীক্ষা:** নিজের (ল্যাব) আর্কিটেকচারের চিত্র এঁকে একক ব্যর্থতার বিন্দুগুলো চিহ্নিত করুন ও প্রতিটির প্রতিকার লিখুন।

পরিশিষ্ট ক

শব্দকোষ (Glossary)

এই শব্দকোষে বইটিতে ব্যবহৃত গুরুত্বপূর্ণ প্রযুক্তিগত পরিভাষাগুলোর বাংলা ব্যাখ্যা দেওয়া হলো। প্রযুক্তিগত নাম ইংরেজিতে রাখা হয়েছে, কারণ প্রোডাকশন পরিবেশে সেগুলোই ব্যবহৃত হয়।

পরিভাষা (Term)	বাংলা ব্যাখ্যা
API	দুটি সফটওয়্যারের মধ্যে যোগাযোগের নিয়ম ও মাধ্যম; এক প্রোগ্রাম অন্য প্রোগ্রামকে সেবা চাইতে পারে।
Access Log	সার্ভারে আসা প্রতিটি অনুরোধের রেকর্ড; কে, কখন, কী চাইল তা থাকে।
Alert	কোনো সীমা ছাড়ালে বা সমস্যা দেখা দিলে স্বয়ংক্রিয় সতর্কবার্তা।
Alert Fatigue	অতিরিক্ত ভুয়া সতর্কতায় মানুষ আসল সতর্কতাও উপেক্ষা করতে শুরু করা।
AMI	অ্যামাজন মেশিন ইমেজ; EC2 VM তৈরির ছাঁচ বা টেমপ্লেট।
Authentication	ব্যবহারকারী বা সিস্টেম আসলেই সে-ই কি না যাচাই করা।
Authorization	যাচাইকৃত ব্যবহারকারী কী কী করতে পারবে তা নির্ধারণ করা।
Availability	সেবা কতটুকু সময় ব্যবহারযোগ্য থাকে তার পরিমাপ।
Backup	তথ্য বা সিস্টেমের অনুলিপি, যাতে হারালে ফেরানো যায়।
Bandwidth	নেটওয়ার্কে নির্দিষ্ট সময়ে কত তথ্য যেতে পারে তার ক্ষমতা।
Blue-Green Deployment	দুটি অভিন্ন পরিবেশের মধ্যে ট্রাফিক সুইচ করে ডিপ্লয় করার কৌশল।
Blameless Post-mortem	ব্যক্তিকে দোষ না দিয়ে ব্যবস্থার ঘাটতি খুঁজে ঘটনা পর্যালোচনা।
Boot	কম্পিউটার চালু হয়ে অপারেটিং সিস্টেম লোড হওয়ার প্রক্রিয়া।
Cache	বারবার ব্যবহৃত তথ্যের দ্রুত অনুলিপি, যাতে আবার হিসাব বা আনা না লাগে।
Canary Deployment	প্রথমে অল্প ব্যবহারকারীর কাছে নতুন সংস্করণ পাঠিয়ে পরীক্ষা করা।
CDN	কনটেন্ট ডেলিভারি নেটওয়ার্ক; ব্যবহারকারীর কাছাকাছি সার্ভার থেকে স্ট্যাটিক ফাইল দেওয়া।
Certificate	ডিজিটাল সনদ যা সার্ভারের পরিচয় প্রমাণ করে ও HTTPS সক্ষম করে।
CI/CD	কন্টিনিউয়াস ইন্টিগ্রেশন ও ডেলিভারি; কোড পরীক্ষা ও ডিপ্লয়ের স্বয়ংক্রিয় ধারা।
CLI	কমান্ড লাইন ইন্টারফেস; লিখিত কমান্ড দিয়ে কম্পিউটার চালানো।
Cloud	ইন্টারনেটের মাধ্যমে ভাড়ায় পাওয়া কম্পিউটিং সম্পদ (সার্ভার, স্টোরেজ)।
Cluster Mode	একাধিক CPU কোরে একই অ্যাপের একাধিক ইনস্ট্যান্স চালানো।
Configuration	সফটওয়্যারের আচরণ নির্ধারণকারী সেটিংস ও মানের সমষ্টি।

Container	অ্যাপ ও তার নির্ভরতা একসাথে বন্দি করা হালকা প্যাকেজিং পদ্ধতি।
CORS	ক্রস-অরিজিন রিসোর্স শেয়ারিং; এক ডোমেইনের পেজ অন্য ডোমেইনের API ডাকতে পারবে কি না তার নিয়ম।
CPU	কম্পিউটারের প্রধান প্রসেসর; হিসাব ও নির্দেশনা চালায়।
Crash	প্রোগ্রাম অপ্রত্যাশিতভাবে থেমে যাওয়া।
Cron	Linux-এ নির্দিষ্ট সময়ে স্বয়ংক্রিয়ভাবে কাজ চালানোর ব্যবস্থা।
Daemon	পেছনে নিরবে চলা সিস্টেম প্রোগ্রাম বা সেবা।
Database	সাজানোভাবে তথ্য সংরক্ষণের ব্যবস্থা।
Dependency	একটি প্রোগ্রাম চালাতে প্রয়োজনীয় অন্য লাইব্রেরি বা প্যাকেজ।
Deployment	কোড প্রোডাকশন সার্ভারে পৌঁছে চালু করার প্রক্রিয়া।
Disk Usage	ডিস্কে কত জায়গা ব্যবহৃত হয়েছে তার পরিমাপ।
DNS	ডোমেইন নামকে IP ঠিকানায় অনুবাদ করার ব্যবস্থা।
Docker	কনটেইনার তৈরি ও চালানোর জনপ্রিয় প্ল্যাটফর্ম।
Domain	ওয়েবসাইটের মানুষের পড়ার মতো নাম, যেমন app.technova.example।
Downtime	সেবা ব্যবহারকারীর কাছে বন্ধ থাকার সময়।
Environment Variable	প্রসেস চালুর সময় দেওয়া নাম-মান জোড়া; কনফিগ রাখার উপায়।
Error Log	সিস্টেম বা অ্যাপের ত্রুটির রেকর্ড।
Failover	প্রধান ব্যবস্থা ভাঙলে বিকল্প ব্যবস্থা স্বয়ংক্রিয়ভাবে দায়িত্ব নেওয়া।
Firewall	নেটওয়ার্ক ট্রাফিক ছাঁকনি; কোন সংযোগ ঢুকতে পারবে তা নিয়ন্ত্রণ করে।
FQDN	সম্পূর্ণ ডোমেইন নাম, যেমন app.technova.example।
Graceful Shutdown	চলমান কাজ শেষ করে শান্তভাবে প্রসেস বন্ধ করা।
Health Check	অ্যাপ বা সেবা জীবিত ও সুস্থ আছে কি না যাচাই।
High Availability	সেবা প্রায় সবসময় চালু রাখার নকশা ও ক্ষমতা।
Hostname	নেটওয়ার্কে একটি মেশিনের নাম।
HTTP	ওয়েবে তথ্য আদান-প্রদানের প্রোটোকল।
HTTPS	এনক্রিপ্ট করা নিরাপদ HTTP; তথ্য গোপন থাকে।
Incident	সেবা ব্যাহত করা অপ্রত্যাশিত ঘটনা।
Incident Commander	ঘটনার সময় সিদ্ধান্ত ও সমন্বয়ের দায়িত্বে থাকা ব্যক্তি।

Instance	চলমান প্রোগ্রামের একটি কপি, বা ক্লাউডে একটি VM।
IP Address	নেটওয়ার্কে মেশিনের সংখ্যাসূচক ঠিকানা।
Latency	অনুরোধ পাঠিয়ে উত্তর পেতে লাগা সময়।
Load Balancer	আসা ট্রাফিক একাধিক সার্ভারে ভাগ করে দেওয়ার ব্যবস্থা।
Log	সিস্টেম বা অ্যাপের ঘটনার কালানুক্রমিক রেকর্ড।
Log Rotation	পুরোনো লগ সংরক্ষণ বা মুছে নতুন লগ শুরু করা, যাতে ডিস্ক না ভরে।
LTS	লং টার্ম সাপোর্ট; দীর্ঘদিন নিরাপত্তা হালনাগাদ পাওয়া সংস্করণ।
Memory Leak	প্রোগ্রাম ব্যবহৃত মেমোরি ফেরত না দিয়ে ধীরে ধীরে সব মেমোরি খেয়ে ফেলা।
MERN	MongoDB, Express, React, Node.js — জাভাস্ক্রিপ্টভিত্তিক ফুল-স্ট্যাক প্রযুক্তি সমষ্টি।
Microservice	বড় অ্যাপকে ছোট ছোট স্বাধীন সেবায় ভাগ করার স্থাপত্য।
Migration	ডাটাবেসের কাঠামো বা তথ্য এক সংস্করণ থেকে অন্য সংস্করণে বদলানো।
Monitoring	সিস্টেমের স্বাস্থ্য, কর্মক্ষমতা ও ত্রুটি নিয়মিত পর্যবেক্ষণ করা।
MongoDB	ডকুমেন্টভিত্তিক NoSQL ডাটাবেস; তথ্য JSON-এর মতো নথিতে রাখে।
Network	যোগাযোগের জন্য সংযুক্ত কম্পিউটারের ব্যবস্থা।
Nginx	দ্রুত ওয়েব সার্ভার ও রিভার্স প্রক্সি সফটওয়্যার।
Node.js	সার্ভারে জাভাস্ক্রিপ্ট চালানোর রানটাইম পরিবেশ।
npm	Node প্যাকেজ ম্যানেজার; লাইব্রেরি ইনস্টল ও পরিচালনার টুল।
Operating System	হার্ডওয়্যার ও সফটওয়্যারের মধ্যে সমন্বয়কারী মূল সফটওয়্যার।
Package	ইনস্টলযোগ্য সফটওয়্যারের একক, যেমন apt প্যাকেজ।
Patch	সফটওয়্যারের ত্রুটি বা নিরাপত্তা ছিদ্র সারানোর ছোট হালনাগাদ।
Permissions	ফাইল বা সম্পদ কে পড়তে, লিখতে বা চালাতে পারবে তার নিয়ম।
PM2	Node.js অ্যাপের প্রসেস ম্যানেজার; চালু রাখা, লগ ও পুনরায় চালুর কাজ করে।
Port	নেটওয়ার্কে নির্দিষ্ট সেবা চেনার নম্বর, যেমন ৮০, ৪৪৩, ৫০০০।
Post-mortem	ঘটনার পর কারণ, শিক্ষা ও পদক্ষেপের লিখিত পর্যালোচনা।
Process	চলমান প্রোগ্রামের একটি উদাহরণ; CPU ও মেমোরি ব্যবহার করে।
Production	আসল ব্যবহারকারীরা ব্যবহার করা জীবন্ত পরিবেশ।
Protocol	যোগাযোগের নিয়মাবলি, যেমন HTTP, SSH।
Proxy	ক্লায়েন্ট ও সার্ভারের মাঝে মধ্যস্থতাকারী সার্ভার।

Public Key	SSH-তে পরিচয় যাচাইয়ে ব্যবহৃত চাবি জোড়ার প্রকাশ্য অংশ।
Private Key	চাবি জোড়ার গোপন অংশ; কখনো কারও সঙ্গে ভাগ করা যায় না।
Rate Limiting	নির্দিষ্ট সময়ে অনুরোধের সংখ্যা সীমিত করা; অপব্যবহার রোধ করে।
Redundancy	ব্যর্থতা সামলাতে অতিরিক্ত অনুলিপি বা বিকল্প ব্যবস্থা রাখা।
Release	ব্যবহারকারীর কাছে পৌঁছে দেওয়া সফটওয়্যারের নির্দিষ্ট সংস্করণ।
Replica	ডাটাবেস বা সেবার অনুলিপি; ব্যর্থতায় দায়িত্ব নিতে পারে।
Repository	কোড সংরক্ষণের জায়গা, যেমন git সংগ্রহস্থল।
Resilience	ভাঙলেও সেবা চালু রাখার ক্ষমতা।
Reverse Proxy	সার্ভারের সামনে বসে অনুরোধ ভেতরের সার্ভারে পাঠানো প্রক্সি।
Rollback	আগের পরিচিত-ভালো সংস্করণে ফেরা।
Rolling Deployment	একবারে একটি করে ইনস্ট্যান্স বদলিয়ে ডিপ্লয় করা।
Root Cause	ঘটনার মূল কারণ; লক্ষণ নয়, গোড়ার সমস্যা।
Root User	Linux-এ সর্বোচ্চ ক্ষমতাসম্পন্ন ব্যবহারকারী।
Runbook	নির্দিষ্ট সমস্যা সারানোর লিখিত ধাপ-নির্দেশিকা।
Scalability	বাড়তি চাপ সামলাতে সক্ষমতা বাড়ানোর গুণ।
Secret	পাসওয়ার্ড, API চাবি, টোকেনের মতো গোপন মান।
Secret Rotation	নিয়মিত বা ফাঁসের পর গোপন মান বদলানো।
Security Group	ক্লাউডে VM-এর নেটওয়ার্ক ফায়ারওয়াল নিয়মের সেট।
Server	অন্য কম্পিউটারকে সেবা দেওয়া কম্পিউটার বা প্রোগ্রাম।
Service	পেছনে চলা সিস্টেম প্রোগ্রাম; systemd দিয়ে পরিচালিত হয়।
Severity (SEV)	ঘটনার তীব্রতার স্তর, যেমন SEV-1 জটিলতম।
Shell	কমান্ড লাইনে নির্দেশনা দেওয়ার মাধ্যম, যেমন bash।
Snapshot	ডিস্ক বা VM-এর নির্দিষ্ট মুহূর্তের অনুলিপি; ফেরানো যায়।
SSH	সিকিউর শেল; এনক্রিপ্ট করা দূরবর্তী লগইন ও কমান্ডের প্রোটোকল।
SSL/TLS	ইন্টারনেট যোগাযোগ এনক্রিপ্ট করার প্রোটোকল; HTTPS-এর ভিত্তি।
Staging	প্রোডাকশনের মতো পরীক্ষার পরিবেশ; ডিপ্লয়ের আগে যাচাইয়ের জায়গা।
Static File	সার্ভারে আগে থেকে তৈরি ফাইল, যেমন HTML, CSS, ছবি।
	বদলায় না এমন স্থায়ী IP ঠিকানা।

Static IP	
Storage	তথ্য সংরক্ষণের মাধ্যম, যেমন ডিস্ক।
Subnet	বড় নেটওয়ার্কের ছোট ভাগ।
sudo	সাধারণ ব্যবহারকারীকে প্রশাসনিক ক্ষমতায় কমান্ড চালাতে দেওয়া।
Systemd	Linux-এর সেবা ও বুট ব্যবস্থাপনার মূল ব্যবস্থা।
Tag	git-এ নির্দিষ্ট কমিটে দেওয়া সংস্করণের চিহ্ন।
Terminal	কমান্ড লাইন ব্যবহারের উইন্ডো বা মাধ্যম।
Throughput	নির্দিষ্ট সময়ে সম্পন্ন কাজ বা তথ্যের পরিমাণ।
Timeout	উত্তরের জন্য অপেক্ষার সর্বোচ্চ সময়; ছাড়ালে ব্যর্থ ধরা হয়।
Token	পরিচয় বা অনুমতি প্রমাণের ডিজিটাল চাবি, যেমন JWT।
Traffic	নেটওয়ার্কে আসা-যাওয়া অনুরোধ ও তথ্যের প্রবাহ।
Troubleshooting	সমস্যার কারণ খুঁজে ধাপে ধাপে সারানোর প্রক্রিয়া।
UFW	আনকমপ্লিকেটেড ফায়ারওয়াল; Ubuntu-র সহজ ফায়ারওয়াল টুল।
Ubuntu Server	সার্ভারের জন্য তৈরি Ubuntu Linux সংস্করণ।
Uptime	সিস্টেম বা সেবা নিরবচ্ছিন্নভাবে চালু থাকার সময়।
URL	ইন্টারনেটে সম্পদের ঠিকানা।
Version Control	কোডের পরিবর্তনের ইতিহাস রাখার ব্যবস্থা, যেমন git।
Virtual Machine (VM)	সফটওয়্যার দিয়ে তৈরি ভার্চুয়াল কম্পিউটার।
VPN	ভার্চুয়াল প্রাইভেট নেটওয়ার্ক; নিরাপদ ব্যক্তিগত সংযোগ।
Vulnerability	সিস্টেমের নিরাপত্তা দুর্বলতা, যা আক্রমণকারী ব্যবহার করতে পারে।
Web Server	HTTP অনুরোধে ওয়েব পেজ বা তথ্য দেওয়া সার্ভার।
Zero-Downtime Deployment	সেবা বন্ধ না করে নতুন সংস্করণ চালু করা।
AB Testing	দুটি সংস্করণের মধ্যে তুলনা করে কোনটি ভালো তা দেখা।
API Gateway	সব API অনুরোধের একক প্রবেশদ্বার; রাউটিং ও নিরাপত্তা সামলায়।
Asynchronous	কাজ শেষের অপেক্ষা না করে এগিয়ে যাওয়া; পরে ফল আসে।
Auto-scaling	চাপ অনুযায়ী স্বয়ংক্রিয়ভাবে সার্ভারের সংখ্যা বাড়ানো-কমানো।
Brute Force Attack	বারবার অনুমান করে পাসওয়ার্ড ভাঙার চেষ্টা।
Changelog	প্রতিটি সংস্করণে কী বদলেছে তার তালিকা।

Connection Pool	ডাটাবেস সংযোগের পুনঃব্যবহারযোগ্য সংগ্রহ; বারবার সংযোগের খরচ বাঁচায়।
Content-Type	HTTP বার্তায় তথ্যের ধরন জানানো হেডার।
Data Loss	তথ্য স্থায়ীভাবে হারিয়ে যাওয়া।
Debugging	ত্রুটি খুঁজে বের করে সারানো।
DDoS Attack	বিপুল ভুয়া ট্রাফিক পাঠিয়ে সেবা অচল করা।
DNS Record	ডোমেইন নামের সঙ্গে IP বা অন্য তথ্যের ম্যাপিং।
Encryption	তথ্যকে গোপন সংকেতে বদলে দেওয়া, যাতে অনুমোদিতরাই পড়তে পারে।
Endpoint	API-র নির্দিষ্ট ঠিকানা যেখানে অনুরোধ পাঠানো হয়।
Error Rate	মোট অনুরোধের মধ্যে ব্যর্থ অনুরোধের অনুপাত।
Feature Flag	কোড ডিপ্লয় না করে ফিচার চালু-বন্ধ করার সুইচ।
File System	ডিস্কে ফাইল সাজিয়ে রাখার ব্যবস্থা।
Firewall Rule	কোন ট্রাফিক অনুমোদিত তা নির্ধারণকারী নিয়ম।
Git	কোডের সংস্করণ নিয়ন্ত্রণের জনপ্রিয় ব্যবস্থা।
GitHub	git সংগ্রহস্থল রাখার ও সহযোগিতার অনলাইন প্ল্যাটফর্ম।
Header	HTTP বার্তার সঙ্গে যাওয়া অতিরিক্ত তথ্য।
Hotfix	জরুরি ত্রুটি সারাতে দ্রুত ছোট সংশোধন।
Load Test	ভারী ট্রাফিকে সিস্টেম কেমন করে তা পরীক্ষা করা।
Middleware	অনুরোধ ও উত্তরের মাঝে চলা কোডের স্তর; যেমন লগিং বা প্রমাণীকরণ।
N+1 Query	একটি তালিকার প্রতিটি আইটেমের জন্য আলাদা ডাটাবেস কোয়েরি — ধীরগতির সাধারণ কারণ।
Outage	সেবা বন্ধ থাকার ঘটনা।
Packet	নেটওয়ার্কে তথ্য পাঠানোর ছোট একক।
PostgreSQL	শক্তিশালী ওপেন-সোর্স রিলেশনাল ডাটাবেস।
Query	ডাটাবেসের কাছে তথ্য চাওয়ার নির্দেশ।
Recovery	ব্যর্থতার পর সেবা বা তথ্য ফিরিয়ে আনা।
Session	ব্যবহারকারীর লগইন অবস্থা ধরে রাখা সময়কাল।
Single Point of Failure (SPOF)	যার একক ব্যর্থতায় পুরো সেবা থামে এমন অংশ।
SSL Certificate	HTTPS-এর জন্য সার্ভারের পরিচয়পত্র।

Swap	RAM পূর্ণ হলে ডিস্কের অস্থায়ী মেমোরি হিসেবে ব্যবহার।
TCP	নির্ভরযোগ্য সংযোগভিত্তিক নেটওয়ার্ক প্রোটোকল।
UDP	দ্রুত কিন্তু অনিশ্চিত ডেটাগ্রাম প্রোটোকল।
Webhook	ঘটনা ঘটলে অন্য সিস্টেমকে স্বয়ংক্রিয়ভাবে জানানো HTTP কল।
Worker Process	মূল প্রক্রিয়ার অধীনে নির্দিষ্ট কাজ করা সহায়ক প্রসেস।
YAML	মানুষের পড়ার মতো কনফিগারেশন লেখার বিন্যাস।
Zone (DNS Zone)	একটি ডোমেইনের সব DNS রেকর্ডের সংগ্রহ।

পরিশিষ্ট খ

তথ্যসূত্র (References)

নিচের তালিকায় এই বইয়ে ব্যবহৃত প্রযুক্তিগুলোর অফিসিয়াল ডকুমেন্টেশন দেওয়া হলো। প্রযুক্তি ও ক্লাউড সেবার বিস্তারিত, হালনাগাদ নিয়ম ও সীমা জানতে সবসময় এই অফিসিয়াল উৎসগুলো দেখুন — তৃতীয় পক্ষের ব্লগ নয়।

1. Ubuntu Server Documentation — https://ubuntu.com/server/docs
2. Linux man-pages (man7.org) — https://man7.org/linux/man-pages/
3. systemd Documentation — https://www.freedesktop.org/software/systemd/man/
4. OpenSSH Manual — https://www.openssh.com/manual.html
5. Nginx Documentation — https://nginx.org/en/docs/
6. Node.js Documentation — https://nodejs.org/en/docs
7. PM2 Documentation — https://pm2.keymetrics.io/docs/usage/quick-start/
8. PostgreSQL Documentation — https://www.postgresql.org/docs/
9. MongoDB Documentation — https://www.mongodb.com/docs/
10. AWS EC2 Documentation — https://docs.aws.amazon.com/ec2/
11. Microsoft Azure Virtual Machines Documentation — https://learn.microsoft.com/en-us/azure/virtual-machines/
12. Google Compute Engine Documentation — https://cloud.google.com/compute/docs
13. IBM Cloud Virtual Servers Documentation — https://cloud.ibm.com/docs/vpc

দ্রষ্টব্য: উপরের লিংকগুলো অফিসিয়াল ডকুমেন্টেশনের অবতরণ পৃষ্ঠা (landing page)। নির্দিষ্ট কোনো বৈশিষ্ট্য, মূল্য বা সীমা জানতে সংশ্লিষ্ট পৃষ্ঠার ভেতরের অংশ দেখুন, কারণ সেগুলো সময়ের সঙ্গে বদলাতে পারে।

পরিশিষ্ট গ — পরবর্তী ধাপ: এখান থেকে কোথায়

এই বইটি শেষ করা মানে প্রোডাকশন সার্ভার চালানোর যাত্রা শেষ হওয়া নয় — এর শুরু। নিচের পথটি অনুসরণ করলে এই বইয়ের জ্ঞান বাস্তব দক্ষতায় পরিণত হবে।

১. ৩০ দিনের অনুশীলন পরিকল্পনা

সপ্তাহ	কাজ	যাচাই
১ম সপ্তাহ	ল্যাব ১–৬ শেষ করুন: VM ইনস্টল, SSH key, ব্যবহারকারী, ডিস্ক ও প্রসেস।	নতুন VM-এ key দিয়ে লগইন, `systemctl status` পরিষ্কার পড়তে পারা।
২য় সপ্তাহ	ল্যাব ৭–১২: systemd সার্ভিস, Nginx, Virtual Host, Reverse Proxy, TLS।	নিজের ডোমেইনে (বা fictional ডোমেইনে) HTTPS-সহ সাইট চালু।
৩য় সপ্তাহ	ল্যাব ১৩–১৮: Node ডেপ্লয়, PM2, ডেটাবেস, ডিস্ক ও পারফরম্যান্স তদন্ত।	PM2-এ চলমান অ্যাপ, `pg_dump` ও `mongodump` থেকে restore।
৪র্থ সপ্তাহ	ল্যাব ১৯–২৪: ফায়ারওয়াল, SSH hardening, ব্যাকআপ, cron, লগ ও ইনসিডেন্ট মহড়া।	সম্পূর্ণ ব্যাকআপ–restore চক্র ও একটি লেখা রানবুক।

২. প্রজেক্ট ও ক্যাপস্টোন

ওয়ার্কবুকের তিনটি মেজর প্রজেক্ট ক্রমানুসারে সম্পূর্ণ করুন — প্রথমে Secure Ubuntu VPS, তারপর Production MERN Deployment, তারপর TechNova Production Server Operations। শেষে ক্যাপস্টোনটি নিজে করুন; সমাধান-নির্দেশিকা শুধু নিজের কাজ যাচাই করতে খুলুন। প্রতিটি প্রজেক্টের ডকুমেন্টেশন (রানবুক, আর্কিটেকচার চিত্র, ব্যাকআপ পরীক্ষার প্রমাণ) আপনার পোর্টফোলিওর অংশ — ভবিষ্যতের নিয়োগকর্তাকে দেখানোর মতো করে লিখুন।

৩. প্রতিদিনের অভ্যাস

- প্রতিদিন সার্ভারের লগ একবার দেখুন — `journalctl --since today -p warning` দিয়ে শুরু করুন।
- প্রতি সপ্তাহে `apt update` ও নিরাপত্তা আপডেট পরীক্ষা করুন; মাসে একবার ব্যাকআপ ফিরিয়ে এনে যাচাই করুন।
- প্রতিটি ঘটনা (incident) লিখে রাখুন: কী হয়েছিল, কীভাবে বুঝলেন, কী করলেন, আবার যাতে না হয় তার জন্য কী বদলালেন।
- কখনো প্রোডাকশনে প্রথমবার কিছু করবেন না — প্রথমে ল্যাব VM-এ করুন, তারপর প্রোডাকশনে।

> **WARNING:** এই বইয়ের কোনো কমান্ডই সরাসরি প্রোডাকশন সার্ভারে পরীক্ষা করবেন না। সবসময় প্রথমে disposable VM বা আলাদা ল্যাব পরিবেশে করুন, স্ন্যাপশট নিন, এবং SSH ও ফায়ারওয়াল বদলানোর আগে বিকল্প লগইন পথ নিশ্চিত করুন।

৪. আরও শেখার দিক

এই সিরিজের পরের খণ্ডগুলোতে আসবে: ক্লাউড প্রোভাইডার-নির্দিষ্ট ইঞ্জিনিয়ারিং (AWS, Azure, GCP, IBM Cloud), কন্টেইনার ও Kubernetes, CI/CD পাইপলাইন, পর্যবেক্ষণ (monitoring) ও সতর্কতা (alerting), এবং Infrastructure as Code। এই খণ্ডের ভিত্তি — Linux, systemd, Nginx, ব্যাকআপ, লগ ও ইনসিডেন্ট ব্যবস্থাপনা — সেগুলোর প্রতিটির নিচে থাকবে।

৫. নিজেকে যাচাই করুন

অ্যাসেসমেন্ট গাইডের ১২০টি MCQ ও চূড়ান্ত পরীক্ষা দিন, তারপর ক্যাপস্টোন রুব্রিক দিয়ে নিজেকে নম্বর দিন। ৮০ বা তার বেশি পেলে আপনি একটি ছোট প্রোডাকশন সার্ভারের দায়িত্ব নিতে প্রস্তুত — তবে প্রথম কয়েকটি প্রোডাকশন কাজ একজন অভিজ্ঞের তত্ত্বাবধানে করুন।